



Oracle Database 23ai: SQL Workshop

Student Guide - Volume I
S1106065GC10

Copyright © 2024, Oracle and/or its affiliates.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Trademark Notice

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

1003222024

For Instructor Use Only.
This document should not be distributed

Contents

O Course Overview

Course Outline O-2

Target Audience O-4

Learning Outcomes O-5

What's Next? I-7

1 Introduction

Objectives 1-2

Overview of Oracle Database 23c and Related Products 1-3

Oracle Database 23c: Focus Areas 1-4

Oracle Database 23c 1-5

Overview of Relational Database Management Concepts and Terminologies 1-7

Relational and Object Relational Database Management Systems 1-8

Data Storage on Different Media 1-9

Relational Database Concept 1-10

Definition of a Relational Database 1-11

Data Models 1-12

Entity Relationship Model 1-13

Entity Relationship Modeling Conventions 1-14

Relating Multiple Tables 1-16

Relational Database Terminology 1-17

Human Resources (HR) Schema and the Tables Used in This Course 1-18

Human Resources (HR) Application 1-19

Tables Used in This Course 1-20

Tables Used in the Course 1-21

Introduction to SQL and Its Development Environments 1-22

Using SQL to Query Your Database 1-23

How SQL Works 1-24

SQL Statements Used in the Course 1-25

Development Environments for SQL in Oracle 1-26

Introduction to Oracle Live SQL 1-27

Oracle Database 23c SQL Documentation and Additional Resources	1-28
Oracle Database Documentation	1-29
Additional Resources for Oracle	1-30
Oracle University: Oracle SQL Training	1-31
Oracle SQL Certification	1-32
Summary	1-33

2 Retrieving Data Using the SQL SELECT Statement

Course Roadmap	2-2
Objectives	2-3
Capabilities of SQL SELECT Statements	2-4
HR Application Scenario	2-5
Writing SQL Statements	2-6
Basic SELECT Statement	2-7
Selecting All Columns	2-8
Executing SQL Statements with Oracle SQL Developer and SQL*Plus	2-9
Column Heading Defaults in SQL Developer and SQL*Plus	2-10
Selecting Specific Columns	2-11
Selecting from dual with Oracle Database	2-12
New Feature in 23c: SELECT Without FROM Clause	2-13
Arithmetic Expressions and NULL Values in the SELECT statement	2-14
Arithmetic Expressions	2-15
Using Arithmetic Operators	2-16
Operator Precedence	2-17
Defining a Null Value	2-18
Null Values in Arithmetic Expressions	2-19
Column Aliases	2-20
Defining a Column Alias	2-21
Using Column Aliases	2-22
Use of the Concatenation Operator, Literal Character Strings, Alternative Quote Operator, and the DISTINCT Keyword	2-23
Concatenation Operator in Oracle	2-24
Literal Character Strings	2-25
Using Literal Character Strings in Oracle	2-26
Alternative Quote (q) Operator in Oracle	2-27
Duplicate Rows	2-28
DESCRIBE Command	2-29
Displaying Table Structure by Using the DESCRIBE Command	2-30
Displaying Table Structure by Using Oracle SQL Developer	2-31
Summary	2-32

3 Restricting and Sorting Data

Course Roadmap 3-2

Objectives 3-3

Limiting Rows 3-4

Limiting Rows by Using a Selection 3-5

Limiting Rows That Are Selected 3-6

Using the WHERE Clause 3-7

Character Strings and Dates 3-8

Comparison Operators 3-9

Using Comparison Operators 3-10

Range Conditions Using the BETWEEN Operator 3-11

Using the IN Operator 3-12

Pattern Matching Using the LIKE Operator 3-13

Combining Wildcard Symbols 3-14

Using NULL Conditions 3-15

Defining Conditions Using Logical Operators 3-16

Using the AND Operator 3-17

Using the OR Operator 3-18

Using the NOT Operator 3-19

Rules of Precedence for Operators in an Expression 3-20

Rules of Precedence 3-21

Sorting Rows Using the ORDER BY Clause 3-23

Using the ORDER BY Clause 3-24

Sorting 3-25

SQL Row Limiting Clause in a Query 3-27

SQL Row Limiting Clause 3-28

Using SQL Row Limiting Clause in a Query in Oracle 3-29

SQL Row Limiting Clause: Example in Oracle 3-30

Substitution Variables in Oracle 3-31

Using the Single-Ampersand Substitution Variable 3-34

Character and Date Values with Substitution Variables 3-36

Specifying Column Names, Expressions, and Text 3-37

Using the Double-Ampersand Substitution Variable 3-38

Using the Ampersand Substitution Variable in SQL*Plus 3-39

Assigning Values to Variables 3-40

Using the DEFINE Command in Oracle 3-41

Using the VERIFY Command in Oracle 3-42

Summary 3-43

4 Using Single-Row Functions to Customize Output

- Course Roadmap 4-2
- Objectives 4-3
- HR Application Scenario 4-4
- Single-Row SQL Functions 4-5
 - SQL Functions 4-6
 - Two Types of SQL Functions 4-7
 - Single-Row Functions 4-8
- Character Functions 4-10
 - Case-Conversion Functions 4-13
 - Using Case-Conversion Functions in WHERE Clauses in Oracle 4-14
 - Character-Manipulation Functions 4-15
 - Using Character-Manipulation Functions 4-16
- Nesting Functions 4-17
 - Nesting Functions: Example 4-19
- Number Functions 4-20
 - Numeric Functions 4-21
 - Using the ROUND Function 4-22
 - Using the TRUNC Function in Oracle 4-23
 - Using the MOD Function 4-24
- Working with Dates in Oracle Databases 4-25
 - RR Date Format in Oracle 4-27
 - Using the SYSDATE Function in Oracle 4-29
 - Using the CURRENT_DATE and CURRENT_TIMESTAMP Functions in Oracle 4-30
 - Arithmetic with Dates in Oracle 4-31
 - Using Arithmetic Operators with Dates in Oracle 4-32
- Date Functions 4-33
 - Date-Manipulation Functions in Oracle 4-34
 - Using Date Functions in Oracle 4-35
 - Using ROUND and TRUNC Functions with Dates in Oracle 4-36
- Summary 4-37

5 Using Conversion Functions and Conditional Expressions

- Course Roadmap 5-2
- Objectives 5-3
- Implicit and Explicit Data Type Conversion 5-4
 - Conversion Functions 5-5
 - Implicit Data Type Conversion of Strings to Numbers 5-6
 - Implicit Data Type Conversion of Numbers to Strings 5-7

- TO_CHAR, TO_DATE, and TO_NUMBER Functions in Oracle 5-8
 - Using the TO_CHAR Function with Dates 5-9
 - Elements of the Date Format Model 5-10
 - Using the TO_CHAR Function with Dates 5-14
 - Using the TO_CHAR Function with Numbers 5-15
 - Using the TO_NUMBER and TO_DATE Functions 5-18
 - Using TO_CHAR and TO_DATE Functions with the RR Date Format 5-20
- General Functions 5-21
 - NVL Function 5-23
 - Using the NVL Function in Oracle 5-24
 - Using the NVL2 Function in Oracle 5-25
 - Using the NULLIF Function 5-26
 - Using the COALESCE Function 5-27
- Conditional Expressions 5-29
 - CASE Expression 5-31
 - Using the CASE Expression 5-32
 - Searched CASE Expression 5-33
 - DECODE Function in Oracle 5-34
 - Using the DECODE Function 5-35
- JSON Functions 5-37
 - JSON_QUERY Function 5-38
 - JSON_TABLE Function 5-39
 - JSON_VALUE Function 5-40
- Summary 5-41

6 Reporting Aggregated Data Using the Group Functions

- Course Roadmap 6-2
- Objectives 6-3
- Group Functions 6-4
 - Types of Group Functions 6-6
 - Group Functions: Syntax 6-7
 - Using the AVG and SUM Functions 6-8
 - Using the MIN and MAX Functions 6-9
 - Using the COUNT Function 6-10
 - Using the DISTINCT Keyword 6-11
 - Group Functions and Null Values in Oracle 6-12
- Grouping Rows 6-13
 - Creating Groups of Data 6-14
 - Creating Groups of Data: GROUP BY Clause Syntax 6-15
 - Using the GROUP BY Clause 6-16
 - Using the GROUP BY Column Position 6-17

- Using the GROUP BY Clause 6-18
- Grouping by More Than One Column 6-19
- Using the GROUP BY Clause on Multiple Columns 6-20
- Illegal Queries Using Group Functions 6-21
- Illegal Queries Using Group Functions in a WHERE Clause 6-22
- Restricting Group Results 6-23
- Restricting Group Results with the HAVING Clause 6-24
- Using the HAVING Clause 6-25
- Nesting Group Functions 6-27
 - Nesting Group Functions in Oracle 6-28
- Summary 6-29

7 Displaying Data from Multiple Tables Using Joins

- Course Roadmap 7-2
- Objectives 7-3
- Types of Joins and Their Syntax 7-4
 - Why Join? 7-5
 - Obtaining Data from Multiple Tables 7-6
 - Types of Joins 7-7
 - Joining Tables Using SQL Syntax 7-8
- Natural Join 7-9
 - Creating Natural Joins 7-10
 - Retrieving Records with Natural Joins 7-11
- Join with the USING Clause 7-12
 - Creating Joins with the USING Clause 7-13
 - Joining Column Names 7-14
 - Retrieving Records with the USING Clause 7-15
 - Qualifying Ambiguous Column Names 7-16
 - Using Table Aliases with the USING Clause in Oracle 7-17
- Join with the ON Clause 7-18
 - Creating Joins with the ON Clause 7-19
 - Retrieving Records with the ON Clause 7-20
 - Creating Three-Way Joins 7-21
 - Applying Additional Conditions to a Join 7-22
- Self-Join 7-23
 - Joining a Table to Itself 7-24
 - Self-Joins Using the ON Clause 7-25

Nonequijoins 7-26

Retrieving Records with Nonequijoins 7-28

Outer Join 7-29

Returning Records with No Direct Match Using OUTER Joins 7-30

INNER Versus OUTER Joins 7-31

LEFT OUTER JOIN 7-32

RIGHT OUTER JOIN 7-33

FULL OUTER JOIN in Oracle 7-34

Cartesian Product 7-35

Cartesian Products 7-36

Generating a Cartesian Product 7-37

Creating Cross Joins 7-38

Summary 7-39

8 Using Subqueries to Solve Queries

Course Roadmap 8-2

Objectives 8-3

Subquery: Types, Syntax, and Guidelines 8-4

Using a Subquery to Solve a Problem 8-5

Subquery Syntax 8-6

Using a Subquery 8-7

Rules and Guidelines for Using Subqueries 8-8

Types of Subqueries 8-9

Single-Row Subqueries 8-10

Executing Single-Row Subqueries 8-12

Using Group Functions in a Subquery 8-13

HAVING Clause with Subqueries 8-14

What is wrong with this statement? 8-15

No Rows Returned by the Inner Query 8-16

Multiple-Row Subqueries 8-17

Using the ANY Operator in Multiple-Row Subqueries 8-19

Using the ALL Operator in Multiple-Row Subqueries 8-20

Multiple-Column Subqueries 8-21

Multiple-Column Subquery: Example 8-23

Null Values in a Subquery 8-24

Summary 8-26

9 Using Set Operators

Course Roadmap 9-2

Objectives 9-3

Set Operators: Types and Guidelines 9-4

Set Operators	9-5
Set Operator Rules	9-6
Oracle Server and Set Operators	9-7
Tables Used in This Lesson	9-8
UNION and UNION ALL Operators	9-13
UNION Operator	9-14
Using the UNION Operator	9-15
UNION ALL Operator	9-16
Using the UNION ALL Operator	9-17
INTERSECT Operator	9-18
Using the INTERSECT Operator	9-20
MINUS Operator	9-21
Using the MINUS Operator	9-23
Matching SELECT Statements	9-24
Matching SELECT Statements in Oracle	9-25
Matching the SELECT Statement: Example in Oracle	9-26
Using the ORDER BY Clause in Set Operations	9-27
Using the ORDER BY Clause in Set Operations in Oracle	9-28
Using the ORDER BY Clause in Set Operations in Oracle: Example	9-29
Summary	9-30

10 Managing Tables Using DML Statements

Course Roadmap	10-2
Objectives	10-3
HR Application Scenario	10-4
Adding New Rows in a Table	10-5
Data Manipulation Language	10-6
Adding a New Row to a Table	10-7
INSERT Statement Syntax	10-8
Inserting New Rows	10-9
INSERT Statement Syntax	10-10
Inserting Rows with Null Values	10-11
Inserting Special Values	10-12
Inserting Specific Date and Time Values	10-13
Creating a Script	10-14
Copying Rows from Another Table	10-15
Changing Data in a Table	10-16
UPDATE Statement Syntax	10-18
Updating Rows in a Table	10-19
Updating Two Columns with a Subquery	10-20
Updating Rows Based on Another Table	10-21

Enhancement in 23c	10-22
RETURNING Clause in 23c: Example	10-23
Removing Rows from a Table	10-24
Removing a Row from a Table	10-25
DELETE Statement	10-26
Deleting Rows from a Table	10-27
Deleting Rows Based on Another Table	10-28
Enhancements to UPDATE and DELETE Statements in 23c	10-29
UPDATE: Example	10-30
DELETE: Example	10-31
TRUNCATE Statement	10-32
Database Transaction Control	10-33
Database Transactions	10-34
Database Transactions: Start and End	10-35
Advantages of the COMMIT and ROLLBACK Statements	10-36
Explicit Transaction Control Statements	10-37
Rolling Back Changes to a Marker	10-38
Implicit Transaction Processing	10-39
State of Data Before COMMIT or ROLLBACK	10-40
State of Data After COMMIT	10-41
Committing Data	10-42
State of Data After ROLLBACK	10-43
State of Data After ROLLBACK: Example	10-44
Statement-Level Rollback	10-45
Read Consistency	10-46
Implementing Read Consistency	10-48
Manual Data Locking	10-49
FOR UPDATE Clause in a SELECT Statement	10-50
FOR UPDATE Clause: Examples	10-51
LOCK TABLE Statement	10-52
Summary	10-53

11 Introduction to Data Definition Language

Course Roadmap	11-2
Objectives	11-3
HR Application Scenario	11-4
Database Objects	11-5
Naming Rules for Tables and Columns	11-7
CREATE TABLE Statement	11-8
Creating Tables	11-10
Staging Tables	11-11

Creating Staging Tables: Methods	11-12
Using Staging Tables: Example	11-13
Data Types	11-14
Datetime Data Types	11-17
SQL*Plus Support for Boolean Data Type in 23c	11-18
Creating a Table with Boolean Type and Inserting Data: Example	11-19
Querying the Table Containing BOOLEAN Values in SQL Developer:	
Example	11-20
Default Output in SQL*Plus: Example	11-21
Using the COLUMN Command to Customize Output: Example	11-22
DEFAULT Option	11-23
23c Enhancement to SQL Statements	11-24
UPDATE Option to the Existing DEFAULT ON NULL Clause: Example	11-25
Updating demo_t and Setting ename to the 'missing name' Value:	
Example	11-26
DEFAULT ON NULL with IDENTITY Columns and Update Column c2	
to <sequence>.nextval: Example	11-27
Constraints: Overview	11-28
Including Constraints	11-29
Constraint Guidelines	11-30
Defining Constraints	11-31
Defining Constraints: Example	11-32
NOT NULL Constraint	11-33
UNIQUE Constraint	11-34
PRIMARY KEY Constraint	11-36
FOREIGN KEY Constraint	11-37
FOREIGN KEY Constraint: Keywords	11-39
CHECK Constraint	11-40
CREATE TABLE: Example	11-41
IF (NOT) EXISTS Enhancement	11-42
IF NOT EXISTS Clause: Example	11-43
Violating Constraints	11-44
Creating a Table by Using a Subquery	11-46
ALTER TABLE Statement	11-49
Adding a Column	11-52
Modifying a Column	11-53
Dropping a Column	11-54
SET UNUSED Option	11-55
Read-Only Tables	11-57

DROP TABLE Statement	11-58
Dropping a Table	11-59
Dropping the DEMO Table If It Exists: Example	11-60
Summary	11-61

12 Introduction to Data Dictionary Views

Objectives	12-2
Introduction to Data Dictionary	12-3
Why data dictionary?	12-4
Data Dictionary	12-5
Data Dictionary Structure	12-6
How to Use Dictionary Views	12-8
USER_OBJECTS and ALL_OBJECTS Views	12-9
USER_OBJECTS View	12-10
Querying the Dictionary Views	12-11
Table Information	12-12
Column Information	12-13
Constraint Information	12-15
USER_CONSTRAINTS: Example	12-16
Querying USER_CONS_COLUMNS	12-17
Adding a Comment to a Table and Querying the Dictionary Views	12-18
Adding Comments to a Table	12-19
Summary	12-20

13 Creating Sequences, Synonyms, and Indexes

Objectives	13-2
Create, Maintain, and Use Sequences	13-3
E-Commerce Scenario	13-4
Database Objects	13-5
Referencing Another User's Tables	13-6
Sequences	13-7
CREATE SEQUENCE Statement: Syntax	13-8
Creating a Sequence	13-10
NEXTVAL and CURRVAL Pseudocolumns	13-11
Using a Sequence	13-13
SQL Column Defaulting Using a Sequence	13-14
Caching Sequence Values	13-15
Modifying a Sequence	13-16
Guidelines for Modifying a Sequence	13-17
Sequence Information	13-18

Create Private and Public Synonyms	13-19
Synonyms	13-20
Creating a Synonym for an Object	13-21
Creating and Removing Synonyms	13-22
Synonym Information	13-23
Create and Maintain Indexes	13-24
Indexes	13-25
How are indexes created?	13-26
Creating an Index	13-27
CREATE INDEX with the CREATE TABLE Statement	13-28
Function-Based Indexes	13-30
Creating Multiple Indexes on the Same Set of Columns	13-31
Creating Multiple Indexes on the Same Set of Columns: Example	13-32
Index Information	13-33
USER_INDEXES: Examples	13-34
Querying USER_IND_COLUMNS	13-35
Removing an Index	13-36
Summary	13-37

14 Creating Views

Objectives	14-2
Overview of Views	14-3
Why views?	14-4
Database Objects	14-5
What is a view?	14-6
Advantages of Views	14-7
Simple Views and Complex Views	14-8
Creating, Modifying, and Retrieving Data from a View	14-9
Creating a View	14-10
Retrieving Data from a View	14-13
Modifying a View	14-14
Creating a Complex View	14-15
View Information	14-16
Data Manipulation Language (DML) Operations on a View	14-17
Rules for Performing DML Operations on a View	14-18
Rules for Performing Modify Operations on a View	14-19
Rules for Performing Insert Operations Through a View	14-20
Using the WITH CHECK OPTION Clause	14-21
Denying DML Operations	14-22

Dropping a View 14-24
 Removing a View 14-25
Summary 14-26

15 Managing Schema Objects

Objectives 15-2
Manage Constraints 15-3
 Adding a Constraint: Syntax 15-4
 Adding a Constraint 15-5
 Dropping a Constraint 15-6
 Dropping a Constraint: ONLINE 15-7
 ON DELETE Clause 15-8
 Cascading Constraints 15-9
 Cascading Constraints: Example 15-10
 Renaming Table Columns and Constraints 15-11
 Disabling Constraints 15-12
 Enabling Constraints 15-13
 Constraint States 15-14
 Deferring Constraints 15-15
 Difference Between INITIALLY DEFERRED and INITIALLY IMMEDIATE 15-16
 DROP TABLE ... PURGE 15-18
Create and Use Temporary Tables 15-19
 Using Temporary Tables 15-20
 Temporary Table 15-21
 Temporary Table: Characteristics 15-22
 Creating a Global Temporary Table 15-23
 Creating a Private Temporary Table 15-24
Creating and Using External Tables 15-25
 External Tables 15-26
 Creating a Directory for the External Table 15-27
 Creating an External Table 15-29
 Creating an External Table by Using ORACLE_LOADER 15-31
 Querying External Tables 15-32
 Creating an External Table by Using ORACLE_DATAPUMP: Example 15-33
Summary 15-34

16 Retrieving Data by Using Subqueries

Objectives 16-2
Retrieving Data by Using a Subquery as a Source 16-3
Writing a Multiple - Column Subquery 16-6
 Multiple-Column Subqueries 16-7

Column Comparisons	16-8
Pairwise Comparison Subquery	16-9
Nonpairwise Comparison Subquery	16-10
Using Scalar Subqueries in SQL	16-11
Scalar Subquery Expressions	16-12
Scalar Subqueries: Examples	16-13
Solving Problems with Correlated Subqueries	16-14
Correlated Subqueries	16-15
Using Correlated Subqueries: Example 1	16-17
Using Correlated Subqueries: Example 2	16-18
Using the EXISTS and NOT EXISTS Operators	16-19
Using the EXISTS Operator	16-20
Find All Departments That Do Not Have Employees: Example	16-22
Using the WITH Clause	16-23
WITH Clause	16-24
WITH Clause: Example	16-25
Recursive WITH Clause	16-26
Recursive WITH Clause: Example	16-27
Summary	16-28

17 Manipulating Data by Using Subqueries

Objectives	17-3
Using Subqueries to Manipulate Data	17-3
Inserting Values by Using a Subquery as a Target	17-5
Inserting by Using a Subquery as a Target (1/2)	17-6
Inserting by Using a Subquery as a Target (2/2)	17-7
Using the WITH CHECK OPTION keyword on DML statements	17-8
Using Correlated Subqueries to Update and Delete Rows	17-11
Correlated UPDATE	17-12
Using Correlated UPDATE	17-13
Correlated DELETE	17-15
Using Correlated DELETE	17-16
Summary	17-17

18 Controlling User Access

Objectives	18-2
Differentiating System Privileges from Object Privileges	18-3
Controlling User Access	18-4
Privileges	18-5
System Privileges	18-6
Creating Users	18-7

User System Privileges	18-8
Granting System Privileges	18-9
Creating a Role	18-10
What is a role?	18-11
Creating and Granting Privileges to a Role	18-12
Changing Your Password	18-13
Object Privileges	18-14
Granting Object Privileges	18-17
Passing On Your Privileges	18-18
Confirming Granted Privileges	18-19
Revoking Object Privileges	18-20
Summary	18-23
Practice 18: Overview	18-24

19 Manipulating Data Using Advanced Queries

Course Roadmap	19-2
Objectives	19-3
Specifying Explicit Default Values in INSERT and UPDATE Statements	19-4
Explicit Default Feature: Overview	19-5
Using Explicit Default Values	19-6
Multitable INSERTs	19-7
E-Commerce Scenario	19-8
Multitable INSERT Statements: Overview	19-9
Types of Multitable INSERT Statements	19-11
Multitable INSERT Statements	19-12
Unconditional INSERT ALL	19-14
Conditional INSERT ALL: Example	19-15
Conditional INSERT ALL	19-16
Conditional INSERT FIRST: Example	19-18
Conditional INSERT FIRST	19-19
Pivoting INSERT	19-20
Merging Rows in a Table	19-23
MERGE Statement	19-24
MERGE Statement Syntax	19-25
Merging Rows: Example	19-26
Performing Flashback Operations	19-29
FLASHBACK TABLE Statement	19-30
Using the FLASHBACK TABLE Statement	19-32

Tracking the Changes in Data Over a Period of Time 19-33

Tracking Changes in Data 19-34

Flashback Query: Example 19-35

Flashback Version Query: Example 19-36

VERSIONS BETWEEN Clause 19-37

Summary 19-38

20 Managing Data in Different Time Zones

Objectives 20-2

CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP 20-3

E-Commerce Scenario 20-4

Time Zones 20-5

TIME_ZONE Session Parameter 20-6

CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP 20-7

Comparing Date and Time in a Session's Time Zone 20-8

DBTIMEZONE and SESSIONTIMEZONE 20-10

TIMESTAMP Data Types 20-11

TIMESTAMP Fields 20-12

Difference Between DATE and TIMESTAMP 20-13

Comparing TIMESTAMP Data Types 20-14

INTERVAL Data Types 20-15

INTERVAL Fields 20-17

INTERVAL YEAR TO MONTH: Example 20-18

INTERVAL DAY TO SECOND Data Type: Example 20-20

Additional Date Functions 20-21

EXTRACT 20-22

TZ_OFFSET 20-23

FROM_TZ 20-25

TO_TIMESTAMP 20-26

TO_YMINTERVAL 20-27

TO_DSINTERVAL 20-28

Daylight Saving Time (DST) 20-29

SYSDATE and SYSTIMESTAMP Data Handling 20-31

Lesson Agenda 20-32

Date and Time in Oracle Databases 20-33

Database Time Versus OS System Time 20-34

Database-Specific Time in Oracle Database 23c 20-35

Database Administrator Configures Database-Specific Time 20-36

Summary 20-37

A Table Descriptions

B Using SQL Developer

- Objectives B-2
- What is Oracle SQL Developer? B-4
- Specifications of SQL Developer B-5
- SQL Developer 17.4.1 Interface B-6
- Creating a Database Connection B-8
- Browsing Database Objects B-11
- Displaying the Table Structure B-12
- Browsing Files B-13
- Creating a Schema Object B-14
- Creating a New Table: Example B-15
- Using SQL Worksheet B-16
- Executing SQL Statements B-19
- Saving SQL Scripts B-20
- Executing Saved Script Files: Method 1 B-21
- Executing Saved Script Files: Method 2 B-22
- Formatting the SQL Code B-23
- Using Snippets B-24
- Using Snippets: Example B-25
- Using the Recycle Bin B-26
- Debugging Procedures and Functions B-27
- Database Reporting B-28
- Creating a User-Defined Report B-29
- External Tools B-30
- Setting Preferences B-31
- Data Modeler in SQL Developer B-32
- Summary B-33

C Using SQL*Plus

- Objectives C-2
- SQL and SQL*Plus Interaction C-3
- SQL Statements Versus SQL*Plus Commands C-4
- SQL*Plus: Overview C-5
- Logging In to SQL*Plus C-6
- Displaying the Table Structure C-7
- SQL*Plus Editing Commands C-9
- Using LIST, n, and APPEND C-11
- Using the CHANGE Command C-12
- SQL*Plus File Commands C-13

Using the SAVE and START Commands C-14
SERVEROUTPUT Command C-15
Using the SQL*Plus SPOOL Command C-16
Using the AUTOTRACE Command C-17
Summary C-18

D Commonly Used SQL Commands

Objectives D-2
Basic SELECT Statement D-3
SELECT Statement D-4
WHERE Clause D-5
ORDER BY Clause D-6
GROUP BY Clause D-7
Data Definition Language D-8
CREATE TABLE Statement D-9
ALTER TABLE Statement D-10
DROP TABLE Statement D-11
GRANT Statement D-12
Privilege Types D-13
REVOKE Statement D-14
TRUNCATE TABLE Statement D-15
Data Manipulation Language D-16
INSERT Statement D-17
UPDATE Statement Syntax D-18
DELETE Statement D-19
Transaction Control Statements D-20
COMMIT Statement D-21
ROLLBACK Statement D-22
SAVEPOINT Statement D-23
Joins D-24
Types of Joins D-25
Qualifying Ambiguous Column Names D-26
Natural Join D-27
Equijoins D-28
Retrieving Records with Equijoins D-29
Add Additional Search Conditions to the ON clause Using AND or WHERE D-30
Retrieving Records with Nonequijoins D-31
Retrieving Records by Using the USING Clause D-32
Retrieving Records by Using the ON Clause D-33
Left Outer Join D-34
Right Outer Join D-35

Full Outer Join D-36
Self-Join: Example D-37
Cross Join D-38
Summary D-39

E Generating Reports by Grouping Related Data

Objectives E-2
Review of Group Functions E-3
Review of the GROUP BY Clause E-4
Review of the HAVING Clause E-5
GROUP BY with ROLLUP and CUBE Operators E-6
ROLLUP Operator E-7
ROLLUP Operator: Example E-8
CUBE Operator E-9
CUBE Operator: Example E-10
GROUPING Function E-11
GROUPING Function: Example E-12
GROUPING SETS E-13
GROUPING SETS: Example E-15
Composite Columns E-17
Composite Columns: Example E-19
Concatenated Groupings E-21
Concatenated Groupings: Example E-22
Summary E-23

F Hierarchical Retrieval

Objectives F-2
Sample Data from the EMPLOYEES Table F-3
Natural Tree Structure F-4
Hierarchical Queries F-5
Walking the Tree F-6
Walking the Tree: From the Bottom Up F-8
Walking the Tree: From the Top Down F-9
Ranking Rows with the LEVEL Pseudocolumn F-10
Formatting Hierarchical Reports Using LEVEL and LPAD F-11
Pruning Branches F-13
Summary F-14

G Writing Advanced Scripts

Objectives G-2
Using SQL to Generate SQL G-3

Creating a Basic Script G-4
Controlling the Environment G-5
The Complete Picture G-6
Dumping the Contents of a Table to a File G-7
Generating a Dynamic Predicate G-9
Summary G-11

H Oracle Database Architectural Components

Objectives H-2
Oracle Database Architecture: Overview H-3
Oracle Database Server Structures H-4
Connecting to the Database H-5
Interacting with an Oracle Database H-6
Oracle Memory Architecture H-8
Process Architecture H-10
Database Writer Process H-12
Log Writer Process H-13
Checkpoint Process H-14
System Monitor Process H-15
Process Monitor Process H-16
Oracle Database Storage Architecture H-17
Logical and Physical Database Structures H-19
Processing a SQL Statement H-21
Processing a Query H-22
Shared Pool H-23
Database Buffer Cache H-25
Program Global Area H-26
Processing a DML Statement H-27
Redo Log Buffer H-29
Rollback Segment H-30
COMMIT Processing H-31
Summary of the Oracle Database Architecture H-33
Summary H-34

I Regular Expression Support

Objectives I-2
What Are Regular Expressions? I-3
Benefits of Using Regular Expressions I-4
Using Regular Expressions Functions and Conditions in SQL and PL/SQL I-5
What are metacharacters? I-6
Using Metacharacters with Regular Expressions I-7

Regular Expressions Functions and Conditions: Syntax I-9
Performing a Basic Search by Using the REGEXP_LIKE Condition I-10
Replacing Patterns by Using the REGEXP_REPLACE Function I-11
Finding Patterns by Using the REGEXP_INSTR Function I-12
Extracting Substrings by Using the REGEXP_SUBSTR Function I-13
Subexpressions I-14
Using Subexpressions with Regular Expression Support I-15
Why access the nth subexpression? I-16
REGEXP_SUBSTR: Example I-17
Using the REGEXP_COUNT Function I-18
Regular Expressions and Check Constraints: Examples I-19
Quiz I-20
Summary I-21

J MySQL Syntax

Objectives J-2
Icons Used in This Course J-3
MySQL: A Modern Database for the Digital Age J-4
High Scalability with MySQL J-5
MySQL-Supported Operating Systems J-6
MySQL Enterprise Edition J-7
Why MySQL Enterprise Edition? J-8
Oracle Premier Support for MySQL J-9
MySQL and Oracle Integration J-10
Development Environments for SQL in MySQL J-11
MySQL Websites J-12
MySQL Community Resources J-13
Oracle University: MySQL Training J-14
MySQL Certification J-15
Selecting All Columns J-16
Executing SQL Statements in MySQL Workbench J-17
Executing SQL Statements in mysql Command-Line Client J-18
Selecting Specific Columns J-19
Selecting Constant Expressions in MySQL J-20
Using Arithmetic Operators J-21
Operator Precedence J-22
Defining a Null Value J-23
Null Values in Arithmetic Expressions J-24
Using Column Aliases J-25
Concatenation Function in MySQL: CONCAT() J-26
Using Literal Character Strings in MySQL J-27

Including a Single Quotation Mark in a String with an Escape Sequence in MySQL	J-28
Duplicate Rows	J-29
Displaying Table Structure by Using the DESCRIBE Command	J-30
Displaying Table Structure by Using MySQL Workbench	J-31
Limiting Rows by Using a Selection	J-32
Using the WHERE Clause	J-33
Character Strings and Dates	J-34
Using Comparison Operators	J-35
Range Conditions Using the BETWEEN Operator	J-36
Using the IN Operator	J-37
Pattern Matching Using the LIKE Operator	J-38
Combining Wildcard Symbols	J-39
Using NULL Conditions	J-40
Using the AND Operator	J-41
Using the OR Operator	J-42
Using the NOT Operator	J-43
Rules of Precedence	J-44
Using the ORDER BY Clause	J-45
Using SQL Row Limiting Clause in a Query in MySQL	J-46
SQL Row Limiting Clause: Example in MySQL	J-47
Using the SET Statement in MySQL	J-48
Case-Conversion Functions	J-49
Case-Insensitive Queries in MySQL	J-50
Using Character-Manipulation Functions	J-51
Nesting Functions: Example	J-52
Numeric Functions	J-53
Using the ROUND Function	J-54
Using the TRUNCATE Function in MySQL	J-55
Using the MOD Function	J-56
Working with Dates in MySQL Databases	J-57
Displaying the Current Date in MySQL	J-58
Date-Manipulation Functions in MySQL	J-59
Using Date Functions in MySQL	J-60
Extracting the Month or Year Portion of Dates in MySQL	J-61
Implicit Data Type Conversion of Strings to Numbers	J-62
Implicit Data Type Conversion of Numbers to Strings	J-63
Using the CAST() Function in Oracle	J-64
Explicit Data Type Conversion of Strings to Numbers in MySQL	J-65
Explicit Data Type Conversion of Numbers to Strings in MySQL	J-66
General Functions	J-67

NVL Function (Oracle) and IFNULL() Function (MySQL)	J-68
Using the IFNULL Function in MySQL	J-69
Using the NULLIF Function	J-70
Using the COALESCE Function	J-71
Using the CASE Expression	J-72
Using the AVG and SUM Functions	J-73
Using the MIN and MAX Functions	J-74
Using the COUNT Function	J-75
Using the DISTINCT Keyword	J-76
Group Functions and Null Values in MySQL	J-77
Using the GROUP BY Clause	J-78
Using the GROUP BY Clause on Multiple Columns	J-80
Illegal Queries Using Group Functions	J-81
Illegal Queries Using Group Functions in a WHERE Clause	J-82
Using the HAVING Clause	J-83
Retrieving Records with Natural Joins	J-85
Retrieving Records with the USING Clause	J-86
Retrieving Records with the ON Clause	J-87
Creating Three-Way Joins	J-88
Self-Joins Using the ON Clause	J-89
Retrieving Records with Nonequijoins	J-90
LEFT OUTER JOIN	J-91
RIGHT OUTER JOIN	J-92
Creating Cross Joins	J-93
Executing Single-Row Subqueries	J-94
Using Group Functions in a Subquery	J-95
HAVING Clause with Subqueries	J-96
What is wrong with this statement?	J-97
No Rows Returned by the Inner Query	J-98
Using the ANY Operator in Multiple-Row Subqueries	J-99
Using the ALL Operator in Multiple-Row Subqueries	J-100
Multiple-Column Subquery: Example	J-101
Null Values in a Subquery	J-102
Using the UNION Operator	J-103
Using the UNION ALL Operator	J-104
Matching SELECT Statements in MySQL	J-105
Matching the SELECT Statement: Example in MySQL	J-106
Using the ORDER BY Clause with UNION in MySQL	J-107
Using the ORDER BY Clause with UNION: Example in MySQL	J-108
Data Manipulation Language	J-109
Adding a New Row to a Table	J-110

INSERT Statement Syntax	J-111
Inserting New Rows: Listing Column Names	J-112
Inserting New Rows: Omitting Column Names	J-113
Inserting Rows with Null Values	J-114
Inserting Special Values in MySQL	J-115
Inserting Specific Date and Time Values in MySQL	J-116
Inserting and Reformatting Specific Date and Time Values in MySQL	J-117
Copying Rows from Another Table	J-118
Changing Data in a Table	J-119
UPDATE Statement Syntax	J-120
Updating Rows in a Table	J-121
Updating Rows Based on Another Table	J-122
Removing a Row from a Table	J-123
DELETE Statement	J-124
Deleting Rows from a Table	J-125
Deleting Rows Based on Another Table	J-126
TRUNCATE Statement	J-127
Multiple-Statement Transactions	J-128
Transaction Diagram	J-129
AUTOCOMMIT and Transaction Control Statements	J-130
Committing Data in a Transaction	J-131
Rolling Back Changes	J-132
Rolling Back Changes to a Marker	J-133
Consistent Reads	J-134
FOR UPDATE Clause in a SELECT Statement	J-135
FOR UPDATE Clause: Examples	J-136
Creating a Database: Syntax	J-137
MySQL Naming Conventions	J-138
CREATE TABLE Statement	J-139
Data Types: Overview	J-140
Numeric Data Types	J-141
Date and Time Data Types	J-142
String Data Types	J-143
Indexes, Keys, and Constraints	J-144
Table Indexes	J-145
Primary Keys	J-146
Unique Key Constraints	J-147
Foreign Key Constraints	J-148
Foreign Key Constraint: Example Tables	J-149
FOREIGN KEY Constraint: Example Statement	J-150
FOREIGN KEY Constraint: Referential Actions	J-151

Secondary Indexes	J-152
Column Options	J-153
Creating a Table Using a Subquery	J-154
Creating a Table Using a Subquery: Example	J-155
ALTER TABLE Statement	J-156
ALTER TABLE Statement: Add, Modify, or Drop Columns	J-157
Adding a Column	J-158
Modifying a Column	J-159
Dropping a Column	J-160
ALTER TABLE Statement: Add an Index or Constraint	J-161
ALTER TABLE to Add a Constraint or Index: Example	J-162
Creating Indexes by Using the CREATE INDEX Statement	J-163
Viewing Index Definitions by Using the SHOW INDEX Statement	J-164
Showing How a Table Was Created with the SHOW CREATE TABLE Statement	J-165
Dropping a Table	J-166
Summary	J-167

For Instructor Use Only.
This document should not be distributed

Course Overview

Oracle Database 23c: SQL Workshop



Course Outline



Retrieving, Restricting, and Sorting Data

- Retrieving Data Using the SQL SELECT Statement
- Restricting and Sorting Data
- Using Single-Row Functions to Customize Output
- Using Conversion Functions and Conditional Expressions

Joins, Subqueries, and Set Operators

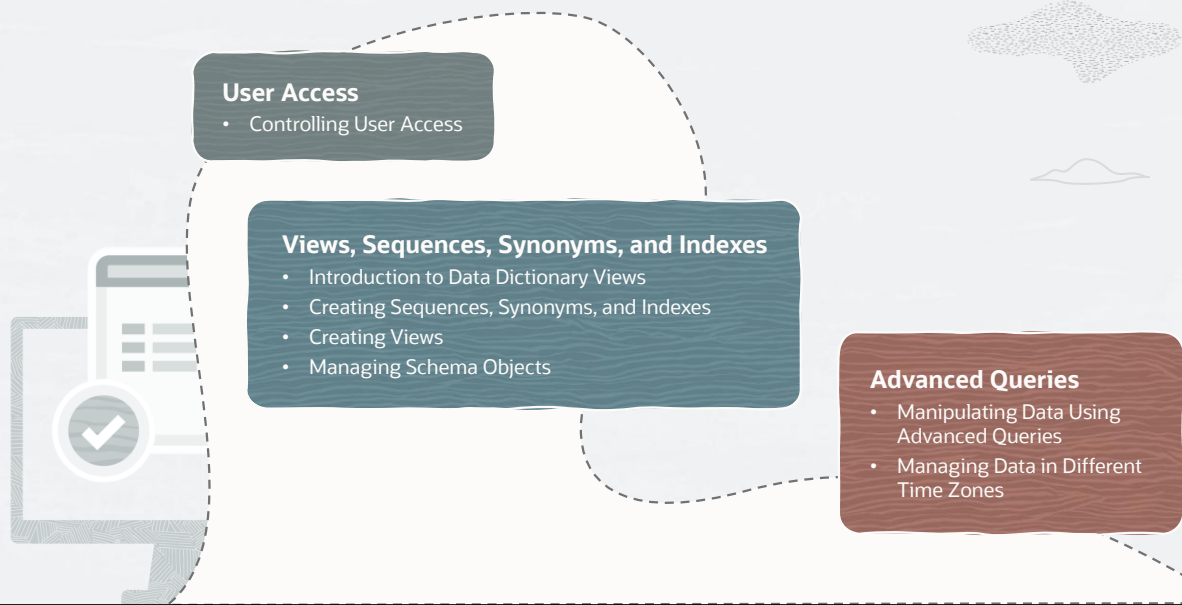
- Reporting Aggregated Data Using the Group Functions
- Displaying Data from Multiple Tables Using Joins
- Using Subqueries to Solve Queries
- Using Set Operators

DML and DDL

- Managing Tables Using DML Statements in Oracle
- Introduction to Data Definition Language in Oracle

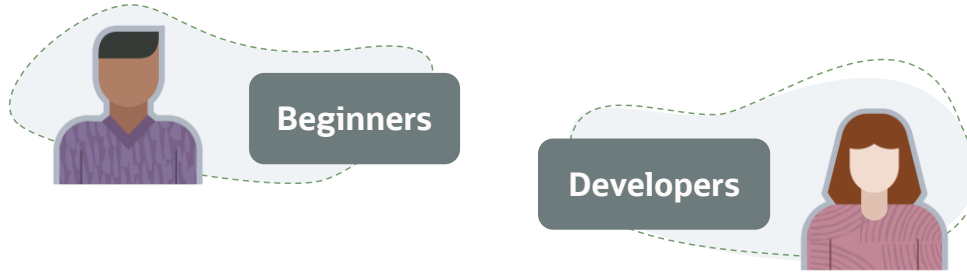
For Instructor Use Only.
This document should not be distributed

Course Outline



For Instructor Use Only.
This document should not be distributed

Target Audience



For Instructor Use Only.
This document should not be distributed

Learning Outcomes



Retrieve Data using the SQL SELECT Statement



Restrict and Sort Data



Customize Output Using Single-Row Functions



Use Conversion Functions and Conditional Expressions



Report Aggregated Data Using the Group Functions

Learning Outcomes



Display Data from Multiple Tables Using Joins



Manage Tables by Using DML Statements



Create Sequences, Synonyms, Indexes, Views, and Schema Objects

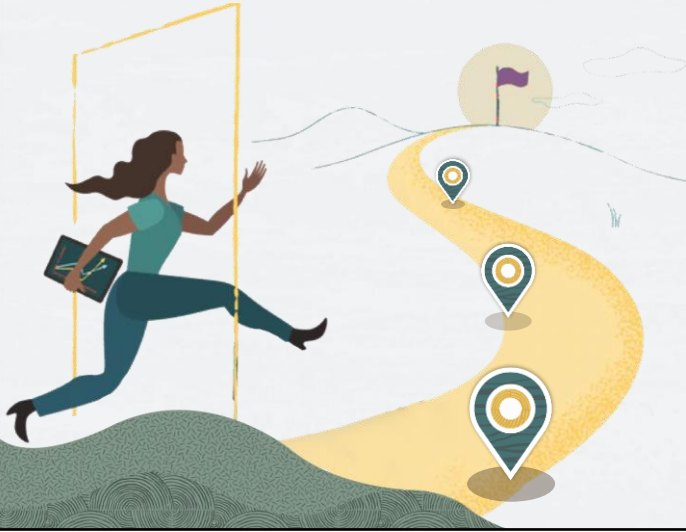


Manage Data by Using Subqueries and Advanced Queries



Control User Access and Manage Data in Different Time Zones

What's Next?



**Oracle Database 23c:
PL/SQL Workshop**

For Instructor Use Only.
This document should not be distributed

For Instructor Use Only.
This document should not be distributed

Introduction



Objectives



Overview of Oracle Database 23c and related products

Overview of relational database management concepts and terminologies

Human Resources (HR) Schema and the tables used in the course

Introduction to SQL and its development environments

Oracle Database 23c SQL documentation and additional resources

In this lesson, you gain an understanding of RDBMS. You are also introduced to Oracle SQL Developer and SQL*Plus as development environments used for executing SQL statements, and for formatting and reporting purposes.

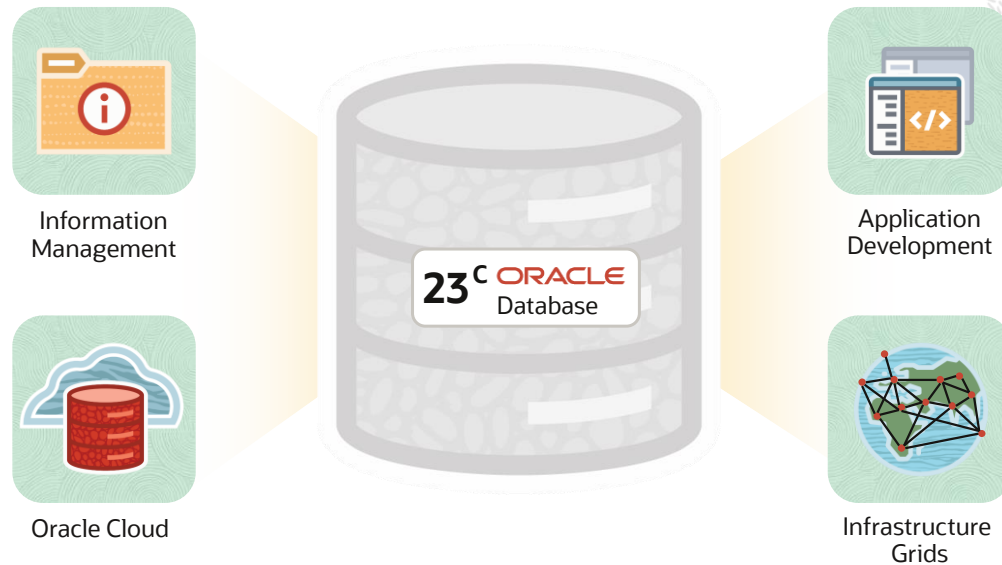
For Instructor Use Only.
This document should not be distributed



Overview of Oracle Database 23c and Related Products

For Instructor Use Only.
This document should not be distributed

Oracle Database 23c: Focus Areas

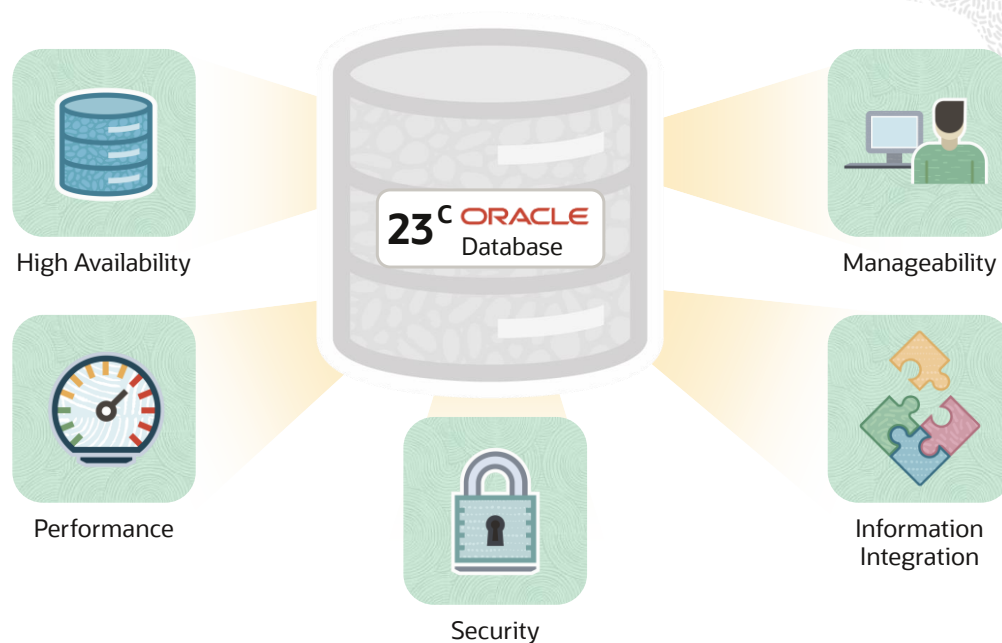


By using Oracle Database 23c, you can utilize the following features across focus areas:

- With the **Infrastructure Grid** technology of Oracle, you can pool low-cost servers and storage to form systems that deliver the highest quality of service in terms of manageability, high availability, and performance. Oracle Database 23c also helps you to consolidate and extend the benefits of grid computing and manage changes in a controlled and cost-effective manner.
- Oracle Database 23c enables **Information Management** by providing capabilities in content management, information integration, and information lifecycle management areas. You can manage content of advanced data types such as Extensible Markup Language (XML), text, spatial, multimedia, medical imaging, and semantic technologies using the features provided by Oracle.
- With Oracle Database 23c, you can manage all the major **Application Development** environments such as PL/SQL, Java/JDBC, .NET, Windows, PHP, SQL Developer, and Application Express.
- You can now plug into **Oracle Cloud** with Oracle Database 23c. This will help you to standardize, consolidate, and automate database services on the cloud.

For Instructor Use Only.
This document should not be distributed

Oracle Database 23c



Imagine you have an organization that needs to support multiple terabytes of information for users who demand fast and secure access to business applications round the clock. The database systems must be reliable and must be able to recover quickly in the event of any kind of failure. Oracle Database 23c is designed to help organizations manage infrastructure grids easily and deliver high-quality service:

- **Manageability:** By using some of the change assurance, management automation, and fault diagnostics features, the database administrators (DBAs) can increase their productivity, reduce costs, minimize errors, and maximize quality of service. Some of the useful features that promote better management are the Database Replay facility, the SQL Performance Analyzer, the Automatic SQL Tuning facility, and Real-Time Database Operations Monitoring.

Enterprise Manager Database Express 23c is a web-based tool for managing Oracle databases. It greatly simplifies database performance diagnostics by consolidating the relevant database performance screens into a view called Database Performance Hub. DBAs get a single, consolidated view of the current real-time and historical view of the database performance across multiple dimensions such as database load, monitored SQL and PL/SQL, and Active Session History (ASH) on a single page for the selected time period.

- **High availability:** By using the high availability features, you can reduce the risk of downtime and data loss. These features improve online operations and enable faster database upgrades.
- **Performance:** By using capabilities such as SecureFiles, Result Caches, and so on, you can greatly improve the performance of your database. Oracle Database 23c enables organizations to manage large, scalable, transactional, and data warehousing systems that deliver fast data access using low-cost modular storage.
- **Security:** Oracle Database 23c helps in protecting your information with unique secure configurations, data encryption and masking, and sophisticated auditing capabilities.

- **Information integration:** You can utilize Oracle Database 23c features to integrate data throughout the enterprise in a better way. You can also manage the changing data in your database by using Oracle Database 23c's advanced information life cycle management capabilities.



Overview of Relational Database Management Concepts and Terminologies

For Instructor Use Only.
This document should not be distributed

Relational and Object Relational Database Management Systems

- Relational model and object relational model
- User-defined data types and objects
- Fully compatible with relational database
- Supports multimedia and large objects
- High-quality database server features



Consider a situation where you put all your stationery in a single drawer without organizing. When you want a pencil immediately, you tend to search the entire drawer, which consumes a lot of time. This can be compared to the normal file storage in your system. But if the amount of information is huge, this system becomes inefficient.

Now imagine that you organize your stationery such that all the pencils/pens go into the first drawer, notepads in the second, gluesticks in third, and so on. Now when you want to fetch a pencil immediately, you will know where to find it efficiently. This can be compared to the relational model of storage.

Oracle Database is a Relational Database Management System (RDBMS). An RDBMS that implements object-oriented features such as user-defined types, inheritance, and polymorphism is called an object relational database management system (ORDBMS). Oracle Database has extended the relational model to an object relational model, making it possible to store complex business models in a relational database.

Some of the advantages are:

- Improved performance and functionality
- Better sharing of runtime data structures
- Larger buffer caches
- Deferrable constraints

Features such as parallel execution of insert, update, and delete operations; partitioning; and parallel-aware query optimization provide benefits to data warehouse applications.

The Oracle model supports client/server and web-based applications that are distributed and multi-tiered.

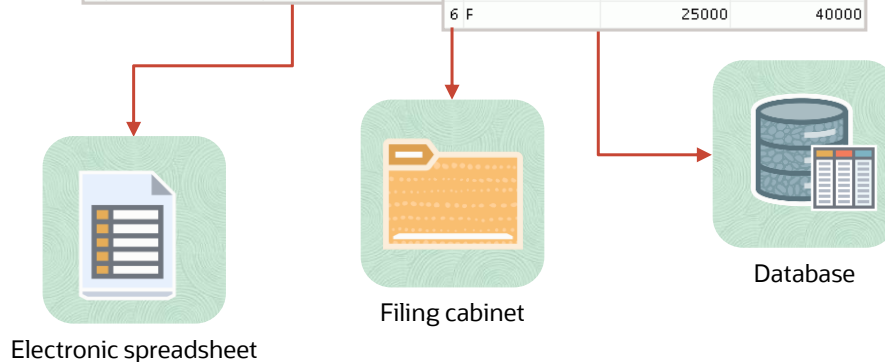
For more information about the relational and object relational model, refer to *Oracle Database Concepts for 23c Database*.

For Instructor Use Only.
This document should not be distributed

Data Storage on Different Media

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping		
4	60	IT	1	A
5	80	Sales	2	B
6	90	Executive	3	C
7	110	Accounting	4	D
8	190	Contracting	5	E
			6	F

	GRADE_LEVEL	LOWEST_SAL	HIGHEST_SAL
1	A	1000	2999
2	B	3000	5999
3	C	6000	9999
4	D	10000	14999
5	E	15000	24999
6	F	25000	40000



Every organization has some information needs. A library keeps a list of members, books, due dates, and fines. A company needs to save information about its employees, departments, and salaries. These pieces of information are called *data*.

Organizations can store data in various media and in different formats, such as a hard copy document in a filing cabinet, or data stored in electronic spreadsheets, or in databases.

A *database* is an organized collection of information.

To manage databases, you need a database management system (DBMS). A DBMS is a program that stores, retrieves, and modifies data in databases on request. There are four main types of databases: *hierarchical*, *network*, *relational*, and (most recently) *object relational*.

Relational Database Concept

- Dr. E. F. Codd proposed the relational model for database systems in 1970.
- It is the basis for RDBMS.
- The relational model consists of:
 - Collection of objects or relations
 - Set of operators to act on the relations
 - Data integrity for accuracy and consistency

The principles of the relational model were first outlined by Dr. E. F. Codd in a June 1970 paper titled *A Relational Model of Data for Large Shared Data Banks*. In this paper, Dr. Codd proposed the relational model for database systems.

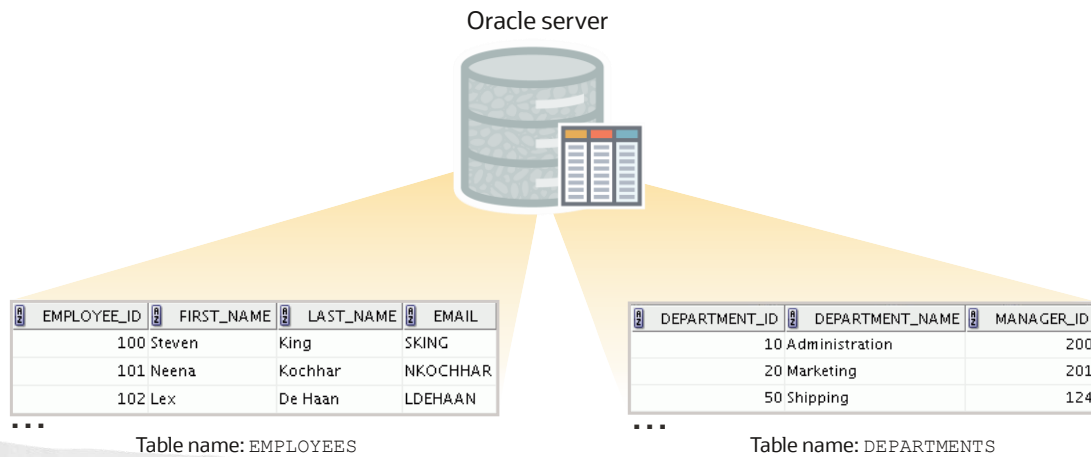
The common models used at that time were hierarchical and network, or even simple flat-file data structures. Relational database management systems (RDBMS) soon became very popular, especially for their ease of use and flexibility in structure. In addition, a number of innovative vendors, such as Oracle, supplemented the RDBMS with a suite of powerful application development and user-interface products, thereby providing a total solution.

Components of the Relational Model

- Collections of objects or relations that store the data
- A set of operators that can act on the relations to produce other relations
- Data integrity for accuracy and consistency

Definition of a Relational Database

A relational database is a collection of relations or two-dimensional tables controlled by the Oracle server.



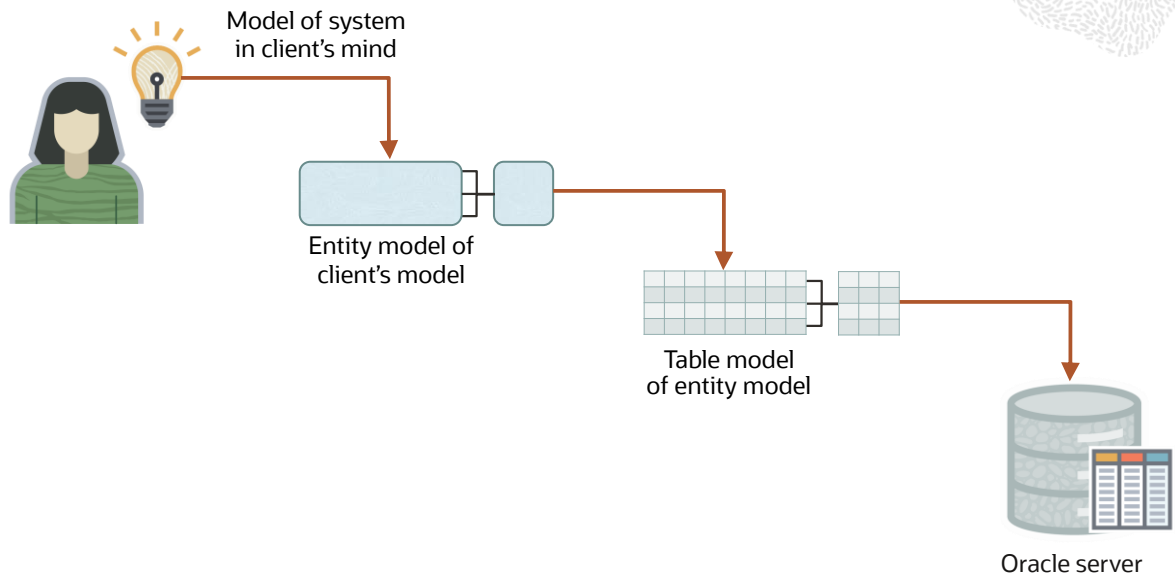
Every organization has information that it must store and manage to meet its requirements.

For example, you might want to store information about all the employees in your company. In a relational database, you create several tables to store different pieces of information about your employees, such as an employee table, a department table, and a salary table.

A relational database uses relations or two-dimensional tables to store information.

But before storing any information in the database, you need to first design a model of how and what data will be stored in the tables. In the following slide, you will learn about different data models that help in visualizing the architecture.

Data Models



Models are the cornerstone of design. Engineers build a model of a car to work out any details before putting it into production. In the same manner, system designers develop models to explore ideas and improve the understanding of database design.

Purpose of Models

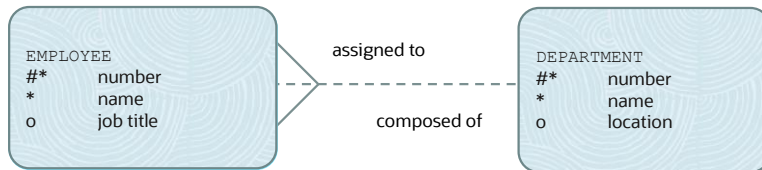
Models help you to communicate the concepts that are in people's minds. They can be used to do the following:

- Communicate
- Categorize
- Describe
- Specify
- Investigate
- Evolve
- Analyze
- Imitate

The objective is to produce a model that fits a multitude of these uses, can be understood by an end user, and contains sufficient details for a developer to build a database system. An Entity-Relationship model is one such data model.

Entity Relationship Model

- Create an entity relationship diagram from business specifications or narratives:



- Scenario:
 - “... Assign one or more employees to a department. . .”
 - “... Some departments do not yet have assigned employees. . .”

Entity relationship model is widely used because in an effective system, data is divided into discrete categories or entities.

An **entity relationship (ER)** model is an illustration of the various entities in a business and the relationships among them. You can derive an ER model from business specifications or narratives and build it during the analysis phase of the system development life cycle. ER models separate the information required by a business from the activities performed within the business. Although businesses can change their activities, the type of information tends to remain constant. Therefore, the data structures also tend to be constant.

Benefits of ER Modeling

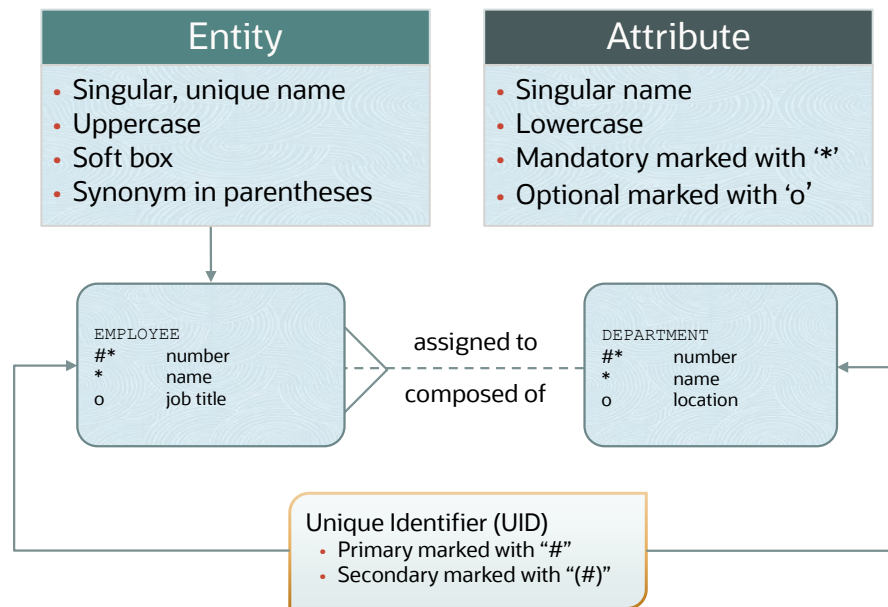
You can:

- Document information for the organization in a clear, precise format
- Provide a clear picture of the scope of the information requirement
- Provide an easily understood pictorial map for database design
- Offer an effective framework for integrating multiple applications

Key Components

- **Entity:** An aspect of significance about which information must be known. For example, departments, employees, and orders are entities.
- **Attribute:** Something that describes or qualifies an entity. For example, in the employee entity, the attributes would be the employee number, name, job title, hire date, department number, and so on. Each of the attributes is either required or optional. This state is called *optionality*.
- **Relationship:** A named association between entities showing optionality and degree. For example, an employee assigned to a department is a relationship between the employee and department entities.

Entity Relationship Modeling Conventions



Entities

To represent an entity in a model, use the following conventions:

- Singular, unique entity name
- Entity name in uppercase
- Soft box
- Optional synonym names in uppercase within parentheses: ()

Attributes

To represent an attribute in a model, use the following conventions:

- Singular name in lowercase
- Asterisk (*) tag for mandatory attributes (that is, values that *must* be known)
- Letter “o” tag for optional attributes (that is, values that *may* be known)

Relationships

Each direction of the relationship contains:

- **A label:** For example, *taught by* or *assigned to*
- **An optionality:** Either *must be* or *maybe*
- **A degree:** Either *one and only one* or *one or more*

Symbol	Description
Dashed line	Optional element indicating “maybe”
Solid line	Mandatory element indicating “must be”
Crow’s foot	Degree element indicating “one or more”
Single line	Degree element indicating “one and only one”

Note: The term *cardinality* is a synonym for the term *degree*.

Each source entity {may be | must be} in relation {one and only one | one or more} with the destination entity.

Note: The convention is to read clockwise.

Unique Identifiers

A unique identifier (UID) is any combination of attributes or relationships, or both, that serves to distinguish occurrences of an entity. Each entity occurrence must be uniquely identifiable.

- Tag each attribute that is part of the UID with a hash sign “#”.
- Tag secondary UIDs with a hash sign in parentheses (#).

For example, employee ID is a unique identifier in `EMPLOYEE` entity since no two employees can have the same employee ID.

For Instructor Use Only.
This document should not be distributed

Relating Multiple Tables

- Each row of data in a table can be uniquely identified by a primary key.
- You can logically relate data from multiple tables using foreign keys.

Table name: EMPLOYEES

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
100	Steven	King	90
101	Neena	Kochhar	90
102	Lex	De Haan	90
103	Alexander	Hunold	60
104	Bruce	Ernst	60
107	Diana	Lorentz	60
124	Kevin	Mourgos	50
141	Trenna	Rajs	50
142	Curtis	Davies	50

Primary key

Foreign key

Table name: DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500
90	Executive	100	1700
110	Accounting	205	1700
190	Contracting	(null)	1700

Primary key

Each table contains data that describes exactly one entity. For example, the `EMPLOYEES` table contains information about employees. Categories of data are listed across the top of each table, and individual records are listed below. Each table can have one **primary key**, which in effect names the row and ensures no duplicate rows exist.

Because data about different entities is stored in different tables, you may need to combine two or more tables to answer a particular question. For example, you may want to know the location of the department where an employee works.

In this scenario, you need information from the `EMPLOYEES` table (which contains data about employees) and the `DEPARTMENTS` table (which contains information about departments). With an RDBMS, you can relate the data in one table to the data in another table by using **foreign keys**. A **foreign key** is a column (or a set of columns) that refers to a primary key in the same table or another table.

You can use the ability to relate data in one table to the data in another table to organize information in separate, manageable units. Employee data can be kept logically distinct from the department data by storing them in separate tables.

Guidelines for Primary Keys and Foreign Keys

- You cannot use duplicate values in a primary key.
- Primary keys generally cannot be changed.
- Foreign keys are based on data values and are purely logical (not physical) pointers.
- A foreign key value must match an existing primary key value or unique key value; otherwise, it must be null.
- A foreign key must reference either a primary key or a unique key column.

Relational Database Terminology

1	2	3	4	5	6
EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	COMMISSION_PCT	DEPARTMENT_ID
100	Steven	King	24000	(null)	90
101	Neena	Kochhar	17000	(null)	90
102	Lex	De Haan	17000	(null)	90
103	Alexander	Hunold	9000	(null)	60
104	Bruce	Ernst	6000	(null)	60
107	Diana	Lorentz	4200	(null)	60
124	Kevin	Mourgos	5800	(null)	50
141	Trenna	Rajs	3500	(null)	50
142	Curtis	Davies	3100	(null)	50
143	Randall	Matos	2600	(null)	50
144	Peter	Vargas	2500	(null)	50
149	Eleni	Zlotkey	10500	0.2	80
174	Ellen	Abel	11000	0.3	80
176	Jonathon	Taylor	8600	0.2	80
178	Kimberely	Grant	7000	0.15	(null)
200	Jennifer	Whalen	4400	(null)	10
201	Michael	Hartstein	13000	(null)	20
202	Pat	Fay	6000	(null)	20
205	Shelley	Higgins	12000	(null)	110
206	William	Gietz	8300	(null)	110

A relational database can contain one or many tables. A *table* is the basic storage structure of an RDBMS. A table holds all the data necessary about something in the real world, such as employees, invoices, or customers.

The slide shows the contents of the `EMPLOYEES` table or *relation*. The numbers indicate the following:

1. A single *row* (or *tuple*) representing all the data required for a particular employee. Each row in a table should be identified by a primary key, which permits no duplicate rows. The order of rows is insignificant; specify the row order when the data is retrieved.
2. A *column* or attribute containing the employee number. The employee number identifies a *unique* employee in the `EMPLOYEES` table. In this example, the employee number column is designated as the *primary key*. A primary key must contain a value and the value must be unique.
3. A column that is not a key value. A column represents one kind of data in a table; in this example, the data is the salaries of all the employees. Column order is insignificant when storing data; specify the column order when the data is retrieved.
4. A column containing the department number, which is also a *foreign key*. A foreign key is a column that defines how tables relate to each other. A foreign key refers to a primary key or a unique key in the same table or in another table. In the example, `DEPARTMENT_ID` uniquely identifies a department in the `DEPARTMENTS` table.
5. A *field* can be found at the intersection of a row and a column. There can be only one value in it.
6. A field may have no value in it. This is called a null value. In the `EMPLOYEES` table, only those employees who have the role of sales representative have a value in the `COMMISSION_PCT` (commission) field.



Human Resources (HR) Schema and the Tables Used in This Course

For Instructor Use Only.
This document should not be distributed

Human Resources (HR) Application



The image shows a mockup of an HR application interface. It features a central dark blue panel with the title "HR Application" in green. Below the title, there is a "Basic Search:" section with a single text input field. Underneath that is an "Advanced Search:" section with four input fields: "First Name", "Last Name", "Emp. ID", and "Department". A green "GO" button is located at the bottom right of the advanced search section. The background is light blue with a pattern of small white dots. There are several icons: a group of people, a single person, and a person with a magnifying glass. A yellow arrow points from the bottom left towards the search fields. In the bottom left corner, there is a tablet icon displaying a bar chart and a pie chart.

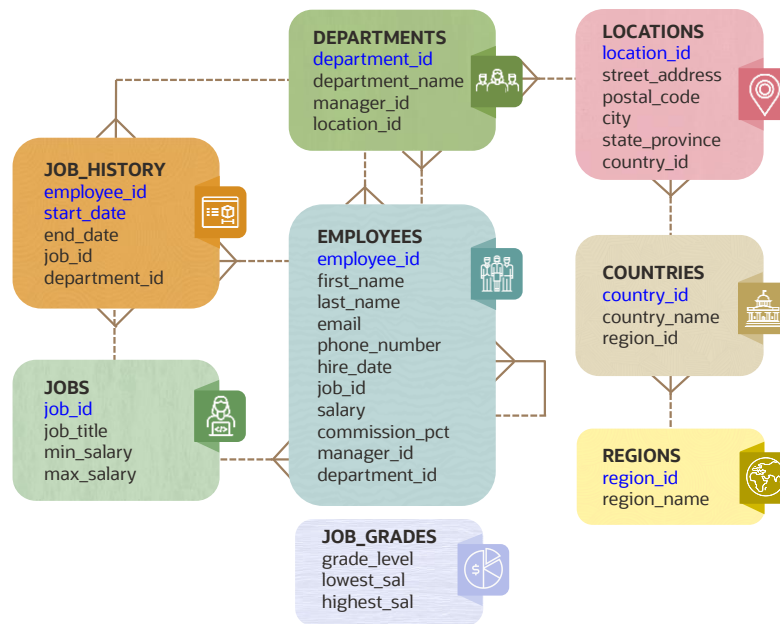
The slide shows an example of a sample HR application, which is generally used by the HR department in an organization. The HR application consists of details of all the employees in the organization whose information is stored in various tables such as `EMPLOYEES`, `DEPARTMENTS`, `JOBS`, `LOCATIONS`, etc. HR managers use this application to search for an employee's details, to update employee records such as when an employee gets promoted, to add new employee records such as when an employee joins the organization, or to delete employee records of employees who have quit. In addition to these, HR managers can use this application to generate a variety of reports, such as the number of employees who have quit and the number of new hires, the average salaries in each department, the list of employees who received a bonus this year, etc.

In this course, we will discuss various scenarios based on the sample HR application. You will also see different examples in the lessons based on HR schema.

Note: Screens like the one shown in this slide are a mockup of an application that could be developed using a tool like APEX or in a programming language like Java or PHP. This course will not be concerned about developing those screens but rather about the database maintaining the data for the HR application. Oracle provides courses on application development in APEX, and MySQL provides training in the MySQL for Developers course on application development in Java, PHP, and Python.

For more information, visit <https://education.oracle.com/home>

Tables Used in This Course



This course uses data from the following tables:

Table Descriptions

- The **EMPLOYEES** table contains information about all the employees, such as their first and last names, job IDs, salaries, hire dates, department IDs, and manager IDs. This table is a child of the **DEPARTMENTS** table.
- The **DEPARTMENTS** table contains information such as the department ID, department name, manager ID, and location ID. This table is the primary key table to the **EMPLOYEES** table.
- The **LOCATIONS** table contains department location information. It contains location ID, street address, city, state province, postal code, and country ID information. It is the primary key table to the **DEPARTMENTS** table and is a child of the **COUNTRIES** table.
- The **COUNTRIES** table contains the country names, country IDs, and region IDs. It is a child of the **REGIONS** table. This table is the primary key table to the **LOCATIONS** table.
- The **REGIONS** table contains region IDs and region names of the various countries. It is a primary key table to the **COUNTRIES** table.
- The **JOB_GRADES** table identifies a salary range per job grade. The salary ranges do not overlap.
- The **JOB_HISTORY** table stores job history of the employees.
- The **JOBS** table contains job titles and salary ranges.

Tables Used in the Course

EMPLOYEES

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY
1	100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24000
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-05	AD_VP	17000
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-01	AD_VP	17000
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-06	AC_MGR	12008
5	104	Bruce	Ernst	BERNST	590.423.4568	21-MAY-07	IT_PROG	6000
6	107	Diana	Lorentz	DLorentz	590.423.5567	07-FEB-07	IT_PROG	4200
7	124	Kevin	Mourgos	KMOURGOS	650.123.5234	16-NOV-07	ST_MAN	5800
8	141	Trenna	Rajs	TRAJS	650.121.8009	17-OCT-03	ST_CLERK	3500
9	142	Curtis	Davies	CDAVIES	650.121.2994	29-JAN-05	ST_CLERK	3100
10	143	Randall	Matos	RMATOS	650.121.2874	15-MAR-06	ST_CLERK	2600
11	144	Peter	Vargas	PVARGAS	650.121.2004	09-JUL-06	ST_CLERK	2500
12	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29-JAN-08	SA_MAN	10500
13	174	Ellen	Abel	EABEL	011.44.1644.429267	11-MAY-04	SA_REP	11000
14	176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	24-MAR-06	SA_REP	8600
15	178	Kimberely	Grant	KGRANT	011.44.1644.429263	24-MAY-07	SA_REP	7000
16	200	Jennifer	Whalen	JWHALEN	515.123.4444	17-SEP-03	AD_ASST	4400
17	201	Michael	Hartstein	MHARTSTE	515.123.5555	17-FEB-04	MK_MAN	13000
18	202	Pat	Fay	PFAY	603.123.6666	17-AUG-05	MK_REP	6000
19	205	Shelley	Higgins	SHIGGINS	515.123.8080	07-JUN-02	AC_MGR	12008
20	206	William	Gietz	WGIEZT	515.123.8181	07-JUN-02	AC_ACCOUNT	8300

JOB_GRADES

	GRADE_LEVEL	LOWEST_SAL	HIGHEST_SAL
1	A	1000	2999
2	B	3000	5999
3	C	6000	9999
4	D	10000	14999
5	E	15000	24999
6	F	25000	40000

DEPARTMENTS

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Shipping	124	1500
4	40	IT	103	1400
5	50	Sales	149	2500
6	60	Executive	100	1700
7	70	Accounting	205	1700
8	80	Contracting	(null)	1700

The following main tables are used in this course:

- **EMPLOYEES** table: Gives details of all the employees
- **DEPARTMENTS** table: Gives details of all the departments
- **JOB_GRADES** table: Gives details of salaries for various grades

Apart from these tables, you will also use the other tables listed in the previous slide, such as the **LOCATIONS** and the **JOB_HISTORY** table.

Note: The structure and data for all the tables are provided in Appendix A.



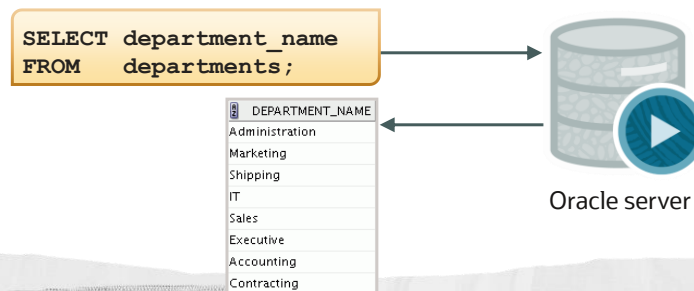
Introduction to SQL and Its Development Environments

For Instructor Use Only.
This document should not be distributed

Using SQL to Query Your Database

Structured query language (SQL) is:

- The ANSI standard language for operating relational databases
- Efficient and easy to learn and use
- Functionally complete
(With SQL, you can define, retrieve, and manipulate data in tables.)



In a relational database, you do not specify the access route to the tables, and you do not need to know how the data is arranged physically.

To access the database, you execute a structured query language (SQL) statement, which is the American National Standards Institute (ANSI) standard language for operating relational databases. SQL is also compliant to ISO Standard (SQL 2016). The Oracle Database complies with the ANSI and ISO standards for SQL (mandatory portion compliance is known as Core SQL, which is at SQL:2016 for the 23c version of the Oracle database).

SQL is a set of statements with which all programs and users access data in an Oracle database. Application programs and Oracle tools often allow users access to the database without using SQL directly, but these applications, in turn, must use SQL when executing the user's request. Oracle Application Express is one such example.

SQL provides statements for a variety of tasks, including:

- Querying data
- Inserting, updating, and deleting rows in a table
- Creating, replacing, altering, and dropping objects
- Controlling access to the database and its objects
- Guaranteeing database consistency and integrity

SQL unifies all of the preceding tasks in one consistent language and enables you to work with data at a logical level.

How SQL Works

SQL:

- Is standalone and powerful
- Processes groups of data
- Lets you work with data at a logical level

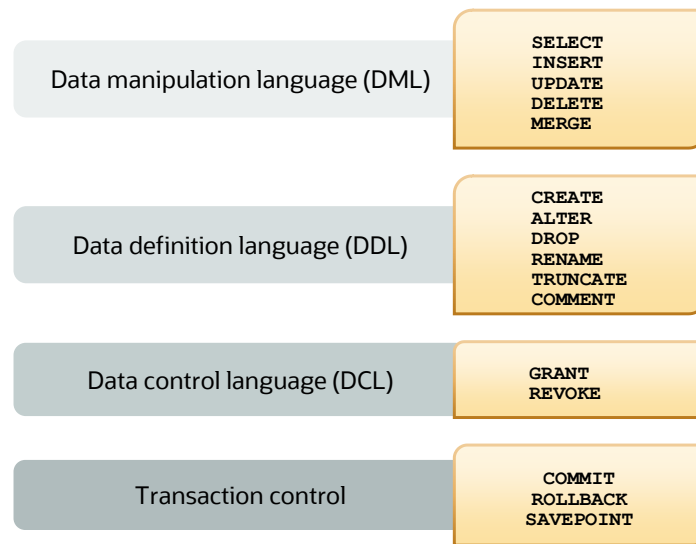


Using SQL benefits all types of users, including application programmers, database administrators, managers, and end users. The purpose of SQL is to provide an interface to a relational database such as Oracle Database. All SQL statements are instructions to the database. Some of the features of SQL are:

- It processes sets of data as groups rather than as individual units.
- It provides automatic navigation to the data.
- It uses statements that are complex and powerful individually and are therefore standalone.

SQL lets you work with data at the logical level. For example, to retrieve a set of rows from a table, you define a condition used to filter the rows. All rows satisfying the condition are retrieved in a single step and can be passed as a unit to the user, to another SQL statement, or to an application. You need not deal with the rows one by one, nor do you have to worry about how they are physically stored or retrieved.

SQL Statements Used in the Course



SQL statements are used to access the database and maintain the data.

SQL statements supported by Oracle comply with industry standards. Oracle Corporation ensures future compliance with evolving standards by actively involving key personnel in SQL standards committees. The industry-accepted committees are ANSI and International Standards Organization (ISO). Both ANSI and ISO have accepted SQL as the standard language for relational databases.

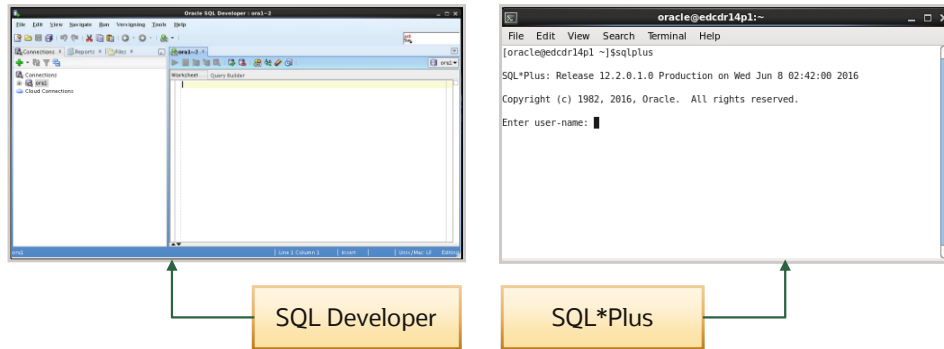
Statements	Description
SELECT INSERT UPDATE DELETE MERGE	Retrieve data from the database, enter new rows, change existing rows, and remove unwanted rows from tables in the database, respectively. Collectively known as <i>Data Manipulation Language</i> (DML)
CREATE ALTER DROP RENAME TRUNCATE COMMENT	Set up, change, and remove data structures from tables. Collectively known as <i>Data Definition Language</i> (DDL)
GRANT REVOKE	Provide or remove access rights to both the Oracle Database and the structures within it.
COMMIT ROLLBACK SAVEPOINT	Manage the changes made by DML statements. Changes to the data can be grouped together into logical transactions.

Development Environments for SQL in Oracle



There are two Oracle development environments presented in this course:

- The primary tool is Oracle SQL Developer.
- The SQL*Plus command-line interface can also be used.



There are various development environments for writing SQL statements. Oracle SQL Developer and Oracle SQL*Plus are the commonly used tools in Oracle.

Oracle SQL Developer

Oracle SQL Developer is a graphical interface for developing SQL. It provides features to view the database components and update values without writing any SQL query. In this course, Oracle SQL Developer is used to create and execute example SQL statements.

Oracle SQL*Plus

If you like working with a command-line interface, you can use Oracle SQL*Plus, which is a command-line based environment.

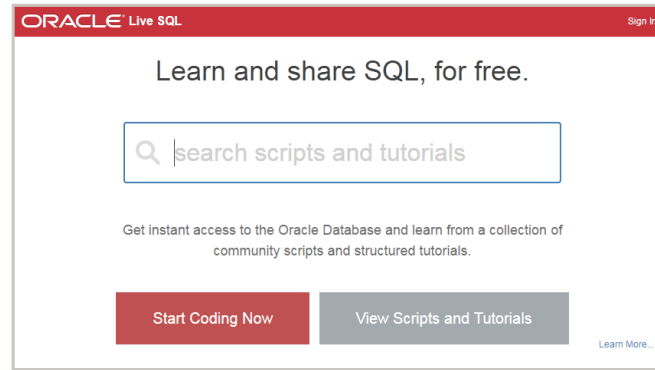
Note

- See Appendix B for information about using Oracle SQL Developer, including simple instructions on the installation process.
- See Appendix C for information about using Oracle SQL*Plus.

Introduction to Oracle Live SQL



- Easy way to learn, access, test, and share SQL and PL/SQL scripts on Oracle Database
- Sign up and use it free of cost



Oracle Live SQL is another environment where you can write and execute SQL statements.

You can now learn SQL without the need to install a database or download any tool. Oracle Live SQL exists to provide the Oracle database community with an easy online way to test and share SQL and PL/SQL application development concepts.

Let's look at some of the features of Oracle Live SQL:

- Provides browser-based SQL worksheet access to an Oracle database schema
- Has the ability to save and share SQL scripts
- Provides a schema browser to view and extend database objects
- Provides access to interactive educational tutorials
- Provides customized data access examples for PL/SQL, Java, PHP, C

You can continue to learn SQL by using Live SQL yourself. All you need is your Oracle Technology Network (OTN) credentials and an interest in learning SQL.

Note: Oracle Live SQL cannot be used to execute the SQL examples without the initial schema setup.

For Instructor Use Only.
This document should not be distributed



Oracle Database 23c SQL Documentation and Additional Resources

For Instructor Use Only.
This document should not be distributed

Oracle Database Documentation



- Oracle Database New Features Guide
- Oracle Database Reference
- Oracle Database SQL Language Reference
- Oracle Database Concepts
- Oracle Database SQL Developer User's Guide

Navigate to <https://docs.oracle.com/en/database/database.html> to access the Oracle Database 23c documentation library.

For Instructor Use Only.
This document should not be distributed

Additional Resources for Oracle

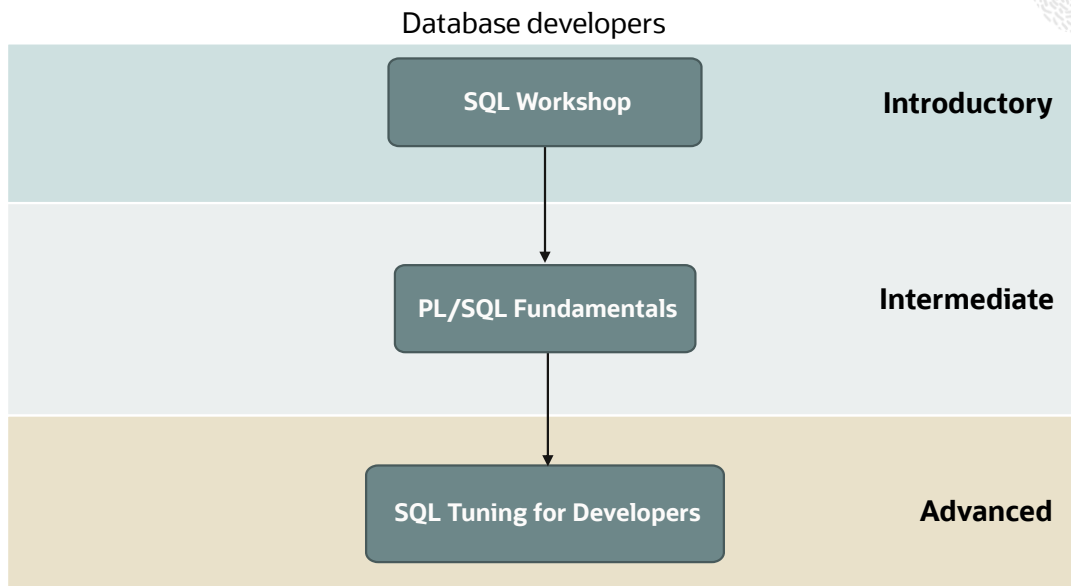


For additional information about Oracle Database 23c, refer to the following:

- Oracle Learning Library:
 - <http://www.oracle.com/goto/oll>
- Oracle Cloud:
 - cloud.oracle.com
- The online SQL Developer Home Page, which is available at:
 - http://www.oracle.com/technology/products/database/sql_developer/index.html
- The SQL Developer tutorial, which is available online at:
 - <http://download.oracle.com/oll/tutorials/SQLDeveloper/index.htm>

For Instructor Use Only.
This document should not be distributed

Oracle University: Oracle SQL Training



For full details about Oracle SQL training options, go to <http://education.oracle.com>. There are different training formats to suit your learning style.

Oracle SQL Certification



- The Oracle Certification Program validates your expertise in the following areas:
 - Oracle Database SQL Certified Associate
 - Oracle Database PL/SQL Certified Associate
 - Oracle Database PL/SQL Certified Professional
- Take the examination at a local Pearson VUE test center:
 - At a time suitable to you
 - In over 175 countries

For full details of SQL and other Oracle Certification options, visit:
<http://education.oracle.com/certification>.

For Instructor Use Only.
This document should not be distributed

Summary



Overview of Oracle Database 23c and related products

Overview of relational database management concepts and terminologies

Human Resources (HR) Schema and the tables used in the course

Introduction to SQL and its development environments

Oracle Database 23c SQL documentation and additional resources

Relational database management systems are composed of objects or relations. They are managed by operations and governed by data integrity constraints.

Oracle Corporation produces products and services to meet your RDBMS needs. The main products are the following:

- Oracle Database, which you use to store and manage information by using SQL
- Oracle Fusion Middleware, which you use to develop, deploy, and manage modular business services that can be integrated and reused
- Oracle Enterprise Manager Grid Control, which you use to manage and automate administrative tasks across sets of systems in a grid environment

SQL

The Oracle server supports ANSI-standard SQL and contains extensions. SQL is the language that is used to communicate with the server to access, manipulate, and control data.

For Instructor Use Only.
This document should not be distributed

For Instructor Use Only.
This document should not be distributed

Retrieving Data Using the SQL `SELECT` Statement



Course Roadmap

Lesson 1: Introduction

**Unit 1: Retrieving, Restricting,
and Sorting Data**

Unit 2: Joins, Subqueries, and Set
Operators

Unit 3: DML and DDL



**Lesson 2: Retrieving Data Using the SQL `SELECT`
statement**

You are here!



Lesson 3: Restricting and Sorting Data



Lesson 4: Using Single-Row Functions to
Customize Output



Lesson 5: Using Conversion Functions and
Conditional Expressions

In Unit 1, you will learn how to query the data from tables, how to query selected records from tables, and also how to sort the data retrieved from the tables.

Objectives



Capabilities of SQL `SELECT` statements

Arithmetic expressions and `NULL` values in the `SELECT` statement

Column aliases

Use of the concatenation operator, literal character strings, alternative quote operator, and the `DISTINCT` keyword

`DESCRIBE` command

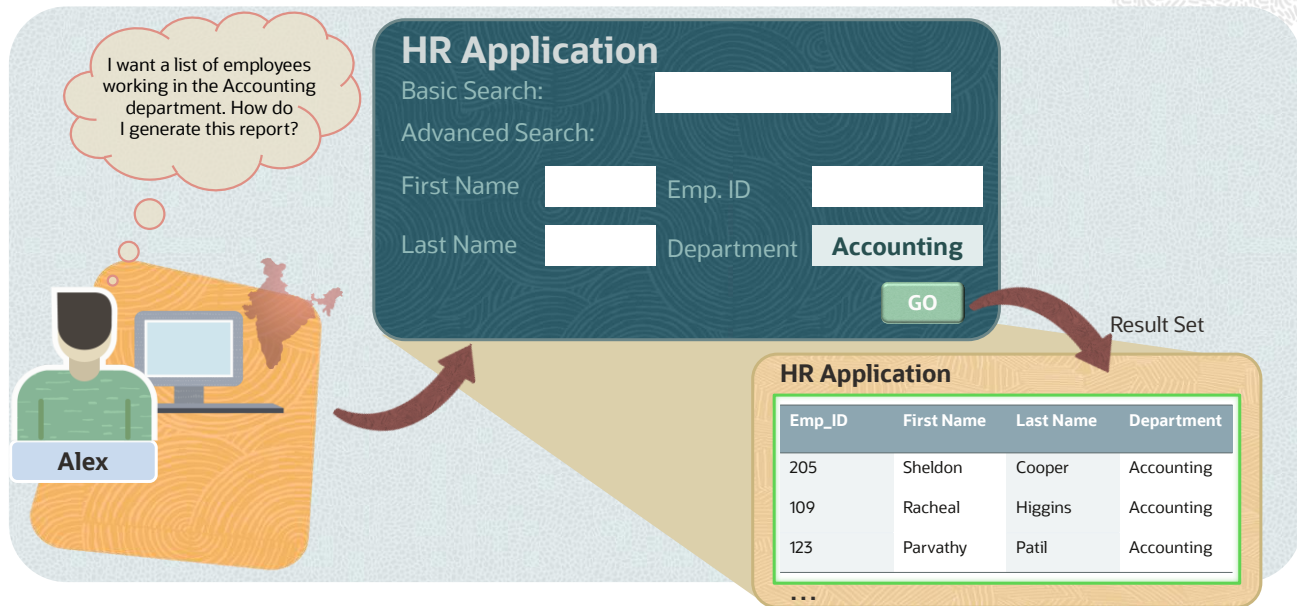
For Instructor Use Only.
This document should not be distributed



Capabilities of SQL SELECT Statements

For Instructor Use Only.
This document should not be distributed

HR Application Scenario



Alex, an HR manager in India, wants a report of all the employees in the Accounting department of the organization. This slide displays a mockup of the screens that the application might display in order to access the database. The HR application consists of a database of all the employees in the organization. So, how does Alex search based on the criteria in the HR application?

Alex finds it very efficient to use the HR application to generate reports. He logs in to the application, enters "Accounting" in the Department field, and clicks Go.

The HR application fires a SQL `SELECT` statement to query the database for all employees in the Accounting department. Alex then sees the result on his application.

Basically, you can query the database for information by writing conditional and complex SQL statements. In this lesson, you will learn more about SQL `SELECT` statements. You will not learn about the application used to develop the interface screens like those shown in the mockup in this slide. Those are covered in Oracle courses on application development like APEX and in the MySQL for Developers course.

Writing SQL Statements

- Not case-sensitive
- Can be entered on one or more lines
- Cannot have abbreviated keywords
- No splitting of keywords across lines
- Clauses usually placed on separate lines
- Indents used to enhance readability
- Uppercase keywords
- Lowercase for all other words



By using the following simple rules and guidelines, you can construct valid statements that are easy to read and edit:

- SQL statements are not case-sensitive (unless indicated).
- SQL statements can be entered on one or many lines.
- Keywords cannot be abbreviated or split across lines.
- Clauses are usually placed on separate lines for readability and ease of editing.
- Indents should be used to make code more readable.
- Keywords typically are entered in uppercase; all other words, such as table names and columns names, are entered in lowercase.

For Instructor Use Only.
This document should not be distributed

Basic SELECT Statement

- `SELECT` identifies the columns to be displayed.
- `FROM` identifies the table containing those columns.

```
SELECT *|{[DISTINCT] column [alias],...}  
FROM table;
```

Selecting from a table →



Use the `SELECT` statement to retrieve data from one or more tables or views in the database.

In its simplest form, a `SELECT` statement must include the following:

- A `SELECT` clause, which specifies the columns to be displayed
- Depending on the database version, a `FROM` clause, which identifies the table containing the columns that are listed in the `SELECT` clause

In the syntax:

<code>SELECT</code>	Is a list of one or more columns
<code>*</code>	Selects all columns
<code>DISTINCT</code>	Suppresses duplicates
<code>column expression</code>	Selects the named column or the expression
<code>alias</code>	Gives different headings to the selected columns
<code>FROM table</code>	Specifies the table containing the columns

Throughout this course, you will see that the words *keyword*, *clause*, and *statement* are used as follows:

- A *keyword* refers to an individual SQL element—for example, `SELECT` and `FROM` are keywords.
- A *clause* is a part of a SQL statement—for example, `SELECT employee_id, last_name`, and so on.
- A *statement* is a combination of two or more clauses—for example, `SELECT * FROM employees`.

Selecting All Columns

Oracle SQL Developer:

```
SELECT *  
FROM departments;
```



	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700
8	190	Contracting	(null)	1700

You can display all columns of data in a table by following the `SELECT` keyword with an asterisk (*). In the example in the slide, the `departments` table contains four columns: `department_id`, `department_name`, `manager_id`, and `location_id`. The table contains eight rows, one for each department.

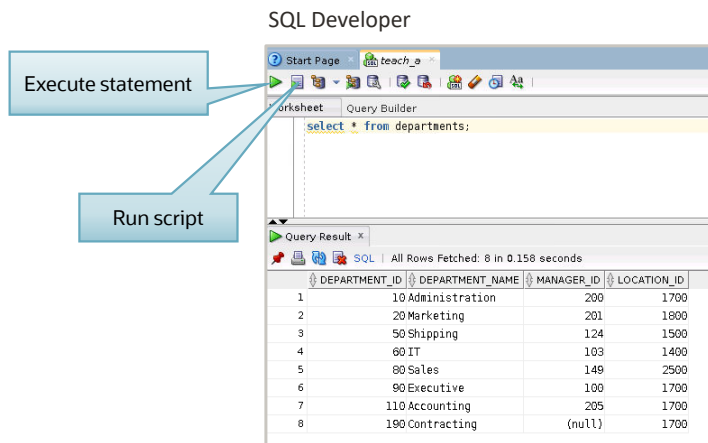
You can also display all columns in the table by listing them after the `SELECT` keyword. For example, the following SQL statement (like the example in the slide) displays all columns and all rows of the `departments` table:

```
SELECT department_id, department_name, manager_id, location_id  
FROM departments;
```

The example in the slide shows the output from executing the displayed statement in Oracle SQL Developer. Before considering other `SELECT` statements, the next few topics explain how to execute statements and what the output looks like in SQL Developer.

Note: In this course, the examples on slides show output of Oracle SQL Developer. Your output should appear like the output shown. Three dots (...) are used where the total output is too large to fit on the slide.

Executing SQL Statements with Oracle SQL Developer and SQL*Plus



SQL*Plus

```
SQL> select * from departments;
```

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
30	Shipping	124	1500
40	IT	103	1400
50	Sales	149	2500
60	Executive	100	1700
70	Accounting	205	1700
80	Contracting		1700

8 rows selected.

```
SQL>
```

In SQL Developer:

- Click the Run Script icon or press [F5] to run the command or commands in the SQL Worksheet.
- You can also click the Execute Statement icon or press [F9] to run a SQL statement in the SQL Worksheet.
 - The Execute Statement icon executes the statement at the cursor in the Enter SQL Statement box, while the Run Script icon executes all the statements in the Enter SQL Statement box.
 - The Execute Statement icon displays the output of the query on the Results tabbed page, whereas the Run Script icon emulates the SQL*Plus display and shows the output on the Script Output tabbed page.

In SQL*Plus:

- In SQL*Plus, terminate the SQL statement with a semicolon and then press [Enter] to run the command.

Column Heading Defaults in SQL Developer and SQL*Plus

- SQL Developer:
 - Default heading alignment: Left-aligned
 - Default heading display: Uppercase
- SQL*Plus:
 - Character and date column headings: Left-aligned
 - Number column headings: Right-aligned
 - Default heading display: Uppercase



In SQL Developer, column headings are displayed in uppercase and are left-aligned.

Run the following SQL statement and observe the column headings in the output:

```
SELECT last_name, hire_date, salary
FROM   employees;
```

You can override the column heading display with an alias. Column aliases are covered later in this lesson.

For Instructor Use Only.
This document should not be distributed

Selecting Specific Columns

Oracle SQL Developer:

```
SELECT department_id, location_id  
FROM departments;
```



	DEPARTMENT_ID	LOCATION_ID
1	10	1700
2	20	1800
3	50	1500
4	60	1400
5	80	2500
6	90	1700
7	110	1700
8	190	1700

You can use the `SELECT` statement to display specific columns of the table by specifying the column names, separated by commas. The example in the slide displays all the department numbers and location numbers from the `departments` table. The order of the results might differ unless you specify an order. You learn to specify an order for the results in the lesson titled “Restricting and Sorting Data.”

In the `SELECT` clause, specify the columns that you want, in the order in which you want them to appear in the output. For example, to display the location before department number (from left to right), you use the following statement:

```
SELECT location_id, department_id  
  
FROM departments;
```

Selecting from dual with Oracle Database

- `dual` is a table automatically created by Oracle Database.
- `dual` has one column called `DUMMY`, of data type `VARCHAR(1)`, and contains one row with a value `x`.

```
SELECT *  
FROM dual;
```

DUMMY
x

```
SELECT SYSDATE  
FROM dual;
```

SYSDATE
1 25-JUN-18

When you install Oracle Database, a `dual` table is automatically created. This table is in the `SYS` user schema but is accessible by the name `dual` to all users. When you display the contents of the `dual` table, you will observe that it has one column, `DUMMY`, defined to be `varchar(1)`, and contains one row with a value `x`.

Selecting from the `dual` table is useful for computing a constant expression with the `SELECT` statement. Because `dual` has only one row, the constant is returned only once. Alternatively, you can select a constant, pseudocolumn, or expression from any table, but the value will be returned as many times as there are rows in the table.

For example, if you want to compute the expression $12 * 4 / 5 + 5 * 8$, use the following statement:

```
SELECT 12*4/5+5*8  
  
FROM dual;
```

New Feature in 23c: SELECT Without FROM Clause

- `dual` is no longer needed in the 23c version of the Oracle Database.
- Create the statement with no `FROM` clause.

```
SELECT 5280 * 1.5;
```

	5280*1.5
1	7920

```
SELECT SYSDATE;
```

	SYSDATE
1	25-JUN-18

Starting in the 23c version of the Oracle Database, the `dual` table is not required when using certain functions or when using certain computations. For example, if you want to compute the expression `12*4/5+5*8` in 23c, use the following statement with no `FROM` clause:

```
SELECT 12*4/5+5*8;
```




Arithmetic Expressions and NULL Values in the `SELECT` statement

For Instructor Use Only.
This document should not be distributed

Arithmetic Expressions

You can create expressions with number and date data by using arithmetic operators.

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide

You may need to modify the way in which data is displayed, or you may want to perform calculations or look at what-if scenarios. All these are possible using arithmetic expressions. An arithmetic expression can contain column names, constant numeric values, and the arithmetic operators.

Arithmetic Operators

The slide lists the arithmetic operators that are available in SQL. You can use arithmetic operators in any clause of a SQL statement (except the `FROM` clause).

Remember that you can use only the addition and subtraction operators with the `DATE` and `TIMESTAMP` data types.

Using Arithmetic Operators

```
SELECT last_name, salary, salary + 300
FROM employees;
```



	LAST_NAME	SALARY	SALARY+300
1	King	24000	24300
2	Kochhar	17000	17300
3	De Haan	17000	17300
4	Hunold	9000	9300
5	Ernst	6000	6300
6	Lorentz	4200	4500
7	Mourgos	5800	6100
8	Rajs	3500	3800
9	Davies	3100	3400
10	Matos	2600	2900

...

The example in the slide uses the addition operator to calculate a salary increase of \$300 for all employees. The slide also displays a `SALARY+300` column in the output.

Note that the resultant calculated column, `SALARY+300`, is not a new column in the `EMPLOYEES` table; it is for display only. By default, the name of a new column comes from the calculation that generated it—in this case, `salary+300`.

Remember that the Oracle server ignores blank spaces before and after the arithmetic operator.

Rules of Precedence

- Multiplication and division occur before addition and subtraction.
- Operators of the same priority are evaluated from left to right.
- Parentheses are used to override the default precedence or to clarify the statement.

Operator Precedence

1

```
SELECT last_name, salary, 12*salary+100
FROM employees;
```



	LAST_NAME	SALARY	12*SALARY+100
1	King	24000	288100
2	Kochhar	17000	204100
3	De Haan	17000	204100
4	Hunold	9000	108100

...

2

```
SELECT last_name, salary, 12*(salary+100)
FROM employees;
```



	LAST_NAME	SALARY	12*(SALARY+100)
1	King	24000	289200
2	Kochhar	17000	205200
3	De Haan	17000	205200
4	Hunold	9000	109200

...

The first example in the slide displays the last name, salary, and annual compensation of employees. It calculates the annual compensation by multiplying the monthly salary with 12, plus a one-time bonus of \$100. Note that multiplication is performed before addition.

Note: Use parentheses to reinforce the standard order of precedence and to improve clarity. For example, the expression in the slide can be written as $(12 * \text{salary}) + 100$ with no change in the result.

Using Parentheses

You can override the rules of precedence by using parentheses to specify the desired order in which the operators are to be executed.

The second example in the slide displays the last name, salary, and annual compensation of employees. It calculates the annual compensation as follows: adding a monthly bonus of \$100 to the monthly salary and then multiplying that subtotal with 12. Because of the parentheses, addition takes priority over multiplication.

Defining a Null Value

- Null is a value that is unavailable, unassigned, unknown, or inapplicable.
- Null is not the same as zero or a blank space.

```
SELECT last_name, job_id, salary, commission_pct  
FROM employees;
```

	LAST_NAME	JOB_ID	SALARY	COMMISSION_PCT
1	King	AD_PRES	24000	(null)
2	Kochhar	AD_VP	17000	(null)
3	De Haan	AD_VP	17000	(null)
...				
12	Zlotkey	SA_MAN	10500	0.2
13	Abel	SA_REP	11000	0.3
14	Taylor	SA_REP	8600	0.2
15	Grant	SA_REP	7000	0.15
...				
18	Fay	MK_REP	6000	(null)
19	Higgins	AC_MGR	12008	(null)
20	Gietz	AC_ACCOUNT	8300	(null)

If a row lacks a data value for a particular column, that value is said to be `NULL` or to contain a null.

You can select columns with `NULL` value in a `SELECT` query and `NULL` values can be part of an arithmetic expression. Any arithmetic expression using `NULL` values results into `NULL`.

Columns of any data type can contain nulls. However, some constraints (`NOT NULL` and `PRIMARY KEY`) prevent nulls from being used in the column.

In the example in the slide, notice that only a sales manager or sales representative can earn a commission in the `COMMISSION_PCT` column of the `EMPLOYEES` table. Other employees are not entitled to earn commissions. A null represents that fact.

You can see that by default, SQL Developer uses the literal, `(null)`, to identify null values.

Null Values in Arithmetic Expressions

Arithmetic expressions containing a null value evaluate to null.

```
SELECT last_name, 12*salary*commission_pct
FROM employees;
```



	LAST_NAME	12*SALARY*COMMISSION_PCT
1	King	(null)
2	Kochhar	(null)
3	De Haan	(null)

...

12	Zlotkey	25200
13	Abel	39600
14	Taylor	20640
15	Grant	12600

...

17	Hartstein	(null)
18	Fay	(null)
19	Higgins	(null)
20	Gietz	(null)

If any column value in an arithmetic expression is null, the result is null. For example, if you attempt to perform division by zero, you get an error. However, if you divide a number by null, the result is a null or unknown.

In the example in the slide, employee King does not get any commission. Because the `commission_pct` column in the arithmetic expression is null, the result is null.



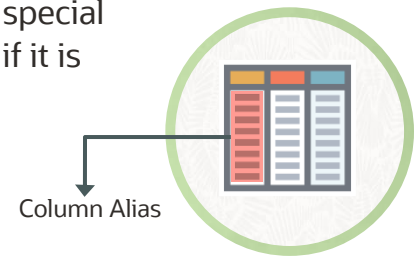
Column Aliases

For Instructor Use Only.
This document should not be distributed

Defining a Column Alias

A column alias:

- Renames a column heading
- Is useful with calculations
- Immediately follows the column name
(there can also be the optional AS keyword between the column name and the alias)
- Requires double quotation marks if it contains spaces or special characters. In Oracle, it requires double quotation marks if it is case-sensitive.



When displaying the result of a query, the results normally use the name of the selected column as the column heading. This heading might not be descriptive and, therefore, might be difficult to understand. You can change a column heading by using a column alias.

Specify the alias after the column in the `SELECT` list using blank space or the optional keyword `AS` as a separator between the column name and the alias.

In Oracle database, by default, alias headings appear in uppercase, so enclose a case-sensitive alias in double quotation marks (" "). In Oracle database, if the alias contains spaces or special characters (such as -, !, _), enclose the alias in double quotation marks (" ").

Using Column Aliases

```
SELECT last_name AS name, commission_pct comm
FROM employees;
```



	NAME	COMM
1	King	(null)
2	Kochhar	(null)
3	De Haan	(null)
4	Hunold	(null)

...

```
SELECT last_name "Name", salary*12 "Annual Salary"
FROM employees;
```



	Name	Annual Salary
1	King	288000
2	Kochhar	204000
3	De Haan	204000
4	Hunold	108000

...

The first example displays the names and the commission percentages of all the employees. Note that the optional `AS` keyword has been used before the column alias `name`. The result of the query is the same whether the `AS` keyword is used or not. Also, note that the SQL statement has the column aliases, `name` and `comm`, in lowercase, whereas the result of the query in Oracle displays the column headings in uppercase. As mentioned in the preceding slide, column headings appear in uppercase by default.

The second example displays the last names and annual salaries of all the employees. Because `Annual Salary` contains a space, it has been enclosed in double quotation marks. Note that the column heading in the output is exactly the same as the column alias.

Use of the Concatenation Operator, Literal Character Strings, Alternative Quote Operator, and the `DISTINCT` Keyword



For Instructor Use Only.
This document should not be distributed

Concatenation Operator in Oracle

The concatenation operator:

- Links columns or character strings to other columns
- Is represented by two vertical bars (||)
- Creates a resultant column that is a character expression

```
SELECT last_name || job_id AS "Employees"  
FROM employees;
```



Employees
1 AbelSA_REP
2 DaviesST_CLERK
3 De HaanAD_VP
4 ErnstIT_PROG
5 FayMK_REP
6 GietzAC_ACCOUNT
7 GrantSA_REP
8 HartsteinMK_MAN

...

You can link columns to other columns, arithmetic expressions, or constant values to create a character expression by using the concatenation operator (||). Columns on either side of the operator are combined to make a single output column.

In the example, LAST_NAME and JOB_ID are concatenated and given the alias Employees. Note that the last name of the employee and the job code are combined to make a single output column.

The AS keyword before the alias name makes the SELECT clause easier to read.

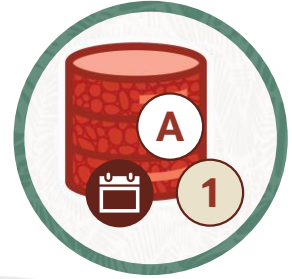
Null Values with the Concatenation Operator

In Oracle databases, if you concatenate a null value with a character string, the result is a character string.

LAST_NAME || NULL results in LAST_NAME.

Literal Character Strings

- A literal is a character, a number, or a date that is included in the `SELECT` statement.
- Date and character literal values must be enclosed within single quotation marks.
- Each character string is output once for each row returned.



A literal is a character, a number, or a date that is included in the `SELECT` list. It is not a column name or a column alias. It is printed for each row returned. Literal strings of free-format text can be included in the query result and are treated the same as a column in the `SELECT` list.

The date and character literals **must** be enclosed within single quotation marks (' '); number literals need not be enclosed in a similar manner.

Using Literal Character Strings in Oracle

```
SELECT last_name || ' is a ' || job_id  
       AS "Employee Details"  
FROM   employees;
```



	Employee Details
1	Abel is a SA_REP
2	Davies is a ST_CLERK
3	De Haan is a AD_VP
4	Ernst is a IT_PROG
5	Fay is a MK_REP
6	Gietz is a AC_ACCOUNT
7	Grant is a SA_REP
8	Hartstein is a MK_MAN
9	Higgins is a AC_MGR
10	Hunold is a IT_PROG
11	King is a AD_PRES

The example in the slide displays the last names and job codes of all employees. The column has the heading Employee Details. Note the spaces between the single quotation marks in the `SELECT` statement. The spaces improve the readability of the output.

In the following example, the last name and salary for each employee are concatenated with a literal, to give the returned rows more meaning:

```
SELECT last_name || ': 1 Month salary = ' || salary Monthly  
FROM   employees;
```

Alternative Quote (q) Operator in Oracle

- Specify your own quotation mark delimiter.
- Select any delimiter.
- Increase readability and usability.

```
SELECT department_name || q'[ Department's Manager Id: ]'  
|| manager_id  
AS "Department and Manager"  
FROM departments;
```

Department and Manager	
1	Administration Department's Manager Id: 200
2	Marketing Department's Manager Id: 201
3	Shipping Department's Manager Id: 124
4	IT Department's Manager Id: 103
5	Sales Department's Manager Id: 149
6	Executive Department's Manager Id: 100
7	Accounting Department's Manager Id: 205
8	Contracting Department's Manager Id:

Many SQL statements use character literals in expressions or conditions. If the literal itself contains a single quotation mark, you can use the quote (q) operator and select your own quotation mark delimiter.

You can choose any convenient delimiter, single-byte or multibyte, or any of the following character pairs: [], { }, (), or < >.

In the example shown, the string contains a single quotation mark, which is normally interpreted as a delimiter of a character string. By using the q operator, however, brackets [] are used as the quotation mark delimiters. The string between the brackets delimiters is interpreted as a literal character string.

Duplicate Rows

The default display of queries is all rows, including duplicate rows.

1
`SELECT department_id
FROM employees;`



	DEPARTMENT_ID
1	90
2	90
3	90
4	60
5	60
6	60
7	50
8	50
...	

2
`SELECT DISTINCT department_id
FROM employees;`



	DEPARTMENT_ID
1	(null)
2	90
3	20
4	110
5	50
6	80
7	60
8	10

Unless you indicate otherwise, SQL displays the results of a query without eliminating the duplicate rows. The first example in the slide displays all the department numbers from the `EMPLOYEES` table. Note that the department numbers are repeated.

To eliminate duplicate rows in the result, include the `DISTINCT` keyword in the `SELECT` clause immediately after the `SELECT` keyword. In the second example in the slide, the `EMPLOYEES` table actually contains 20 rows, but there are only seven unique department numbers in the table.

You can specify multiple columns after the `DISTINCT` qualifier. The `DISTINCT` qualifier affects all the selected columns, and the result is every distinct combination of the columns.

```
SELECT DISTINCT department_id, job_id  
FROM employees;
```



DESCRIBE Command

For Instructor Use Only.
This document should not be distributed

Displaying Table Structure by Using the DESCRIBE Command

Syntax:

```
DESCRIBE tablename
```

Example:

```
DESCRIBE employees
```



```
DESCRIBE Employees
Name          Null    Type
-----
EMPLOYEE_ID   NOT NULL NUMBER(6)
FIRST_NAME    VARCHA VARCHAR2(20)
LAST_NAME     NOT NULL VARCHA VARCHAR2(25)
EMAIL         NOT NULL VARCHA VARCHAR2(25)
PHONE_NUMBER  VARCHA VARCHAR2(20)
HIRE_DATE     NOT NULL DATE
JOB_ID        NOT NULL VARCHA VARCHAR2(10)
SALARY        NUMBER(8,2)
COMMISSION_PCT NUMBER(2,2)
MANAGER_ID    NUMBER(6)
DEPARTMENT_ID NUMBER(4)
```

In the syntax, *tablename* is the name of any existing table, view, or synonym that is accessible to the user.

The example in the slide displays information about the structure of the `employees` table using the `DESCRIBE` command.

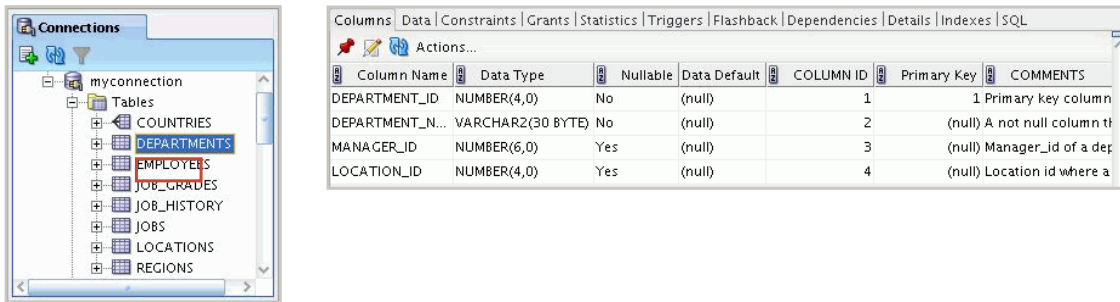
In the resulting display, *Null* indicates that the values for this column may be unknown. `NOT NULL` in SQL Developer indicates that a column must contain data. *Type* displays the data type for a column.

Data types are explained in the lesson titled “Introduction to Data Definition Language.”

Data Type	Description
NUMBER (<i>p</i> , <i>s</i>)	Number value having a maximum number of digits <i>p</i> , with <i>s</i> digits to the right of the decimal point
VARCHAR2 (<i>s</i>)	Variable-length character value of maximum size <i>s</i>
DATE	Date and time value between January 1, 4712 B.C. and December 31, A.D. 9999

Displaying Table Structure by Using Oracle SQL Developer

- Use the `DESCRIBE` command to display the structure of a table.
- Alternatively, select the table in the Connections tree and use the Columns tab to view the table structure.



You can display the structure of a table in Oracle SQL Developer by using the `DESCRIBE` command. The command displays the column names and the data types, and it shows you whether a column *must* contain data (that is, whether the column has a `NOT NULL` constraint).

In the syntax, *table name* is the name of any existing table, view, or synonym that is accessible to the user.

Using the SQL Developer GUI interface, you can select the table in the Connections tree and use the Columns tab to view the table structure.

Note: `DESCRIBE` is a SQL*Plus command supported by SQL Developer. It is abbreviated as `DESC`.

Summary



Capabilities of SQL `SELECT` statements

Arithmetic expressions and `NULL` values in the `SELECT` statement

Column aliases

Use of the concatenation operator, literal character strings, alternative quote operator, and the `DISTINCT` keyword

`DESCRIBE` command

In this lesson, you should have learned how to retrieve data from a database table with the `SELECT` statement.

```
SELECT  *|{[DISTINCT] column [alias],...}
FROM    table;
```

In the syntax:

<code>SELECT</code>	Is a list of one or more columns
<code>*</code>	Selects all columns
<code>DISTINCT</code>	Suppresses duplicates
<code>column expression</code>	Selects the named column or the expression
<code>alias</code>	Gives different headings to the selected columns
<code>FROM table</code>	Specifies the table containing the columns

Restricting and Sorting Data



Course Roadmap

Lesson 1: Introduction

**Unit 1: Retrieving, Restricting,
and Sorting Data**

Unit 2: Joins, Subqueries, and Set
Operators

Unit 3: DML and DDL



Lesson 2: Retrieving Data Using the SQL `SELECT`
Statement



Lesson 3: Restricting and Sorting Data

You are here!



Lesson 4: Using Single-Row Functions to
Customize Output



Lesson 5: Using Conversion Functions and
Conditional Expressions

In Unit 1, you will learn how to query the data from tables, how to query selected records from tables, and also how to sort the data retrieved from the tables.

Objectives



Limiting rows

Rules of precedence for operators in an expression

Sorting rows using the `ORDER BY` clause

SQL row limiting clause in a query

Substitution variables in Oracle

Assigning values to variables

When retrieving data from a database, you may need to:

- Restrict the rows of data that are displayed
- Specify the order in which the rows are displayed

This lesson explains the SQL statements that you use to perform the actions listed in the slide.



Limiting Rows

For Instructor Use Only.
This document should not be distributed

Limiting Rows by Using a Selection

```
SELECT employee_id, last_name, job_id, department_id
FROM employees;
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90
4	103	Hunold	IT_PROG	60
5	104	Ernst	IT_PROG	60
6	107	Lorentz	IT_PROG	60

...

What if you want to retrieve all employees in department 90, but not other departments?



	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90

In the example in the slide, assume that you want to display all the employees in department 90. The rows with a value of 90 in the `DEPARTMENT_ID` column are the only ones that are returned. This method of restriction is the basis of the `WHERE` clause in SQL.

Limiting Rows That Are Selected

- Restrict the rows that are returned by using the `WHERE` clause:

```
SELECT *|{[DISTINCT] column [alias],...}  
FROM   table  
[WHERE logical expression(s)];
```

- The `WHERE` clause follows the `FROM` clause.



You can restrict the rows that are returned from the query by using the `WHERE` clause. A `WHERE` clause contains a condition that must be met, and it directly follows the `FROM` clause. If the condition is true, the row meeting the condition is returned.

In the syntax:

`WHERE`
logical expression

Restricts the query to rows that meet a condition
Is composed of column names, constants, and a comparison operator. It specifies a combination of one or more expressions and Boolean operators and returns a value of `TRUE`, `FALSE`, or `KNOWN`.

The `WHERE` clause can compare literals and values in columns, arithmetic expressions, or functions. It consists of three elements:

- Column name
- Comparison condition
- Column name, constant, or list of values

Using the WHERE Clause

```
SELECT employee_id, last_name, job_id, department_id
FROM   employees
WHERE  department_id = 90 ;
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90

In the example, the `SELECT` statement retrieves the employee ID, last name, job ID, and department number of all employees who are in department 90.

Note: You cannot use column alias in the `WHERE` clause.

Character Strings and Dates

- Character strings and date values are enclosed within single quotation marks (' ').
- Character values are case-sensitive, and date values are format-sensitive.
- The default display format for date is DD-MON-RR.

```
SELECT last_name, job_id, department_id
FROM   employees
WHERE  last_name = 'Whalen' ;
```



LAST_NAME	JOB_ID	DEPARTMENT_ID
1 Whalen	AD_ASST	10

```
SELECT last_name
FROM   employees
WHERE  hire_date = '17-OCT-11' ;
```



LAST_NAME
1 Rajs

You must enclose character strings and dates in the `WHERE` clause within single quotation marks (' '). Number constants, however, need not be enclosed within single quotation marks.

Remember that all character searches are case-sensitive. In the following example, observe that no rows are returned because the `EMPLOYEES` table stores all the last names in mixed case:

```
SELECT last_name, job_id, department_id
FROM   employees
WHERE  last_name = 'WHALEN' ;
```

Oracle databases store dates in an internal numeric format, representing the century, year, month, day, hours, minutes, and seconds. The default date display is in the `DD-MON-RR` format in Oracle.

Note: For details about the `RR` format and about changing the default date format, see the lesson titled “Using Single-Row Functions to Customize Output.” Also, you learn about the use of single-row functions such as `UPPER` and `LOWER` to override case sensitivity in the same lesson.

Comparison Operators

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to
BETWEEN ...AND...	Between two values (inclusive)
IN(set)	Match any of a list of values
LIKE	Match a character pattern
IS NULL	Is a null value

You can use comparison operators in conditions that compare one expression with another value or expression. They are used in the `WHERE` clause in the following format:

Syntax

```
... WHERE expr operator value
```

Example

```
... WHERE hire_date = '01-JAN-05'
```

```
... WHERE salary >= 6000
```

```
... WHERE last_name = 'Smith'
```

Remember, an alias cannot be used in the `WHERE` clause.

Note: The symbols `!=` and `^=` can also represent the *not equal to* condition.

Using Comparison Operators

Let's look at some examples:

```
SELECT last_name, salary
FROM employees
WHERE salary <= 3000 ;
```



	LAST_NAME	SALARY
1	Matos	2600
2	Vargas	2500

```
SELECT *
FROM employees
WHERE last_name = 'Abel' ;
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	174 Ellen	Abel	EABEL	011.44.1644.429267	11-MAY-12	SA_REP	11000	0.3	149	80

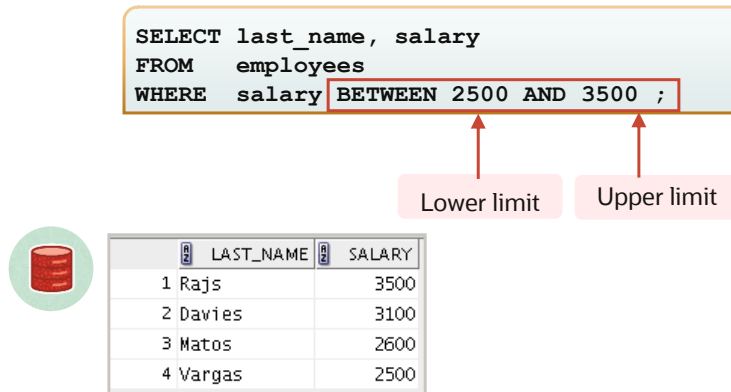


In the first example in the slide, the `SELECT` statement retrieves the last name and salary from the `employees` table for any employee whose salary is less than or equal to \$3,000. Observe that there is an explicit value supplied to the `WHERE` clause. The explicit value of 3000 is compared to the salary value in the `salary` column of the `employees` table.

In the second example, the `SELECT` statement retrieves all rows where the last name is `Abel`. Because `*` is used in the `SELECT` statement, all fields from the `employees` table appear in the result set.

Range Conditions Using the BETWEEN Operator

You can use the BETWEEN operator to display rows based on a range of values:



```
SELECT last_name, salary
FROM   employees
WHERE  salary BETWEEN 2500 AND 3500 ;
```

Lower limit Upper limit

	LAST_NAME	SALARY
1	Rajs	3500
2	Davies	3100
3	Matos	2600
4	Vargas	2500

You can display rows based on a range of values using the BETWEEN operator. The range that you specify contains a lower limit and an upper limit.

The SELECT statement in the slide returns rows from the employees table for any employee whose salary is between \$2,500 and \$3,500.

Values that are specified with the BETWEEN operator are inclusive. Remember, you must specify the lower limit first.

You can also use the BETWEEN operator on character values:

```
SELECT last_name FROM employees
WHERE last_name BETWEEN 'King' AND 'Whalen';
```

Using the IN Operator

Use the IN operator to test for values in a list:

```
SELECT employee_id, last_name, salary, manager_id
FROM   employees
WHERE  manager_id IN (100, 101, 201)
```



	EMPLOYEE_ID	LAST_NAME	SALARY	MANAGER_ID
1	101	Kochhar	17000	100
2	102	De Haan	17000	100
3	124	Mourgos	5800	100
4	149	Zlotkey	10500	100
5	201	Hartstein	13000	100
6	200	Whalen	4400	101
7	205	Higgins	12008	101
8	202	Fay	6000	201

You can use the IN operator to test for values among a specified set of values. The condition defined using the IN operator is also known as the *membership condition*.

The example in the slide displays employee IDs, last names, salaries, and manager's employee IDs for all the employees whose manager's employee ID is 100, 101, or 201.

Note: The set of values can be specified in any random order—for example, (201,100,101).

The IN operator can be used with any data type. The following example returns rows from the employees table, for any employee whose last name is included with the IN operator:

```
SELECT employee_id, manager_id, department_id
FROM   employees
WHERE  last_name IN ('Hartstein', 'Vargas');
```

If characters or dates are used in a list, they must be enclosed within single quotation marks (' ').

Pattern Matching Using the LIKE Operator

- You can use the `LIKE` operator to perform wildcard searches of valid string patterns.
- The search conditions can contain either literal characters or numbers:
 - `%` denotes zero or more characters.
 - `_` denotes one character.

```
SELECT first_name
FROM   employees
WHERE  first_name LIKE 'S%';
```



	FIRST_NAME
1	Shelley
2	Steven

You may not always know the exact value to search for. You can select rows that match a character pattern by using the `LIKE` operator. The character pattern-matching operation is referred to as a *wildcard* search. Two symbols can be used to construct the search string.

Symbol	Description
<code>%</code>	Represents any sequence of zero or more characters
<code>_</code>	Represents any single character

The `SELECT` statement in the slide returns the first name from the `employees` table for any employee whose first name begins with the letter “S.” Note the uppercase “S.” Consequently, names beginning with a lowercase “s” are not returned.

The `LIKE` operator can be used as a shortcut for some `BETWEEN` comparisons. In Oracle, the following example displays the last names and hire dates of all employees who joined between January 2015 and December 2015:

```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date LIKE '%15'
```


Combining Wildcard Symbols

- You can combine the two wildcard symbols (% , _) with literal characters for pattern matching:

```
SELECT last_name  
FROM employees  
WHERE last_name LIKE '_o%';
```



	LAST_NAME
1	Kochhar
2	Lorentz
3	Mourgos

- You can use the ESCAPE identifier to search for the actual % and _ symbols.

The % and _ symbols can be used in any combination with literal characters. The example in the slide displays the names of all employees whose last names have the letter “o” as the second character.

When you need to have an exact match for the actual % and _ characters, use the `ESCAPE` identifier. For example, in Oracle, to find the last name and job ID of all the employees whose job ID contains 'SA_', use the following statement:

```
SELECT last_name, job_id  
FROM employees  
WHERE job_id LIKE 'SA\_%' ESCAPE '\';
```

Using NULL Conditions

You can use the `IS NULL` operator to test for NULL values in a column.

```
SELECT last_name, manager_id
FROM   employees
WHERE  manager_id IS NULL ;
```



LAST_NAME	MANAGER_ID
1 King	(null)

There are two types of NULL conditions:

- `IS NULL` tests for NULL values
- `IS NOT NULL` tests for values that are not NULL

A null value means that the value is unavailable, unassigned, unknown, or inapplicable. Therefore, you cannot test with `=`, because a null cannot be equal or unequal to any value. The example in the slide retrieves the `last_name` and `manager_id` of all employees who do not have a manager.

Here is another example: To display the last name, job ID, and commission for all employees who are *not* entitled to receive a commission, use the following SQL statement:

```
SELECT last_name, job_id, commission_pct
FROM   employees
WHERE  commission_pct IS NULL;
```

Defining Conditions Using Logical Operators

You can use the logical operators to filter the result set based on more than one condition or invert the result set.

Operator	Meaning
AND	Returns TRUE if both component conditions are true
OR	Returns TRUE if either component condition is true
NOT	Returns TRUE if the condition is false

A logical condition combines the results of two or more component conditions to produce a single result based on those conditions, or it inverts the result of a single condition. A row is returned only if the overall result of the condition is true.

Three logical operators are available in SQL:

- AND
- OR
- NOT

All the examples so far have specified only one condition in the `WHERE` clause. You can use several conditions in a single `WHERE` clause using the `AND` and `OR` operators.

Using the AND Operator

AND requires both the component conditions to be true:

```
SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary >= 10000
AND job_id LIKE '%MAN%' ;
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
1	149	Zlotkey	SA_MAN	10500
2	201	Hartstein	MK_MAN	13000

In the example, both the component conditions must be true for any record to be selected. Therefore, only those employees who have a job title that contains the string 'MAN' *and* earn \$10,000 or more are selected.

All character searches are case-sensitive, that is, no rows are returned if 'MAN' is not uppercase. Further, character strings must be enclosed within quotation marks.

AND Truth Table

The following table shows the results of combining two expressions with AND:

AND	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

Using the OR Operator

OR requires either component condition to be true:

```
SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary >= 10000
OR job_id LIKE '%MAN%' ;
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
1	100	King	AD_PRES	24000
2	101	Kochhar	AD_VP	17000
3	102	De Haan	AD_VP	17000
4	124	Mourgos	ST_MAN	5800
5	149	Zlotkey	SA_MAN	10500
6	174	Abel	SA_REP	11000
7	201	Hartstein	MK_MAN	13000
8	205	Higgins	AC_MGR	12008

In the example, either component condition can be true for any record to be selected. Therefore, any employee who has a job ID that contains the string 'MAN' or earns \$10,000 or both is selected.

OR Truth Table

The following table shows the results of combining two expressions with OR:

OR	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

Using the NOT Operator

NOT is used to negate a condition:

```
SELECT last_name, job_id
FROM employees
WHERE job_id
      NOT IN ('IT_PROG', 'ST_CLERK',
'SA_REP');
```



	LAST_NAME	JOB_ID
1	De Haan	AD_VP
2	Fay	MK_REP
3	Gietz	AC_ACCOUNT
4	Hartstein	MK_MAN
5	Higgins	AC_MGR
6	King	AD_PRES
7	Kochhar	AD_VP
8	Mourgos	ST_MAN
9	Whalen	AD_ASST
10	Zlotkey	SA_MAN

The example in the slide displays the last name and job ID of all employees whose job ID *is not* IT_PROG, ST_CLERK, or SA_REP.

NOT Truth Table

The following table shows the result of applying the NOT operator to a condition:

NOT	TRUE	FALSE	NULL
	FALSE	TRUE	TRUE/FALSE



Rules of Precedence for Operators in an Expression

For Instructor Use Only.
This document should not be distributed

Rules of Precedence



Order	Operator
1	Arithmetic operators
2	Concatenation operator
3	Comparison conditions
4	IS [NOT] NULL, LIKE, [NOT] IN
5	[NOT] BETWEEN
6	Not equal to
7	NOT logical operator
8	AND logical operator
9	OR logical operator

You can use parentheses to override rules of precedence.

The rules of precedence determine the order in which expressions are evaluated and calculated. The table in the slide lists the default order of precedence. However, you can override the default order by using parentheses around the expressions that you want to calculate first.

Rules of Precedence

1

```
SELECT last_name, department_id, salary
FROM employees
WHERE department_id = 60
OR department_id = 80
AND salary > 10000;
```



	LAST_NAME	DEPARTMENT_ID	SALARY
1	Hunold	60	9000
2	Ernst	60	6000
3	Lorentz	60	4200
4	Zlotkey	80	10500
5	Abel	80	11000

2

```
SELECT last_name, department_id, salary
FROM employees
WHERE (department_id = 60
OR department_id = 80)
AND salary > 10000;
```



	LAST_NAME	DEPARTMENT_ID	SALARY
1	Zlotkey	80	10500
2	Abel	80	11000

1. Precedence of the AND Operator: Example

- In this example, there are two conditions:
 - o The first condition is that the department ID is 80 and the salary is greater than \$10,000.
 - o The second condition is that the department ID is 60.
- Therefore, the SELECT statement reads as follows:
 - o "Select the row if an employee's department ID is 80 and earns more than \$10,000, or if the employee's department ID is 60."

2. Using Parentheses: Example

- In this example, there are two conditions:
 - o The first condition is that the department ID is 80 or 60.
 - o The second condition is that the salary is greater than \$10,000.
- Therefore, the SELECT statement reads as follows:
 - o "Select the row if an employee's department ID is 80 or 60, and if the employee earns more than \$10,000."



Sorting Rows Using the ORDER BY Clause

For Instructor Use Only.
This document should not be distributed

Using the ORDER BY Clause

You can sort the retrieved rows with the ORDER BY clause:

- ASC: Ascending order, default
- DESC: Descending order

```
SELECT last_name, job_id, department_id, hire_date
FROM employees
ORDER BY hire_date ;
```



	LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
1	De Haan	AD_VP	90	13-JAN-09
2	Kochhar	AD_VP		21-SEP-09
3	Higgins	AC_MGR	110	07-JUN-10
4	Gietz	AC_ACCOUNT	110	07-JUN-10
5	King	AD_PRES		17-JUN-11
6	Whalen	AD_ASST	10	17-SEP-11
7	Rajs	ST_CLERK	50	17-OCT-11

...

The order of rows that are returned in a query result is undefined. The ORDER BY clause can be used to sort the rows. You can specify an expression, an alias, or a column position as the sort condition. You can specify multiple expressions in the ORDER BY clause. Rows are first sorted based on their values for the first expression. Rows with the same value for the first expression are then sorted based on their values for the second expression, and so on.

Syntax

```
SELECT          expr
FROM            table
[WHERE          condition(s)]
[ORDER BY       {column, expr, numeric_position} [ASC|DESC]];
```

In the syntax:

ORDER BY	Specifies the order in which the retrieved rows are displayed
ASC	Orders the rows in ascending order (this is the default order)
DESC	Orders the rows in descending order

If the ORDER BY clause is not used, the sort order is undefined, and the Oracle server may not fetch rows in the same order for the same query twice. Use the ORDER BY clause to display the rows in a specific order.

Sorting

- Sorting in descending order:

```
SELECT last_name, job_id, department_id, hire_date
FROM employees
ORDER BY department_id DESC ;
```

- Sorting by column alias:

```
SELECT employee_id, last_name, salary*12 annsal
FROM employees
ORDER BY annsal ;
```

The default sort order is ascending. Let's look at the other characteristics:

- Numeric values are displayed with the lowest values first (for example, 1 to 999).
- Date values are displayed with the earliest value first (for example, 01-JAN-92 before 01-JAN-95 in Oracle).
- Character values are displayed in the alphabetical order (for example, "A" first and "Z" last).
- In Oracle, null values are displayed last for ascending sequences and first for descending sequences.
- You can also sort by a column that is not in the `SELECT` list.

Examples

1. To reverse the order in which the rows are displayed, specify the `DESC` keyword after the column name in the `ORDER BY` clause. The example in the slide sorts the result by the `department_id`.
2. You can also use a column alias in the `ORDER BY` clause. The example in the slide sorts the data by annual salary.

Note: In Oracle, use the keywords `NULLS FIRST` or `NULLS LAST` to specify whether returned rows containing null values should appear first or last in the ordering sequence.

Sorting

- Sorting by using the column's numeric position:

```
SELECT last_name, job_id, department_id, hire_date
FROM employees
ORDER BY 3;
```

3

- Sorting by multiple columns:

```
SELECT last_name, department_id, salary
FROM employees
ORDER BY department_id, salary DESC;
```

4

Examples

3. You can sort query results by specifying the numeric position of the column in the `SELECT` clause. The example in the slide sorts the result by the `department_id` as this column is at the third position in the `SELECT` clause.
4. You can sort query results by more than one column. You list the columns (or `SELECT` list column sequence numbers) in the `ORDER BY` clause, delimited by commas. The results are ordered by the first column, then the second, and so on for as many columns as the `ORDER BY` clause includes. If you want any results sorted in descending order, your `ORDER BY` clause must use the `DESC` keyword directly after the name or the number of the relevant column. The result of the query example shown in the slide is sorted by `department_id` in ascending order and also by salary in descending order.



SQL Row Limiting Clause in a Query

For Instructor Use Only.
This document should not be distributed

SQL Row Limiting Clause

- You can use the row limiting clause to limit the rows that are returned by a query.
- You can use this clause to implement Top-N reporting.



Limiting the number of rows returned by a query can be valuable for reporting, analysis, data browsing, and other tasks.

With the SQL `SELECT` syntax, you can limit the number of rows that are returned in the result set using the row limiting clause.

Queries that order data and then limit row output are widely used and are often referred to as Top-N queries. Top-N queries sort their result set and then return only the first n rows.

The SQL row limiting clause has certain limitations. These limitations are explained in the lesson titled “Managing Tables Using DML Statements and Transactions.”

Using SQL Row Limiting Clause in a Query in Oracle



You specify `row_limiting_clause` in the SQL `SELECT` statement by placing it after the `ORDER BY` clause.

Syntax:

```
SELECT ...  
  FROM ...  
  [ WHERE ... ]  
  [ ORDER BY ... ]  
  [OFFSET offset { ROW | ROWS }]  
  [FETCH { FIRST | NEXT } [{ row_count | percent PERCENT }] { ROW  
  | ROWS }  
  { ONLY | WITH TIES }]
```

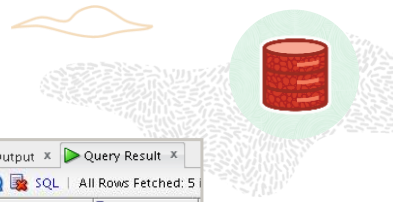
You specify `row_limiting_clause` in the SQL `SELECT` statement by placing it after the `ORDER BY` clause. Note that an `ORDER BY` clause is required if you want to sort the rows for consistency.

- **OFFSET:** Use this clause to specify the number of rows to skip before row limiting begins. The value for offset must be a number. If you specify a negative number, offset is treated as 0. If you specify `NULL` or a number greater than or equal to the number of rows that are returned by the query, 0 rows are returned.
- **ROW | ROWS:** Use these keywords interchangeably. They are provided for semantic clarity.
- **FETCH:** Use this clause to specify the number of rows or percentage of rows to return.
- **FIRST | NEXT:** Use these keywords interchangeably. They are provided for semantic clarity.
- **row_count | *percent* PERCENT:** Use `row_count` to specify the number of rows to return. Use *percent* `PERCENT` to specify the percentage of the total number of selected rows to return. The value for percent must be a number.
- **ONLY | WITH TIES:** Specify `ONLY` to return exactly the specified number of rows or percentage of rows. Specify `WITH TIES` to return additional rows with the same sort key as the last row fetched. You must specify the `ORDER BY` clause for additional rows to be returned.

For Instructor Use Only.
This document should not be distributed

SQL Row Limiting Clause: Example in Oracle

```
SELECT employee_id, first_name
FROM employees
ORDER BY employee_id
FETCH FIRST 5 ROWS ONLY;
```



EMPLOYEE_ID	FIRST_NAME
1	100 Steven
2	101 Neena
3	102 Lex
4	103 Alexander
5	104 Bruce

```
SELECT employee_id, first_name
FROM employees
ORDER BY employee_id
OFFSET 5 ROWS FETCH NEXT 5 ROWS ONLY;
```



EMPLOYEE_ID	FIRST_NAME
1	107 Diana
2	124 Kevin
3	141 Trena
4	142 Curtis
5	143 Randall

The first example in the slide returns the first five employees after sorting the rows in ascending order of the `employee_id`.

The second example in the slide returns the next set of five employees after sorting the rows in ascending order of the `employee_id`.

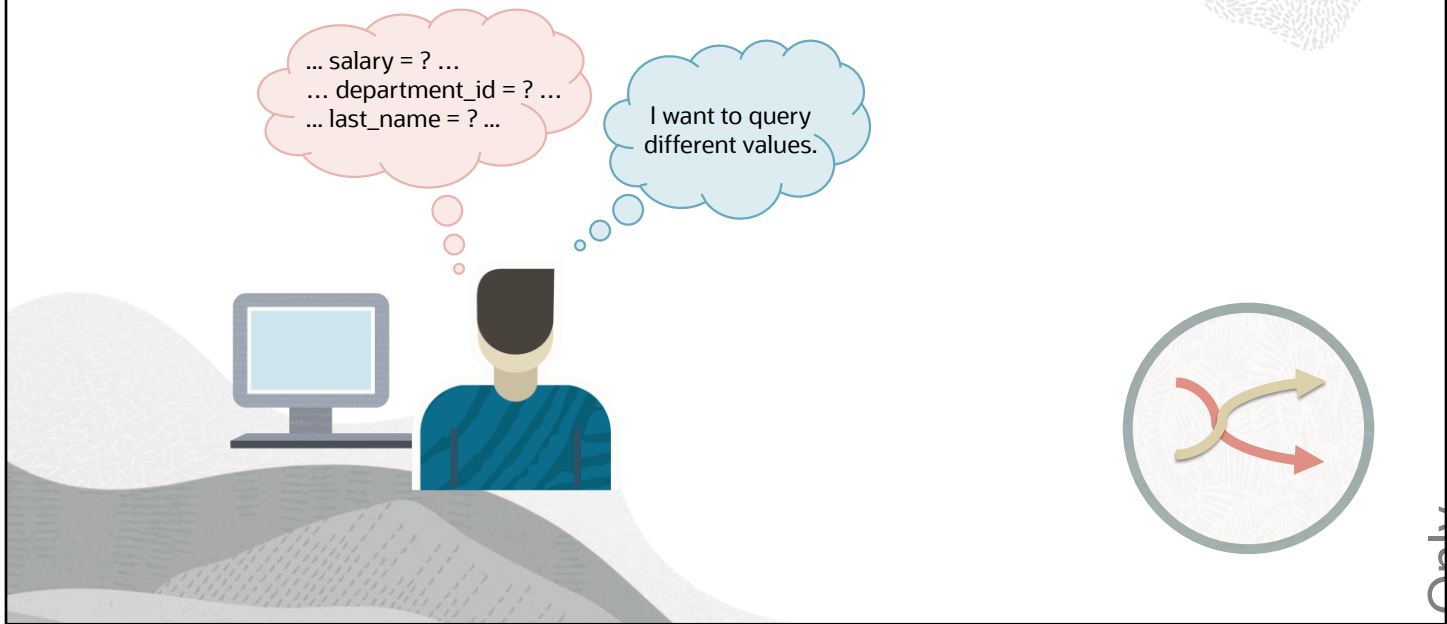
Note: If `employee_id` is assigned sequentially by the date when the employee joined the organization, these examples give us the top 5 employees and then employees 6-10, all in terms of seniority.



Substitution Variables in Oracle

For Instructor Use Only.
This document should not be distributed

Substitution Variables in Oracle



So far, all the SQL statements were executed with predetermined columns, conditions, and their values. Suppose that you want a query that lists the employees with various jobs and not just those whose `job_ID` is `SA_REP`. You can edit the `WHERE` clause to provide a different value each time you run the command, but there is also an easier way in Oracle.

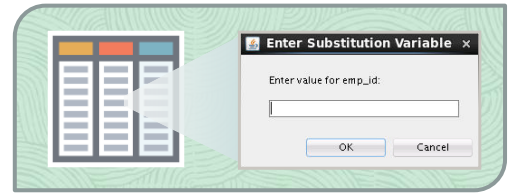
By using a **substitution variable** in place of the exact values in the `WHERE` clause, you can run the same query for different values.

You can create reports that prompt users to supply their own values to restrict the range of data returned, by using substitution variables. You can embed *substitution variables* in a command file or in a single SQL statement. A variable can be thought of as a container in which values are temporarily stored. When the statement is run, the stored value is substituted.

Substitution Variables in Oracle



- Use substitution variables to:
 - Temporarily store values with single-ampersand (&) and double-ampersand (&&) substitution
- Use substitution variables to supplement:
 - WHERE conditions
 - ORDER BY clauses
 - Column expressions
 - Table names
 - Entire SELECT statements



You can use single-ampersand (&) substitution variables to temporarily store values.

You can also predefine variables by using the `DEFINE` command. `DEFINE` creates and assigns a value to a variable.

Restricted Ranges of Data: Examples

- Reporting figures only for the current quarter or specified date range
- Reporting on data relevant only to the user requesting the report
- Displaying personnel only within a given department

Other Interactive Effects

Interactive effects are not restricted to direct user interaction with the `WHERE` clause. You can also use it for:

- Obtaining input values from a file rather than from a person
- Passing values from one SQL statement to another

Note: Both Oracle SQL Developer and SQL*Plus support substitution variables and the `DEFINE`/`UNDEFINE` commands.

Using the Single-Ampersand Substitution Variable



Use a variable prefixed with an ampersand (&) to prompt the user for a value:

```
SELECT employee_id, last_name, salary, department_id
FROM   employees
WHERE  employee_id = &employee_num ;
```

Enter Substitution Variable X

Enter value for employee_num:

OK Cancel



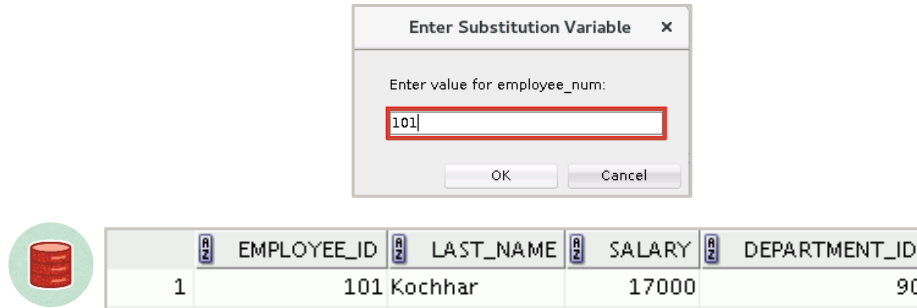
When running a report, users often want to restrict the data that is returned dynamically. SQL*Plus and SQL Developer provide this flexibility with user variables. Use an ampersand (&) to identify each variable in your SQL statement. However, you do not need to define the value of each variable

Notation	Description
<i>&user_variable</i>	Indicates a variable in a SQL statement; if the variable does not exist, SQL*Plus or SQL Developer prompts the user for a value (the new variable is discarded after it is used)

The example in the slide creates a SQL Developer substitution variable for an employee number. When the statement is executed, SQL Developer prompts the user for an employee number and then displays the employee number, last name, salary, and department number for that employee.

With the single ampersand, the user is prompted every time the command is executed if the variable does not exist.

Using the Single-Ampersand Substitution Variable



The screenshot shows a dialog box titled "Enter Substitution Variable" with a close button (X). Inside, it says "Enter value for employee_num:" and has a text input field containing "101". Below the input field are "OK" and "Cancel" buttons. To the left of the dialog is a small database icon. Below the dialog is a table with the following data:

	EMPLOYEE_ID	LAST_NAME	SALARY	DEPARTMENT_ID
1	101	Kochhar	17000	90

When SQL Developer detects that the SQL statement contains an ampersand, you are prompted to enter a value for the substitution variable that is named in the SQL statement.

After you enter a value and click the OK button, the results are displayed.

Note that if the substitution variable is referenced twice, even in the same command, then you are prompted twice. Different values can be entered at each prompt.

Character and Date Values with Substitution Variables



Use single quotation marks for date and character values:

```
SELECT last_name, department_id, salary*12
FROM   employees
WHERE  job_id = '&job_title' ;
```

Enter Substitution Variable X

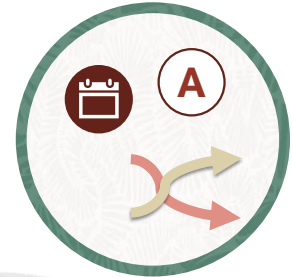
Enter value for job_title:

IT_PROG

OK Cancel



	LAST_NAME	DEPARTMENT_ID	SALARY*12
1	Hunold	60	108000
2	Ernst	60	72000
3	Lorentz	60	50400



In a `WHERE` clause, you must enclose date and character values within single quotation marks. The same rule applies to the substitution variables.

Enclose the variable with single quotation marks within the SQL statement itself.

The slide shows a query to retrieve the employee names, department numbers, and annual salaries of all employees based on the job title value of the SQL Developer substitution variable.

Specifying Column Names, Expressions, and Text

```
SELECT employee_id, last_name, job_id, &column_name
FROM employees
WHERE &condition
ORDER BY &order_column ;
```

Enter Substitution Variable x

Enter value for column_name:

OK Cancel

Enter Substitution Variable x

Enter value for condition:

OK Cancel

Enter Substitution Variable x

Enter value for order_column:

OK Cancel

You can use the substitution variables not only in the `WHERE` clause of a SQL statement but also as substitution for column names, expressions, or text.

Example

The example in the slide displays the employee number, last name, job title, and any other column that is specified by the user at run time, from the `EMPLOYEES` table. For each substitution variable in the `SELECT` statement, you are prompted to enter a value and then click OK to proceed.

If you do not enter a value for the substitution variable, you get an error when you execute the preceding statement.

Note: A substitution variable can be used anywhere in the `SELECT` statement.

Using the Double-Ampersand Substitution Variable



Use double ampersand (&&) if you want to reuse the variable value without prompting the user each time:

```
SELECT  employee_id, last_name, job_id, &&column_name
FROM    employees
ORDER BY &column_name ;
```

Enter Substitution Variable x

Enter value for column_name:

department_id

OK Cancel



	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	200	Whalen	AD_ASST	10
2	201	Hartstein	MK_MAN	20
3	202	Fay	MK_REP	20

...



You can use the double-ampersand (&&) substitution variable if you want to reuse the variable value without prompting the user each time. The user sees the prompt for the value only once.

In the example in the slide, the user is asked to give the value for the variable `column_name` only once. The value that is supplied by the user (`department_id`) is used for both display and ordering of data. If you run the query again, you will not be prompted for the value of the variable.

SQL Developer stores the value that is supplied by using the `DEFINE` command; it uses it again whenever you reference the variable name. After a user variable is in place, you need to use the `UNDEFINE` command to delete it:

```
UNDEFINE column_name;
```

Using the Ampersand Substitution Variable in SQL*Plus



```
oracle@ ~  
File Edit View Search Terminal Help  
SQL> SELECT employee_id, last_name, salary, department_id  
2 from employees  
3 where employee_id = &employee_num;  
Enter value for employee_num: 101
```

```
oracle@ ~  
File Edit View Search Terminal Help  
SQL> SELECT employee_id, last_name, salary, department_id  
2 from employees  
3 where employee_id = &employee_num;  
Enter value for employee_num: 101  
old 3: where employee_id = &employee_num  
new 3: where employee_id = 101  
  
EMPLOYEE_ID LAST_NAME          SALARY DEPARTMENT_ID  
-----  
101 Kochhar          17000          90  
  
SQL>
```

The example in the slide creates a SQL*Plus substitution variable for an employee number. When the statement is executed, SQL*Plus prompts you for an employee number and then displays the employee number, last name, salary, and department number for that employee.



Assigning Values to Variables

For Instructor Use Only.
This document should not be distributed

Using the DEFINE Command in Oracle



- Use the `DEFINE` command to create a variable and assign a value to it.
- Use the `UNDEFINE` command to remove a variable.

```
DEFINE employee_num = 200  
  
SELECT employee_id, last_name, salary, department_id  
FROM   employees  
WHERE  employee_id = &employee_num ;  
  
UNDEFINE employee_num
```



EMPLOYEE_ID	LAST_NAME	SALARY	DEPARTMENT_ID
1	200 Whalen	4400	10

The example shown in the slide creates a substitution variable for an employee number by using the `DEFINE` command. At run time, this displays the employee number, name, salary, and department number for that employee.

Because the variable is created using the SQL Developer `DEFINE` command, the user is not prompted to enter a value for the employee number. Instead, the variable value is automatically substituted in the `SELECT` statement.

The `EMPLOYEE_NUM` substitution variable is present in the session until the user undefines it or exits the SQL Developer session.

Using the VERIFY Command in Oracle



Use the `VERIFY` command to toggle the display of the substitution variable, both before and after SQL Developer replaces substitution variables with values:

```
SET VERIFY ON
SELECT employee_id, last_name, salary
FROM   employees
WHERE  employee_id = &employee_num;
```



Enter Substitution Variable

Enter value for employee_num:

200

OK Cancel

Script Output x | Task completed in 6.496 seconds

old:	SELECT employee_id, last_name, salary	
	FROM employees	
	WHERE employee_id = &employee_num	
new:	SELECT employee_id, last_name, salary	
	FROM employees	
	WHERE employee_id = 200	
	EMPLOYEE_ID LAST_NAME	SALARY

	200 Whalen	4400

You can use the `VERIFY` command to confirm the changes in the SQL statement. Setting `SET VERIFY ON` forces SQL Developer to display the text of a command after it replaces substitution variables with values.

To see the `VERIFY` output, you should use the Run Script (F5) icon in the SQL Worksheet. SQL Developer displays the text of a command after it replaces substitution variables with values, on the Script Output tab as shown in the slide.

The example in the slide displays the new value of the `EMPLOYEE_ID` column in the SQL statement followed by the output.

SQL*Plus System Variables

SQL*Plus uses various system variables that control the working environment. One of the variables is `VERIFY`. To obtain a complete list of all the system variables, you can issue the `SHOW ALL` command on the SQL*Plus command prompt.

Summary



Limiting rows

Rules of precedence for operators in an expression

Sorting rows using the `ORDER BY` clause

SQL row limiting clause in a query

Substitution variables in Oracle

Assigning values to variables

In this lesson, you should have learned about restricting and sorting rows that are returned by the `SELECT` statement. You should also have learned how to implement various operators and conditions.

For Instructor Use Only.
This document should not be distributed

For Instructor Use Only.
This document should not be distributed

Using Single-Row Functions to Customize Output



Course Roadmap

Lesson 1: Introduction

**Unit 1: Retrieving, Restricting,
and Sorting Data**

Unit 2: Joins, Subqueries, and Set
Operators

Unit 3: DML and DDL



Lesson 2: Retrieving Data Using the SQL `SELECT`
Statement



Lesson 3: Restricting and Sorting Data



**Lesson 4: Using Single-Row Functions to
Customize Output**

You are here!



Lesson 5: Using Conversion Functions and
Conditional Expressions

In Unit 1, you will learn how to query the data from tables, how to query selected records from tables, and also how to sort the data retrieved from the tables.

Objectives



Single-row SQL functions

Character functions

Nesting functions

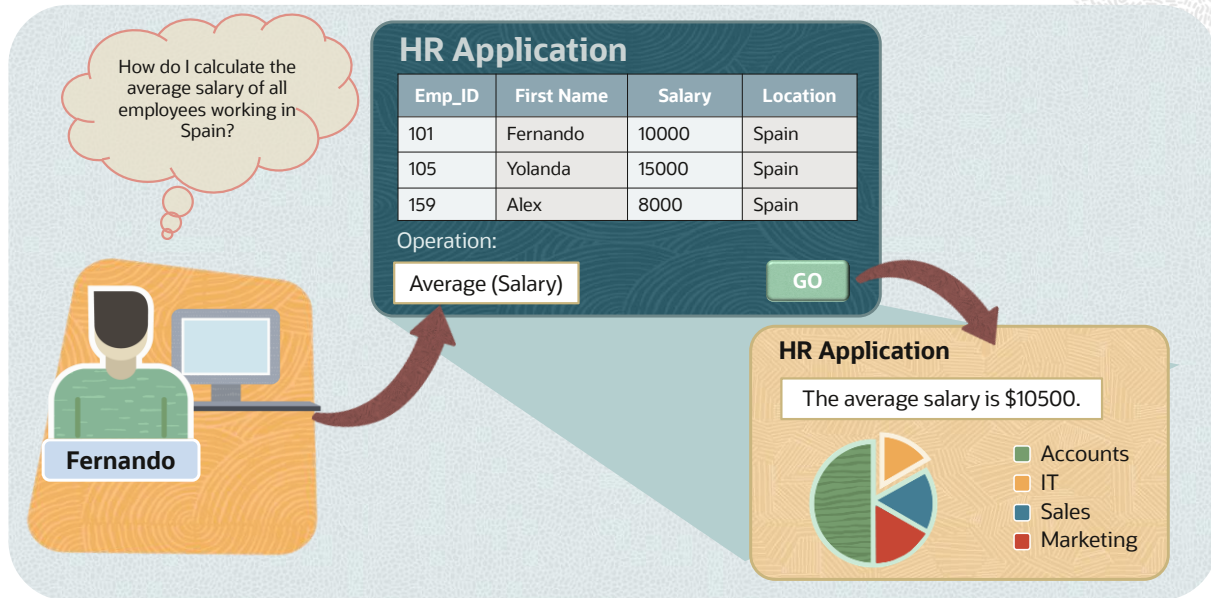
Number functions

Working with dates in Oracle Databases

Date functions

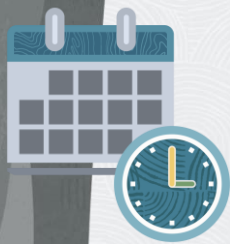
Functions make the basic query block more powerful, and they are used to manipulate data values. This is the first of two lessons that explore functions. It focuses on single-row character, number, and date functions.

HR Application Scenario



Consider a scenario where Fernando, an HR manager in Spain, wants to calculate the average salary across various departments of all employees working in Spain. In order to generate such information, Fernando has to enter conditions such as country name (Spain) and get the list of employees working in Spain. Then he has to enter the operation to be performed on the values (in this case, `AVERAGE` salary). The HR application queries the database conditionally and then applies the mathematical operation to calculate the average salary. The results are returned to Fernando, along with a chart for analysis.

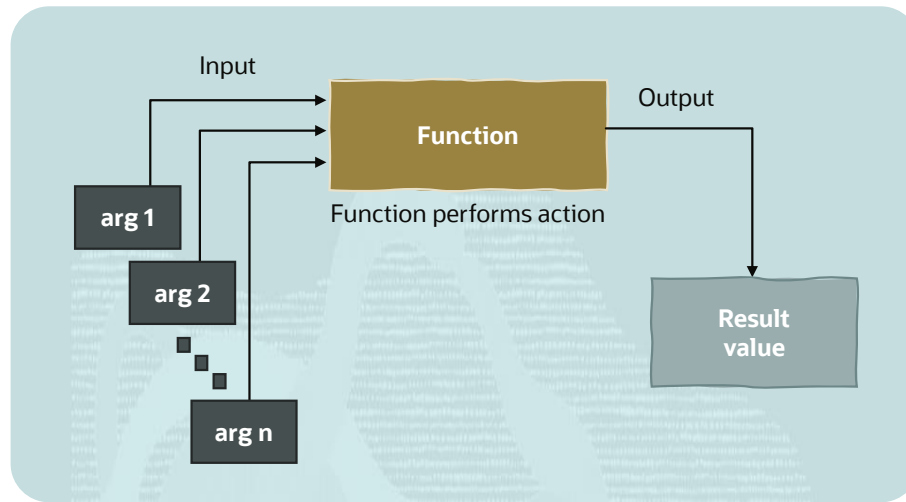
The operations performed on the values returned by a SQL query are called functions. There are different types of functions, which are useful when you want to apply some kind of customization on the values returned by the query. In the following lessons, you will learn about the different types of functions.



Single-Row SQL Functions

For Instructor Use Only.
This document should not be distributed

SQL Functions

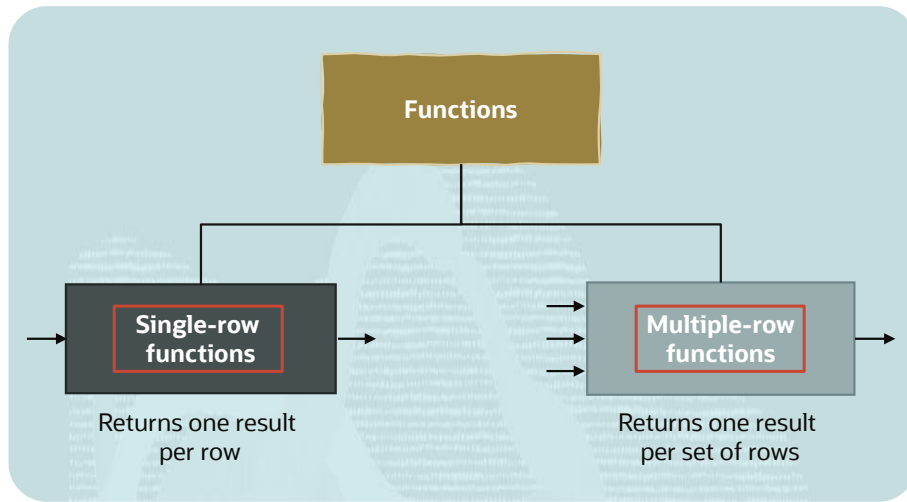


Functions are a very powerful feature of SQL. You can use functions to:

- Perform calculations on data
- Modify individual data items
- Manipulate output for groups of rows
- Format dates and numbers for display
- Convert column data types

SQL functions sometimes take arguments and always return a value.

Two Types of SQL Functions



There are two types of functions:

- Single-row functions
- Multiple-row functions

Single-Row Functions

These functions operate on single rows only and return one result per row. There are different types of single-row functions. This lesson covers the following functions:

- Character
- Number
- Date

Multiple-Row Functions

Functions can manipulate groups of rows to give one result per group of rows. These functions are also known as *group functions* (covered in the lesson titled “Reporting Aggregated Data Using the Group Functions”).

Note: There are many other functions available in Oracle databases. Not all can be covered in this course. This course explains how functions work and provides examples of some of the most commonly used function. Check the database documentation for other functions you might need for your application.

Single-Row Functions

Single-row functions:

- Manipulate data items
- Accept arguments and return one value
- Act on each row that is returned
- Return one result per row
- Might modify the data type
- Can be nested
- Accept arguments that can be a column or an expression

```
function_name[(arg1, arg2,...)]
```



You can use single-row functions to manipulate data items. They accept one or more arguments and return one value for each row that is returned by the query. An argument can be one of the following:

- User-supplied constant
- Variable value
- Column name
- Expression

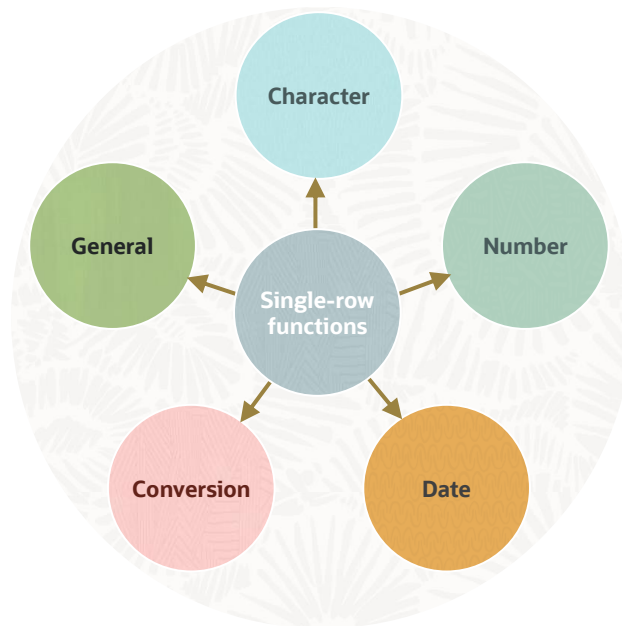
Features of single-row functions include the following:

- Act on each row that is returned in the query
- Return one result per row
- Possibly return a data value of a different type than the one that is referenced
- Possibly expect one or more arguments
- Can be used in `SELECT`, `WHERE`, and `ORDER BY` clauses; can be nested

In the syntax:

<i>function_name</i>	Is the name of the function
<i>arg1, arg2</i>	Is any argument to be used by the function. This can be represented by a column name or expression.

Single-Row Functions



In this lesson, you will learn about the following single-row functions:

- **Character functions:** Accept character input and can return both character and number values.
- **Number functions:** Accept numeric input and return numeric values.
- **Date functions:** Operate on values of the `DATE` data type.

You will learn about the following single-row functions in the lesson titled “Using Conversion Functions and Conditional Expressions”:

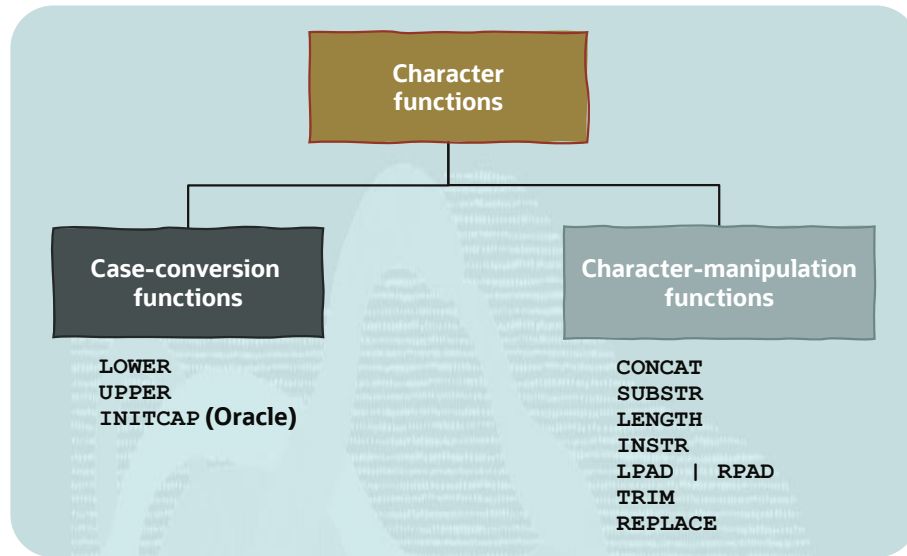
- **Conversion functions:** Convert a value from one data type to another.
- **General functions:** These functions take any data type and can also handle NULLs.
- **JSON functions:** These functions allow you to query and generate JSON data.



Character Functions

For Instructor Use Only.
This document should not be distributed

Character Functions



Single-row character functions accept character data as input and can return both character and numeric values. Character functions can be divided into the following:

- Case-conversion functions
- Character-manipulation functions

Function	Purpose
<code>LOWER(column expression)</code>	Converts alpha character values to lowercase
<code>UPPER(column expression)</code>	Converts alpha character values to uppercase
<code>INITCAP(column expression)</code>	Converts alpha character values to uppercase for the first letter of each word; all other letters in lowercase. This function is only available in Oracle SQL.
<code>CONCAT(column1 expression1, column2 expression2)</code>	Concatenates the first character value to the second character value; equivalent to concatenation operator ()
<code>SUBSTR(column expression,m[,n])</code>	Returns specified characters from character value starting at character position <i>m</i> , <i>n</i> characters long (If <i>m</i> is negative, the count starts from the end of the character value. If <i>n</i> is omitted, all characters to the end of the string are returned.)

Note: The functions discussed in this lesson are only some of the available functions.

Function	Purpose
<code>LENGTH(column expression)</code>	Returns the number of characters in the expression
<code>INSTR(column expression, 'string', [,m], [n])</code>	Returns the numeric position of a named string. Optionally, you can provide a position <i>m</i> to start searching, and the occurrence <i>n</i> of the string. <i>m</i> and <i>n</i> default to 1, meaning start the search at the beginning of the string and report the first occurrence.
<code>LPAD(column expression, n, 'string')</code>	Returns an expression left-padded to length of <i>n</i> characters with a character expression
<code>RPAD(column expression, n, 'string')</code>	Returns an expression right-padded to length of <i>n</i> characters with a character expression
<code>TRIM(leading trailing both, trim_character FROM trim_source)</code>	Enables you to trim leading or trailing characters (or both) from a character string. If <i>trim_character</i> or <i>trim_source</i> is a character literal, you must enclose it in single quotation marks.
<code>REPLACE(text, search_string, replacement_string)</code>	Searches a text expression for a character string and, if found, replaces it with a specified replacement string

Case-Conversion Functions

- You can use the `LOWER` and `UPPER` functions to convert the case of character strings.
- For example:

```
SELECT last_name, UPPER(last_name), job_id, LOWER(job_id)
FROM   employees
WHERE  department_id = 90;
```



	LAST_NAME	UPPER(LAST_NAME)	JOB_ID	LOWER(JOB_ID)
1	King	KING	AD_PRES	ad_pres
2	Kochhar	KOCHHAR	AD_VP	ad_vp
3	De Haan	DE HAAN	AD_VP	ad_vp

`LOWER`, `UPPER`, and `INITCAP` are the three case-conversion functions.

- `LOWER`: Converts mixed-case or uppercase character strings to lowercase
- `UPPER`: Converts mixed-case or lowercase character strings to uppercase
- `INITCAP`: (Oracle) Converts the first letter of each word to uppercase and the remaining letters to lowercase

The slide shows an example of using `LOWER` and `UPPER` functions.

Using Case-Conversion Functions in WHERE Clauses in Oracle

- Display the employee number, name, and department number for employee Higgins:



```
SELECT employee_id, last_name, department_id
FROM employees
WHERE last_name = 'higgins';
```

0 rows selected

```
SELECT employee_id, last_name, department_id
FROM employees
WHERE LOWER(last_name) = 'higgins';
```



EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
1	205 Higgins	110

The example in the slide displays the employee number, name, and department number of employee Higgins.

The WHERE clause of the first SQL statement specifies the employee name as `higgins`. Because all the data in the `EMPLOYEES` table is stored in proper case, the name `higgins` does not find a match in the table, and no rows are selected.

The WHERE clause of the second SQL statement converts the `LAST_NAME` column to lowercase for comparison purposes. Because both names are now lowercase, a match is found and one row is selected. The WHERE clause can be rewritten in the following manner to produce the same result:

```
...WHERE last_name = 'Higgins'
```

The name in the output appears as it was stored in the database. To display the name in uppercase, use the `UPPER` function in the `SELECT` statement.

```
SELECT employee_id, UPPER(last_name), department_id
FROM employees
WHERE INITCAP(last_name) = 'Higgins';
```

Note: You can use functions such as `UPPER` and `LOWER` with ampersand substitution. For example, use `UPPER('&job_title')` so that the user does not have to enter the job title in a specific case.

Character-Manipulation Functions

You can use these functions to manipulate character strings:

Function	Result
CONCAT('Hello', 'World')	HelloWorld
SUBSTR('HelloWorld',1,5)	Hello
LENGTH('HelloWorld')	10
INSTR('HelloWorld', 'W')	6
LPAD(24000,10,'*')	*****24000
RPAD(24000, 10, '*')	24000*****

CONCAT, SUBSTR, LENGTH, INSTR, LPAD, and RPAD are the character-manipulation functions that you will learn in this lesson.

- **CONCAT:** Joins values together (you are limited to using two parameters with CONCAT)
- **SUBSTR:** Extracts a string of determined length
- **LENGTH:** Shows the length of a string as a numeric value
- **INSTR:** Finds the numeric position of a named character
- **LPAD:** Returns an expression left-padded to the length of *n* characters with a character expression
- **RPAD:** Returns an expression right-padded to the length of *n* characters with a character expression

Using Character-Manipulation Functions

1

```
SELECT last_name, CONCAT('Job category is ', job_id)
AS Job FROM employees
WHERE SUBSTR(job_id, 4) = 'REP';
```



LAST_NAME	JOB
1 Abel	Job category is SA_REP
2 Fay	Job category is MK_REP
3 Grant	Job category is SA_REP
4 Taylor	Job category is SA_REP

2

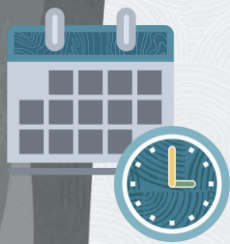
```
SELECT employee_id, CONCAT(first_name, last_name) NAME,
LENGTH(last_name), INSTR(last_name, 'a') "Contains 'a'?"
FROM employees
WHERE SUBSTR(last_name, -1, 1) = 'n';
```



EMPLOYEE_ID	NAME	LENGTH(LAST_NAME)	Contains 'a'?
1	102 LexDe Haan	7	5
2	200 JenniferWhalen	6	3
3	201 MichaelHartstein	9	2

The first example in the slide displays employee last names and job IDs for all employees who have the string, REP, contained in the job ID, starting at the fourth position of the job ID.

The second SQL statement in the slide displays data such as employee ID, concatenated first name and last name, length of the last name, and the position of the first occurrence of the letter 'a' in the last name for those employees whose last names end with the letter "n."

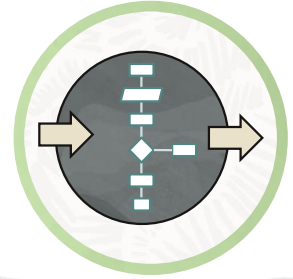
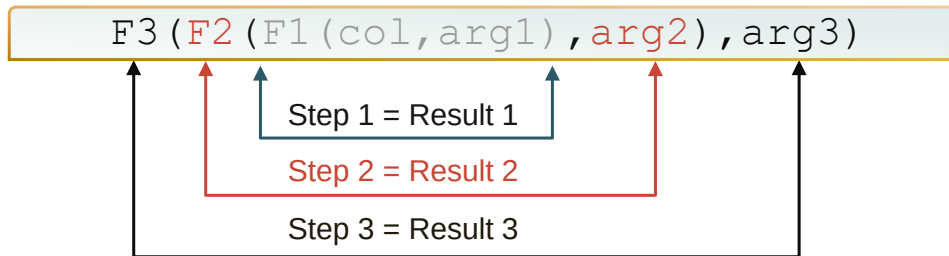


Nesting Functions

For Instructor Use Only.
This document should not be distributed

Nesting Functions

- Single-row functions can be nested to any level.
- Nested functions are evaluated from the deepest level to the least deep level.



Single-row functions can be nested to any depth. Nested functions are evaluated from the innermost level to the outermost level. Some examples follow to show you the flexibility of these functions.

Nesting Functions: Example

```
SELECT last_name,  
       UPPER(CONCAT(SUBSTR(LAST_NAME, 1, 8), ' US'))  
FROM   employees  
WHERE  department_id = 60;
```



	LAST_NAME	UPPER(CONCAT(SUBSTR(LAST_NAME,1,8),'_US'))
1	Hunold	HUNOLD_US
2	Ernst	ERNST_US
3	Lorentz	LORENTZ_US

The example in the slide displays the last names of employees in department 60. The evaluation of the SQL statement involves three steps:

1. The inner function retrieves the first eight characters of the last name.

Result1 = SUBSTR(LAST_NAME, 1, 8)

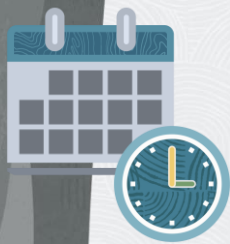
2. The outer function concatenates the result with _US.

Result2 = CONCAT(Result1, '_US')

3. The outermost function converts the results to uppercase.

Result3 = UPPER(Result2)

Result3 is displayed. The entire expression becomes the column heading because no column alias was given.



Number Functions

For Instructor Use Only.
This document should not be distributed

Numeric Functions

- **ROUND:** Rounds value to a specified decimal
- **TRUNC (Oracle):** Truncates value to a specified decimal
- **CEIL:** Returns the smallest whole number greater than or equal to a specified number
- **FLOOR:** Returns the largest whole number equal to or less than a specified number
- **MOD:** Returns remainder of division

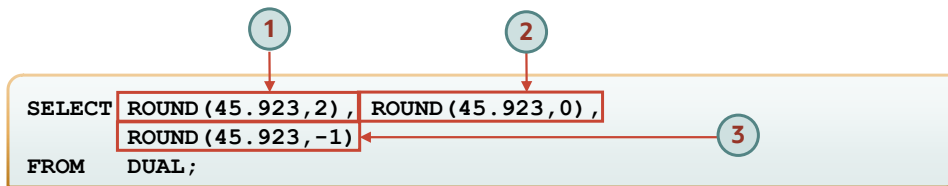
Function	Result
ROUND (45.926, 2)	45.93
TRUNC (45.926, 2)	45.92
CEIL (2.83)	3
FLOOR (2.83)	2
MOD (1600, 300)	100

Numeric functions accept numeric input and return numeric values. This section describes some of the numeric functions.

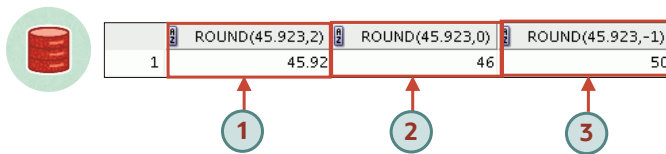
Function	Purpose
ROUND (<i>column expression</i> , <i>n</i>)	Rounds the column, expression, or value to <i>n</i> decimal places or, if <i>n</i> is omitted, no decimal places (If <i>n</i> is negative, numbers to the left of decimal point are rounded.)
TRUNC (<i>column expression</i> , <i>n</i>) TRUNCATE (<i>column expression</i> , <i>n</i>)	Truncates the column, expression, or value to <i>n</i> decimal places or, if <i>n</i> is omitted, <i>n</i> defaults to zero
MOD (<i>m</i> , <i>n</i>)	Returns the remainder of <i>m</i> divided by <i>n</i>

Note: This list contains only some of the available numeric functions. More information is available in the Oracle documentation.

Using the ROUND Function



```
SELECT ROUND (45.923, 2), ROUND (45.923, 0),  
       ROUND (45.923, -1)  
FROM   DUAL;
```



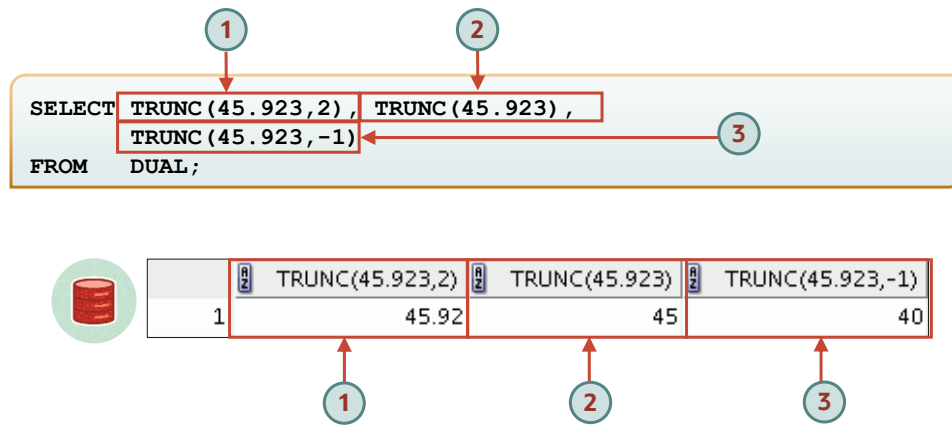
	1	2	3
1	45.92	46	50

The `ROUND` function rounds the column, expression, or value to n decimal places. If the second argument is 0 or is missing, the value is rounded to zero decimal places. If the second argument is 2, the value is rounded to two decimal places. Conversely, if the second argument is -2, the value is rounded to two decimal places to the left (rounded to the nearest unit of 100).

Recall `DUAL` Table in Oracle

In Oracle, the `DUAL` table is owned by the user `SYS` and can be accessed by all users. It contains one column, `DUMMY`, and one row with the value `X`. The `DUAL` table is useful when you want to return a value only once (for example, the value of a constant, pseudocolumn, or expression that is not derived from a table with user data). The `DUAL` table is generally used for completeness of the `SELECT` clause syntax, because, in Oracle, both `SELECT` and `FROM` clauses are mandatory in versions of the database prior to 23c, and several calculations do not need to select from the actual tables. Starting in 23c, the `FROM` clause may not be required depending on the statement that is executed and therefore may not use the `DUAL` table.

Using the TRUNC Function in Oracle



The **TRUNC** function truncates the column, expression, or value to *n* decimal places.

The **TRUNC** function works with arguments similar to those of the **ROUND** function.

If the second argument is 0 or is missing, the value is truncated to zero decimal places.

If the second argument is 2, the value is truncated to two decimal places.

Conversely, if the second argument is -2, the value is truncated to two decimal places to the left.

If the second argument is -1, the value is truncated to one decimal place to the left.

Using the MOD Function

Display the employee records where the employee_id is an even number:

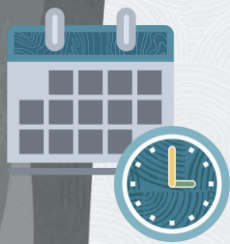
```
SELECT employee_id AS Even_Numbers, last_name
FROM employees
WHERE MOD(employee_id,2) = 0;
```



EVEN_NUMBERS	LAST_NAME
1	174 Abel
2	142 Davies
3	102 De Haan
4	104 Ernst
5	202 Fay
6	206 Gietz
7	178 Grant
8	100 King
9	124 Mourgös
10	176 Taylor
11	144 Vargas
12	200 Whalen

The MOD function finds the remainder of the first argument divided by the second argument. The example in the slide displays employee records where the employee_id is an even number.

Note: The MOD function is often used to determine whether a value is odd or even.



Working with Dates in Oracle Databases

23c agenda topic

Working with Dates in Oracle Databases



- The Oracle Database stores dates in an internal numeric format: century, year, month, day, hours, minutes, and seconds.
- The default date display format is `DD-MON-RR`.

```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date < '01-FEB-2013';
```



	LAST_NAME	HIRE_DATE
1	King	17-JUN-11
2	Kochhar	21-SEP-09
3	De Haan	13-JAN-09

...

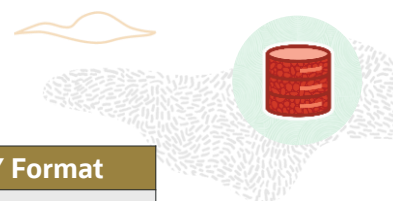
The Oracle Database stores dates in an internal numeric format, representing the century, year, month, day, hours, minutes, and seconds.

- The default display and input format for any date is `DD-MON-RR`. Valid Oracle dates are between January 1, 4712 B.C., and December 31, 9999 A.D.
 - Enables you to store 21st-century dates in the 20th century by specifying only the last two digits of the year
 - Enables you to store 20th-century dates in the 21st century in the same way

In the example in the slide, the `HIRE_DATE` column output is displayed in the default format `DD-MON-RR`. However, dates are not stored in the database in this format. All the components of the date and time are stored. So, although a `HIRE_DATE` such as 17-JUN-11 is displayed as day, month, and year, there is also *time* and *century* information associated with the date. The complete date might be June 17, 2011, 5:10:43 PM.

For Instructor Use Only.
This document should not be distributed

RR Date Format in Oracle



Current Year	Specified Date	RR Format	YY Format
1995	27-OCT-95	1995	1995
1995	27-OCT-17	2017	1917
2001	27-OCT-17	2017	2017
2001	27-OCT-95	1995	2095

If the specified two-digit year is:			
		0-49	50-99
If two digits of the current year are:	0-49	The return date is in the current century.	The return date is in the century before the current one.
	50-99	The return date is in the century after the current one.	The return date is in the current century.

The RR date format is similar to the YY element, but you can use it to specify different centuries. Use the RR date format element instead of YY so that the century of the return value varies according to the specified two-digit year and the last two digits of the current year. The table in the slide summarizes the behavior of the RR element.

Current Year	Given Date	Interpreted (RR)	Interpreted (YY)
1994	27-OCT-95	1995	1995
1994	27-OCT-17	2017	1917
2001	27-OCT-17	2017	2017
2048	27-OCT-52	1952	2052
2051	27-OCT-47	2147	2047

For Instructor Use Only.
This document should not be distributed

Centuries and the Year 2000

When a record with a date column is inserted into a table, the *century* information is picked up from the `SYSDATE` function. However, when the date column is displayed on the screen, the century component is not displayed (by default). You will learn how to use the `SYSDATE` function in the next slide.

The `DATE` data type uses 2 bytes for the year information, one for century and one for year. The century value is always included, whether or not it is specified or displayed. In this case, `RR` determines the default value for century on `INSERT`.

For Instructor Use Only.
This document should not be distributed

Using the SYSDATE Function in Oracle



Use the `SYSDATE` function to get:

- Date
- Time

```
SELECT sysdate  
FROM dual;
```



SYSDATE
1 28-JUN-18

`SYSDATE` is a date function that returns the system date. You can use `SYSDATE` just as you would use any other column name. For example, you can display the system date by selecting `SYSDATE` from a table. It is customary to select `SYSDATE` from a public table called `DUAL`.

Note: `SYSDATE` returns the current date and time set for the operating system on which the database resides. Therefore, if you are in a place in Australia and connected to a remote database in a location in the United States (U.S.), the `SYSDATE` function will return the U.S. date and time. In such a case, to get the local time, you can use the `CURRENT_DATE` function that returns the current date in the session time zone.

Using the CURRENT_DATE and CURRENT_TIMESTAMP Functions in Oracle



- CURRENT_DATE returns the current date from the user session.

```
SELECT SESSIONTIMEZONE, CURRENT_DATE FROM DUAL;
```



SESSIONTIMEZONE	CURRENT_DATE
1 UTC	28-JUN-18

- CURRENT_TIMESTAMP returns the current date and time from the user session.

```
SELECT SESSIONTIMEZONE, CURRENT_TIMESTAMP FROM DUAL;
```

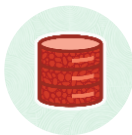


SESSIONTIMEZONE	CURRENT_TIMESTAMP
1 UTC	28-JUN-18 10.06.46.187191000 AM UTC

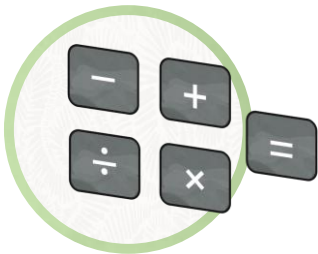
The CURRENT_DATE and CURRENT_TIMESTAMP functions return the current date and current time stamp, respectively.

Note: The SESSIONTIMEZONE function returns the value of the current session's time zone. The return type is a time zone offset (a character type in the format ' [+|-] TZh:TzM ') or a time zone region name, depending on how the user specified the session time zone value in the most recent ALTER SESSION statement. The example in the slide shows that the session time zone is Universal timezone. Observe that the database time zone is different from the current session's time zone.

Arithmetic with Dates in Oracle



- Add or subtract a number from a date for a resultant date value.
- Subtract two dates to find the number of days between those dates.
- Add hours to a date by dividing the number of hours by 24.



Because the database stores dates as numbers, you can perform calculations using arithmetic operators such as addition and subtraction. You can add and subtract number constants as well as dates.

You can perform the following operations:

Operation	Result	Description
date + number	Date	Adds a number of days to a date
date – number	Date	Subtracts a number of days from a date
date – date	Number of days	Subtracts one date from another
date + number/24	Date	Adds a number of hours to a date

For Instructor Use Only.
This document should not be distributed

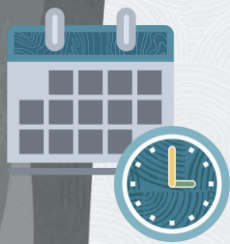
Using Arithmetic Operators with Dates in Oracle

```
SELECT last_name, (SYSDATE-hire_date)/7 AS WEEKS
FROM   employees
WHERE  department_id = 90;
```



	LAST_NAME	WEEKS
1	King	478.871917989417989417989417989418
2	Kochhar	360.729060846560846560846560846561
3	De Haan	605.300489417989417989417989417989

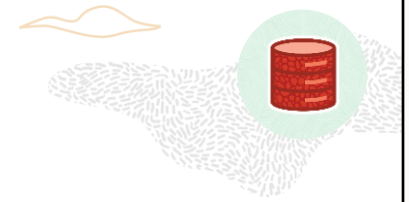
The example in the slide displays the last name and the number of weeks employed for all employees in department 90. It subtracts the date on which the employee was hired from the current date (SYSDATE) and divides the result by 7 to calculate the number of weeks that a worker has been employed.



Date Functions

For Instructor Use Only.
This document should not be distributed

Date-Manipulation Functions in Oracle



Function	Result
MONTHS_BETWEEN	Number of months between two dates
ADD_MONTHS	Add calendar months to date
NEXT_DAY	Date of the next occurrence of the specified day
LAST_DAY	Last day of the month
ROUND	Round date
TRUNC	Truncate date



Date functions operate on Oracle dates. All date functions return a value of the `DATE` data type except `MONTHS_BETWEEN`, which returns a numeric value.

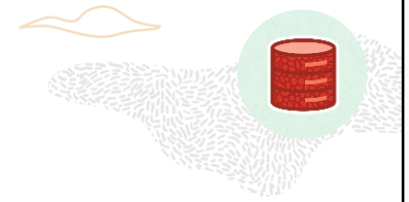
- **MONTHS_BETWEEN**(*date1*, *date2*): Finds the number of months between *date1* and *date2*. The result can be positive or negative. If *date1* is later than *date2*, the result is positive; if *date1* is earlier than *date2*, the result is negative. The noninteger part of the result represents a portion of the month.
- **ADD_MONTHS**(*date*, *n*): Adds *n* number of calendar months to *date*. The value of *n* must be an integer and can be negative.
- **NEXT_DAY**(*date*, '*char*'): Finds the date of the next specified day of the week ('*char*') following *date*. The value of *char* may be a number representing a day or a character string.
- **LAST_DAY**(*date*): Finds the date of the last day of the month that contains *date*

The preceding list is a subset of the available date functions. `ROUND` and `TRUNC` number functions can also be used to manipulate the date values as shown below:

- **ROUND**(*date*[, '*fmt*']): Returns *date* rounded to the unit that is specified by the format model *fmt*. If the format model *fmt* is omitted, *date* is rounded to the nearest day.
- **TRUNC**(*date*[, '*fmt*']): Returns *date* with the time portion of the day truncated to the unit that is specified by the format model *fmt*. If the format model *fmt* is omitted, *date* is truncated to the nearest day.

The format models are covered in detail in the lesson titled “Using Conversion Functions and Conditional Expressions.”

Using Date Functions in Oracle



Function	Result
MONTHS_BETWEEN ('01-SEP-18', '11-JAN-17')	19.6774194
ADD_MONTHS ('31-JAN-16', 1)	'29-FEB-16'
NEXT_DAY ('01-JUN-16', 'FRIDAY')	'08-JUN-18'
LAST_DAY ('01-APR-16')	'30-APR-18'

In the example in the slide, the `ADD_MONTHS` function adds one month to the supplied date value “31-JAN-16” and returns “29-FEB-16.” The function recognizes the year 2016 as a leap year and, therefore, returns the last day of the February month. If you change the input date value to “31-JAN-18,” the function returns “28-FEB-18.”

For example, display the employee number, hire date, number of months employed, six-month review date, first Friday after hire date, and the last day of the hire month for all employees who have been employed for fewer than 150 months.

```
SELECT employee_id, hire_date, MONTHS_BETWEEN (SYSDATE, hire_date)
       TENURE, ADD_MONTHS (hire_date, 6) REVIEW, NEXT_DAY (hire_date,
       'FRIDAY'), LAST_DAY(hire_date)
FROM employees WHERE MONTHS_BETWEEN (SYSDATE, hire_date) < 150;
```

Using ROUND and TRUNC Functions with Dates in Oracle



Assumption: The below functions were run on **29-JUN-18**.

Function	Result
ROUND (SYSDATE, 'MONTH')	01-JUL-18
ROUND (SYSDATE, 'YEAR')	01-JAN-18
TRUNC (SYSDATE, 'MONTH')	01-JUN-18
TRUNC (SYSDATE, 'YEAR')	01-JAN-18

The `ROUND` and `TRUNC` functions can be used for number and date values. When used with dates, these functions round or truncate to the specified format model. Therefore, you can round dates to the nearest year or month. If the format model is month, dates 1-15 result in the first day of the current month. Dates 16-31 result in the first day of the next month. If the format model is year, months 1-6 result in January 1 of the current year. Months 7-12 result in January 1 of the next year.

Example

Compare the hire dates for all employees who started in 2010. Display the employee number, hire date, and starting month using the `ROUND` and `TRUNC` functions.

```
SELECT employee_id, hire_date,  
       ROUND(hire_date, 'MONTH'), TRUNC(hire_date, 'MONTH')  
FROM   employees  
WHERE  hire_date LIKE '%10';
```

Summary



Single-row SQL functions

Character functions

Nesting functions

Number functions

Working with dates in Oracle Databases

Date functions

You have learned about several different types of functions and how to use them.

For Instructor Use Only.
This document should not be distributed

Using Conversion Functions and Conditional Expressions



Course Roadmap

Lesson 1: Introduction

**Unit 1: Retrieving, Restricting,
and Sorting Data**

Unit 2: Joins, Subqueries, and Set
Operators

Unit 3: DML and DDL



Lesson 2: Retrieving Data Using the SQL `SELECT`
Statement



Lesson 3: Restricting and Sorting Data



Lesson 4: Using Single-Row Functions to
Customize Output



**Lesson 5: Using Conversion Functions and Conditional
Expressions**

You are here!

In Unit 1, you will learn how to query the data from tables, how to query selected records from tables, and also how to sort the data retrieved from the tables.

Objectives



Implicit and explicit data type conversion

TO_CHAR, TO_DATE, and TO_NUMBER functions in Oracle

General functions

Conditional expressions

JSON functions

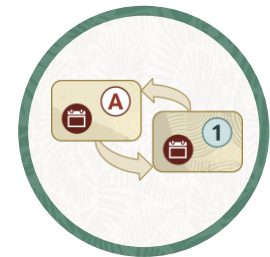
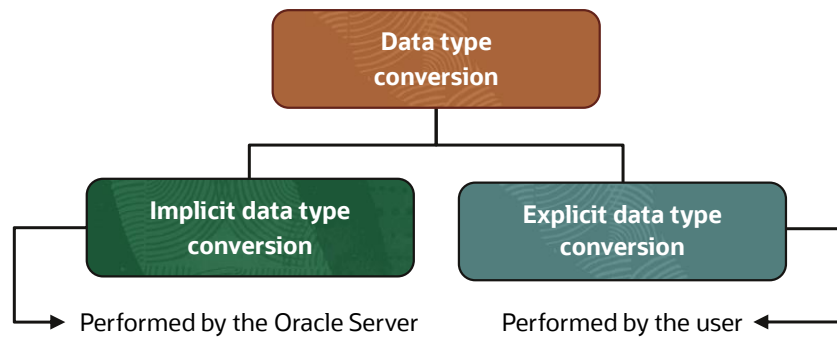
This lesson focuses on functions that convert data from one type to another (for example, conversion from character data to numeric data) and discusses the conditional expressions in SQL `SELECT` statements.



Implicit and Explicit Data Type Conversion

For Instructor Use Only.
This document should not be distributed

Conversion Functions



In some situations, the database server is provided data of one type when it expects data of a different data type. This can happen when providing output, when making comparisons in a `WHERE` clause, or when using a function that expects a specific data type. When this happens, the database can automatically convert the data to the expected data type. This data type conversion can be done **implicitly** by the Oracle server or **explicitly** by the user.

Implicit data type conversions work according to the rules explained in the following slides.

Explicit data type conversions are performed by using the conversion functions. Conversion functions convert a value from one data type to another.

Note: Although implicit data type conversion is available, it is recommended that you do the explicit data type conversion to ensure the reliability of your SQL statements.

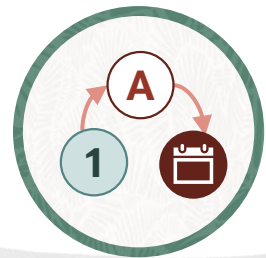
Implicit Data Type Conversion of Strings to Numbers

In expressions, the database can automatically convert strings to numbers. In this example, two strings are concatenated and then implicitly converted to a number for comparison with the numeric department ID in the `WHERE` clause.

```
SELECT first_name, last_name, department_id
FROM employees WHERE department_id < CONCAT('9', '0');
```



	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	Ellen	Abel	80
2	Curtis	Davies	50
3	Bruce	Ernst	60
4	Pat	Fay	20
5	Michael	Hartstein	20
6	Alexander	Hunold	60
7	Diana	Lorentz	60
8	Randall	Matos	50
9	Kevin	Mourgos	50
10	Trenna	Rajs	50
11	Jonathon	Taylor	80
12	Peter	Vargas	50
13	Jennifer	Whalen	10
14	Eleni	Zlotkey	80



The server can automatically perform data type conversion in an expression from `VARCHAR2` or `CHAR` to a numeric or date data type.

For example, the expression `hire_date > '01-JAN-90'` results in the implicit conversion from the string to a date. Therefore, a character expression or value can be implicitly converted to a number or date data type in an expression.

Note: `CHAR` to numeric data type conversions succeed only if the character string provided represents a valid number.

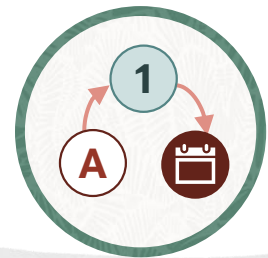
Implicit Data Type Conversion of Numbers to Strings

In expressions, the database can automatically convert numbers to strings. In this example, the salary column is converted to a string to determine if it contains a character.

```
SELECT first_name, last_name, salary
FROM employees
WHERE INSTR(salary, '5') > 0;
```



	FIRST_NAME	LAST_NAME	SALARY
1	Kevin	Mourgos	5800
2	Trenna	Rajs	3500
3	Peter	Vargas	2500
4	Eleni	Zlotkey	10500



The Oracle server can automatically perform data type conversion in an expression from NUMBER or DATE to VARCHAR2 or CHAR.

In general, the Oracle server uses the rule for expressions when a data type conversion is needed. For example, the expression `job_id = 2` results in the implicit conversion of the number 2 to the string "2" because `job_id` is a `VARCHAR(2)` column.



TO_CHAR, TO_DATE, and TO_NUMBER Functions in Oracle

For Instructor Use Only.
This document should not be distributed

Using the TO_CHAR Function with Dates



Example:

```
TO_CHAR(date[, 'format_model'])
```

```
SELECT employee_id, TO_CHAR(hire_date, 'MM/YY') Month_Hired
FROM   employees
WHERE  last_name = 'Higgins';
```

	EMPLOYEE_ID	MONTH_HIRED
1		205 06/10



You can convert a datetime data type to a value of VARCHAR2 data type using TO_CHAR in Oracle in the format specified by the *format_model*.

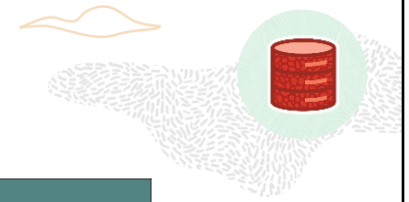
A format model is a character literal that describes the format of datetime stored in a character string, for example, the datetime format model for the string '11-Nov-2000' is 'DD-Mon-YYYY'. You can use the TO_CHAR function to convert a date from its default format to the one that you specify.

Here are some of the guidelines to follow while using TO_CHAR:

- Enclose the format model within single quotation marks. The format model is case-sensitive.
- Ensure that you separate the date value from the format model with a comma. The format model can include any valid date format element.
- The names of days and months in the output are automatically padded with blanks.
- Use the fill mode *fm* element to remove padded blanks or to suppress leading zeros.

For Instructor Use Only.
This document should not be distributed

Elements of the Date Format Model



Element	Result
YYYY	Full year in numbers
YEAR	Year spelled out (in English)
MM	Two-digit value for the month
MONTH	Full name of the month
MON	Three-letter abbreviation of the month
DY	Three-letter abbreviation of the day of the week
DAY	Full name of the day of the week
DD	Numeric day of the month

You can refer to the complete list of date format model elements in the following page.

Element	Description
SCC or CC	Century; server prefixes B.C. date with -
Years in dates YYYY or SYYYY	Year; server prefixes B.C. date with -
YYY or YY or Y	Last three, two, or one digit of the year
Y,YYY	Year with comma in this position
IYYY, IYY, IY, I	Four-, three-, two-, or one-digit year based on the ISO standard
SYEAR or YEAR	Year spelled out; server prefixes B.C. date with -
BC or AD	Indicates B.C. or A.D. year
B.C. or A.D.	Indicates B.C. or A.D. year using periods
Q	Quarter of year
MM	Month: two-digit value
MONTH	Name of the month padded with blanks to a length of nine characters
MON	Name of the month, three-letter abbreviation
RM	Roman numeral month
WW or W	Week of the year or month
DDD or DD or D	Day of the year, month, or week
DAY	Name of the day padded with blanks to a length of nine characters
DY	Name of the day; three-letter abbreviation
J	Julian day; the number of days since December 31, 4713 B.C.
IW	Weeks in the year from ISO standard (1 to 53)

For Instructor Use Only.
This document should not be distributed

Elements of the Date Format Model



- Time elements help you format the time portion of the date:

HH24:MI:SS AM	15:45:32 PM
---------------	-------------

- Add character strings by enclosing them within double quotation marks:

DD "of" MONTH	12 of OCTOBER
---------------	---------------

- Number suffixes help in spelling out numbers:

ddspth	fourteenth
--------	------------

Use the formats that are listed in the following tables to display time information and literals and to change numerals to spelled numbers:

Element	Description
AM or PM	Meridian indicator
A.M. or P.M.	Meridian indicator with periods
HH or HH12	12 hour format
HH24	24 hour format
MI	Minute (0–59)
SS	Second (0–59)
SSSSS	Seconds past midnight (0–86399)

Other Formats

Element	Description
/ . ,	Punctuation is reproduced in the result.
“of the”	Quoted string is reproduced in the result.

Specifying Suffixes to Influence Number Display

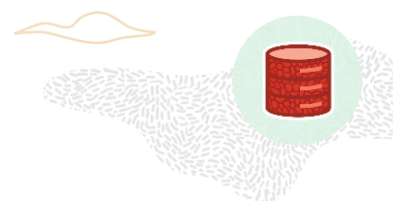
Element	Description
TH	Ordinal number (for example, DDTH for 4TH)
SP	Spelled-out number (for example, DDSP for FOUR)
SPTH or THSP	Spelled-out ordinal numbers (for example, DDSPTH for FOURTH)

For Instructor Use Only.
This document should not be distributed

Using the TO_CHAR Function with Dates

```
SELECT last_name,  
       TO_CHAR(hire_date, 'fmDD Month YYYY')  
       AS HIREDATE  
FROM   employees;
```

	LAST_NAME	HIREDATE
1	King	17 June 2011
2	Kochhar	21 September 2009
3	De Haan	13 January 2009
4	Hunold	3 January 2014
5	Ernst	21 May 2015
6	Lorentz	7 February 2015
7	Mourgos	16 November 2015
8	Rajs	17 October 2011
...		



The SQL statement in the slide displays the last names and hire dates for all the employees. Observe that the hire date appears as 17 June 2011.

Example

Modify the example in the slide to display the dates in a format that appears as “Seventeenth of June 2011 12:00:00 AM.”

```
SELECT last_name,  
       TO_CHAR(hire_date,  
               'fmDdspth "of" Month YYYY fmHH:MI:SS AM')  
       AS HIREDATE  
FROM   employees;
```

Notice that the month follows the format model specified; in other words, the first letter is capitalized and the rest are in lowercase.

Using the TO_CHAR Function with Numbers



These are some of the format elements that you can use with the TO_CHAR function to display a number value as a character:

```
TO_CHAR(number[, 'format_model'])
```

Element	Result
9	Represents a number
0	Forces a zero to be displayed
\$	Places a floating dollar sign
£	Uses the floating local currency symbol
.	Prints a decimal point
,	Prints a comma as a thousands indicator

When working with number values, such as character strings, you should convert the numbers to the character data type using the TO_CHAR function. This function translates a value of NUMBER data type to VARCHAR2 data type.

This technique is especially useful with concatenation.

Number Format Elements

If you are converting a number to the character data type, you can use the following format elements:

Element	Description	Example	Result
9	Numeric position (number of 9s determine display width)	999999	1234
0	Display leading zeros	099999	001234
\$	Floating dollar sign	\$999999	\$1234
L	Floating local currency symbol	L999999	FF1234
D	Returns the decimal character in the specified position. The default is a period (.).	9999D99	1234.00
.	Decimal point in position specified	999999.99	1234.00
G	Returns the group separator in the specified position. You can specify multiple group separators in a number format model.	9G999	1,234
,	Comma in position specified	999,999	1,234
MI	Minus signs to right (negative values)	999999MI	1234-
PR	Parenthesize negative numbers	999999PR	<1234>
EEEE	Scientific notation (format must specify four Es)	99.999EEEE	1.234E+03
U	Returns in the specified position the “Euro” (or other) dual currency	U9999	€1234
V	Multiply by 10 <i>n</i> times (<i>n</i> = number of 9s after V)	9999V99	123400
S	Returns the negative or positive value	S9999	-1234 or +1234
B	Display zero values as blank, not 0	B9999.99	1234.00

For Instructor Use Only.
This document should not be distributed

Using the TO_CHAR Function with Numbers



Let's look at an example:

```
SELECT TO_CHAR(salary, '$99,999.00') SALARY
FROM   employees
WHERE  last_name = 'Ernst';
```

	SALARY
1	\$6,000.00



Good to know:

- The Oracle server displays a string of number signs (#) in place of a whole number whose digits exceed the number of digits provided in the format model.
- The Oracle server rounds the stored decimal value to the number of decimal places provided in the format model.

For Instructor Use Only.
This document should not be distributed

Using the TO_NUMBER and TO_DATE Functions



- Convert a character string to a number format using the TO_NUMBER function:

```
TO_NUMBER(char[, 'format_model'])
```

- Convert a character string to a date format using the TO_DATE function:

```
TO_DATE(char[, 'format_model'])
```

You may want to convert a character string to either a number or a date. To accomplish this task, use the TO_NUMBER or TO_DATE functions. The format model that you select is based on the previously demonstrated format elements.

The `fx` modifier in these functions specifies the exact match for the character argument and date format model of a TO_DATE function:

- Punctuation and quoted text in the character argument must exactly match (except for case) the corresponding parts of the format model.
- The character argument cannot have extra blanks. Without `fx`, the Oracle server ignores extra blanks.
- Numeric data in the character argument must have the same number of digits as the corresponding element in the format model. Without `fx`, the numbers in the character argument can omit leading zeros.

Example

Display the name and hire date for all employees who started on May 24, 2015. There are two spaces after the month *May* and before the number *24* in the following example. Because the `fx` modifier is used, an exact match is required and the spaces after the word *May* are not recognized:

```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date = TO_DATE('May  24, 2015', 'fxMonth DD, YYYY');
```

The resulting error output looks like this:

```
ORA-01858: a non-numeric character was found where a numeric was expected
01858. 00000 - "a non-numeric character was found where a numeric was expected"
*Cause:   The input data to be converted using a date format model was
          incorrect. The input data did not contain a number where a number was
          required by the format model.
*Action:  Fix the input data or the date format model to make sure the
          elements match in number and type. Then retry the operation.
```

To see the output, correct the query by deleting the extra space between “May” and “24.”

```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date = TO_DATE('May 24, 2015', 'fxMonth DD, YYYY');
```


Using TO_CHAR and TO_DATE Functions with the RR Date Format



To find employees hired before 2010, use the RR date format, which produces the correct result if the command is run now or before the year 2049:

```
SELECT last_name, TO_CHAR(hire_date, 'DD-Mon-YYYY')
FROM employees
WHERE hire_date < TO_DATE('01 Jan, 10', 'DD Mon,RR');
```

LAST_NAME	TO_CHAR(HIRE_DATE,'DD-MON-YYYY')
1 Kochhar	21-Sep-2009
2 De Haan	13-Jan-2009

To find employees who were hired before 2010, the RR format can be used. Because the current year is greater than 1999, the RR format interprets the year portion of the date from 2000 to 2049.

Alternatively, the following command also results in the same rows being selected because the YY format interprets the year portion of the date in the current century (2010).

```
SELECT last_name, TO_CHAR(hire_date, 'DD-Mon-yyyy')
FROM employees
WHERE hire_date < '01-Jan-10';
```

Notice that the same rows are retrieved from the preceding query.

The general tip is to use the YYYY format instead of RR format to avoid confusion.



General Functions

For Instructor Use Only.
This document should not be distributed

General Functions

The following functions pertain to using nulls and can be used with any data type:

```
NVL (expr1, expr2)
```

```
NVL2 (expr1, expr2, expr3)
```

```
NULLIF (expr1, expr2)
```

```
COALESCE (expr1, expr2,  
          ...,  
          exprn)
```



These functions work with any data type and pertain to the use of null values in the expression list.

Function	Description
NVL	Converts a null value to an actual value
NVL2	If <code>expr1</code> is not null, NVL2 returns <code>expr2</code> . If <code>expr1</code> is null, NVL2 returns <code>expr3</code> . The argument <code>expr1</code> can have any data type.
NULLIF	Compares two expressions and returns null if they are equal; returns the first expression if they are not equal
COALESCE	Returns the first non-null expression in the expression list

Note: For more information about the hundreds of functions available, see the “Functions” section in *Oracle Database SQL Language Reference* for 23c database.

NVL Function

Converts a null value to an actual value:

- Data types that can be used are date, character, and number.
- Data types for both expressions must match.
- Examples:

Oracle
NVL(commission_pct,0)
NVL(hire_date,'01-JAN-97')
NVL(job_id,'No Job Yet')

NVL (expr1, expr2)



To convert a null value to an actual value, use the `NVL` function or `IFNULL` function.

Syntax

`NVL (expr1, expr2)`

`IFNULL (expr1, expr2)`

In the syntax:

- *expr1* is the source value or expression that may contain a null
- *expr2* is the target value for converting the null

You can use the `NVL` function with any data type, but the return value is always the same as the data type of *expr1*.

NVL Conversions for Various Data Types

Data Type	Conversion Example
NUMBER	<code>NVL (number_column, 9)</code>
DATE	<code>NVL (date_column, '01-JAN-95')</code>
CHAR or VARCHAR2	<code>NVL (character_column, 'Unavailable')</code>

Using the NVL Function in Oracle



```
SELECT last_name, salary, NVL(commission_pct, 0),  
       (salary*12) + (salary*12*NVL(commission_pct, 0)) AN_SAL  
FROM employees;
```

	LAST_NAME	SALARY	NVL(COMMISSION_PCT,0)	AN_SAL
1	King	24000	0	288000
2	Kochhar	17000	0	204000
3	De Haan	17000	0	204000
4	Hunold	9000	0	108000
5	Ernst	6000	0	72000
6	Lorentz	4200	0	50400
7	Mourgos	5800	0	69600
8	Rajs	3500	0	42000
9	Davies	3100	0	37200
10	Matos	2600	0	31200

To calculate the annual compensation of all employees, you need to multiply the monthly salary by 12 and then add the commission percentage to the result:

```
SELECT last_name, salary, commission_pct,  
       (salary*12) + (salary*12*commission_pct) AN_SAL  
FROM employees;
```

Notice that the annual compensation is calculated for only those employees who earn a commission. If any column value in an expression is null, the result is null. To calculate values for all employees, you must convert the null value to a number before applying the arithmetic operator. In the example in the slide, the NVL function is used to convert null values to zero.

Using the NVL2 Function in Oracle

NVL2 (expr1, expr2, expr3)

```
SELECT last name, salary, commission_pct,
       NVL2(commission_pct,
            'SAL+COMM', 'SAL') income
FROM   employees WHERE department_id IN (50, 80);
```

	LAST_NAME	SALARY	COMMISSION_PCT	INCOME
1	Mourgos	5800	(null)	SAL
2	Rajs	3500	(null)	SAL
3	Davies	3100	(null)	SAL
4	Matos	2600	(null)	SAL
5	Vargas	2500	(null)	SAL
6	Zlotkey	10500	0.2	SAL+COMM
7	Abel	11000	0.3	SAL+COMM
8	Taylor	8600	0.2	SAL+COMM

The NVL2 function examines the first expression. If the first expression is not null, the NVL2 function returns the second expression. If the first expression is null, the third expression is returned.

Syntax

NVL2(*expr1*, *expr2*, *expr3*)

In the syntax:

- *expr1* is the source value or expression that may contain a null
- *expr2* is the value that is returned if *expr1* is not null
- *expr3* is the value that is returned if *expr1* is null

In the example shown in the slide, the `COMMISSION_PCT` column is examined. If a value is detected, the text literal value of `SAL+COMM` is returned. If the `COMMISSION_PCT` column contains a null value, the text literal value of `SAL` is returned.

Note: The argument *expr1* can be of any data type, but *expr2* and *expr3* should be of the same data type.

Using the NULLIF Function

NULLIF (expr1, expr2)

```
SELECT first_name, LENGTH(first_name) AS expr1,
       last_name, LENGTH(last_name) AS expr2,
       NULLIF(LENGTH(first_name), LENGTH(last_name)) AS Result
FROM employees;
```



	FIRST_NAME	expr1	LAST_NAME	expr2	RESULT
1	Ellen	5	Abel	4	5
2	Curtis	6	Davies	6	(null)
3	Lex	3	De Haan	7	3
4	Bruce	5	Ernst	5	(null)
5	Pat	3	Fay	3	(null)
6	William	7	Gietz	5	7
7	Kimberely	9	Grant	5	9
8	Michael	7	Hartstein	9	7
9	Shelley	7	Higgins	7	(null)

...

1

2

3

The NULLIF function compares two expressions.

Syntax

NULLIF (expr1, expr2)

In the syntax:

- NULLIF compares *expr1* and *expr2*. If they are equal, the function returns null. If they are not, the function returns *expr1*. However, you cannot specify the literal NULL for *expr1*.

In the example shown in the slide, the length of the first name in the EMPLOYEES table is compared with the length of the last name in the EMPLOYEES table. When the lengths of the names are equal, a null value is displayed. When the lengths of the names are not equal, the length of the first name is displayed.

Using the COALESCE Function

- The advantage of the COALESCE function over the NVL or IFNULL functions is that the COALESCE function can take multiple alternative values.
- If the first expression is not null, the COALESCE function returns that expression; otherwise, it does a COALESCE of the remaining expressions.

```
COALESCE (expr1, expr2, ..., exprn)
```

The COALESCE function returns the first non-null expression in the list.

Syntax

```
COALESCE (expr1, expr2, ... exprn)
```

In the syntax:

- *expr1* is the value returned if this expression is not null
- *expr2* is the value returned if the first expression is null and this expression is not null
- *exprn* is the value returned if the preceding expressions are null
- Note that all expressions must be of the same data type.

Using the COALESCE Function

```
SELECT last name, salary, commission pct,  
COALESCE((salary+(commission pct*salary)), salary+2000)  
AS New_Salary  
FROM employees;
```



	LAST_NAME	SALARY	COMMISSION_PCT	NEW_SALARY
1	King	24000	(null)	26000
2	Kochhar	17000	(null)	19000
3	De Haan	17000	(null)	19000
4	Hunold	9000	(null)	11000
5	Ernst	6000	(null)	8000
6	Lorentz	4200	(null)	6200
7	Mourgos	5800	(null)	7800
8	Rajs	3500	(null)	5500
9	Davies	3100	(null)	5100
10	Matos	2600	(null)	4600
11	Vargas	2500	(null)	4500
12	Zlotkey	10500	0.2	12600
13	Abel	11000	0.3	14300
14	Taylor	8600	0.2	10320
15	Grant	7000	0.15	8050
16	Whalen	4400	(null)	6400
17	Hartstein	13000	(null)	15000
18	Fay	6000	(null)	8000
19	Higgins	12008	(null)	14008
20	Gietz	8300	(null)	10300

In the example shown in the slide, for the employees who do not get any commission, your organization wants to give a salary increment of \$2,000 and, for employees who get commission, the query should compute the new salary that is equal to the existing salary added to the commission amount.

Note: Examine the output. For employees who do not get any commission, the New Salary column shows the salary incremented by \$2,000, and for employees who get commission, the New Salary column shows the computed commission amount added to the salary.



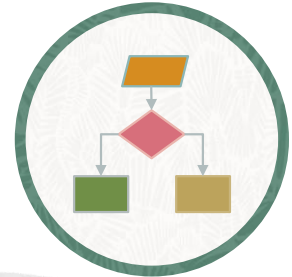
Conditional Expressions

For Instructor Use Only.
This document should not be distributed

Conditional Expressions

Help provide the use of `IF-THEN-ELSE` logic within a SQL statement

- You can use the following methods:
 - `CASE` expression
 - Searched `CASE` expression
 - `DECODE` function



The two methods that are used to implement conditional processing (`IF-THEN-ELSE` logic) in a SQL statement are the `CASE` expression and the `DECODE` function.

Note: The `CASE` expression complies with the ANSI SQL. The `DECODE` function is specific to Oracle syntax.

CASE Expression

Facilitates conditional inquiries by doing the work of an `IF-THEN-ELSE` statement:

```
CASE expr WHEN comparison_expr1 THEN return_expr1
      [WHEN comparison_expr2 THEN return_expr2
      WHEN comparison_exprn THEN return_exprn
      ELSE else_expr]
END
```

CASE expressions allow you to use the `IF-THEN-ELSE` logic in SQL statements without having to invoke procedures.

In a simple CASE expression, the Oracle server searches for the first `WHEN ... THEN` pair for which `expr` is equal to `comparison_expr` and returns `return_expr`. If none of the `WHEN ... THEN` pairs meet this condition, and if an `ELSE` clause exists, the Oracle server returns `else_expr`. Otherwise, the Oracle server returns a null. You cannot specify the literal `NULL` for all the `return_exprs` and the `else_expr`.

The expressions `expr` and `comparison_expr` must be of the same data type, which can be `CHAR`, `VARCHAR2`, `NCHAR`, `NVARCHAR2`, `NUMBER`, `BINARY_FLOAT`, or `BINARY_DOUBLE` or must all have a numeric data type. All of the return values (`return_expr`) must be of the same data type.

Using the CASE Expression

```
SELECT last_name, job_id, salary,  
       CASE job_id WHEN 'IT_PROG' THEN 1.10*salary  
                  WHEN 'ST_CLERK' THEN 1.15*salary  
                  WHEN 'SA_REP' THEN 1.20*salary  
                  ELSE salary END AS REVISED_SALARY  
FROM employees;
```



	LAST_NAME	JOB_ID	SALARY	REVISED_SALARY
1	King	AD_PRES	24000	24000
...				
4	Hunold	IT_PROG	9000	9900
5	Ernst	IT_PROG	6000	6600
6	Lorentz	IT_PROG	4200	4620
7	Mourgos	ST_MAN	5800	5800
8	Rajs	ST_CLERK	3500	4025
9	Davies	ST_CLERK	3100	3565
10	Matos	ST_CLERK	2600	2990
11	Vargas	ST_CLERK	2500	2875
...				
13	Abel	SA_REP	11000	13200
14	Taylor	SA_REP	8600	10320
15	Grant	SA_REP	7000	8400

In the SQL statement in the slide, the value of `JOB_ID` is decoded.

If `JOB_ID` is `IT_PROG`, the salary increase is 10%.

If `JOB_ID` is `ST_CLERK`, the salary increase is 15%.

If `JOB_ID` is `SA_REP`, the salary increase is 20%.

For all other job roles, there is no increase in salary.

The same statement can be written with the `DECODE` function.

Searched CASE Expression

```
CASE
  WHEN condition1 THEN use_expression1
  WHEN condition2 THEN use_expression2
  WHEN condition3 THEN use_expression3
  ELSE default_use_expression
END
```

```
SELECT last_name,salary,
       (CASE WHEN salary<5000 THEN 'Low'
             WHEN salary<10000 THEN 'Medium'
             WHEN salary<20000 THEN 'Good'
             ELSE 'Excellent'
          END) AS qualified_salary
FROM employees;
```

In a searched CASE expression, the search occurs from left to right until an occurrence of the listed condition is found, and then it returns the return expression. If no condition is found to be true, and if an ELSE clause exists, the return expression in the ELSE clause is returned; otherwise, a NULL is returned. The searched CASE expression evaluates the conditions independently under each of the WHEN options.

The difference between the CASE expression and the searched CASE expression is that in a searched CASE expression, you specify a condition or predicate instead of a comparison_expression after the WHEN keyword.

For both simple and searched CASE expressions, all of the return_exprs must either have the same data type CHAR, VARCHAR2, NCHAR, NVARCHAR2, NUMBER, BINARY_FLOAT, or BINARY_DOUBLE or all must have a numeric data type.

The code in the slide is an example of the searched CASE expression.

DECODE Function in Oracle



Facilitates conditional inquiries by doing the work of a `CASE` expression or an `IF-THEN-ELSE` statement:

```
DECODE(col|expression, search1, result1  
      [, search2, result2, ..., ]  
      [, default])
```



The `DECODE` function decodes an expression in a way similar to the `IF-THEN-ELSE` logic that is used in various languages. The `DECODE` function decodes *expression* after comparing it to each *search* value. If the expression is the same as *search*, *result* is returned.

If the default value is omitted, a null value is returned where a search value does not match any of the result values.

Using the DECODE Function

```
SELECT last_name, job_id, salary,  
       DECODE(job_id, 'IT_PROG', 1.10*salary,  
                'ST_CLERK', 1.15*salary,  
                'SA_REP', 1.20*salary,  
                salary)  
       REVISSED_SALARY  
FROM   employees;
```

	LAST_NAME	JOB_ID	SALARY	REVISSED_SALARY
...				
4	Hunold	IT_PROG	9000	9900
5	Ernst	IT_PROG	6000	6600
6	Lorentz	IT_PROG	4200	4620
7	Mourgos	ST_MAN	5800	5800
8	Rajs	ST_CLERK	3500	4025
9	Davies	ST_CLERK	3100	3565
10	Matos	ST_CLERK	2600	2990
11	Vargas	ST_CLERK	2500	2875
12	Zlotkey	SA_MAN	10500	10500
...				
13	Abel	SA_REP	11000	13200
14	Taylor	SA_REP	8600	10320
15	Grant	SA_REP	7000	8400

In the SQL statement in the slide, the value of `JOB_ID` is tested.

If `JOB_ID` is `IT_PROG`, the salary increase is 10%.

If `JOB_ID` is `ST_CLERK`, the salary increase is 15%.

If `JOB_ID` is `SA_REP`, the salary increase is 20%.

For all other job roles, there is no increase in salary.

The same statement can be expressed in pseudocode as an IF-THEN-ELSE statement:

```
IF job_id = 'IT_PROG'      THEN salary = salary*1.10  
IF job_id = 'ST_CLERK'    THEN salary = salary*1.15  
IF job_id = 'SA_REP'      THEN salary = salary*1.20  
ELSE salary = salary
```


Using the DECODE Function



Display the applicable tax rate for each employee in department 80:

```
SELECT last name, salary,
       DECODE (TRUNC(salary/2000, 0),
               0, 0.00,
               1, 0.09,
               2, 0.20,
               3, 0.30,
               4, 0.40,
               5, 0.42,
               6, 0.44,
               0.45) TAX RATE
FROM   employees
WHERE  department_id = 80;
```

This slide shows another example using the DECODE function. In this example, you determine the tax rate for each employee in department 80 based on the monthly salary. The tax rates are as follows:

Monthly Salary Range	Tax Rate
\$0.00–1,999.99	00%
\$2,000.00–3,999.99	09%
\$4,000.00–5,999.99	20%
\$6,000.00–7,999.99	30%
\$8,000.00–9,999.99	40%
\$10,000.00–11,999.99	42%
\$12,200.00–13,999.99	44%
\$14,000.00 or greater	45%



JSON Functions

For Instructor Use Only.
This document should not be distributed

JSON_QUERY Function

The SQL/JSON function `JSON_QUERY` finds one or more specified JSON values in JSON data and returns the values in a character string.

```
SELECT JSON_QUERY('{a:100, b:200, c:300}', '$') AS value  
FROM DUAL;
```

VALUE

{ "a" : 100, "b" : 200, "c" : 300 }

The following query returns the context item or the specified string of JSON data. The path expression matches a single JSON object, which does not require an array wrapper. Note that the JSON data is converted to strict JSON syntax in the returned value—that is, the object property names are enclosed in double quotation marks.

Note: See Appendix C in [Oracle Database Globalization Support Guide](#) for the collation derivation rules, which define the collation assigned to the character value returned by `JSON_QUERY`.

JSON_TABLE Function

The SQL/JSON function `JSON_TABLE` creates a relational view of JSON data.

```
SELECT JSON_QUERY('{a:100, b:200, c:300}', '$') AS value  
FROM DUAL;
```

VALUE

{ "a" : 100, "b" : 200, "c" : 300 }

It maps the result of a JSON data evaluation into relational rows and columns. You can query the result returned by the function as a virtual relational table using SQL. The main purpose of `JSON_TABLE` is to create a row of relational data for each object inside a JSON array and output JSON values from within that object as individual SQL column values.

You must specify `JSON_TABLE` only in the `FROM` clause of a `SELECT` statement. The function first applies a path expression, called a **SQL/JSON row path expression**, to the supplied JSON data. The JSON value that matches the row path expression is called a **row source** in that it generates a row of relational data. The `COLUMNS` clause evaluates the row source, finds specific JSON values within the row source, and returns those JSON values as SQL values in individual columns of a row of relational data.

Note: See Appendix C in [Oracle Database Globalization Support Guide](#) for the collation derivation rules, which define the collation assigned to each character data type column in the table generated by `JSON_TABLE`.

JSON_VALUE Function

The SQL/JSON function `JSON_QUERY` finds one or more specified JSON values in JSON data and returns the values in a character string.

```
SELECT JSON_VALUE('{a:100}', '$.a') AS value
FROM DUAL;
```

```
VALUE
-----
100
```

The example query listed returns the value of the member with a property name. Because the `RETURNING` clause is not specified, the value is returned as a `VARCHAR2(4000)` data type.

Note: See Appendix C in [Oracle Database Globalization Support Guide](#) for the collation derivation rules, which define the collation assigned to the value returned by this function when it is a character value.

Summary



Implicit and explicit data type conversion

TO_CHAR, TO_DATE, TO_NUMBER functions in Oracle

General functions

Conditional expressions

JSON functions

Remember the following:

- Conversion functions can convert character, date, and numeric values: TO_CHAR, TO_DATE, TO_NUMBER
- There are several functions that pertain to nulls, including NVL, NVL2, NULLIF, and COALESCE.
- The IF-THEN-ELSE logic can be applied within a SQL statement by using the CASE expression, searched CASE, or the DECODE function.

For Instructor Use Only.
This document should not be distributed

For Instructor Use Only.
This document should not be distributed

Reporting Aggregated Data Using the Group Functions



Course Roadmap

Lesson 1: Introduction

Unit 1: Retrieving, Restricting, and Sorting Data

Unit 2: Joins, Subqueries, and Set operators

Unit 3: DML and DDL

▶ **Lesson 6: Reporting Aggregated Data Using the Group Functions**

You are here!

▶ Lesson 7: Displaying Data from Multiple Tables Using Joins

▶ Lesson 8: Using Subqueries to Solve Queries

▶ Lesson 9: Using Set Operators

In Unit 2, you will learn about SQL statements to query and display data from multiple tables using Joins. You will also learn to use subqueries when the condition is unknown, use group functions to aggregate data, and use set operators.

Objectives



Group functions

Grouping rows

Nesting group functions

In this lesson, you will further learn about functions. The lesson focuses on obtaining summary information (such as averages) for groups of rows. You will discuss how to group rows in a table into smaller sets and how to specify search criteria for groups of rows.



Group Functions

For Instructor Use Only.
This document should not be distributed

Group Functions

Group functions operate on sets of rows to give one result per group.

EMPLOYEES

	DEPARTMENT_ID	SALARY
1	10	4400
2	20	13000
3	20	6000
4	110	12000
5	110	8300
6	90	24000
7	90	17000
8	90	17000
9	60	9000
10	60	6000
...		
18	80	11000
19	80	8600
20	(null)	7000

Maximum salary in
EMPLOYEES table

MAX(SALARY)
24000



Unlike single-row functions, group functions operate on sets of rows to give one result per group. These sets can comprise the entire table or groups of rows in the table.

Types of Group Functions

- AVG
- COUNT*
- MAX
- MIN
- SUM
- LISTAGG
- STDDEV
- VARIANCE



Each of the functions accepts an argument. The following table identifies the options that you can use in the syntax:

Function	Description
AVG ([DISTINCT <u>ALL</u>] <i>n</i>)	Average value of <i>n</i> , ignoring null values
COUNT ([DISTINCT ALL] <i>expr</i>)	Number of rows where <i>expr</i> evaluates to something other than null (count all selected rows using *, including duplicates and rows with nulls)
MAX ([DISTINCT ALL] <i>expr</i>)	Maximum value of <i>expr</i> , ignoring null values
MIN ([DISTINCT <u>ALL</u>] <i>expr</i>)	Minimum value of <i>expr</i> , ignoring null values
STDDEV ([DISTINCT <u>ALL</u>] <i>n</i>)	Standard deviation of <i>n</i> , ignoring null values
SUM ([DISTINCT <u>ALL</u>] <i>n</i>)	Sum values of <i>n</i> , ignoring null values
LISTAGG	Orders data within each group specified in the ORDER BY clause and then concatenates the values of the measure column
VARIANCE ([DISTINCT <u>ALL</u>] <i>n</i>)	Variance of <i>n</i> , ignoring null values

***Note:** Materialized views based on COUNT(DISTINCT) functions can provide enhanced performance by using bitmap-based operations on integer columns.

Starting with Oracle Database Release 19c, you can create materialized views based on SQL aggregate functions that use bitmap representation to express the computation of COUNT(DISTINCT) operations. These functions include BITMAP_BUCKET_NUMBER, BITMAP_BIT_POSITION and BITMAP_CONSTRUCT_AGG.

Group Functions: Syntax

```
SELECT  group_function(column), ...  
FROM    table  
[WHERE  condition];
```

Group all rows in a column



The group function is placed after the `SELECT` keyword. You may have multiple group functions separated by commas.

Syntax:

```
group_function([DISTINCT|ALL] expr)
```

Let us look at a few guidelines for using the group functions:

- `DISTINCT` makes the function consider only nonduplicate values; `ALL` makes it consider every value, including duplicates. The default is `ALL` and, therefore, does not need to be specified.
- The data types for the functions with an `expr` argument may be `CHAR`, `VARCHAR2`, `NUMBER`, or `DATE`.
- All group functions ignore null values. To substitute a value for null values, use the `NVL`, `NVL2`, `COALESCE`, `CASE`, or `DECODE` functions.

Using the AVG and SUM Functions

You can use the AVG and SUM functions for numeric data.

```
SELECT AVG(salary), MAX(salary),  
       MIN(salary), SUM(salary)  
FROM   employees  
WHERE  job_id LIKE '%REP%';
```



	AVG(SALARY)	MAX(SALARY)	MIN(SALARY)	SUM(SALARY)
1	8150	11000	6000	32600

You can use the AVG, SUM, MIN, and MAX functions against the columns that can store numeric data. The example in the slide displays the average, highest, lowest, and sum of monthly salaries for all sales representatives.

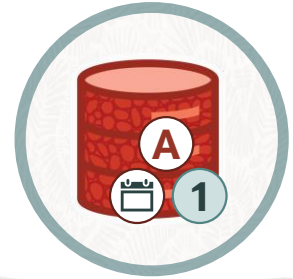
Using the MIN and MAX Functions

You can use MIN and MAX for numeric, character, and date data types.

```
SELECT MIN(last_name), MAX(last_name),  
       MIN(hire_date), MAX(hire_date)  
FROM   employees;
```



	MIN(LAST_NAME)	MAX(LAST_NAME)	MIN(HIRE_DATE)	MAX(HIRE_DATE)
1	Abel	Zlotkey	13-JAN-09	29-JAN-16



You can use the MAX and MIN functions for numeric, character, and date data types. The example in the slide displays the employee last name that is first and last in an alphabetic list of all employees and the date hired for the most senior and most junior employees.

Note: The AVG, SUM, VARIANCE, and STDDEV functions can be used only with numeric data types. MAX and MIN cannot be used with LOB or LONG data types.

Using the COUNT Function

- COUNT (*) returns the number of rows in a table:

```
SELECT COUNT(*)  
FROM employees  
WHERE department_id = 50;
```



COUNT(*)	
1	5

- COUNT (expr) returns the number of rows with non-null values for expr:

```
SELECT COUNT(commission_pct)  
FROM employees  
WHERE department_id = 50;
```



COUNT(COMMISSION_PCT)	
1	0

You can use the COUNT function in the following three formats:

- COUNT (*)
- COUNT (expr)
- COUNT (DISTINCT expr)

COUNT (*) returns the number of rows in a table that satisfy the criteria of the SELECT statement, including duplicate rows and rows containing null values in any of the columns. If a WHERE clause is included in the SELECT statement, COUNT (*) returns the number of rows that satisfy the condition in the WHERE clause.

In contrast, COUNT (expr) returns the number of non-null values that are in the column identified by expr.

COUNT (DISTINCT expr) returns the number of unique, non-null values that are in the column identified by expr.

Examples

1. The first example in the slide displays the number of employees in department 50.
2. The second example in the slide displays the number of employees in department 50 who can earn a commission.

Using the DISTINCT Keyword

- `COUNT (DISTINCT expr)` returns the number of distinct non-null values of *expr*.
- To display the number of distinct department values in the `EMPLOYEES` table:

```
SELECT COUNT(DISTINCT department_id)
FROM employees;
```



COUNT(DISTINCTDEPARTMENT_ID)	
1	7

Use the `DISTINCT` keyword to suppress the counting of any duplicate values in a column.

The example in the slide displays the number of distinct department values that are in the `EMPLOYEES` table.

Group Functions and Null Values in Oracle



- Group functions ignore null values in the column:

```
SELECT AVG(commission_pct)
FROM employees;
```

1



	Avg(COMMISSION_PCT)
1	0.2125

- The NVL function forces group functions to include null values:

```
SELECT AVG(NVL(commission_pct, 0))
FROM employees;
```

2



	Avg(NVL(COMMISSION_PCT,0))
1	0.0425

All group functions ignore null values in the column.

However, the NVL function forces group functions to include null values.

Examples

- The average is calculated based on *only* those rows in the table in which a valid value is stored in the `COMMISSION_PCT` column. The average is calculated as the total commission that is paid to all employees divided by the number of employees receiving a commission (four).
- The average is calculated based on *all* rows in the table, regardless of whether null values are stored in the `COMMISSION_PCT` column. The average is calculated as the total commission that is paid to all employees divided by the total number of employees in the company (20).

For Instructor Use Only.
This document should not be distributed



Grouping Rows

For Instructor Use Only.
This document should not be distributed

Creating Groups of Data

EMPLOYEES

	DEPARTMENT_ID	SALARY
1	10	4400
2	20	13000
3	20	6000
4	50	2500
5	50	2600
6	50	3100
7	50	3500
8	50	5800
9	60	9000
10	60	6000
11	60	4200
12	80	11000
13	80	8600
...		
18	110	8300
19	110	12000
20	(null)	7000

4400
9500
3500
6400
10033

Average salary in the
EMPLOYEES table for
each department

	DEPARTMENT_ID	AVG(SALARY)
1	(null)	7000
2	20	9500
3	90	19333.333333333333...
4	110	10150
5	50	3500
6	80	10033.333333333333...
7	10	4400
8	60	6400

Until this point in the discussion, you have observed that all group functions have treated the table as one large group of information. At times, however, you need to divide the table of information into smaller groups. This can be done by using the `GROUP BY` clause.

Creating Groups of Data: GROUP BY Clause Syntax

You can divide the rows in a table into smaller groups by using the `GROUP BY` clause.

```
SELECT    column, group_function(column)
FROM      table
[WHERE    condition]
[GROUP BY group_by_expression or position]
[ORDER BY column];
```



You can use the `GROUP BY` clause to divide the rows in a table into groups. You can then use the group functions to return summary information for each group.

In the syntax:

group_by_expression

Specifies the columns whose values determine the basis for grouping rows

Let us look at some guidelines for using the `GROUP BY` clause:

- If you include a group function in a `SELECT` clause, you cannot select an individual column as well, *unless* the individual column appears in the `GROUP BY` clause. You receive an error message if you fail to include the column list in the `GROUP BY` clause.
- Using a `WHERE` clause, you can exclude rows before dividing them into groups.
- You can substitute `column` with an expression in the `SELECT` statement.
- You must include the `columns` in the `GROUP BY` clause.
- You cannot use a column alias in the `GROUP BY` clause unless you are using Oracle Database version 23c or later.

Using the GROUP BY Clause

All the columns in the `SELECT` list that are not in group functions must be in the `GROUP BY` clause.

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id ;
```

```
SELECT department_id deptid, AVG(salary)
FROM employees
GROUP BY deptid ;
```



	DEPARTMENT_ID	Avg(SALARY)
1	(null)	7000
2	90	19333.3333333333333333333333333333
3	20	9500
4	110	10150
5	50	3500
6	80	10033.3333333333333333333333333333
7	60	6400
8	10	4400

When using the `GROUP BY` clause, ensure that all columns in the `SELECT` list that are not group functions are included in the `GROUP BY` clause. The example in the slide displays the department number and the average salary for each department. Here is how this `SELECT` statement, containing a `GROUP BY` clause, is evaluated:

- The `SELECT` clause specifies the columns to be retrieved as follows:
 - Department number column in the `EMPLOYEES` table
 - The average of all salaries in the group that you specified in the `GROUP BY` clause
- The `FROM` clause specifies the tables that the database must access: the `EMPLOYEES` table.
- The `WHERE` clause specifies the rows to be retrieved. Because there is no `WHERE` clause, all rows are retrieved by default.
- The `GROUP BY` clause specifies how the rows should be grouped. The rows are grouped by department number, so the `AVG` function that is applied to the salary column calculates the average salary for each department.

Note: To order the query results in ascending or descending order, include the `ORDER BY` clause in the query.

Using the GROUP BY Column Position

The `GROUP BY` column position is supported with the `GROUP_BY_POSITION_ENABLED` parameter.

```
ALTER SYSTEM SET group_by_position_enabled = true;
```

```
ALTER SESSION SET group_by_position_enabled = true;
```

```
SELECT department id, AVG(salary)
FROM employees
GROUP BY 1;
```

[illegible]

Starting with Oracle Database 23c, when using the `GROUP BY` clause, you can group by the column position. This requires that the `GROUP_BY_POSITION_ENABLED` parameter is set to `true`. This offers the same functionality that is available in other systems.

Note: The parameter may be set at either the system or the session level depending on your permissions.

Using the GROUP BY Clause

The `GROUP BY` column does not have to be in the `SELECT` list.

```
SELECT    AVG(salary)
FROM      employees
GROUP BY  department_id ;
```

[illegible]

You do not have to include the `GROUP BY` column in the `SELECT` clause. For example, the `SELECT` statement in the slide displays the average salaries for each department without displaying the respective department numbers. Without the department numbers, however, the results do not look meaningful.

You can also use the group function in the `ORDER BY` clause:

- `SELECT department_id, AVG(salary)`
- `FROM employees`
- `GROUP BY department_id`
- `ORDER BY AVG(salary);`

Grouping by More Than One Column

EMPLOYEES

	DEPARTMENT_ID	JOB_ID	SALARY
1	10	AD_ASST	4400
2	20	MK_MAN	13000
3	20	MK_REP	6000
4	50	ST_CLERK	2500
5	50	ST_CLERK	2600
6	50	ST_CLERK	3100
7	50	ST_CLERK	3500
8	50	ST_MAN	5800
9	60	IT_PROG	9000
10	60	IT_PROG	6000
11	60	IT_PROG	4200
12	80	SA_REP	11000
13	80	SA_REP	8600
14	80	SA_MAN	10500
...			
19	110	AC_MGR	12000
20	(null)	SA_REP	7000

Add the salaries in the EMPLOYEES table for each job, grouped by department.

	DEPARTMENT_ID	JOB_ID	SUM(SALARY)
1	110	AC_ACCOUNT	8300
2	110	AC_MGR	12008
3	10	AD_ASST	4400
4	90	AD_PRES	24000
5	90	AD_VP	34000
6	60	IT_PROG	19200
7	20	MK_MAN	13000
8	20	MK_REP	6000
9	80	SA_MAN	10500
10	80	SA_REP	19600
11	(null)	SA_REP	7000
12	50	ST_CLERK	11700
13	50	ST_MAN	5800

Sometimes, you need to see results for groups within groups. The slide shows a report that displays the total salary that is paid to each job title in each department.

The EMPLOYEES table is grouped first by the department number and then by the job title within that grouping. For example, the four stock clerks in department 50 are grouped together, and a single result (total salary) is produced for all stock clerks in the group.

The following SELECT statement returns the result shown in the slide:

- SELECT department_id, job_id, SUM(salary)
- FROM employees
- GROUP BY department_id, job_id
- ORDER BY job_id;

Using the GROUP BY Clause on Multiple Columns

```
SELECT department_id, job_id, SUM(salary)
FROM employees
WHERE department_id > 40
GROUP BY department_id, job_id
ORDER BY department_id;
```



	DEPARTMENT_ID	JOB_ID	SUM(SALARY)
1	50	ST_CLERK	11700
2	50	ST_MAN	5800
3	60	IT_PROG	19200
4	80	SA_MAN	10500
5	80	SA_REP	19600
6	90	AD_PRES	24000
7	90	AD_VP	34000
8	110	AC_ACCOUNT	8300
9	110	AC_MGR	12008



You can return summary results for groups and subgroups by listing multiple GROUP BY columns. The GROUP BY clause groups rows but does not guarantee the order of the result set. To order the groupings, use the ORDER BY clause.

In the example in the slide, the SELECT statement that contains a GROUP BY clause is evaluated as follows:

- The SELECT clause specifies the columns to be retrieved:
 - DEPARTMENT_ID in the EMPLOYEES table
 - JOB_ID in the EMPLOYEES table
 - The sum of all salaries in the group that you specified in the GROUP BY clause
- The FROM clause specifies the tables that the database must access: the EMPLOYEES table.
- The WHERE clause reduces the result set to those rows where DEPARTMENT_ID is greater than 40.
- The GROUP BY clause specifies how you must group the resulting rows:
 - First, the rows are grouped by DEPARTMENT_ID.
 - Second, the rows are grouped by JOB ID in the DEPARTMENT_ID groups.
- The ORDER BY clause sorts the results by DEPARTMENT_ID.

Note: The SUM function is applied to the salary column for all job IDs in the result set in each DEPARTMENT_ID group. Also, note that the SA_REP row is not returned. The DEPARTMENT_ID for this row is NULL and, therefore, does not meet the WHERE condition.

Illegal Queries Using Group Functions

Any column or expression in the `SELECT` list that is not an aggregate function must be in the `GROUP BY` clause:

```
SELECT department_id, COUNT(last_name)
FROM employees;
```

1

A `GROUP BY` clause must be added to count the last names for each `department_id`.

ORA-00937: not a single-group group function
00937. 00000 - "not a single-group group function"



```
SELECT department_id, job_id, COUNT(last_name)
FROM employees
GROUP BY department_id;
```

2

Either add `job_id` in the `GROUP BY` clause or remove the `job_id` column from the `SELECT` list.

ORA-00979: not a GROUP BY expression
00979. 00000 - "not a GROUP BY expression"



1. Whenever you use a mixture of individual items (`DEPARTMENT_ID`) and group functions (`COUNT`) in the same `SELECT` statement, you must include a `GROUP BY` clause that specifies the individual items (in this case, `DEPARTMENT_ID`). If the `GROUP BY` clause is missing, the error message “not a single-group group function” appears and an asterisk (*) points to the offending column. You can correct the error in the first example in the slide by adding the `GROUP BY` clause:
 - `SELECT department_id, COUNT(last_name)`
 - `FROM employees`
 - `GROUP BY department_id;`
2. Any column or expression in the `SELECT` list that is not an aggregate function must be in the `GROUP BY` clause. In the second example in the slide, `JOB_ID` is neither in the `GROUP BY` clause nor is it being used by a group function, so there is a “not a `GROUP BY` expression” error. You can correct the error in the second slide example by adding `JOB_ID` in the `GROUP BY` clause.
 - `SELECT department_id, job_id, COUNT(last_name)`
 - `FROM employees`
 - `GROUP BY department_id, job_id;`

Illegal Queries Using Group Functions in a WHERE Clause

- You use the `HAVING` clause to restrict groups using a group function.
- You cannot use group functions in the `WHERE` clause.

```
SELECT department_id, AVG(salary)
FROM employees
WHERE AVG(salary) > 8000
GROUP BY department_id;
```



ORA-00934: group function is not allowed here
00934. 00000 - "group function is not allowed here"
*Cause:
*Action:
Error at Line: 3 Column: 9

Cannot use a group
function in a `WHERE` clause

You cannot use a group function in a `WHERE` clause. The `SELECT` statement in the example in the slide results in an error because it uses the `AVG()` function in the `WHERE` clause to restrict the display of the average salaries of those departments that have an average salary greater than \$8,000.

However, you can correct the error in the example by putting the group function in the `HAVING` clause to restrict groups:

- `SELECT department_id, AVG(salary)`
- `FROM employees`
- `GROUP BY department_id`
- `HAVING AVG(salary) > 8000;`

Restricting Group Results

EMPLOYEES

	DEPARTMENT_ID	SALARY
1	10	4400
2	20	13000
3	20	6000
4	50	2500
5	50	2600
6	50	3100
7	50	3500
8	50	5800
9	60	9000
10	60	6000
11	60	4200
12	80	11000
13	80	8600
...		
18	110	8300
19	110	12000
20	(null)	7000

The maximum salary per department when it is greater than \$10,000

	DEPARTMENT_ID	MAX(SALARY)
1	20	13000
2	90	24000
3	110	12000
4	80	11000

You use the `HAVING` clause to restrict groups in the same way that you use the `WHERE` clause to restrict the rows that you select. To find the maximum salary in each of the departments that have a maximum salary greater than \$10,000, you need to do the following:

1. Find the maximum salary for each department by grouping by department number.
2. Restrict the groups to the departments with a maximum salary greater than \$10,000.

Restricting Group Results with the HAVING Clause

When you use the `HAVING` clause, the Oracle server restricts groups:

- Rows are grouped.
- The group function is applied.
- Groups matching the `HAVING` clause are displayed.

```
SELECT    column, group_function
FROM      table
[WHERE    condition]
[GROUP BY group by expression or position]
[HAVING   group_condition]
[ORDER BY column];
```

You use the `HAVING` clause to specify the groups that are to be displayed, thus further restricting the groups on the basis of aggregate information.

The Oracle server performs the following steps when you use the `HAVING` clause:

- Rows are grouped.
- The group function is applied to the group.
- The groups that match the criteria in the `HAVING` clause are displayed.

In the syntax, *group_condition* restricts the groups of rows returned to those groups for which the specified condition is true.

The `HAVING` clause can precede the `GROUP BY` clause, but it is recommended that you place the `GROUP BY` clause first because it is more logical. Groups are formed and group functions are calculated before the `HAVING` clause is applied to the groups in the `SELECT` list.

Note: The `WHERE` clause restricts rows, whereas the `HAVING` clause restricts groups.

Using the HAVING Clause

```
SELECT department_id, MAX(salary)
FROM employees
GROUP BY department_id
HAVING MAX(salary) > 10000 ;
```



	DEPARTMENT_ID	MAX(SALARY)
1	90	24000
2	20	13000
3	110	12008
4	80	11000

The example in the slide displays the department numbers and maximum salaries for departments with a maximum salary greater than \$10,000.

You can use the `GROUP BY` clause without using a group function in the `SELECT` list. If you restrict rows based on the result of a group function, you must have a `GROUP BY` clause as well as the `HAVING` clause.

The following example displays the department numbers and average salaries for departments with a maximum salary greater than \$10,000:

- `SELECT department_id, AVG(salary)`
- `FROM employees`
- `GROUP BY department_id`
- `HAVING max(salary) > 10000;`

Using the HAVING Clause

1

```
SELECT  job_id, SUM(salary) PAYROLL
FROM    employees
WHERE   job_id NOT LIKE '%REP%'
GROUP BY job_id
HAVING  SUM(salary) > 13000
ORDER BY SUM(salary);
```

2

```
SELECT  job_id, SUM(salary) PAYROLL
FROM    employees
WHERE   job_id NOT LIKE '%REP%'
GROUP BY job_id
HAVING  PAYROLL > 13000
ORDER BY SUM(salary);
```



	JOB_ID	PAYROLL
1	IT_PROG	19200
2	AD_PRES	24000
3	AD_VP	34000

The first example in the slide displays the `JOB_ID` and total monthly salary for each job that has a total payroll exceeding \$13,000. The example excludes sales representatives and sorts the list by the total monthly salary. The second example shows the use of the column alias in the `HAVING` clause.

Note: Using the column alias is available since Oracle Database 23c. It is not supported in earlier releases.



Nesting Group Functions

For Instructor Use Only.
This document should not be distributed



```
1 MAX(AVG(SALARY))
```

Note that the `GROUP BY` clause is mandatory when nesting group functions.

Summary



Group functions

Grouping rows

Nesting group functions

There are several group functions available in SQL, such as `AVG`, `COUNT`, `MAX`, `MIN`, `SUM`, `LISTAGG`, `STDDEV`, and `VARIANCE`.

You can create subgroups by using the `GROUP BY` clause. Further, groups can be restricted using the `HAVING` clause.

Place the `HAVING` and `GROUP BY` clauses after the `WHERE` clause in a statement. You can have either the `GROUP BY` clause or the `HAVING` clause first, as long as they follow the `WHERE` clause. Place the `ORDER BY` clause at the end.

The Oracle server evaluates the clauses in the following order:

1. If the statement contains a `WHERE` clause, the server establishes the candidate rows.
2. The server identifies the groups that are specified in the `GROUP BY` clause.
3. The `HAVING` clause further restricts result groups that do not meet the group criteria in the `HAVING` clause.

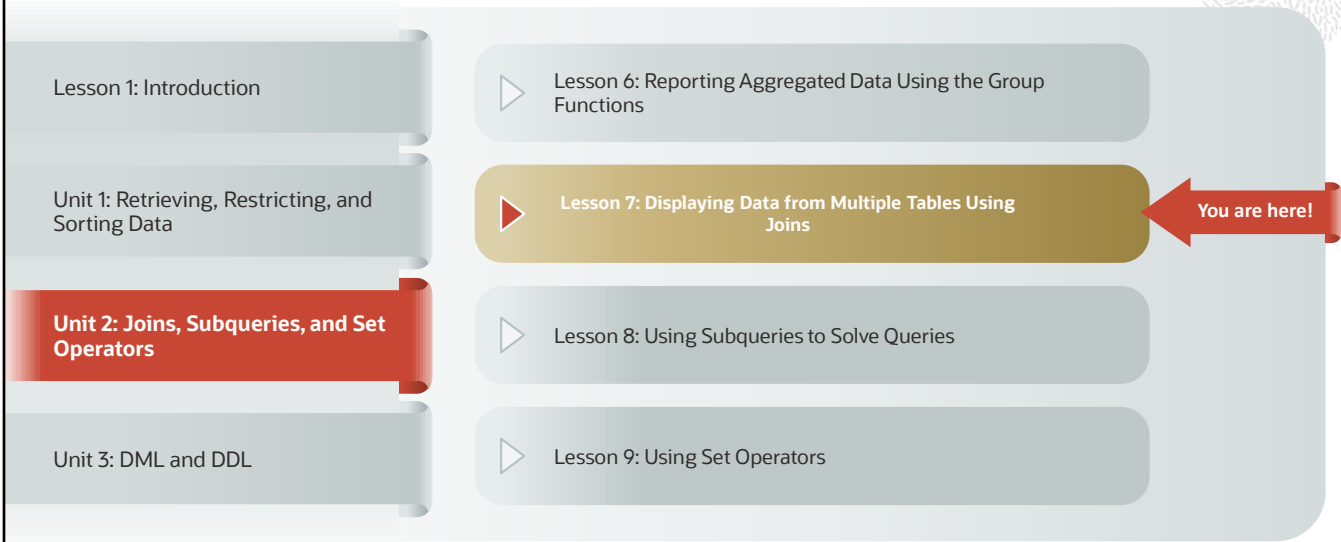
Note: For a complete list of group functions, see *Oracle Database SQL Language Reference* for 23c database.

For Instructor Use Only.
This document should not be distributed

Displaying Data from Multiple Tables Using Joins



Course Roadmap



In Unit 2, you will learn to use:

- SQL statements to query and display data from multiple tables using Joins
- Subqueries when the condition is unknown
- Group functions to aggregate data
- Set operators

Objectives



Types of JOINS and their syntax

Natural join

Join with the `USING` clause

Join with the `ON` clause

Self-join

Nonequijoins

Outer join

Cartesian product

In this lesson, you learn how to obtain data from more than one table. A *join* is used to view information from multiple tables. Therefore, you can *join* tables together to view information from more than one table.

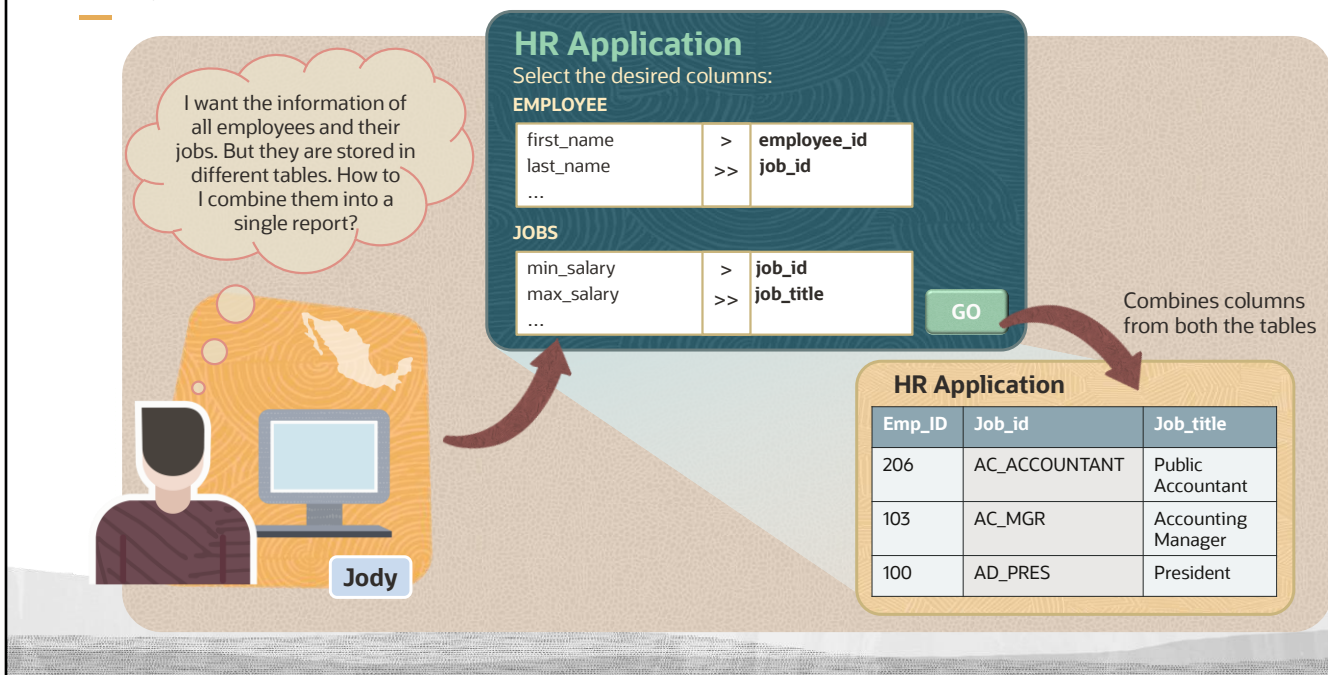
Note: For information about joins, refer to the “SQL Queries and Subqueries: Joins” section in *Oracle Database SQL Language Reference* for 23c database.



Types of Joins and Their Syntax

For Instructor Use Only.
This document should not be distributed

Why Join?



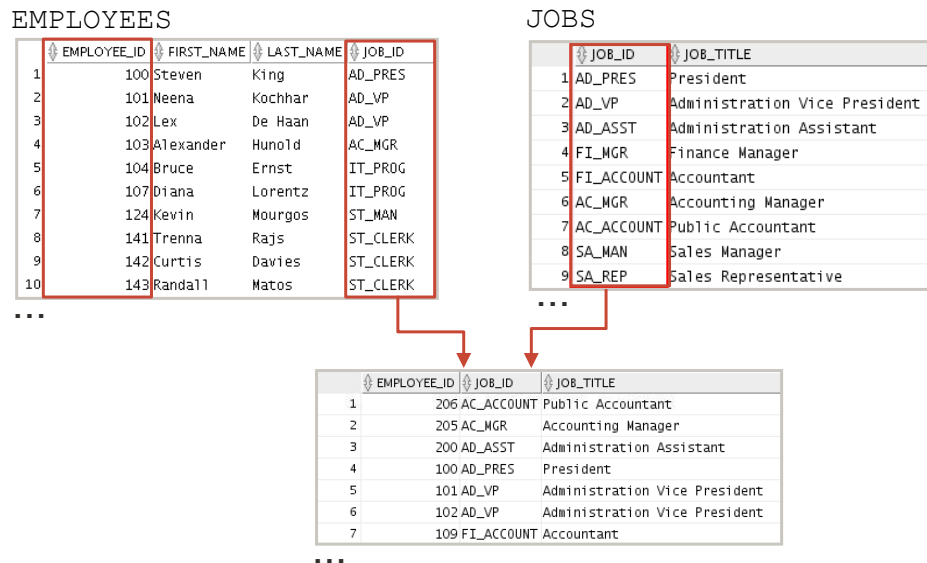
Jody, an HR Manager located in Mexico, wants a report of all employees working in the organization, their respective job IDs, and job titles. You cannot get this report by querying a single table because the employee information is in one table and the job information in another table.

What is the solution for Jody's query?

You will need to write a SQL statement to fetch the required information from two different tables. This SQL statement will fetch the employee IDs from the `EMPLOYEE` table and Job Title from the `JOBS` table by using `JOB ID` as the common column. The result of the SQL Query is a single report consisting of the following columns: Employee ID, Job ID, and Job title.

The following slides explain Joins in detail, and how to use them.

Obtaining Data from Multiple Tables



Sometimes you need to use data from more than one table. A join is a query that combines rows from two or more tables or views. Oracle Database performs a join whenever multiple tables appear in the **FROM** clause of the query.

In the example in the slide, the report displays data from two separate tables:

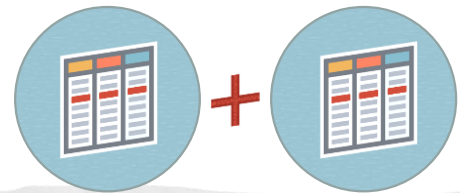
- Employee IDs exist in the **EMPLOYEES** table.
- Job IDs exist in both the **EMPLOYEES** and **JOBS** tables.
- Job titles exist in the **JOBS** table.

To produce the report, you need to link the **EMPLOYEES** and **JOBS** tables and access data from both of them.

Types of Joins

Joins that are compliant with the `ANSI` standard:

- Natural join with the `NATURAL JOIN` clause
- Join with the `USING` clause
- Join with the `ON` clause
- OUTER joins:
 - `LEFT OUTER JOIN`
 - `RIGHT OUTER JOIN`
 - `FULL OUTER JOIN`
- Cross joins



A join is a query that combines rows from two or more tables or views. The types of joins are listed above that are compliant with the `ANSI` standard.

Joining Tables Using SQL Syntax

Use a join to query data from more than one table:

```
SELECT table1.column, table2.column
FROM   table1
[NATURAL JOIN table2] |
[JOIN table2 USING (column_name)] |
[JOIN table2 ON (table1.column_name = table2.column_name)] |
[LEFT|RIGHT|FULL OUTER JOIN table2
 ON (table1.column_name = table2.column_name)] |
[CROSS JOIN table2];
```

In the syntax:

- `table1.column` denotes the table and the column from which data is retrieved
- `NATURAL JOIN` joins two tables based on the same column name
- `JOIN table2 USING column_name` performs an equijoin based on the column name
- `JOIN table2 ON table1.column_name = table2.column_name` performs an equijoin based on the condition in the `ON` clause
- `LEFT/RIGHT/FULL OUTER` is used to perform OUTER joins
- `CROSS JOIN` returns a Cartesian product from the two tables

For more information, see the section titled “SELECT” in *Oracle Database SQL Language Reference* for 18c database.



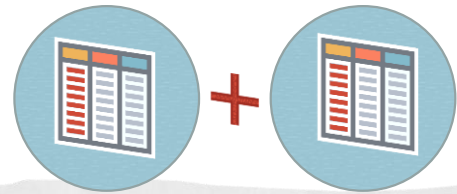
Natural Join

For Instructor Use Only.
This document should not be distributed

Creating Natural Joins

- The `NATURAL JOIN` clause is based on all the columns that have the same name in two tables.
- It selects rows from the two tables that have equal values in all matched columns.
- If the columns having the same names have different data types, an error is returned.

```
SELECT * FROM table1 NATURAL JOIN table2;
```



You can join tables automatically based on the columns in the two tables that have matching data types and names. You do this by using the `NATURAL JOIN` clause.

Note: The join can happen on only those columns that have the same names and data types in both tables. If the columns have the same name but different data types, the `NATURAL JOIN` clause causes an error.

Retrieving Records with Natural Joins

```
SELECT employee_id, first_name, job_id, job_title
from employees NATURAL JOIN jobs;
```



EMPLOYEE_ID	FIRST_NAME	JOB_ID	JOB_TITLE
1	206 William	AC_ACCOUNT	Public Accountant
2	205 Shelley	AC_MGR	Accounting Manager
3	200 Jennifer	AD_ASST	Administration Assistant
4	100 Steven	AD PRES	President
5	102 Lex	AD_VP	Administration Vice President
6	101 Neena	AD_VP	Administration Vice President
7	103 Alexander	IT_PROG	Programmer
8	104 Bruce	IT_PROG	Programmer
9	107 Diana	IT_PROG	Programmer
10	201 Michael	MK_MAN	Marketing Manager
11	202 Pat	MK_REP	Marketing Representative
12	149 Eleni	SA_MAN	Sales Manager
13	174 Ellen	SA_REP	Sales Representative
14	178 Kimbberly	SA_REP	Sales Representative
15	176 Jonathon	SA_REP	Sales Representative
16	143 Randall	ST_CLERK	Stock Clerk
17	142 Curtis	ST_CLERK	Stock Clerk
18	141 Trena	ST_CLERK	Stock Clerk
19	144 Peter	ST_CLERK	Stock Clerk
20	124 Kevin	ST_MAN	Stock Manager

In the example in the slide, observe that the JOBS table is joined to the EMPLOYEES table by the JOB_ID column, which is the only column of the same name in both tables. If other common columns were present, the join would have used them all.

Natural Joins with a WHERE Clause

Additional restrictions on a natural join are implemented by using a WHERE clause. The following example limits the rows of output to those with a DEPARTMENT_ID equal to 20 or 50:

```
SELECT  department_id, department_name,
        location_id, city
FROM    departments
NATURAL JOIN locations
WHERE   department_id IN (20, 50);
```




Join with the `USING` Clause

For Instructor Use Only.
This document should not be distributed

Creating Joins with the USING Clause

Use the `USING` clause to:

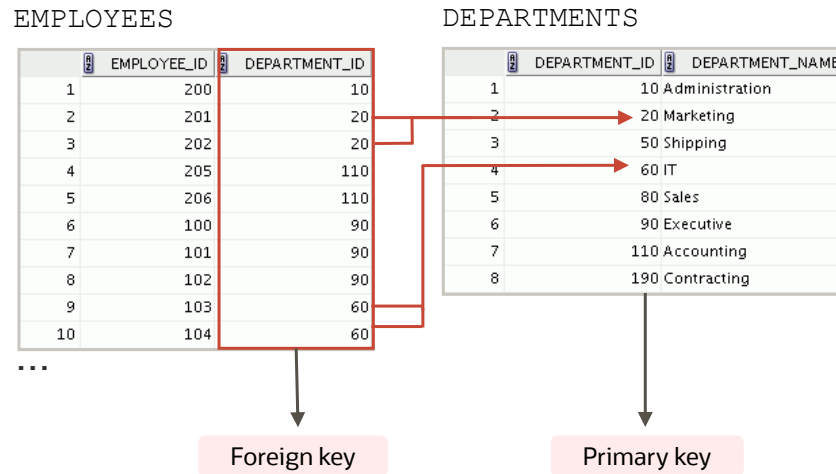
- Specify the columns for the equijoin, if several columns have the same names but the data types do not match
- Match only one column when more than one column matches



Natural joins use all columns with matching names and data types to join the tables. You can use the `USING` clause to specify only those columns that should be used for an equijoin.

An equijoin is a join containing an equality operator. A natural join is a type of equijoin.

Joining Column Names



To determine an employee's department name, you compare the value in the `DEPARTMENT_ID` column in the `EMPLOYEES` table with the `DEPARTMENT_ID` values in the `DEPARTMENTS` table.

The relationship between the `EMPLOYEES` and `DEPARTMENTS` tables is an *equijoin*; that is, values in the `DEPARTMENT_ID` column in both the tables must be equal. Frequently, this type of join involves primary and foreign key complements.

Retrieving Records with the USING Clause

```
SELECT employee_id, last_name,  
       location_id, department_id  
FROM   employees JOIN departments  
       USING (department_id) ;
```



EMPLOYEE_ID	LAST_NAME	LOCATION_ID	DEPARTMENT_ID
1	200 Whalen	1700	10
2	201 Hartstein	1800	20
3	202 Fay	1800	20
4	144 Vargas	1500	50
5	143 Matos	1500	50
6	142 Davies	1500	50
7	141 Raju	1500	50
8	124 Mourgos	1500	50
...			
18	206 Gietz	1700	110
19	205 Higgins	1700	110

In the example in the slide, the `DEPARTMENT_ID` columns in the `EMPLOYEES` and `DEPARTMENTS` tables are joined, and thus `LOCATION_ID` of the department where an employee works is shown.

Note that the `EMPLOYEE_ID` and `LAST_NAME` columns belong to the `EMPLOYEES` table, whereas `LOCATION_ID` belongs to the `DEPARTMENTS` table and the `DEPARTMENT_ID` column belongs to both these tables.

Qualifying Ambiguous Column Names

- Use table prefixes to:
 - Qualify column names that are in multiple tables
 - Increase the speed of parsing of a statement
- Use table aliases, instead of full table name prefixes.
 - Table alias gives a table a shorter name.
- Use column aliases to distinguish columns that have identical names but reside in different tables.



When joining two or more tables, you need to qualify the names of the columns with the table name to avoid ambiguity. Without table prefixes, the `DEPARTMENT_ID` column in the `SELECT` list could be from either the `DEPARTMENTS` table or the `EMPLOYEES` table. It is necessary to add the table prefix to execute your query. If there are no common column names between the two tables, there is no need to qualify the columns. Basically, using the table prefix increases the speed of parsing of the statement, because you tell the Oracle server exactly where to find the columns.

However, qualifying column names with table names can be time-consuming, particularly if the table names are lengthy. Instead, you can use *table aliases*. Just as a column alias gives a column another name, a table alias gives a table another name. Table aliases help to keep SQL code smaller; therefore, use less memory.

The table name is specified in full, followed by a space, and then the table alias. For example, the `EMPLOYEES` table can be given an alias of `e` and the `DEPARTMENTS` table an alias of `d`.

Guidelines

- Table aliases can be up to 30 characters in length, but shorter aliases are better than longer ones.
- If a table alias is used for a particular table name in the `FROM` clause, that table alias must be substituted for the table name throughout the `SELECT` statement.
- Table aliases should be meaningful.
- The table alias is valid for only the current `SELECT` statement.

Using Table Aliases with the USING Clause in Oracle

- Do not qualify a column that is used in the `NATURAL` join or a join with a `USING` clause.
- If the same column is used elsewhere in the SQL statement, do not alias it.

```
SELECT l.city, d.department_name
FROM   locations l JOIN departments d
USING (location_id)
WHERE  d.location_id = 1400;
```



ORA-25154: column part of USING clause cannot have qualifier
25154. 00000 - "column part of USING clause cannot have qualifier"
*Cause: Columns that are used for a named-join (either a `NATURAL` join or a join with a `USING` clause) cannot have an explicit qualifier.
*Action: Remove the qualifier.
Error at Line: 4 Column: 6

When joining with the `USING` clause in Oracle Database, you cannot qualify a column that is used in the `USING` clause itself. Furthermore, if that column is used anywhere in the SQL statement, you cannot alias it. For example, in the query mentioned in the slide, you should not alias the `LOCATION_ID` column in the `WHERE` clause because the column is used in the `USING` clause.

The columns that are referenced in the `USING` clause should not have a qualifier (table name or alias) anywhere in the SQL statement. For example, the following statement is valid:

```
SELECT l.city, d.department_name
FROM   locations l JOIN departments d USING (location_id)
WHERE  location_id = 1400;
```

The columns that are common in both the tables, but not used in the `USING` clause, must be prefixed with a table alias; otherwise, you get the “column ambiguously defined” error.

In the following statement, `manager_id` is present in both the `employees` and `departments` table; if `manager_id` is not prefixed with a table alias, it gives a “column ambiguously defined” error.

The following statement is valid:

```
SELECT first_name, d.department_name, d.manager_id
FROM   employees e JOIN departments d USING (department_id)
WHERE  department_id = 50;
```



Join with the ON Clause

For Instructor Use Only.
This document should not be distributed

Creating Joins with the ON Clause

- The join condition for the natural join is basically an equijoin of all columns with the same name.
- Use the `ON` clause to:
 - Specify arbitrary conditions or specify the columns to join
 - Separate the join condition from other search conditions
- The `ON` clause makes code easy to understand.



Use the `ON` clause to specify a join condition. With this, you can specify join conditions separate from any search or filter conditions in the `WHERE` clause.

Retrieving Records with the ON Clause

```
SELECT e.employee_id, e.last_name, e.department_id,  
       d.department_id, d.location_id  
FROM   employees e JOIN departments d  
ON     (e.department_id = d.department_id);
```



	EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_ID_1	LOCATION_ID
1	200	Whalen	10	10	1700
2	201	Hartstein	20	20	1800
3	202	Fay	20	20	1800
4	124	Mourgos	50	50	1500
5	144	Vargas	50	50	1500
6	143	Matos	50	50	1500
7	142	Davies	50	50	1500
8	141	Rajs	50	50	1500
9	107	Lorentz	60	60	1400
10	104	Ernst	60	60	1400
11	103	Hunold	60	60	1400

...

In this example, the `DEPARTMENT_ID` columns in the `EMPLOYEES` and `DEPARTMENTS` table are joined using the `ON` clause. Wherever a department `ID` in the `EMPLOYEES` table equals a department `ID` in the `DEPARTMENTS` table, the row is returned. The table alias is necessary to qualify the matching column names.

You can also use the `ON` clause to join columns that have different names. The parentheses around the joined columns, as in the example in the slide `(e.department_id = d.department_id)`, is optional. So, even `ON e.department_id = d.department_id` will work.

Note: When you use the Execute Statement icon to run the query, SQL Developer suffixes a “_1” to differentiate between the two `DEPARTMENT_ID`s.

Creating Three-Way Joins

```
SELECT employee_id, city, department_name
FROM   employees e
JOIN   departments d
ON     d.department_id = e.department_id
JOIN   locations l
ON     d.location_id = l.location_id;
```



	EMPLOYEE_ID	CITY	DEPARTMENT_NAME
1	100	Seattle	Executive
2	101	Seattle	Executive
3	102	Seattle	Executive
4	103	Southlake	IT
5	104	Southlake	IT
6	107	Southlake	IT
7	124	South San Francisco	Shipping
8	141	South San Francisco	Shipping
9	142	South San Francisco	Shipping

...

A three-way join is a join of three tables. Here, the first join to be performed is `employees JOIN departments`. The first join condition can reference columns in `employees` and `departments` but cannot reference columns in `locations`. The second join condition can reference columns from all three tables.

Note: The code example in the slide can also be accomplished with the `USING` clause:

```
SELECT e.employee_id, l.city, d.department_name
FROM   employees e
JOIN   departments d
      USING (department_id)
JOIN   locations l
      USING (location_id);
```

Applying Additional Conditions to a Join

Use the **AND** clause or the **WHERE** clause to apply additional conditions:

```
SELECT e.employee_id, e.last_name, e.department_id,  
       d.department_id, d.location_id  
FROM   employees e JOIN departments d  
ON     (e.department_id = d.department_id)  
AND    e.manager_id = 149 ;
```

OR

```
SELECT e.employee_id, e.last_name, e.department_id,  
       d.department_id, d.location_id  
FROM   employees e JOIN departments d  
ON     (e.department_id = d.department_id)  
WHERE  e.manager_id = 149 ;
```

You can apply additional conditions to the join.

The example shown performs a join on the `EMPLOYEES` and `DEPARTMENTS` tables and, in addition, displays only employees who have a manager `ID` of 149. To add additional conditions to the `ON` clause, you can add `AND` clauses. Alternatively, you can use a `WHERE` clause to apply additional conditions.

Both the queries produce the same output.



Self-Join

For Instructor Use Only.
This document should not be distributed

Joining a Table to Itself

EMPLOYEES (WORKER)

EMPLOYEE_ID	LAST_NAME	MANAGER_ID
200	Whalen	101
201	Hartstein	100
202	Fay	201
205	Higgins	101
206	Gietz	205
100	King	(null)
101	Kochhar	100
102	De Haan	100
103	Hunold	102
104	Ernst	103

...

EMPLOYEES (MANAGER)

EMPLOYEE_ID	LAST_NAME
200	Whalen
201	Hartstein
202	Fay
205	Higgins
206	Gietz
100	King
101	Kochhar
102	De Haan
103	Hunold
104	Ernst

...

MANAGER_ID in the WORKER table is equal to
EMPLOYEE_ID in the MANAGER table.

Sometimes you need to join a table to itself. To find the name of each employee's manager, you need to join the `EMPLOYEES` table to itself or perform a self-join. For example, to find the name of Ernst's manager, you need to find:

- Ernst in the `EMPLOYEES` table by looking at the `LAST_NAME` column
- The manager number for Ernst by looking at the `MANAGER_ID` column. Ernst's manager number is 103.
- The name of the manager with `EMPLOYEE_ID` 103 by looking at the `LAST_NAME` column. Hunold's employee number is 103, so Hunold is Ernst's manager.

In this process, you look in the table twice. The first time you look in the table to find Ernst in the `LAST_NAME` column and the `MANAGER_ID` value of 103. The second time you look in the `EMPLOYEE_ID` column to find 103 and the `LAST_NAME` column to find Hunold.

Self-Joins Using the ON Clause

```
SELECT worker.last_name emp, manager.last_name mgr
FROM   employees worker JOIN employees manager
ON     (worker.manager_id = manager.employee_id);
```



	EMP	MGR
1	Hunold	De Haan
2	Fay	Hartstein
3	Gietz	Higgins
4	Lorentz	Hunold
5	Ernst	Hunold
6	Zlotkey	King
7	Mourgos	King
8	Kochhar	King

...



The ON clause can also be used to join columns that have different names, within the same table or in a different table.

The example shown is a self-join of the EMPLOYEES table, based on the EMPLOYEE_ID and MANAGER_ID columns.



Nonequijoins

For Instructor Use Only.
This document should not be distributed

Nonequijoins

EMPLOYEES

	LAST_NAME	SALARY
1	Whalen	4400
2	Hartstein	13000
3	Fay	6000
4	Higgins	12000
5	Gietz	8300
6	King	24000
7	Kochhar	17000
8	De Haan	17000
9	Hunold	9000
10	Ernst	6000
...		
19	Taylor	8600
20	Grant	7000

JOB_GRADES

	GRADE_LEVEL	LOWEST_SAL	HIGHEST_SAL
1	A	1000	2999
2	B	3000	5999
3	C	6000	9999
4	D	10000	14999
5	E	15000	24999
6	F	25000	40000

The JOB_GRADES table defines the LOWEST_SAL and HIGHEST_SAL range of values for each GRADE_LEVEL.

Therefore, the GRADE_LEVEL column can be used to assign grades to each employee based on their salary.

A nonequijoin is a join condition containing something other than an equality operator.

The relationship between the EMPLOYEES table and the JOB_GRADES table is an example of a nonequijoin. The SALARY column in the EMPLOYEES table ranges between the values in the LOWEST_SAL and HIGHEST_SAL columns of the JOB_GRADES table. Therefore, each employee can be graded based on their salary. The relationship is obtained using an operator other than the equality (=) operator.

Retrieving Records with Nonequijoins

```
SELECT e.last_name, e.salary, j.grade_level
FROM employees e JOIN job_grades j
ON e.salary
   BETWEEN j.lowest_sal AND j.highest_sal;
```



	LAST_NAME	SALARY	GRADE_LEVEL
1	Vargas	2500	A
2	Matos	2600	A
3	Davies	3100	B
4	Rajs	3500	B
5	Lorentz	4200	B
6	Whalen	4400	B
7	Mourgos	5800	B
8	Ernst	6000	C
9	Fay	6000	C
10	Grant	7000	C

The example in the slide creates a nonequijoin to evaluate an employee's salary grade. The salary must be *between* any pair of the low and high salary ranges.

It is important to note that all employees appear exactly once when this query is executed. No employee is repeated in the list. There are two reasons for this:

- None of the rows in the `JOB_GRADES` table contain grades that overlap. That is, the salary value for an employee can lie only between the low-salary and high-salary values of one of the rows in the salary grade table.
- All of the employees' salaries lie within the limits provided by the `JOB_GRADES` table. That is, no employee earns less than the lowest value contained in the `LOWEST_SAL` column or more than the highest value contained in the `HIGHEST_SAL` column.

Note: Other conditions (such as `<=` and `>=`) can be used, but `BETWEEN` is the simplest. Remember to specify the low value first and the high value last when using the `BETWEEN` condition. The Oracle server translates the `BETWEEN` condition to a pair of `AND` conditions. Therefore, using `BETWEEN` has no performance benefits, but should be used only for logical simplicity.

Table aliases have been specified in the example in the slide for performance reasons, not because of possible ambiguity.



Outer Join

For Instructor Use Only.
This document should not be distributed

Returning Records with No Direct Match Using OUTER Joins

DEPARTMENTS

	DEPARTMENT_NAME	DEPARTMENT_ID
1	Administration	10
2	Marketing	20
3	Shipping	50
4	IT	60
5	Sales	80
6	Executive	90
7	Accounting	110
8	Contracting	190

Equijoin with EMPLOYEES

	DEPARTMENT_ID	LAST_NAME
1	10	Whalen
2	20	Hartstein
3	20	Fay
4	110	Higgins
5	110	Gietz
6	90	King
7	90	Kochhar
8	90	De Haan
9	60	Hunold
10	60	Ernst

There are no employees in department 190.

Employee "Grant" has not been assigned a department ID.

Therefore, the above two records do not appear in the equijoin result.

If a row does not satisfy a join condition, the row does not appear in the query result.

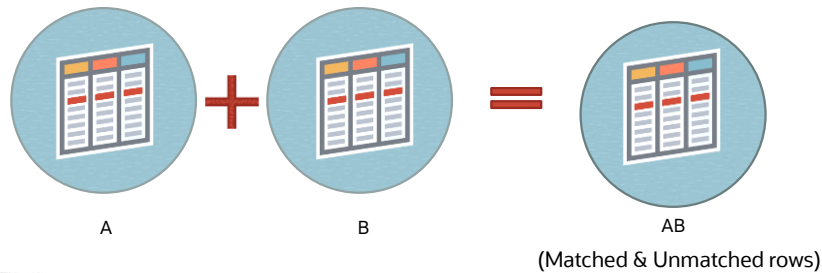
In the example in the slide, a simple equijoin condition is used on the EMPLOYEES and DEPARTMENTS tables to return the result on the right. The result set does not contain the following:

- Department ID 190, because there are no employees with that department ID recorded in the EMPLOYEES table
- The employee with the last name of Grant, because this employee has not been assigned a department ID

To return the department record that does not have any employees, or employees that do not have an assigned department, you can use an OUTER join.

INNER Versus OUTER Joins

- The join of two tables returning only matched rows is called an `INNER` join.
- A join between two tables that returns the results of the `INNER` join as well as the unmatched rows from the left (or right) table is called a `LEFT` (or `RIGHT`) `OUTER` join.
- In Oracle, a join between two tables that returns the results of an `INNER` join as well as the results of a left and right join is a `FULL OUTER JOIN`.



Joining tables with the `NATURAL JOIN`, `USING`, or `ON` clause results in an `INNER` join. Any unmatched rows are not displayed in the output. To return the unmatched rows, you can use an `OUTER` join. An `OUTER` join returns all rows that satisfy the join condition and also returns some or all of those rows from one table for which no rows from the other table satisfy the join condition.

There are three types of `OUTER` joins:

- `LEFT OUTER`
- `RIGHT OUTER`
- `FULL OUTER`

LEFT OUTER JOIN

```
SELECT e.last_name, e.department_id, d.department_name
FROM   employees e LEFT OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```



	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Whalen	10	Administration
2	Fay	20	Marketing
3	Hartstein	20	Marketing
4	Vargas	50	Shipping
5	Matos	50	Shipping
...			
16	Kochhar	90	Executive
17	King	90	Executive
18	Gietz	110	Accounting
19	Higgins	110	Accounting
20	Grant	(null)	(null)

This query retrieves all the rows in the `EMPLOYEES` table, which is the left table, even if there is no match in the `DEPARTMENTS` table.

Basically, a `LEFT OUTER JOIN` will return all the rows that match between the two tables on the given condition and also return the unmatched rows of the left table, that is, the table mentioned first in the SQL statement.

RIGHT OUTER JOIN

```
SELECT e.last_name, d.department_id, d.department_name
FROM   employees e RIGHT OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```



	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Whalen	10	Administration
2	Hartstein	20	Marketing
3	Fay	20	Marketing
4	Davies	50	Shipping
5	Vargas	50	Shipping
6	Rajs	50	Shipping
7	Mourgos	50	Shipping
8	Matos	50	Shipping
...			
18	Higgins	110	Accounting
19	Gietz	110	Accounting
20	(null)	190	Contracting

This query retrieves all the rows in the `departments` table, which is the table at the right, even if there is no match in the `employees` table.

Basically, a `RIGHT OUTER JOIN` will return all the rows that match between the two tables on the given condition and also return the unmatched rows of the right table, that is, the table mentioned second in the SQL statement.

FULL OUTER JOIN in Oracle



```
SELECT e.last_name, d.department_id, d.department_name
FROM   employees e FULL OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```



	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	King	90	Executive
2	Kochhar	90	Executive
3	De Haan	90	Executive
4	Hunold	60	IT

...

15	Grant	(null)	(null)
16	Whalen	10	Administration
17	Hartstein	20	Marketing
18	Fay	20	Marketing
19	Higgins	110	Accounting
20	Gietz	110	Accounting
21	(null)	190	Contracting

This query retrieves all rows in the `EMPLOYEES` table, even if there is no match in the `DEPARTMENTS` table. It also retrieves all rows in the `DEPARTMENTS` table, even if there is no match in the `EMPLOYEES` table.

Basically, a `FULL OUTER JOIN` will return all the rows that match between the two tables on the given condition and also return the unmatched rows of both the tables.



Cartesian Product

For Instructor Use Only.
This document should not be distributed

Cartesian Products

A Cartesian product:

- Is a join of every row of one table to every row of another table
- Generates a large number of rows and the result is rarely useful



A Cartesian product tends to generate a large number of rows and the result is rarely useful. You should, therefore, always include a valid join condition unless you have a specific need to combine all rows from all tables.

Cartesian products are useful for some tests when you need to generate a large number of rows to simulate a reasonable amount of data.

Generating a Cartesian Product

EMPLOYEES (20 rows)

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
1	200 Whalen	10
2	201 Hartstein	20
3	202 Fay	20
4	205 Higgins	110
...		
19	176 Taylor	80
20	178 Grant	(null)

DEPARTMENTS (8 rows)

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
1	10 Administration	1700
2	20 Marketing	1800
3	50 Shipping	1500
4	60 IT	1400
5	80 Sales	2500
6	90 Executive	1700
7	110 Accounting	1700
8	190 Contracting	1700

Cartesian product:
20 x 8 = 160 rows

EMPLOYEE_ID	DEPARTMENT_ID	LOCATION_ID
1	200	10
2	201	20
...		
21	200	10
22	201	20
...		
159	176	80
160	178	(null)

A Cartesian product is generated if a join condition is omitted. The example in the slide displays the employee ID, department ID, and location ID from the `EMPLOYEES` and `DEPARTMENTS` tables. Because no join condition was specified, all rows (20 rows) from the `EMPLOYEES` table are joined with all rows (8 rows) in the `DEPARTMENTS` table, thereby generating 160 rows in the output.

Creating Cross Joins

- A `CROSS JOIN` is a `JOIN` operation that produces a Cartesian product of two tables.
- To create a Cartesian product, specify `CROSS JOIN` in your `SELECT` statement.

```
SELECT last_name, department_name
FROM   employees
CROSS JOIN departments ;
```



	LAST_NAME	DEPARTMENT_NAME
1	Abel	Administration
2	Davies	Administration
3	De Haan	Administration
4	Ernst	Administration
5	Fay	Administration
...		
158	Vargas	Contracting
159	Whalen	Contracting
160	Zlotkey	Contracting

The example in the slide produces a Cartesian product of `EMPLOYEES` and `DEPARTMENTS` tables.

It is a good practice to explicitly state `CROSS JOIN` in your `SELECT` statement when you intend to create a Cartesian product. Therefore, it is very clear that you intend for this to happen and it is not the result of missing joins.

Summary



Types of JOINS and their syntax

Natural join

Join with the USING clause

Join with the ON clause

Self-join

Nonequijoins

Outer join

Cartesian product

There are multiple ways to join tables.

Types of Joins

- Equijoins
- Nonequijoins
- OUTER joins
- Self-joins
- Cross joins
- Natural joins
- Full (or two-sided) OUTER joins

Cartesian Products

A Cartesian product results in the display of all combinations of rows. This is done by either omitting the WHERE clause or specifying the CROSS JOIN clause.

Table Aliases

- Table aliases speed up database access.
- They can help to keep SQL code smaller by conserving memory.
- They are sometimes mandatory to avoid column ambiguity.

For Instructor Use Only.
This document should not be distributed

Using Subqueries to Solve Queries



Course Roadmap

Lesson 1: Introduction

Unit 1: Retrieving, Restricting, and
Sorting Data

**Unit 2: Joins, Subqueries, and Set
Operators**

Unit 3: DML and DDL



Lesson 6: Reporting Aggregated Data Using the Group
Functions



Lesson 7: Displaying Data from Multiple Tables
Using Joins



Lesson 8: Using Subqueries to Solve Queries

You are here!



Lesson 9: Using Set Operators

In Unit 2, you will learn to use:

- SQL statements to query and display data from multiple tables by using joins
- Subqueries when the condition is unknown
- Group functions to aggregate data
- Set operators

Objectives



Subquery: Types, syntax, and guidelines

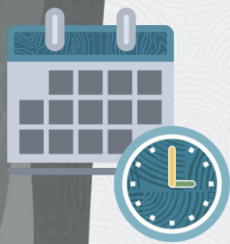
Single-row subqueries

Multiple-row subqueries

Multiple-column subqueries

Null values in a subquery

In this lesson, you learn about the more advanced features of the `SELECT` statement. You can write subqueries in the `WHERE` clause of another SQL statement to obtain values based on an unknown conditional value. This lesson also covers single-row subqueries, multiple-row subqueries, and multiple-column subqueries.



Subquery: Types, Syntax, and Guidelines

For Instructor Use Only.
This document should not be distributed

Using a Subquery to Solve a Problem



Suppose the HR manager wants a report of all employees who were hired after Davies. The HR manager submits a request for the report to the IT department.

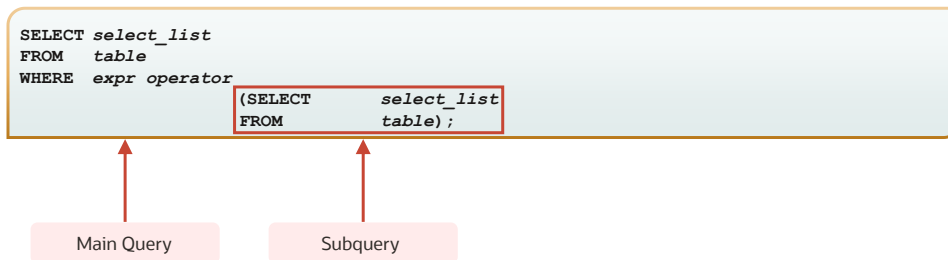
To solve this problem, the IT manager needs *two* queries: one query to find when Davies was hired and another to find who were hired after Davies.

The IT manager can solve this problem by combining the two queries, placing one query *inside* the other query.

The inner query (or *subquery*) returns a value that is used by the outer query (or *main query*).

Subquery Syntax

- The subquery (inner query) executes before the main query (outer query).
- The result of the subquery is used by the main query.



You can build powerful statements out of simple ones by using subqueries. They can be very useful when you need to select rows from a table with a condition that depends on the data in the table itself.

A subquery is a `SELECT` statement that is embedded in the clause of another `SELECT` statement.

You can place the subquery in a number of SQL clauses, including the following:

- `WHERE` clause
- `HAVING` clause
- `FROM` clause

In the syntax:

operator includes a comparison condition such as `>`, `=`, or `IN`

The subquery is often referred to as a nested `SELECT`, sub-`SELECT`, or inner `SELECT` statement. The subquery generally executes first, and its output is used to complete the query condition for the main (or outer) query.

Using a Subquery



```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date > (SELECT hire_date
                   FROM   employees
                   WHERE  last_name = 'Davies');
```

In the example in the slide, the inner query determines the hire date of the employee, Davies. The outer query takes the result of the inner query and uses this result to display all the employees who were hired after Davies.

Rules and Guidelines for Using Subqueries

- Enclose subqueries in parentheses.
- Place subqueries on the right side of the comparison condition for readability. (However, the subquery can appear on either side of the comparison operator.)
- Use single-row operators with single-row subqueries and multiple-row operators with multiple-row subqueries.



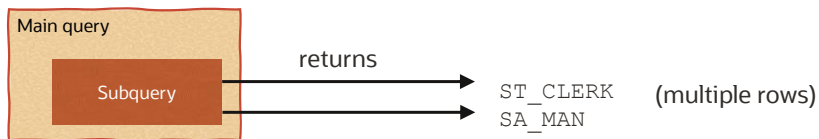
Two classes of comparison conditions are used in subqueries: single-row operators and multiple-row operators.

Types of Subqueries

- Single-row subquery

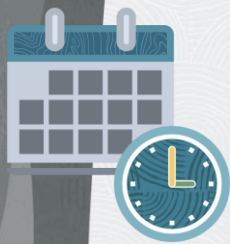


- Multiple-row subquery



- **Single-row subqueries:** Queries that return only one row from the inner `SELECT` statement
- **Multiple-row subqueries:** Queries that return more than one row from the inner `SELECT` statement

Note: There are also multiple-column subqueries, which are queries that return more than one column from the inner `SELECT` statement. They are covered in another section in this course.



Single-Row Subqueries

For Instructor Use Only.
This document should not be distributed

Single-Row Subqueries

- Return only one row
- Use single-row comparison operators:

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to



A single-row subquery is one that returns one row from the inner `SELECT` statement. This type of subquery uses a single-row operator. The table in the slide lists single-row operators.

Example

Display the employees whose job ID is the same as that of employee 141:

```
SELECT last_name, job_id
FROM   employees
WHERE  job_id =
        (SELECT job_id
         FROM   employees
         WHERE  employee_id = 141);
```

	LAST_NAME	JOB_ID
1	Rajs	ST_CLERK
2	Davies	ST_CLERK
3	Matos	ST_CLERK
4	Vargas	ST_CLERK

Executing Single-Row Subqueries

```
SELECT last_name, job_id, salary
FROM employees
WHERE job_id = (SELECT job_id
                FROM employees
                WHERE last_name = 'Taylor')
AND salary > (SELECT salary
              FROM employees
              WHERE last_name = 'Taylor');
```



	LAST_NAME	JOB_ID	SALARY
1	Abel	SA_REP	11000

A `SELECT` statement can be considered as a query block. The example in the slide displays employees who do the same job as “Taylor” but earn more salary than him.

The example consists of three query blocks: the outer query and two inner queries. The inner query blocks are executed first, producing the query results `SA_REP` and `8600`, respectively. The outer query block is then processed and uses the values that were returned by the inner queries to complete its search conditions.

Both inner queries return single values (`SA_REP` and `8600`, respectively), so this SQL statement is called a single-row subquery.

Note: The outer and inner queries can get data from different tables.

Using Group Functions in a Subquery

```
SELECT last_name, job_id, salary
FROM employees
WHERE salary = (SELECT MIN(salary)
                FROM employees);
```



	LAST_NAME	JOB_ID	SALARY
1	Vargas	ST_CLERK	2500

You can display data from a main query by using a group function in a subquery to return a single row. The subquery is in parentheses and is placed after the comparison condition.

The example in the slide displays the employee last name, job ID, and salary of all employees whose salary is equal to the minimum salary. The `MIN` group function returns a single value (2500) to the outer query.

HAVING Clause with Subqueries

The database server:

- Executes the subqueries first
- Returns the result into the HAVING clause of the main query

```
SELECT department_id, MIN(salary)
FROM employees
GROUP BY department_id
HAVING MIN(salary) > (SELECT MIN(salary)
                     FROM employees
                     WHERE department_id = 50);
```

2500



	DEPARTMENT_ID	MIN(SALARY)
1	{null}	7000
2	90	17000
3	20	6000
4	110	8300
5	80	8600
6	60	4200
7	10	4400

You can use subqueries not only in the WHERE clause but also in the HAVING clause. The server executes the subquery, and the results are returned into the HAVING clause of the main query.

The SQL statement in the slide displays all the departments that have a minimum salary greater than the minimum salary of department 30.

Another Example:

Find the job with the lowest average salary.

```
SELECT job_id, AVG(salary)
FROM employees
GROUP BY job_id
HAVING AVG(salary) = (SELECT MIN(AVG(salary))
                     FROM employees
                     GROUP BY job_id);
```

What is wrong with this statement?

```
SELECT employee_id, last_name
FROM employees
WHERE salary =
    (SELECT MIN(salary)
     FROM employees
     GROUP BY department_id);
```



ORA-01427: single-row subquery returns more than one row
01427. 00000 - "single-row subquery returns more than one row"
*Cause:
*Action:

Single-row operator with multiple-row subquery

A common error with subqueries occurs when more than one row is returned for a single-row subquery.

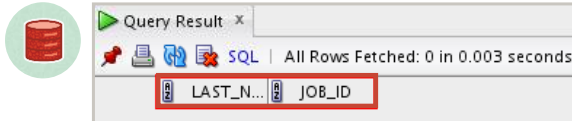
In the SQL statement in the slide, the subquery contains a `GROUP BY` clause, which implies that the subquery will return multiple rows, one for each group that it finds. In this case, the results of the subquery are 4400, 6000, 2500, 4200, 7000, 17000, and 8300.

The outer query takes those results and uses them in its `WHERE` clause. The `WHERE` clause contains an equal (`=`) operator, a single-row comparison operator that expects only one value. The `=` operator cannot accept more than one value from the subquery and, therefore, generates the error.

To correct this error, change the `=` operator to `IN`.

No Rows Returned by the Inner Query

```
SELECT last_name, job_id
FROM employees
WHERE job_id = (SELECT job_id
                FROM jobs
                WHERE job_title = 'Architect');
```

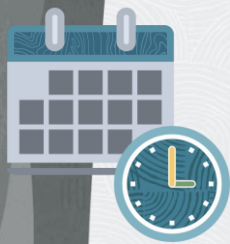


The subquery returns no rows because there is no job with the title "Architect."

Another common problem occurs with subqueries when no rows are returned by the inner query.

In the SQL statement in the slide, the subquery contains a `WHERE` clause. Presumably, the intention is to find the employee who works as an "Architect." The statement is correct, but selects no rows when executed because there is no job titled "Architect." Therefore, the subquery returns no rows.

The outer query takes the results of the subquery (null) and uses these results in its `WHERE` clause. The outer query finds no employee with a job `ID` equal to `NULL` and so returns no rows. Even if a job existed with a value of null, the row is not returned because comparison of two null values yields a null; therefore, the `WHERE` condition is not true.



Multiple-Row Subqueries

For Instructor Use Only.
This document should not be distributed

Multiple-Row Subqueries

- Return more than one row
- Use multiple-row comparison operators:

Operator	Meaning
IN	Equal to any member in the list
ANY	Must be preceded by =, !=, >, <, <=, >=. This returns TRUE if at least one element exists in the result set of the subquery for which the relation is TRUE.
ALL	Must be preceded by =, !=, >, <, <=, >=. This returns TRUE if the relation is TRUE for all elements in the result set of the subquery.

Subqueries that return more than one row are called multiple-row subqueries. You use a multiple-row operator, instead of a single-row operator, with a multiple-row subquery. The multiple-row operator expects one or more values:

```
SELECT last_name, salary, department_id
FROM   employees
WHERE  salary IN (SELECT   MIN(salary)
                  FROM     employees
                  GROUP BY department_id);
```

Example

Find the employees who earn the same salary as the minimum salary of any department.

The inner query is executed first, producing a query result. The main query block is then processed and uses the values that were returned by the inner query to complete its search condition. In fact, the main query appears to the database server as follows:

```
SELECT last_name, salary, department_id
FROM   employees
WHERE  salary IN (2500, 4200, 4400, 6000, 7000, 8300,
                 8600, 17000);
```

Using the ANY Operator in Multiple-Row Subqueries

```
SELECT employee_id, last_name, job_id, salary
FROM   employees
WHERE  salary < ANY (SELECT salary
                     FROM   employees
                     WHERE  job_id = 'IT_PROG')
AND    job_id <> 'IT_PROG';
```



EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
1	144 Vargas	ST_CLERK	2500
	143 Matos	ST_CLERK	2600
	142 Davies	ST_CLERK	3100
	141 Rajs	ST_CLERK	3500
5	200 Whalen	AD_ASST	4400
...			
9	206 Gietz	AC_ACCOUNT	8300
10	176 Taylor	SA_REP	8600



The ANY operator (and its synonym, the SOME operator) compares a value to *each* value returned by a subquery. The example in the slide displays employees who are not IT programmers and whose salary is less than that of any IT programmer. The maximum salary that a programmer earns is \$9,000.

- <ANY means less than the maximum.
- >ANY means more than the minimum.
- =ANY is equivalent to IN.

Using the ALL Operator in Multiple-Row Subqueries

```
SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary < ALL
      (SELECT salary
       FROM employees
       WHERE job_id = 'IT_PROG')
AND job_id <> 'IT_PROG';
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
1	141	Rajs	ST_CLERK	3500
2	142	Davies	ST_CLERK	3100
3	143	Matos	ST_CLERK	2600
4	144	Vargas	ST_CLERK	2500



The ALL operator compares a value to *every* value returned by a subquery. The example in the slide displays employees who are not IT programmers and whose salary is less than that of all IT programmers.

- >ALL means more than the maximum.
- <ALL means less than the minimum.
- The NOT operator can be used with IN, ANY, and ALL operators.



Multiple-Column Subqueries

For Instructor Use Only.
This document should not be distributed

Multiple-Column Subqueries

- A multiple-column subquery returns more than one column to the outer query.
- Column comparisons in multiple column comparisons can be pairwise or nonpairwise.
- A multiple-column subquery can also be used in the `FROM` clause of a `SELECT` statement.
- Syntax:

```
SELECT column, column, ...  
FROM table  
WHERE (column1, column2, ...) IN  
      (SELECT column1, column2, ...  
       FROM table  
       WHERE condition);
```

A multiple-column subquery returns more than one column to the outer query and can be listed in the outer query's `FROM`, `WHERE`, or `HAVING` clause.

If you want to compare two or more columns, you must write a compound `WHERE` clause using logical operators. Multiple-column subqueries enable you to combine duplicate `WHERE` conditions into a single `WHERE` clause.

`IN` operator is used to check a value within a set of values. The list of values may come from the results returned by a subquery.

Syntax:

```
SELECT column, column, ...  
FROM table  
WHERE (column, column, ...) IN  
      (SELECT column, column, ...  
       FROM table  
       WHERE condition);
```

Multiple-Column Subquery: Example

Display all the employees with the lowest salary in each department.

```
SELECT first_name, department_id, salary
FROM employees
WHERE (salary, department_id) IN
    (SELECT min(salary), department_id
     FROM employees
     GROUP BY department_id)
ORDER BY department_id;
```



	FIRST_NAME	DEPARTMENT_ID	SALARY
1	Jennifer	10	4400
2	Pat	20	6000
3	Peter	50	2500
4	Diana	60	4200
5	Jonathon	80	8600
6	Neena	90	17000
7	Lex	90	17000
8	William	110	8300

The example in the slide shows a multiple-column subquery in which the subquery returns more than one column.

The inner query is executed first, and it returns the lowest `salary` and `department_id` for each department. The main query block is then processed and uses the values that were returned by the inner query to complete its search condition.

Note: The employee with the first name Kimberly is not returned in the result because the `department_id` for the employee is `NULL`.

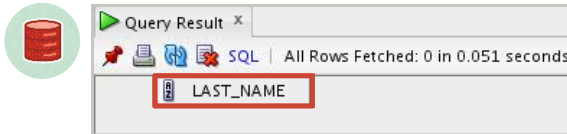


Null Values in a Subquery

For Instructor Use Only.
This document should not be distributed

Null Values in a Subquery

```
SELECT emp.last_name
FROM   employees emp
WHERE  emp.employee_id NOT IN
      (SELECT mgr.manager_id
       FROM   employees mgr);
```



The subquery returns no rows because one of the values returned by a subquery is null.

The SQL statement in the slide attempts to display all the employees who do not have any subordinates. Logically, this SQL statement should have returned 12 rows. However, the SQL statement does not return any rows. One of the values returned by the inner query is a null value and, therefore, the entire query returns no rows.

The reason is that all conditions that compare a null value result in a null. So whenever null values are likely to be part of the results set of a subquery, do not use the `NOT IN` operator. The `NOT IN` operator is equivalent to `<> ALL`.

Notice that the null value as part of the results set of a subquery is not a problem if you use the `IN` operator. The `IN` operator is equivalent to `=ANY`. For example, to display the employees who have subordinates, use the following SQL statement:

```
SELECT emp.last_name
FROM   employees emp
WHERE  emp.employee_id IN
      (SELECT mgr.manager_id
       FROM   employees mgr);
```

Alternatively, a `WHERE` clause can be included in the subquery to display all employees who do not have any subordinates:

```
SELECT last_name FROM employees
WHERE  employee_id NOT IN
      (SELECT manager_id
       FROM   employees
       WHERE  manager_id IS NOT NULL);
```

Summary



Subquery: Types, syntax, and guidelines

Single-row subqueries

Multiple-row subqueries

Multiple-column subqueries

Null values in a subquery

In this lesson, you should have learned how to use subqueries. A subquery is a `SELECT` statement that is embedded in the clause of another SQL statement. Subqueries are useful when a query is based on a search criterion with unknown intermediate values.

Subqueries have the following characteristics:

- Can pass one row of data to a main statement that contains a single-row operator, such as `=`, `<>`, `>`, `>=`, `<`, or `<=`
- Can pass multiple rows of data to a main statement that contains a multiple-row operator, such as `IN`
- Are processed first by the Oracle server, after which the `WHERE` or `HAVING` clause uses the results
- Can contain group functions

For Instructor Use Only.
This document should not be distributed

Using Set Operators



Course Roadmap

Lesson 1: Introduction

Unit 1: Retrieving, Restricting, and
Sorting Data

**Unit 2: Joins, Subqueries, and Set
Operators**

Unit 3: DML and DDL



Lesson 6: Reporting Aggregated Data Using the Group
Functions



Lesson 7: Displaying Data from Multiple Tables
Using Joins



Lesson 8: Using Subqueries to Solve Queries



Lesson 9: Using Set Operators

You are here!

In Unit 2, you will learn to use:

- SQL statements to query and display data from multiple tables using Joins
- Subqueries when the condition is unknown
- Group functions to aggregate data
- Set operators

Objectives



Set operators: Types and guidelines

Tables used in this lesson

UNION and UNION ALL operators

INTERSECT operator

MINUS operator

Matching SELECT statements

Using the ORDER BY clause in set operations

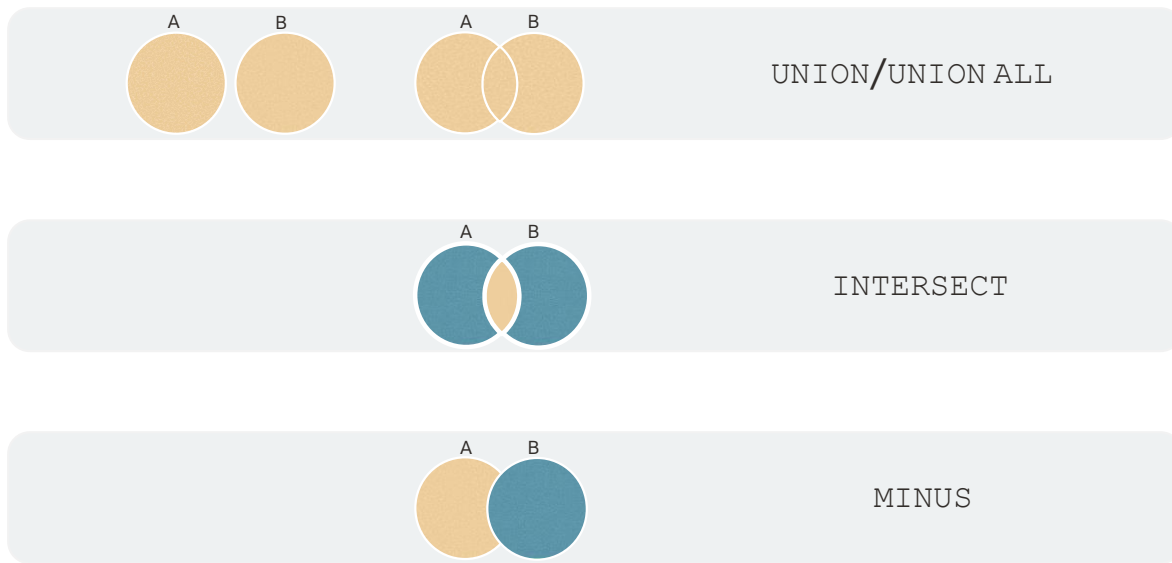
In this lesson, you will learn how to write queries by using set operators.



Set Operators: Types and Guidelines

For Instructor Use Only.
This document should not be distributed

Set Operators



Set operators combine the results of two or more component queries into one result. Queries containing set operators are called *compound queries*.

Operator	Returns
UNION	Rows from both queries after eliminating duplications
UNION ALL	Rows from both queries, including all duplications
INTERSECT	Rows that are common to both queries
MINUS	Rows in the first query that are not present in the second query

All set operators have equal precedence. If a SQL statement contains multiple set operators, the Oracle server evaluates them from left (top) to right (bottom), if no parentheses explicitly specify another order. You should use parentheses to specify the order of evaluation explicitly in queries that use the `INTERSECT` operator with other set operators.

Set Operator Rules

- The expressions in the `SELECT` lists must match in number.
- The data type of each column in the subsequent query must match the data type of its corresponding column in the first query.
- Parentheses can be used to alter the sequence of execution.
- The `ORDER BY` clause can appear only at the very end of the statement.



For Instructor Use Only.
This document should not be distributed

- The expressions in the `SELECT` lists of the queries must match in number and data type. When you use the `UNION`, `UNION ALL`, `INTERSECT`, and `MINUS` operators, ensure that the `SELECT` lists have the same number and data type of columns. The data type sometimes may not be exactly the same. In such cases, the column in the second query must be in the same data type group (such as numeric or character) as the corresponding column in the first query.
- You can use the set operators in subqueries.
- You should use parentheses to specify the order of evaluation in queries that use the `INTERSECT` operator with other set operators. According to SQL standards, the `INTERSECT` operator has greater precedence than the other set operators.

Oracle Server and Set Operators



- Duplicate rows are automatically eliminated except in `UNION ALL`.
- Column names from the first query appear in the result.
- The output is sorted in ascending order by default, except in `UNION ALL`.



The `SELECT` lists of a compound query must match the corresponding expressions in number and data type. If component queries select character data, the data type of the return values is determined as follows:

- If both queries select values of `CHAR` data type, of equal length, the returned values have the `CHAR` data type of that length. If the queries select values of `CHAR` with different lengths, the returned value is `VARCHAR2` with the length of the larger `CHAR` value.
- If either or both of the queries select values of `VARCHAR2` data type, the returned values have the `VARCHAR2` data type.
- If component queries select numeric data, the data type of the return values is determined by numeric precedence.
- If all queries select values of the `NUMBER` type, the returned values have the `NUMBER` data type.
- In queries using set operators, the Oracle server does not perform implicit conversion across data type groups. Therefore, if the corresponding expressions of component queries resolve to both character data and numeric data, the Oracle server returns an error.



Tables Used in This Lesson

For Instructor Use Only.
This document should not be distributed

Tables Used in This Lesson

The tables used in this lesson are:

- `employees`: Provides details about all current employees
- `retired_employees`: Provides details about all past employees



Two tables are used in this lesson: `employees` and `retired_employees`.

You are already familiar with the `employees` table that stores employee details, such as a unique identification number, email address, job identification (such as `ST_CLERK`, `SA_REP`, and so on), salary, manager, and so on.

`retired_employees` stores the details of the employees who have left the company.

The structure of and data from the `employees` and `retired_employees` tables are shown on the following pages.


```
DESCRIBE employees
```

Name	Null	Type
EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

```
SELECT employee_id, last_name, job_id, hire_date, department_id
FROM employees;
```

	EMPLOYEE_ID	LAST_NAME	JOB_ID	HIRE_DATE	DEPARTMENT_ID
1	100	King	AD_PRES	17-JUN-11	90
2	101	Kochhar	AD_VP	21-SEP-09	90
3	102	De Haan	AD_VP	13-JAN-09	90
4	103	Hunold	IT_PROG	03-JAN-14	60
5	104	Ernst	IT_PROG	21-MAY-15	60
6	107	Lorentz	IT_PROG	07-FEB-15	60
7	124	Mourgos	ST_MAN	16-NOV-15	50
8	141	Rajs	ST_CLERK	17-OCT-11	50
9	142	Davies	ST_CLERK	29-JAN-13	50
10	143	Matos	ST_CLERK	15-MAR-14	50
11	144	Vargas	ST_CLERK	09-JUL-14	50
12	149	Zlotkey	SA_MAN	29-JAN-16	80
13	174	Abel	SA_REP	11-MAY-12	80
14	176	Taylor	SA_REP	24-MAR-14	80
15	178	Grant	SA_REP	24-MAY-15	(null)
16	200	Whalen	AD_ASST	17-SEP-11	10
17	201	Hartstein	MK_MAN	17-FEB-12	20
18	202	Fay	MK_REP	17-AUG-13	20
19	205	Higgins	AC_MGR	07-JUN-10	110
20	206	Gietz	AC_ACCOUNT	07-JUN-10	110

```
DESCRIBE retired_employees
```

Name	Null	Type
EMPLOYEE_ID		NUMBER(7)
FIRST_NAME		VARCHAR2(20)
LAST_NAME		VARCHAR2(20)
EMAIL		VARCHAR2(25)
RETIRED_DATE		DATE
JOB_ID		VARCHAR2(20)
SALARY		NUMBER(8,2)
MANAGER_ID		NUMBER(4)
DEPARTMENT_ID		NUMBER(6)

```
SELECT * FROM retired_employees;
```

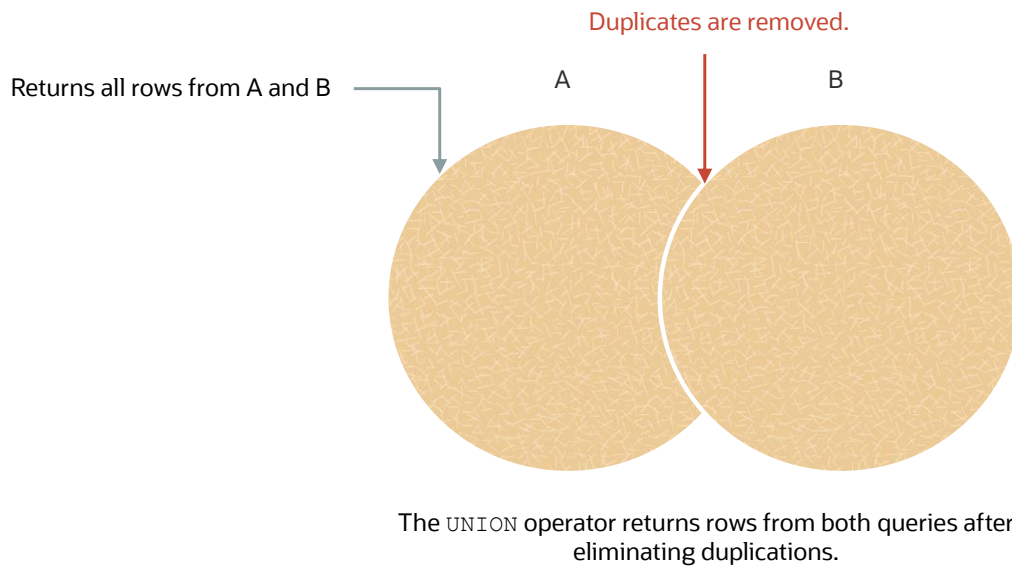
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	RETIRED_DATE	JOB_ID	SALARY	MANAGER_ID	DEPARTMENT_ID
1	301	Rick	Dayle	RDAYLE	18-MAR-10	AD_PRES	8000	124	90
2	302	Meena	Rac	MRAC	21-SEP-11	AD_VP	11000	149	90
3	303	Mex	Haan	MHAAN	13-JAN-10	AD_VP	9500	149	80
4	304	Alexandera	Runold	ARUNOLD	03-JAN-11	IT_PROG	7500	124	60
5	305	Bruk	Ernst	BERNST	21-MAY-10	IT_PROG	6000	149	60
6	306	Dravid	Aust	DAUST	25-JUN-09	IT_PROG	4800	124	60
7	307	Raj	Patil	RPATIL	05-FEB-12	IT_PROG	4800	201	60
8	308	Rahul	Bose	RBOSE	17-AUG-12	FI_MGR	12008	124	100
9	309	Dany	Fav	DFAV	16-AUG-11	FI_ACCOUNT	9000	101	100
10	310	James	Ken	JKHEN	28-SEP-10	FI_ACCOUNT	8200	101	90
11	311	Shana	Garg	SGARG	30-SEP-10	FI_ACCOUNT	7700	201	100
12	312	Peter	Jois	PJOIS	07-JUN-14	FI_ACCOUNT	7800	124	100
13	313	Lui	Pops	LPOPS	07-DEC-10	FI_ACCOUNT	6900	201	100
14	314	Del	Raph	DRAPH	07-DEC-12	PU_MAN	11000	101	30
15	315	Alex	Khurl	AKHURL	18-MAY-11	PU_CLERK	3100	149	30



UNION and UNION ALL Operators

For Instructor Use Only.
This document should not be distributed

UNION Operator



The UNION operator returns all rows that are selected by either query. Use the UNION operator to return all rows from multiple tables and eliminate any duplicate rows.

Guidelines

- The number of columns being selected must be the same.
- The data types of the columns being selected must be in the same data type group (such as numeric or character).
- The names of the columns need not be identical.
- UNION operates over all of the columns being selected.
- NULL values are not ignored during duplicate checking.
- By default, the output is sorted in ascending order of the columns of the SELECT clause.

Using the UNION Operator

Display the job details of all the current and retired employees.
Display each job only once.

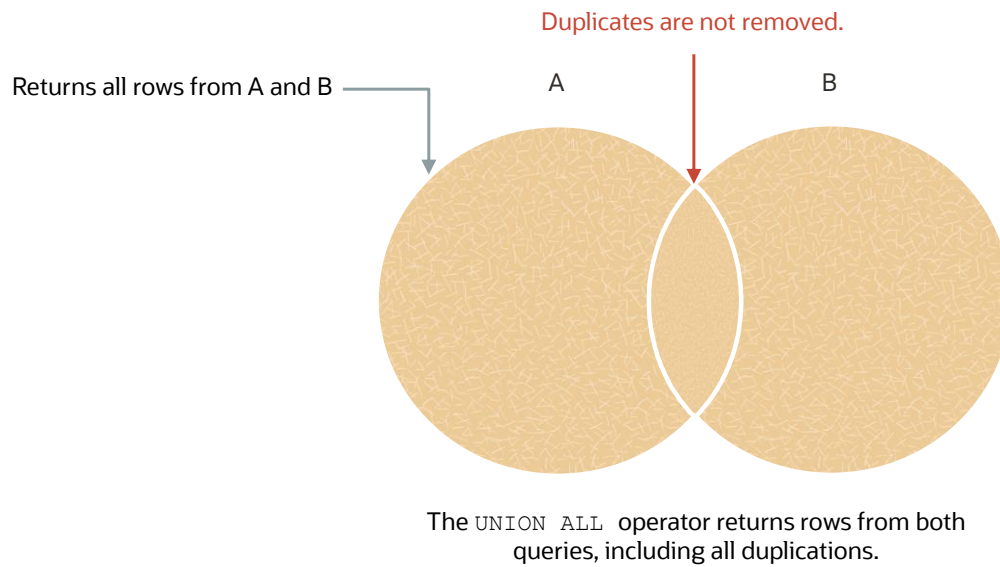
```
SELECT job_id
FROM employees
UNION
SELECT job_id
FROM retired_employees;
```



JOB_ID
1 AC_ACCOUNT
2 AC_MGR
3 AD_ASST
4 AD PRES
5 AD_VP
6 FI_ACCOUNT
7 FI_MGR
8 IT_PROG
9 MK_MAN
10 MK_REP
11 PU_CLERK
12 PU_MAN
13 SA_MAN
14 SA_REP
15 ST_CLERK
16 ST_MAN

The UNION operator eliminates any duplicate records. If records that occur in both the EMPLOYEES and the RETIRED_EMPLOYEES tables are identical, the records are displayed only once.

UNION ALL Operator



Use the UNION ALL operator to return all rows from multiple queries.

Guidelines

The guidelines for UNION and UNION ALL are the same, with the following two exceptions that pertain to UNION ALL: Unlike UNION, duplicate rows are not eliminated and the output is not sorted by default.

Using the UNION ALL Operator

Display the jobs and departments of all current and previous employees.

```
SELECT job_id, department_id
FROM   employees
UNION ALL
SELECT job_id, department_id
FROM   retired_employees
ORDER BY job_id;
```

JOB_ID	DEPARTMENT_ID
1 AC_ACCOUNT	110
2 AC_MGR	110
3 AD_ASST	10
4 AD_PRES	90
5 AD_PRES	90
6 AD_VP	90
7 AD_VP	80
8 AD_VP	90
9 AD_VP	90



28 SA_REP	80
29 SA_REP	80
30 SA_REP	(null)
31 ST_CLERK	50
32 ST_CLERK	50
33 ST_CLERK	50
34 ST_CLERK	50
35 ST_MAN	50

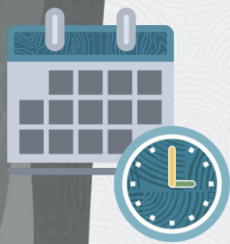
...

In the example in the slide, 35 rows are selected. The combination of the two tables totals to 35 rows. The UNION ALL operator does not eliminate duplicate rows. UNION returns all distinct rows selected by either query. UNION ALL returns all rows selected by either query, including all duplicates.

Consider the query in the slide, now written with the UNION clause:

```
SELECT job_id, department_id
FROM   employees
UNION
SELECT job_id, department_id
FROM   retired_employees
ORDER BY job_id;
```

The preceding query returns 19 rows. This is because it eliminates all the duplicate rows.



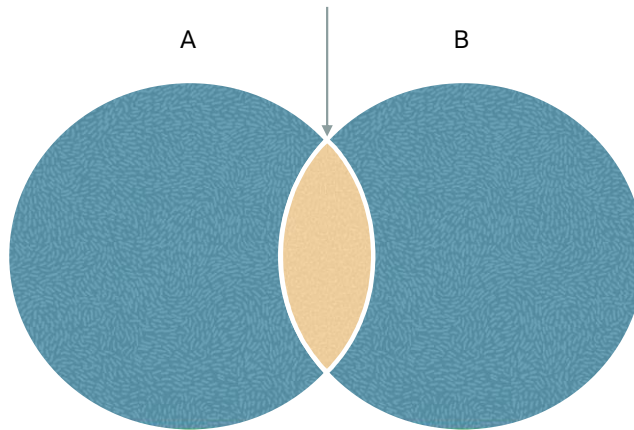
INTERSECT Operator

For Instructor Use Only.
This document should not be distributed

INTERSECT Operator



Returns all rows common to A and B



The `INTERSECT` operator returns rows that are common to both queries.

Use the `INTERSECT` operator to return all rows that are common to multiple queries.

Guidelines

- The number of columns and the data types of the columns being selected by the `SELECT` statements in the queries must be identical in all the `SELECT` statements used in the query. The names of the columns, however, need not be identical.
- Reversing the order of the intersected tables does not alter the result.
- `INTERSECT` does not ignore `NULL` values.

Using the INTERSECT Operator



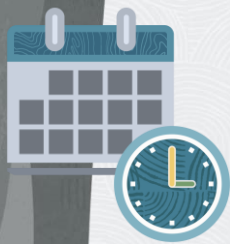
Display the common manager IDs and department IDs of current and previous employees.

```
SELECT manager_id, department_id
FROM employees
INTERSECT
SELECT manager_id, department_id
FROM retired_employees;
```



	MANAGER_ID	DEPARTMENT_ID
1	149	80

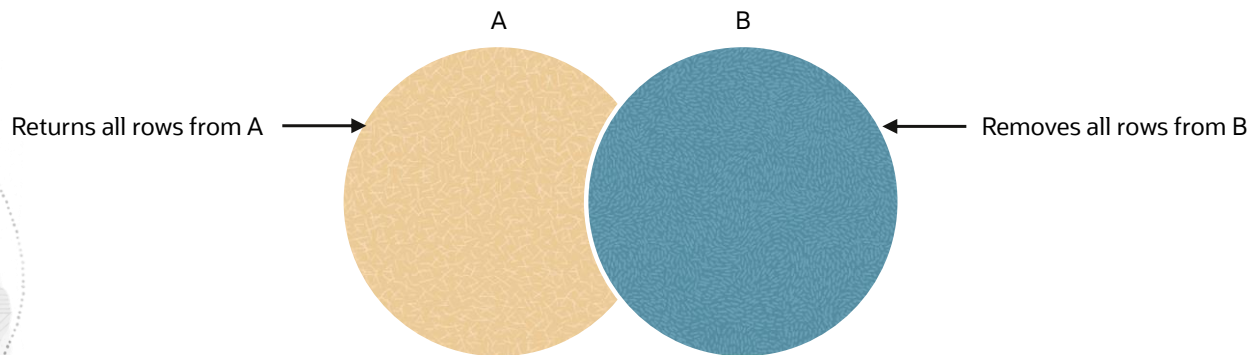
In the example in this slide, the query returns only those records that have the same values in the selected columns in both tables.



MINUS Operator

For Instructor Use Only.
This document should not be distributed

MINUS Operator



The `MINUS` operator returns all the distinct rows selected by the first query, but not present in the second query result set.

Use the `MINUS` operator to return all distinct rows selected by the first query, but not present in the second query result set (the first `SELECT` statement `MINUS` the second `SELECT` statement).

Note: The number of columns must be the same and the data types of the columns being selected by the `SELECT` statements in the queries must belong to the same data type group in all the `SELECT` statements used in the query. The names of the columns, however, need not be identical.

Using the MINUS Operator



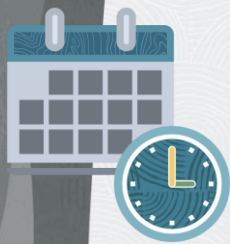
Display the manager IDs and Job IDs of employees whose managers have never managed retired employees in the sales department.

```
SELECT manager_id, job_id
FROM employees
WHERE department_id = 80
MINUS
SELECT manager_id, job_id
FROM retired_employees
WHERE department_id = 80;
```



	MANAGER_ID	JOB_ID
1	100	SA_MAN
2	149	SA_REP

In the example in the slide, the manager IDs and job IDs in the RETIRED_EMPLOYEES table are subtracted from those in the EMPLOYEES table. The results set displays the employees remaining after the subtraction; they are represented by rows that exist in the EMPLOYEES table, but do not exist in the RETIRED_EMPLOYEES table. These are the records of the employees who work in the sales department and are being managed by managers who have not managed any of the retired employees.



Matching SELECT Statements

For Instructor Use Only.
This document should not be distributed

Matching SELECT Statements in Oracle



You must match the data type (using the `TO_CHAR` function or any other conversion functions) when columns do not exist in one or the other table.

```
SELECT location_id, department_name "Department",  
       TO_CHAR(NULL) "Warehouse location"  
FROM departments  
UNION  
SELECT location_id, TO_CHAR(NULL) "Department",  
       state_province  
FROM locations;
```

Because the expressions in the `SELECT` lists of the queries must match in number, you can use the dummy columns and the data type conversion functions to comply with this rule.

To match the column list explicitly, you can insert `NULL` columns at the missing positions so as to match the count and data type of selected columns in each `SELECT` statement.

In the slide, the name `Warehouse location` is given as the dummy column heading. The `TO_CHAR` function is used in the first query to match the `VARCHAR2` data type of the `state_province` column that is retrieved by the second query. Similarly, the `TO_CHAR` function in the second query is used to match the `VARCHAR2` data type of the `department_name` column that is retrieved by the first query.

Matching the SELECT Statement: Example in Oracle



Using the UNION operator, display the employee name, job ID, and hire date of all employees.

```
SELECT FIRST_NAME, JOB_ID, hire_date "HIRE_DATE"
FROM employees
UNION
SELECT FIRST_NAME, JOB_ID, TO_DATE(NULL) "HIRE_DATE"
FROM retired_employees;
```



	FIRST_NAME	JOB_ID	HIRE_DATE
1	Alex	PU_CLERK	(null)
2	Alexander	IT_PROG	03-JAN-14
3	Alexandera	IT_PROG	(null)
4	Bruce	IT_PROG	21-MAY-15
5	Bruk	IT_PROG	(null)
6	Curtis	ST_CLERK	29-JAN-13
7	Dany	FI_ACCOUNT	(null)
8	Del	PU_MAN	(null)

The EMPLOYEES and RETIRED_EMPLOYEES tables have several columns in common (for example, EMPLOYEE_ID, JOB_ID, and DEPARTMENT_ID). However, what if you want the query to display FIRST_NAME, JOB_ID, and HIRE_DATE by using the UNION operator, knowing that HIRE_DATE exists only in the EMPLOYEES table?

The code example in the slide matches the FIRST_NAME and JOB_ID columns in the EMPLOYEES and RETIRED_EMPLOYEES tables. NULL is added to the RETIRED_EMPLOYEES SELECT statement to match the HIRE_DATE column in the EMPLOYEES SELECT statement.

In the results shown in the slide, each row in the output that corresponds to a record from the RETIRED_EMPLOYEES table contains a NULL in the HIRE_DATE column.



Using the ORDER BY Clause in Set Operations

For Instructor Use Only.
This document should not be distributed

Using the ORDER BY Clause in Set Operations in Oracle



- The ORDER BY clause can appear only once at the end of the compound query.
- Component queries cannot have individual ORDER BY clauses.
- The ORDER BY clause recognizes only the columns of the first SELECT query.
- By default, the first column of the first SELECT query is used to sort the output in ascending order.



You can use the ORDER BY clause only once in a compound query. Place the ORDER BY clause at the end of the query. The ORDER BY clause accepts the column name or an alias. By default, the output is sorted in ascending order in the first column of the first SELECT query.

Note: The ORDER BY clause does not recognize the column names of the second SELECT query. To avoid confusion over column names, it is a common practice to ORDER BY column positions.

For example, in the following statement, the output will be shown in ascending order of job_id.

```
SELECT employee_id, job_id, salary
FROM   employees
UNION
SELECT employee_id, job_id, 0
FROM   retired_employees
ORDER BY 2;
```

If you omit ORDER BY, by default, the output will be sorted in ascending order of employee_id. You cannot use the columns from the second query to sort the output.

Using the ORDER BY Clause in Set Operations in Oracle: Example

Display the employee ID and job ID of all current and retired employees, sorted by job ID.

```
SELECT employee_id, job_id
FROM   employees
UNION
SELECT employee_id, job_id
FROM   retired_employees
ORDER BY 2;
```



	EMPLOYEE_ID	JOB_ID
1	206	AC_ACCOUNT
2	205	AC_MGR
3	200	AD_ASST
4	100	AD_PRES
5	301	AD_PRES
6	101	AD_VP
7	102	AD_VP
8	302	AD_VP
9	303	AD_VP
10	309	FI_ACCOUNT
11	310	FI_ACCOUNT
12	311	FI_ACCOUNT
13	312	FI_ACCOUNT
14	313	FI_ACCOUNT

...

28	174	SA_REP
29	176	SA_REP
30	178	SA_REP
31	141	ST_CLERK
32	142	ST_CLERK
33	143	ST_CLERK
34	144	ST_CLERK
35	124	ST_MAN

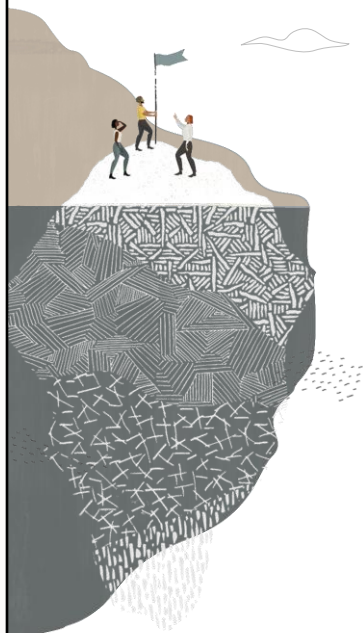


In the example shown in the slide, the output is sorted in the ascending order of `job_id`.

Note: The `ORDER BY` clause does not recognize the column names of the second `SELECT` query. To avoid confusion over column names, it is a common practice to `ORDER BY` column positions.

If you omit `ORDER BY`, by default, the output will be sorted in ascending order of `employee_id`. You cannot use the columns from the second query to sort the output.

Summary



Set operators: Types and guidelines

Tables used in this lesson

UNION and UNION ALL operators

INTERSECT operator

MINUS operator

Matching SELECT statements

Using the ORDER BY clause in set operations

The `UNION` operator returns all the distinct rows selected by each query in the compound query. Use the `UNION` operator to return all rows from multiple tables and eliminate any duplicate rows.

Use the `UNION ALL` operator to return all rows from multiple queries. Unlike the case with the `UNION` operator, duplicate rows are not eliminated, and the output is not sorted by default.

Use the `INTERSECT` operator to return all rows that are common to multiple queries.

Use the `MINUS` operator to return rows returned by the first query that are not present in the second query.

Remember to use the `ORDER BY` clause only at the very end of the compound statement.

Make sure that the corresponding expressions in the `SELECT` lists match in number and data type.

For Instructor Use Only.
This document should not be distributed

Managing Tables Using DML Statements



Course Roadmap

Lesson 1: Introduction

Unit 1: Retrieving, Restricting, and
Sorting Data

Unit 2: Joins, Subqueries, and Set
Operators

Unit 3: DML and DDL



Lesson 10: Managing Tables Using DML Statements

You are here!



Lesson 11: Introduction to Data Definition Language

In Unit 3, you learn how to manage data in tables by using data manipulation language (DML) statements. You also learn how to create and manage database objects by using data definition language (DDL) statements.

Objectives



Adding new rows in a table

Changing data in a table

Removing rows from a table

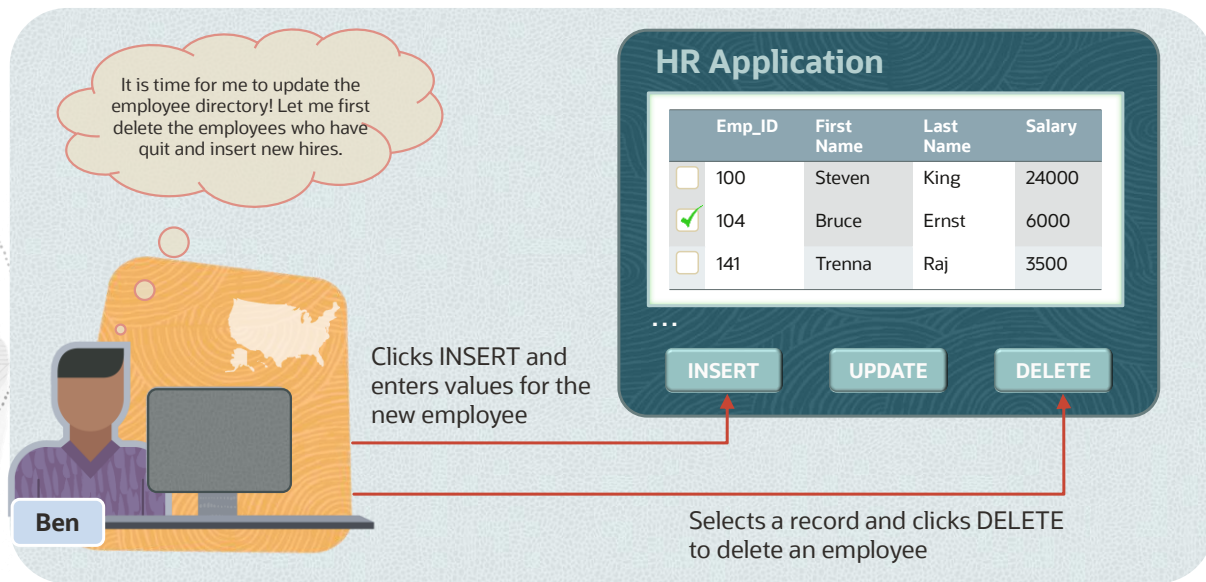
Database transaction control

Read consistency

Manual Data Locking

In this lesson, you learn how to use data manipulation language (DML) statements to insert rows into a table, update existing rows in a table, and delete existing rows from a table. You also learn how to control transactions with the `COMMIT`, `SAVEPOINT`, and `ROLLBACK` statements.

HR Application Scenario



Ben is an HR Manager in the USA. Ben wants to update the outdated employee list in his organization because he has hired new employees recently and a few employees have left the organization.

Ben logs in to the HR application and selects the ex-employee records and clicks `DELETE`. He then clicks `INSERT` and enters the details of new hires and clicks `SAVE`. The employee list is now updated.

When the HR Manager performs these transactions in the HR application, data manipulation language (DML) statements are used in the background. DML statements modify the data in the tables. In this lesson, you learn about DML statements and how to use them.



Adding New Rows in a Table

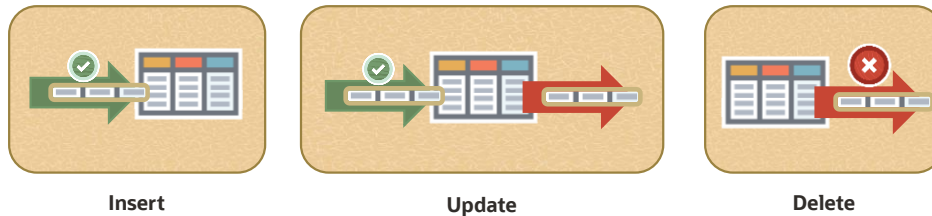
For Instructor Use Only.
This document should not be distributed

Data Manipulation Language

A DML statement is executed when you:

- Add new rows to a table
- Modify existing rows in a table
- Remove existing rows from a table

A *transaction* consists of a collection of DML statements that form a logical unit of work.



Data manipulation language (DML) is a core part of SQL. When you want to add, update, or delete data in the database, you execute a DML statement. A collection of DML statements that form a logical unit of work is called a *transaction*.

Consider a banking database. When a bank customer transfers money from a savings account to a checking account, the transaction might consist of three separate operations: decreasing the savings account, increasing the checking account, and recording the transaction in the transaction journal. The database server must guarantee that all the three SQL statements are performed to maintain the accounts in proper balance. When something prevents one of the statements in the transaction from executing, the other statements of the transaction must be undone.

Note: Most of the DML statements in this lesson assume that no constraints on the table are violated. Constraints are discussed later in this course.

Adding a New Row to a Table

DEPARTMENTS 70 Public Relations 100 1700 New row

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700
8	190	Contracting	(null)	1700

Insert a new row into the
DEPARTMENTS table.

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	70	Public Relations	100	1700
2	10	Administration	200	1700
3	20	Marketing	201	1800
4	50	Shipping	124	1500
5	60	IT	103	1400
6	80	Sales	149	2500
7	90	Executive	100	1700
8	110	Accounting	205	1700
9	190	Contracting	(null)	1700

The graphic in the slide illustrates the addition of a new department record to the DEPARTMENTS table.

INSERT Statement Syntax

- Add new rows to a table by using the `INSERT` statement.

```
INSERT INTO table [(column [, column...])]  
VALUES (value [, value...]);
```

- With this syntax, only one row is inserted at a time.



You can add new rows to a table by issuing the `INSERT` statement.

In the syntax:

<i>table</i>	Is the name of the table
<i>column</i>	Is the name of the column in the table to populate
<i>value</i>	Is the corresponding value for the column

Note: This statement with the `VALUES` clause adds only one row at a time to a table.

Inserting New Rows

- Insert a new row containing values for each column.
- List values in the default order of the columns in the table.
- Optionally, list the columns in the `INSERT` clause.

```
INSERT INTO departments(department_id,  
                        department_name, manager_id, location_id)  
VALUES (70, 'Public Relations', 100, 1700);
```

```
1 row inserted.
```

- Enclose character and date values within single quotation marks.

Because you can insert a new row that contains values for each column, the column list is not required in the `INSERT` clause. However, if you do not use the column list, the values must be listed according to the default order of the columns in the table, and a value must be provided for each column.

```
DESCRIBE departments
```

For clarity, use the column list in the `INSERT` clause.

Enclose character and date values within single quotation marks; however, it is not recommended that you enclose numeric values within single quotation marks.

INSERT Statement Syntax

- Add new rows to a table by using the VALUES constructor with an INSERT statement starting in 23c.

```
INSERT INTO table [(column [, column...])]
VALUES (value [, value...], (value [, value...]), ... ;
```

- With this syntax, more than one row is inserted at a time.

```
INSERT into hr.departments(department_id, manager_id, location_id)
VALUES
(900, 100, 1700),
(910, 101, 1800),
(920, 102, 1700);
```



Table Values Constructor

- Is a clause constructed as comma-separated groupings of values
- Is similar to methods used within other database engines
- Can be used with SELECT, INSERT, and MERGE statements
- Can also be used with JOIN statements:

```
SELECT e.first_name, e.last_name
FROM (VALUES (197, 124), (198, 124), (200, 101))
      my_colleagues(employee_id, manager_id)
JOIN hr.employees e ON (e.employee_id =
                        my_colleagues.employee_id
                        AND e.manager_id = my_colleagues.manager_id);
```

Benefits:

- Compatibility with other vendors
- Less code
- More readable code
- No need for UNION ALL construct

Inserting Rows with Null Values

- Implicit method: Omit the column from the column list.

```
INSERT INTO departments (department_id,  
                        department_name)  
VALUES (30, 'Purchasing');
```

1 row inserted.

- Explicit method: Specify the `NULL` keyword in the `VALUES` list.

```
INSERT INTO departments  
VALUES (100, 'Finance', NULL, NULL);
```

1 row inserted.

Method	Description
Implicit	Omit the column from the column list.
Explicit	Specify the <code>NULL</code> keyword in the <code>VALUES</code> list; specify the empty string (' ') in the <code>VALUES</code> list for character strings and dates.

Be sure that you can use null values in the targeted column by verifying the `NULL` status with the `DESCRIBE` command.

The Oracle server automatically enforces all data types, data ranges, and data integrity constraints. If a column is not explicitly listed, a null value is inserted in the new row unless you have default values for the missing columns that are used.

Common errors that can occur when you are inserting are in the following order:

- Mandatory value missing for a `NOT NULL` column
- Duplicate value violating any unique or primary key constraint
- Any value violating a `CHECK` constraint
- Referential integrity maintained for foreign key constraint
- Data type mismatches or values too wide to fit in a column

Note: Use of the column list is recommended because it makes the `INSERT` statement more readable and reliable, or less prone to mistakes.

Inserting Special Values

The `CURRENT_DATE` function records the current date and time in Oracle.

```
INSERT INTO employees (employee_id,
                       first_name, last_name,
                       email, phone_number,
                       hire_date, job_id, salary,
                       commission_pct, manager_id,
                       department_id)
VALUES (113,
       'Louis', 'Popp',
       'LPOPP', '515.124.4567',
       CURRENT_DATE, 'AC_ACCOUNT', 6900,
       NULL, 205, 110);
```

1 row inserted.

You can use functions to enter special values in your table.

The example in the slide records information for the employee Popp in the `EMPLOYEES` table. It supplies the current date and time in the `HIRE_DATE` column. It uses the `CURRENT_DATE` function that returns the current date in the session time zone. You can also use the `USER` function when inserting rows in a table. The `USER` function records the current username.

Confirming Additions to the Table

```
SELECT employee_id, last_name, job_id, hire_date, commission_pct
FROM   employees
WHERE  employee_id = 113;
```

Inserting Specific Date and Time Values

- Add a new employee.

```
INSERT INTO employees
VALUES
(114,
'Den', 'Raphealy',
'DRAPHEAL', '515.127.4561',
TO_DATE('FEB 3, 2016', 'MON DD, YYYY'),
'SA_REP', 11000, 0.2, 100, 70);
```

1 row inserted.

- Verify your addition.

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	Den	Raphealy	DRAPHEAL	515.127.4561	03-FEB-16	SA_REP	11000	0.2	100	70

The DD-MON-RR format is generally used to insert a date value. With the RR format, the system provides the correct century automatically.

You may also supply the date value in the DD-MON-YYYY format. This is recommended because it clearly specifies the century and does not depend on the internal RR format logic of specifying the correct century.

If you want to enter the date in a format other than the default format (for example, with another century or a specific time), you must use the TO_DATE function.

The example in the slide records information for employee Raphealy in the EMPLOYEES table. It sets the HIRE_DATE column to be February 3, 2016.

Creating a Script

- Use the & substitution in a SQL statement to prompt for values.
- & is a placeholder for the variable value.

```
INSERT INTO departments
      (department_id, department_name, location_id)
VALUES (&department_id, '&department_name', &location);
```

Enter Substitution Variable x

Enter value for department_id:

OK Cancel

Enter Substitution Variable x

Enter value for department_name:

OK Cancel

Enter Substitution Variable x

Enter value for location:

OK Cancel

You can save commands with substitution variables to a file and execute the commands in the file. The example in the slide records information for a department in the `DEPARTMENTS` table.

Run the script file and you are prompted for input for each of the ampersand (&) substitution variables. After entering a value for the substitution variable, click the OK button. The values that you input are then substituted into the statement. This enables you to run the same script file over and over, but supply a different set of values each time you run it.

Copying Rows from Another Table

- Write your `INSERT` statement with a subquery:

```
INSERT INTO sales_reps(id, name, salary, commission_pct)
SELECT employee_id, last_name, salary, commission_pct
FROM employees
WHERE job_id LIKE '%REP%';
```

5 rows inserted.

- Do not use the `VALUES` clause.
- Match the number of columns in the `INSERT` clause to those in the subquery.
- Insert all the rows returned by the subquery in the `sales_reps` table.

You can use the `INSERT` statement to add rows to a table where the values are derived from existing tables. In the example in the slide, for the `INSERT INTO` statement to work, you must have already created the `sales_reps` table by using the `CREATE TABLE` statement. `CREATE TABLE` is discussed in the lesson titled “Introduction to Data Definition Language.”

In place of the `VALUES` clause, you use a subquery.

Syntax

```
INSERT INTO table [ column (, column) ] subquery;
```

In the syntax:

<i>table</i>	Is the name of the table
<i>column</i>	Is the name of the column in the table to populate
<i>subquery</i>	Is the subquery that returns rows to the table

The number of columns and their data types in the column list of the `INSERT` clause must match the number of values and their data types in the subquery. Zero or more rows are added depending on the number of rows returned by the subquery. To create a copy of the rows of a table, use `SELECT *` in the subquery:

```
INSERT INTO copy_emp
SELECT *
FROM employees;
```



Changing Data in a Table

For Instructor Use Only.
This document should not be distributed

Changing Data in a Table

EMPLOYEES

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	MANAGER_ID	COMMISSION_PCT	DEPARTMENT_ID
100	Steven	King	24000	(null)	(null)	90
101	Neena	Kochhar	17000	100	(null)	90
102	Lex	De Haan	17000	100	(null)	90
103	Alexander	Hunold	9000	102	(null)	60
104	Bruce	Ernst	6000	103	(null)	60
107	Diana	Lorentz	4200	103	(null)	60
124	Kevin	Mourgos	5800	100	(null)	50

Update rows in the EMPLOYEES table:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	MANAGER_ID	COMMISSION_PCT	DEPARTMENT_ID
100	Steven	King	24000	(null)	(null)	90
101	Neena	Kochhar	17000	100	(null)	90
102	Lex	De Haan	17000	100	(null)	90
103	Alexander	Hunold	9000	102	(null)	80
104	Bruce	Ernst	6000	103	(null)	80
107	Diana	Lorentz	4200	103	(null)	80
124	Kevin	Mourgos	5800	100	(null)	50

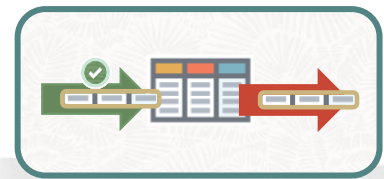
The slide illustrates changing the department number for employees in department 60 to department 80.

UPDATE Statement Syntax

- Modify existing values in a table with the `UPDATE` statement:

```
UPDATE      table
SET         column = value [, column = value, ...]
[WHERE      condition];
```

- Update more than one row at a time (if required).



You can modify the existing values in a table by using the `UPDATE` statement.

In the syntax:

<code>table</code>	Is the name of the table
<code>column</code>	Is the name of the column in the table to populate
<code>value</code>	Is the corresponding value or subquery for the column
<code>Condition</code>	Identifies the rows to be updated and is composed of column names, expressions, constants, subqueries, and comparison operators

Confirm the update operation by querying the table to display the updated rows.

For more information, see the section on “`UPDATE`” in *Oracle Database SQL Language Reference* for 23c database.

Note: In general, use the primary key column in the `WHERE` clause to identify a single row for update. Using other columns can unexpectedly cause several rows to be updated. For example, identifying a single row in the `EMPLOYEES` table by last name may return more than one having the same last name.

Updating Rows in a Table

- Values for a specific row or rows are modified if you specify the `WHERE` clause:

```
UPDATE employees
SET    department_id = 50
WHERE  employee_id = 113;
```

1 row updated.

- Values for all the rows in the table are modified if you omit the `WHERE` clause:

```
UPDATE    copy_emp
SET       department_id = 110;
```

22 rows updated

- Specify `SET column_name= NULL` to update a column value to `NULL`.

The `UPDATE` statement modifies the values of a specific row or rows if the `WHERE` clause is specified. The example in the slide shows the transfer of employee 113 (Popp) to department 50.

If you omit the `WHERE` clause, values for all the rows in the table are modified. Examine the updated rows in the `COPY_EMP` table.

```
SELECT last_name, department_id
FROM    copy_emp;
```

For example, an employee who was an `SA_REP` has now changed his job to an `IT_PROG`. Therefore, his `JOB_ID` needs to be updated and the commission field needs to be set to `NULL`.

```
UPDATE employees
SET job_id = 'IT_PROG', commission_pct = NULL
WHERE employee_id = 114;
```

Note: The `COPY_EMP` table has the same data as the `EMPLOYEES` table.

Updating Two Columns with a Subquery

Update employee 103's job and salary to match those of employee 205.

```
UPDATE employees
SET (job_id,salary) = (SELECT job_id,salary
                        FROM employees
                        WHERE employee_id = 205)
WHERE employee_id = 103;
```

1 row updated.

You can update multiple columns in the SET clause of an UPDATE statement by writing multiple subqueries. The syntax is as follows:

```
UPDATE table
SET column =
    (SELECT column
     FROM table
     WHERE condition)
[ ,
  column =
    (SELECT column
     FROM table
     WHERE condition)]
[WHERE condition ] ;
```

Updating Rows Based on Another Table

Use the subqueries in the `UPDATE` statements to update row values in a table based on values from another table:

```
UPDATE copy_emp
SET    department_id = (SELECT department_id
                        FROM employees
                        WHERE employee_id = 100)
WHERE  job_id        = (SELECT job_id
                        FROM employees
                        WHERE employee_id = 200);
```

1 row updated.

You can use the subqueries in the `UPDATE` statements to update values in a table. The example in the slide updates the `COPY_EMP` table based on the values from the `EMPLOYEES` table. It changes the department number to employee 100's current department number for all the employees whose job ID is the same as employee 200's job ID.

Enhancement in 23c

RETURNING clause enhancement for UPDATE

- Fetch rows affected by the last DML operation.
- Perform arithmetic on columns affected by a transaction.
- Retrieve the old and new values for any column affected.

Benefits

- ✓ Offers SQL developers similar functionality to PL/SQL code



This enhancement allows developers to fetch rows affected by the last DML operation; for example, fetching the primary key after an `INSERT`. It can also be used to perform arithmetic on columns affected by a transaction.

Supports a `RETURNING INTO` clause for the following DML statements:

- `INSERT`
- `UPDATE`
- `DELETE`

RETURNING Clause in 23c: Example

Retrieving the old and new values

```
variable l_id number
variable l_new_sal number
variable l_old_sal number
EXECUTE :l_id:=100;
UPDATE employees set salary=salary*2 where employee_id =
    :l_id
RETURNING new salary, old salary INTO :l_new_sal,
    :l_old_sal;
PRINT l_new_sal
PRINT l_old_sal
```

For Instructor Use Only.
This document should not be distributed



Removing Rows from a Table

For Instructor Use Only.
This document should not be distributed

Removing a Row from a Table

DEPARTMENTS

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700
8	190	Contracting	(null)	1700

Delete a row from the DEPARTMENTS table:

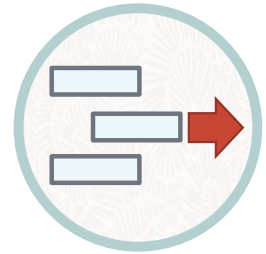
	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700

The contracting department has been removed from the DEPARTMENTS table (assuming no constraints on the DEPARTMENTS table are violated), as shown in the graphic in the slide.

DELETE Statement

You can remove existing rows from a table by using the `DELETE` statement:

```
DELETE [FROM]    table
[WHERE    condition];
```



In the syntax:

<code>table</code>	Is the name of the table
<code>condition</code>	Identifies the rows to be deleted and is composed of column names, expressions, constants, subqueries, and comparison operators

Note: If no rows are deleted, the message “0 rows deleted” is returned (on the Script Output tab in SQL Developer).

For more information, see the section on “`DELETE`” in *Oracle Database SQL Language Reference* for 23c database.

Deleting Rows from a Table

- Specific rows are deleted if you specify the `WHERE` clause:

```
DELETE FROM departments
WHERE department_name = 'Finance';
1 row deleted.
```

- All rows in the table are deleted if you omit the `WHERE` clause:

```
DELETE FROM copy_emp;
22 rows deleted
```

You can delete specific rows by specifying the `WHERE` clause in the `DELETE` statement. The first example in the slide deletes the accounting department from the `DEPARTMENTS` table. You can confirm the delete operation by trying to display the deleted rows by using the `SELECT` statement. The query returns 0 rows.

```
SELECT  *
FROM    departments
WHERE   department_name = 'Finance';
```

However, if you omit the `WHERE` clause, all rows in the table are deleted. The second example in the slide deletes all rows from the `COPY_EMP` table, because no `WHERE` clause was specified.

Example

Remove rows identified in the `WHERE` clause.

```
DELETE FROM employees WHERE employee_id = 113;
DELETE FROM departments WHERE department_id IN (30, 40);
```


Deleting Rows Based on Another Table

Use the subqueries in the `DELETE` statements to remove rows from a table based on values from another table:

```
DELETE FROM employees
WHERE department_id IN
    (SELECT department_id
     FROM departments
     WHERE department_name
       LIKE '%Public%');
```

1 row deleted.

You can use the subqueries to delete rows from a table based on values from another table. The example in the slide deletes all the employees in a department, where the department name contains the string `Public`.

The subquery searches the `DEPARTMENTS` table to find the department number based on the department name containing the string `Public`. The subquery then feeds the department number to the main query, which deletes rows of data from the `EMPLOYEES` table.

Enhancements to UPDATE and DELETE Statements in 23c

Direct joins for UPDATE and DELETE

- Update one table using values from another.
- Limit the rows changed in DELETE & UPDATE.

Benefits

- ✓ Makes the Oracle Database more developer friendly
- ✓ Increases SQL standard support
- ✓ Establishes parity with other non-Oracle systems



Direct join syntax is easier and more intuitive than the other existing methods for updating (or deleting) data in one table with data from another. This also allows for limiting rows to update or delete.

Developers in other technologies such as PostgreSQL, SQL Server, Snowflake, MySQL, DB2 are now able to use this feature in Oracle.

Use Case Examples

- Remove/change all rows for customers in a certain region.
- Aggregating child totals into parent row.
- Setting current/most recent value on one table using values from another.
- Copying values from a temporary/staging table into the main table in ETL.

UPDATE: Example

Setting the employees' current salary to the maximum salary value in the JOBS table for their job

```
UPDATE hr.employees e
set     e.salary = j.max_salary
from    hr.jobs j
where   e.job_id = j.job_id;
```

DELETE: Example

Removing the `job_history` rows for all employees currently in department 10

```
DELETE hr.job_history h
from   hr.employees e
where  e.employee_id = h.employee_id
and    e.department_id = 10;
```

1 row deleted.

TRUNCATE Statement

- Removes all rows from a table, leaving the table empty and the table structure intact
- Is a data definition language (DDL) statement rather than a DML statement; cannot be undone
- Syntax:

```
TRUNCATE TABLE table_name;
```

- Example:

```
TRUNCATE TABLE copy_emp;
```

You can use the `TRUNCATE` statement to quickly remove all rows from a table or cluster efficiently. Removing rows with the `TRUNCATE` statement is faster than removing them with the `DELETE` statement for the following reasons:

- The `TRUNCATE` statement is a data definition language (DDL) statement and generates no rollback information. Rollback information is covered later in this lesson.

If the table is the parent of a referential integrity constraint, you cannot truncate the table. You need to disable the constraint before issuing the `TRUNCATE` statement. You will learn more about DDL statements and disabling constraints in the lesson titled “Introduction to Data Definition Language.”



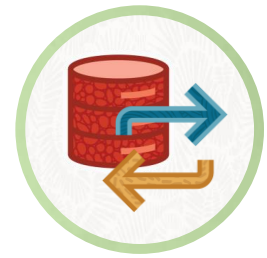
Database Transaction Control

For Instructor Use Only.
This document should not be distributed

Database Transactions

A database transaction consists of one of the following:

- DML statements that constitute one consistent change to the data
- One DDL statement
- One data control language (DCL) statement



Transactions give you more flexibility and control when changing data, and the Oracle server ensures data consistency in the event of user process failure or system failure.

Transactions consist of DML statements that constitute one consistent change to the data. For example, a transfer of funds between two accounts should include the debit in one account and the credit to another account of the same amount. Both actions should either fail or succeed together; the credit should not be committed without the debit.

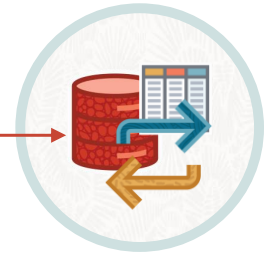
Transaction Types

Type	Description
Data manipulation language (DML)	Consists of any number of DML statements that the Oracle server treats as a single entity or a logical unit of work
Data definition language (DDL)	Consists of only one DDL statement
Data control language (DCL)	Consists of only one DCL statement

Database Transactions: Start and End

- Begin when the first DML SQL statement is executed
- End with one of the following events:
 - COMMIT or ROLLBACK statement is issued.
 - DDL or DCL statement executes (automatic commit).
 - User exits SQL Developer or SQL*Plus.
 - System crashes.

Database transaction on table/s



When does a database transaction start and end?

A transaction begins when the first DML statement is encountered and ends when one of the following occurs:

- A COMMIT or ROLLBACK statement is issued.
- A DDL statement, such as CREATE, is issued.
- A DCL statement is issued.
- The user exits SQL Developer or SQL*Plus.
- A machine fails or the system crashes.

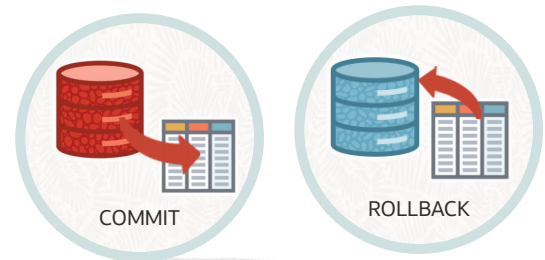
After one transaction ends, the next executable SQL statement automatically starts the next transaction.

A DDL statement or a DCL statement is automatically committed and, therefore, implicitly ends a transaction.

Advantages of the COMMIT and ROLLBACK Statements

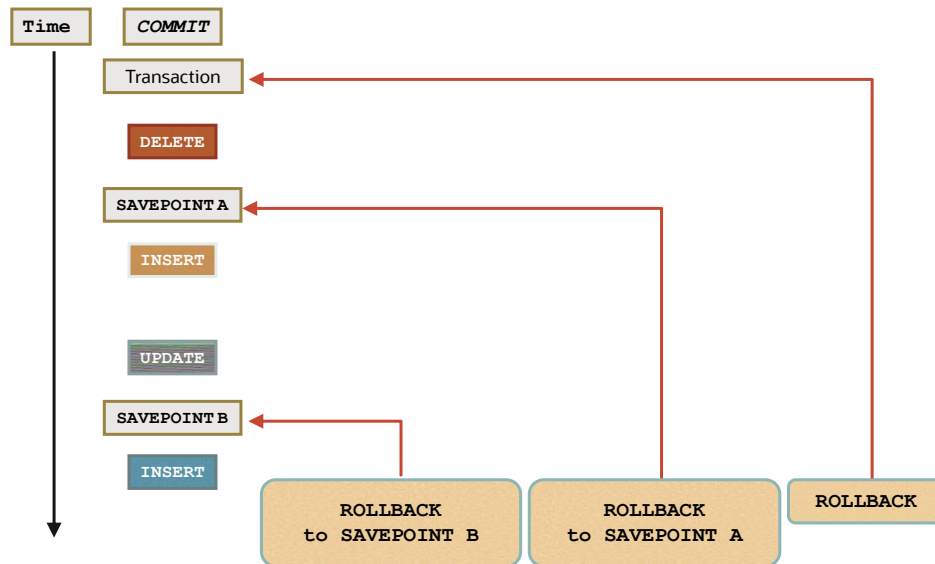
By using `COMMIT` and `ROLLBACK` statements, you can:

- Ensure data consistency
- Preview data changes before making changes permanent
- Group logically related operations



With the `COMMIT` and `ROLLBACK` statements, you have control over making changes to the data permanent.

Explicit Transaction Control Statements



You can control the logic of transactions by using the `COMMIT`, `SAVEPOINT`, and `ROLLBACK` statements.

Statement	Description
COMMIT	<code>COMMIT</code> ends the current transaction by making all pending data changes permanent.
<code>SAVEPOINT name</code>	SAVEPOINT <i>name</i> marks a savepoint within the current transaction.
ROLLBACK	<code>ROLLBACK</code> ends the current transaction by discarding all pending data changes.
<code>ROLLBACK TO SAVEPOINT name</code>	<code>ROLLBACK TO <savepoint></code> rolls back the current transaction to the specified savepoint, thereby discarding any changes and/or savepoints that were created after the savepoint to which you are rolling back. If you omit the <code>TO SAVEPOINT</code> clause, the <code>ROLLBACK</code> statement rolls back the entire transaction. Because savepoints are logical, there is no way to list the savepoints that you have created.

Note: You cannot `COMMIT` to a `SAVEPOINT`. `SAVEPOINT` is not ANSI-standard SQL.

Rolling Back Changes to a Marker

- Create a marker in the current transaction by using the `SAVEPOINT` statement.
- Roll back to that marker by using the `ROLLBACK TO SAVEPOINT` statement.

```
UPDATE...  
SAVEPOINT update_done;  
SAVEPOINT update_done  
  
INSERT...  
ROLLBACK TO update_done;  
Rollback complete.
```

Roll back to this point.

You can create a marker in the current transaction by using the `SAVEPOINT` statement, which divides the transaction into smaller sections. You can then discard pending changes back to that marker by using the `ROLLBACK TO SAVEPOINT` statement.

Note that if you create a second savepoint with the same name as an earlier savepoint, the earlier savepoint is deleted.

Implicit Transaction Processing

- An automatic commit occurs when:
 - A DDL statement is issued
 - A DCL statement is issued
 - There is a normal exit from SQL Developer or SQL*Plus, without explicitly issuing `COMMIT` or `ROLLBACK` statements
- An automatic rollback occurs when there is an abnormal termination of SQL Developer or SQL*Plus, or a system failure.



Status	Circumstances
Automatic commit	A DDL statement or DCL statement is issued; SQL Developer or SQL*Plus exited normally, without explicitly issuing <code>COMMIT</code> or <code>ROLLBACK</code> commands.
Automatic rollback	Abnormal termination of SQL Developer or SQL*Plus, or system failure.

Note: In SQL*Plus, the `AUTOCOMMIT` command can be toggled `ON` or `OFF`. If set to `ON`, each individual DML statement is committed as soon as it is executed. You cannot roll back the changes. If set to `OFF`, the `COMMIT` statement can still be issued explicitly. Also, the `COMMIT` statement is issued when a DDL statement is issued or when you exit SQL*Plus. The `SET AUTOCOMMIT ON/OFF` command is skipped in SQL Developer. DML is committed on a normal exit from SQL Developer only if you have the Autocommit preference enabled. To enable Autocommit, perform the following:

- From the Tools menu, select Preferences. In the Preferences dialog box, expand Database and select Worksheet Parameters.
- In the right pane, select the “**Autocommit in SQL Worksheet**” option. Click OK.

System Failures

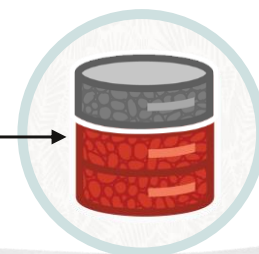
When a transaction is interrupted by a system failure, the entire transaction is automatically rolled back. This prevents the error from causing unwanted changes to the data and returns the tables to the state at the time of the last commit. In this way, the Oracle server protects the integrity of the tables.

In SQL Developer, a normal exit from the session is accomplished by selecting Exit from the File menu. In SQL*Plus, a normal exit is accomplished by entering the `EXIT` command at the prompt. Closing the window is interpreted as an abnormal exit.

State of Data Before COMMIT or ROLLBACK

- You can recover the data of the previous state.
- You can review the results of the DML operations by using the `SELECT` statement in the current session.
- Other sessions *cannot* view the results of the DML statements issued by the current session.
- The affected rows are locked; other sessions cannot change the data in the affected rows.

Current session changes stored in buffer



Every data change made during the transaction is temporary until the transaction is committed.

The state of the data before `COMMIT` or `ROLLBACK` statements are issued can be described as follows:

- Data manipulation operations primarily affect the database buffer; therefore, the previous state of the data can be recovered.
- The current session can review the results of the data manipulation operations by querying the tables.
- Other sessions cannot view the results of the data manipulation operations made by the current session. The Oracle server institutes read consistency to ensure that each session sees data as it existed at the last commit.
- The affected rows are locked; other sessions cannot change the data in the affected rows.

For Instructor Use Only.
This document should not be distributed

State of Data After COMMIT

- Data changes are saved in the database.
- The previous state of the data is overwritten.
- All sessions can view the results.
- Locks on the affected rows are released; those rows are available for other sessions to manipulate.
- All savepoints are erased.



Make all pending changes permanent by using the `COMMIT` statement. The following happens after a `COMMIT` statement:

- Data changes are written to the database.
- The previous state of the data is no longer available with normal SQL queries.
- All sessions can view the results of the transaction.
- The locks on the affected rows are released; the rows are now available for other sessions to perform new data changes.
- All savepoints are erased.

Committing Data

- Make the changes:

```
DELETE FROM employees
WHERE employee_id = 113;
1 row deleted.

INSERT INTO departments
VALUES (290, 'Corporate Tax', NULL, 1700);
1 row inserted.
```

- Commit the changes:

```
COMMIT;
Commit complete.
```

In the example in the slide, a row is deleted from the `EMPLOYEES` table and a new row is inserted into the `DEPARTMENTS` table. The changes are saved by issuing the `COMMIT` statement.

Example

Remove departments 290 and 300 from the `DEPARTMENTS` table and update a row in the `EMPLOYEES` table. Save the data change.

```
DELETE FROM departments
WHERE department_id IN (290, 300);

UPDATE employees
SET department_id = 80
WHERE employee_id = 206;

COMMIT;
```

State of Data After ROLLBACK

Discard all pending changes by using the `ROLLBACK` statement:

- Data changes are undone.
- Previous state of the data is restored.
- Locks on the affected rows are released.

```
DELETE FROM copy_emp;  
ROLLBACK ;
```



Discard all pending changes by using the `ROLLBACK` statement, which results in the following:

- Data changes are undone.
- The previous state of the data is restored.
- Locks on the affected rows are released.

State of Data After ROLLBACK: Example

```
DELETE FROM test;
4 rows deleted.

ROLLBACK;
Rollback complete.

DELETE FROM test WHERE id = 100;
1 row deleted.

SELECT * FROM test WHERE id = 100;
No rows selected.

COMMIT;
Commit complete.
```

While attempting to remove a record from the `TEST` table, you may accidentally empty the table. However, you can correct the mistake by if you roll back, reissue a proper statement, and make the data change permanent with `COMMIT` statement.

Note: Refer to `code_ex_10.sql` and `code_10_cretest.sql` in the `/home/oracle/labs/Dev3PDB/code_ex` folder to create the test table.

Statement-Level Rollback

- If a single DML statement fails during execution, only that statement is rolled back.
- The Oracle server implements an implicit savepoint.
- All other changes are retained.
- The user should terminate transactions explicitly by executing a `COMMIT` or `ROLLBACK` statement.



If a single DML statement fails during the execution of a transaction, its effect is undone by a statement-level implicit rollback; however, the changes made by the previous DML statements in the transaction are not discarded. These can be committed or rolled back explicitly by the user.

The Oracle server issues an implicit commit before and after any DDL statement. So, even if your DDL statement does not execute successfully, you cannot roll back the previous statement because the server issued a commit.

Terminate your transactions explicitly by executing a `COMMIT` or `ROLLBACK` statement.



Read Consistency

For Instructor Use Only.
This document should not be distributed

Read Consistency

- Read consistency guarantees a consistent view of data at all times.
- Changes made by one user do not conflict with the changes made by another user.
- Read consistency ensures that, on the same data:
 - Readers do not wait for writers
 - Writers do not wait for readers
 - Writers wait for writers



Database users access the database in two ways:

- Read operations (`SELECT` statement)
- Write operations (`INSERT`, `UPDATE`, `DELETE` statements)

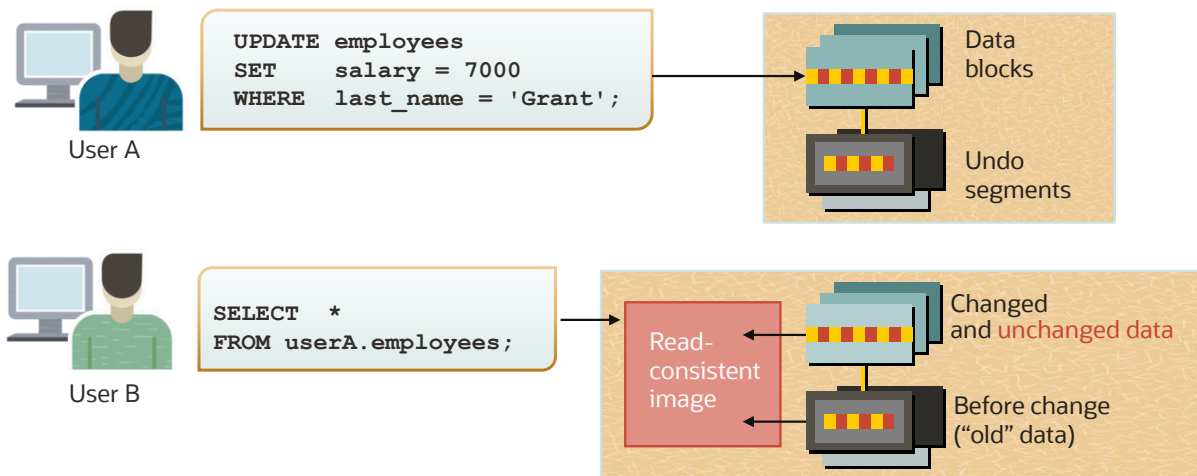
You need read consistency so that:

- The database reader and writer are ensured a consistent view of data
- Readers do not view data that is in the process of being changed
- Writers are ensured that the changes to the database are done in a consistent manner
- Changes made by one writer do not disrupt or conflict with the changes being made by another writer

The purpose of read consistency is to ensure that each user sees data as it existed at the last commit, before a DML operation started.

Note: The same user can log in to different sessions. Each session maintains read consistency in the manner described above, even if they are the same users.

Implementing Read Consistency



Read consistency is an automatic implementation. It keeps a partial copy of the database in the undo segments. The read-consistent image is constructed from the committed data in the table and the old data that is being changed and is not yet committed from the undo segment.

When an insert, update, or delete operation is made on the database, the Oracle server takes a copy of the data before it is changed and writes it to an *undo segment*.

All readers, except the one who issued the change, see the database as it existed before the changes started; they view the undo segment's "snapshot" of the data.

Before the changes are committed to the database, only the user who is modifying the data sees the database with the alterations. Everyone else sees the snapshot in the undo segment. This guarantees that readers read consistent data that is not currently undergoing change.

When a DML statement is committed, the change made to the database becomes visible to anyone issuing a `SELECT` statement *after* the commit is done. The space occupied by the *old* data in the undo segment file is freed for reuse.

If the transaction is rolled back, the changes are undone:

- The original, older version of the data in the undo segment is written back to the table.
- All users see the database as it existed before the transaction began.



Manual Data Locking

For Instructor Use Only.
This document should not be distributed

FOR UPDATE Clause in a SELECT Statement

- It locks the rows in the `EMPLOYEES` table where `job_id` is `SA_REP`.

```
SELECT employee_id, salary, commission_pct, job_id
FROM employees
WHERE job_id = 'SA_REP'
FOR UPDATE
ORDER BY employee_id;
```

- Lock is released only when you issue a `ROLLBACK` or a `COMMIT`.
- If the `SELECT` statement attempts to lock a row that is locked by another user, the database waits until the row is available and then returns the results of the `SELECT` statement.

When you issue a `SELECT` statement against the database to query some records, no locks are placed on the selected rows. In general, this is required because the number of records locked at any given time is (by default) kept to the absolute minimum: only those records that have been changed but not yet committed are locked. Even then, others will be able to read those records as they appeared before the change (the “before image” of the data). There are times, however, when you may want to lock a set of records even before you change them in your program. Oracle offers the `FOR UPDATE` clause of the `SELECT` statement to perform this locking.

When you issue a `SELECT . . . FOR UPDATE` statement, RDBMS automatically obtains exclusive row-level locks on all the rows identified by the `SELECT` statement, thereby holding the records “for your changes only.” No one else will be able to change any of these records until you perform a `ROLLBACK` or a `COMMIT`.

You can append the optional keyword `NOWAIT` to the `FOR UPDATE` clause to tell the Oracle server not to wait if the table has been locked by another user. In this case, control will be returned immediately to your program or to your SQL Developer environment so that you can perform other work, or simply wait for a period of time before trying again. Without the `NOWAIT` clause, your process will block until the table is available, when the locks are released by the other user through the issue of a `COMMIT` or a `ROLLBACK` command.

FOR UPDATE Clause: Examples

- You can use the `FOR UPDATE` clause in a `SELECT` statement against multiple tables.

```
SELECT e.employee_id, e.salary, e.commission_pct
FROM employees e JOIN departments d
USING (department_id)
WHERE job_id = 'ST_CLERK'
AND location_id = 1500
FOR UPDATE
ORDER BY e.employee_id;
```

- Rows from both the `EMPLOYEES` and `DEPARTMENTS` tables are locked.
- Use `FOR UPDATE OF column_name` to qualify the column that you intend to change; then only the rows from that specific table are locked.

In the example in the slide, the statement locks rows in the `EMPLOYEES` table with `JOB_ID` set to `ST_CLERK` and `LOCATION_ID` set to 1500 and locks rows in the `DEPARTMENTS` table with departments in `LOCATION_ID` set as 1500.

You can use the `FOR UPDATE OF column_name` to qualify the column that you intend to change. The `OF` list of the `FOR UPDATE` clause does not restrict you to changing only those columns of the selected rows. Locks are still placed on all rows; if you simply state `FOR UPDATE` in the query and do not include one or more columns after the `OF` keyword, the database will lock all identified rows across all the tables listed in the `FROM` clause.

The following statement locks only those rows in the `EMPLOYEES` table with `ST_CLERK` located in `LOCATION_ID` 1500. No rows are locked in the `DEPARTMENTS` table:

```
SELECT e.employee_id, e.salary, e.commission_pct
FROM employees e JOIN departments d
USING (department_id)
WHERE job_id = 'ST_CLERK' AND location_id = 1500
FOR UPDATE OF e.salary
ORDER BY e.employee_id;
```

In the following example, the database is instructed to wait for five seconds for the row to become available and then return control to you.

```
SELECT employee_id, salary, commission_pct, job_id
FROM employees
WHERE job_id = 'SA_REP'
FOR UPDATE WAIT 5
ORDER BY employee_id;
```


LOCK TABLE Statement

- Use the `LOCK TABLE` statement to lock one or more tables in a specified mode.
- This manually overrides automatic locking.
- Tables are locked until you use `COMMIT` or `ROLLBACK`.

```
LOCK TABLE table_name  
IN [ROW SHARE/ROW EXCLUSIVE/SHARE UPDATE/SHARE/  
    SHARE ROW EXCLUSIVE/ EXCLUSIVE] MODE  
[NOWAIT];
```



You can use the `LOCK TABLE` statement to manually lock the tables and override automatic locking. The table must be in your schema or you must have the `LOCK ANY TABLE` privilege.

The *lockmode* clause:

<code>SHARE</code>	Permits concurrent queries but prevents update on the locked table
<code>EXCLUSIVE</code>	Permits queries on the locked table but prevents any other activity

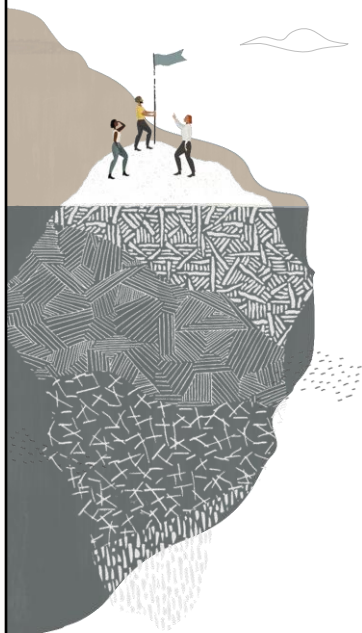
For more information about the other *lockmode* clauses, refer to the `LOCK TABLE` statement in *Oracle Database SQL Language Reference* for 23c database.

You can append the optional keyword `NOWAIT` to the `LOCK TABLE` statement to tell the Oracle server not to wait if the table has been locked by another user.

For example, the following statement locks the `EMPLOYEES` table in exclusive mode but does not wait if another user has already locked the table.

```
LOCK TABLE employees  
IN EXCLUSIVE MODE  
NOWAIT;
```

Summary



Adding new rows in a table

Changing data in a table

Removing rows from a table

Database transaction control

Read consistency

Manual Data Locking

In this lesson, you should have learned how to manipulate data in the Oracle database by using the `INSERT`, `UPDATE`, `DELETE`, and `TRUNCATE` statements, as well as how to control data changes by using the `COMMIT`, `SAVEPOINT` and `ROLLBACK` statements. You should have also learned how to use the `FOR UPDATE` clause of the `SELECT` statement to lock rows for your changes only.

Remember that the Oracle server guarantees a consistent view of data at all times.

For Instructor Use Only.
This document should not be distributed

Introduction to Data Definition Language



Course Roadmap

Lesson 1: Introduction

Unit 1: Retrieving, Restricting and
Sorting Data

Unit 2: Joins, Subqueries, and Set
Operators

Unit 3: DML and DDL



Lesson 10: Managing Tables Using DML
Statements



**Lesson 11: Introduction to Data Definition
Language**

You are here!

In Unit 3, you learn how to create and manage database objects using data definition language (DDL) statements. You will also learn how to manage data in the tables by using data manipulation language (DML) statements.

Objectives



Database objects

CREATE TABLE statement

Data types

Overview of constraints

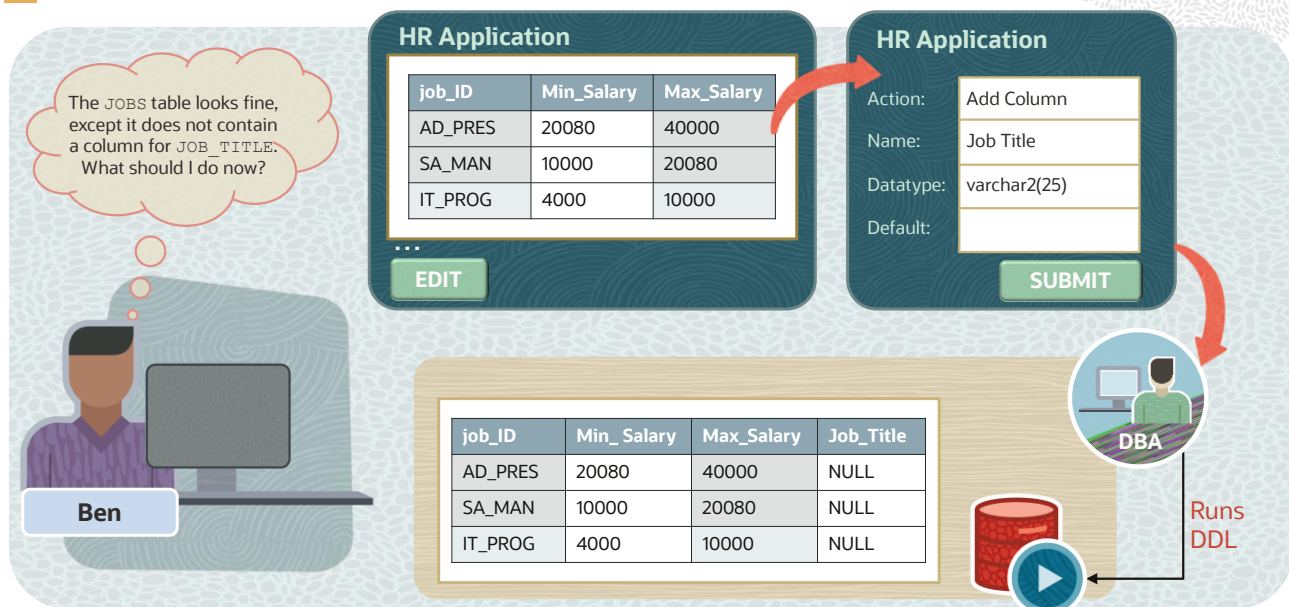
Creating a table by using a subquery

ALTER TABLE statement

DROP TABLE statement

In this lesson, you are introduced to the data definition language (DDL) statements. You learn the basics of creating simple tables, altering them, and removing them. The data types available in DDL are shown and schema concepts are introduced. Constraints are discussed in this lesson. Exception messages that are generated from violating constraints during DML operations are shown and explained.

HR Application Scenario



Consider a scenario where Bob, an HR Manager, is creating tables to store all the employee information in the database. While creating the `JOBS` table to store all the job information, he forgets to create a column for the job title. So what should Bob do now?

Should he drop the table and create the `JOBS` table again? No!

Bob can alter the table and add a new column (`JOB_TITLE`) to the existing `JOBS` table. The statements that allow you to modify the structure of database objects are called data definition language (DDL) statements. Usually, the permission to execute DDL statements is given only to the admin.

Bob submits the request to alter the `JOBS` table structure along with the details. The DBA receives the request and constructs an appropriate SQL statement to alter the `JOBS` table.

A new column called `Job_Title` is added to the `JOBS` table. The value for `Job_Title` remains `NULL` until Bob goes to the application and updates the values for all the records.

In this lesson, you learn more about DDL statements.



Database Objects

For Instructor Use Only.
This document should not be distributed

Database Objects

Object	Description
Table	Is the basic unit of storage; composed of rows
View	Logically represents subsets of data from one or more tables
Sequence	Generates numeric values
Index	Improves the performance of some queries
Synonym	Gives alternative name to an object



The Oracle Database can contain multiple data objects. Remember to outline each object in the database design so that it can be created during the build stage of database development.

- **Table:** Stores data
- **View:** Is a subset of data from one or more tables
- **Sequence:** Generates numeric values
- **Index:** Improves the performance of some queries
- **Synonym:** Gives an alternative name to an object

Oracle Table Structures

- You can create tables at any time, even when users are using the database.
- You do not need to specify the size of a table. The size is ultimately defined by the amount of space allocated to the database as a whole. It is important, however, to estimate how much space a table will use over time.
- You can also modify the table structure online.

Naming Rules for Tables and Columns

Ensure that the table names and column names:

- Begin with a letter
- Are no more than 128 characters long
- Contain only A–Z, a–z, 0–9, _, \$, and #
- Do not duplicate the name of another object owned by the same user
- Are not Oracle server–reserved words



Note: Oracle Database prior to version 12.2 limit identifier names, such as table names, column names, and primary key names, to 30 characters. Oracle Database 12.2 and later have a default limit of 128 characters.

Name the database tables and columns according to the standard rules for naming any Oracle database object.

- Table names and column names must begin with a letter and be no more than 128 characters long.
- Names must contain only the characters A–Z, a–z, 0–9, _ (underscore), \$, and # (legal characters, but their use is discouraged).
- Names must not duplicate the name of another object owned by the same Oracle server user.
- Names must not be an Oracle server–reserved word.
 - You may also use quoted identifiers to represent the name of an object. A quoted identifier begins and ends with double quotation marks ("""). If you name a schema object using a quoted identifier, you must use the double quotation marks whenever you refer to that object. Quoted identifiers can be reserved words, although this is not recommended.

Naming Guidelines

Use descriptive names for tables and other database objects.

Note: Names are not case-sensitive. For example, `EMPLOYEES` is treated to be the same name as `eMpLoYeeS` or `EMPLOYEEs`. However, quoted identifiers are case-sensitive.

For more information, see the “Schema Object Names and Qualifiers” section in the *Oracle Database SQL Language Reference* for 23c database.



CREATE TABLE Statement

For Instructor Use Only.
This document should not be distributed

CREATE TABLE Statement

- You must have:
 - CREATE TABLE privilege
 - Storage area

```
CREATE TABLE [schema.]table
    (column datatype [DEFAULT expr][, ...]);
```

- You specify:
 - Table name
 - Column name, column data type, and column size



You create tables to store data by executing the SQL `CREATE TABLE` statement. This statement is one of the DDL statements that are a subset of the SQL statements used to create, modify, or remove Oracle Database structures. These statements have an immediate effect on the database, and they also record information in the data dictionary. The data dictionary is an important set of read-only tables that provide database information.

To create a table, a user must have the `CREATE TABLE` privilege and a storage area in which to create objects. The database administrator (DBA) uses data control language (DCL) statements to grant privileges to users.

In the syntax:

<i>schema</i>	Is the same as the owner's name
<i>table</i>	Is the name of the table
<code>DEFAULT expr</code>	Specifies a default value if a value is omitted in the <code>INSERT</code> statement
<i>column</i>	Is the name of the column
<i>datatype</i>	Is the column's data type and length

Note: The `CREATE ANY TABLE` privilege is needed to create a table in any schema other than the user's schema.

Creating Tables

- Create the table:

```
CREATE TABLE dept
  (deptno      NUMBER(2) ,
   dname       VARCHAR2(14) ,
   loc         VARCHAR2(13) ,
   create_date DATE DEFAULT SYSDATE) ;
```

table DEPT created.

- Confirm table creation:

```
DESCRIBE dept
```

DESCRIBE dept	
Name	Null Type

DEPTNO	NUMBER(2)
DNAME	VARCHAR2(14)
LOC	VARCHAR2(13)
CREATE_DATE	DATE

The example in the slide creates the DEPT table with four columns: DEPTNO, DNAME, LOC, and CREATE_DATE.

The CREATE_DATE column has a default value. If a value is not provided for an INSERT statement, the system date is automatically inserted.

To confirm that the table was created, run the DESCRIBE command.

Because creating a table is a DDL statement, an automatic commit takes place when this statement is executed.

Note: You can view the list of tables that you own by querying the data dictionary, as shown in the following example:

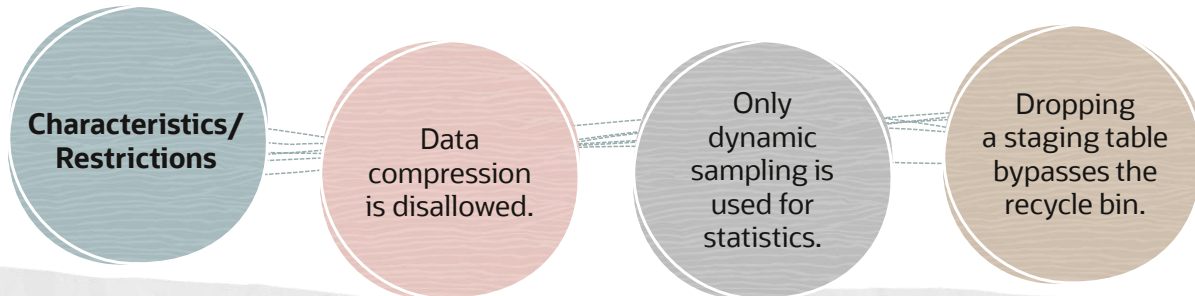
```
select table_name from user_tables;
```

Using data dictionary views, you can also find information about other database objects, such as views, indexes, and so on. You will learn about data dictionaries in detail in the lesson titled “Manipulating Data Using Advanced Queries” in the course.

Staging Tables

Are a new type of table in 23c that are optimized for loading data into a data warehouse

- Can be created as new tables, or existing tables can be altered for staging
- Typically created during the data load process
- Are either dropped or truncated after data has been inserted into production tables



Staging tables help customers to **avoid slow data loads**.

The default table attributes for new tables in a data warehouse are often not well-suited for fast data loading.

- **Best practice:** Tables in data warehouses should use compression by default. We should be compressing almost all tables in a data warehouse, except the staging tables.

Staging tables make it simple for customers to indicate which tables will be used for data loading, without having to set any other table attributes.

Benefits include:

- Help to improve data warehouse performance for data loading
- Are easy to use for nonexpert Oracle users and third-party tools
- Avoid unnecessary tuning and debugging for data loading
- Are especially useful for Autonomous Data Warehouse

Creating Staging Tables: Methods

Staging tables created on demand

- A data-load job process creates one or more staging tables and loads data.
- Data from the staging tables is transformed and then inserted into production tables.
- Staging tables are then truncated or deleted.

Existing tables used for data loading

- Existing tables are altered for staging.
- Data is loaded into existing tables.
- After data load is complete, existing tables are altered as not for staging.
- Statistics are gathered on existing tables.
- Existing tables can be recompressed when convenient time.

In this slide, we are discussing the two different methods for creating staging tables:

- Creation of a new staging table
- Altering an existing table for staging

Using Staging Tables: Example

How to use staging tables:

Create a new staging table:

```
CREATE TABLE staging_table (col1 number, col2 varchar2(100)) FOR STAGING;
```

Alter an existing table:

```
ALTER TABLE staging_table FOR STAGING;
```

When creating a new staging table, no other storage attributes are needed.

This feature is also available in Autonomous Data Warehouse and documented at the following location:

<https://docs.oracle.com/en/cloud/paas/autonomous-database/adbsa/appendix-autonomous-database-data-warehouse-workload.html#GUID-251BE943-338B-462A-9E65-C79B608B65D6>



Data Types

For Instructor Use Only.
This document should not be distributed

Data Types

Data Type	Description
VARCHAR2 (<i>size</i>)	Variable-length character data
CHAR (<i>size</i>)	Fixed-length character data
NUMBER (<i>p</i> , <i>s</i>)	Variable-length numeric data
DATE	Date and time values
LONG	Variable-length character data (up to 2 GB)
CLOB	Maximum size is (4 gigabytes - 1) * (DB_BLOCK_SIZE).
RAW and LONG RAW	Raw binary data
BLOB	Maximum size is (4 gigabytes - 1) * (DB_BLOCK_SIZE initialization parameter (8 TB to 128 TB)).
BFILE	Binary data stored in an external file (up to 4 GB)
ROWID	A base-64 number system representing the unique address of a row in its table
BOOLEAN	The BOOLEAN data type comprises the distinct truth values True and False. Unless prohibited by a NOT NULL constraint, the Boolean data type also supports the truth value UNKNOWN as the null value.

When you identify a column for a table, you need to provide a data type for the column. In addition to the built-in data types listed in the slide, Oracle Database uses many data types internally that are visible via the `DUMP` function. There are several data types available.

Data Type	Description
<code>VARCHAR2</code> (<i>size</i>)	Variable-length character data (a maximum <i>size</i> must be specified; minimum <i>size</i> is 1) Maximum size is: 32767 bytes if <code>MAX_SQL_STRING_SIZE</code> = <code>EXTENDED</code> 4000 bytes if <code>MAX_SQL_STRING_SIZE</code> = <code>LEGACY</code>
<code>CHAR</code> [<i>size</i>]	Fixed-length character data of length <i>size</i> bytes (default and minimum <i>size</i> is 1; maximum <i>size</i> is 2,000.)
<code>NUMBER</code> [<i>p</i> , <i>s</i>]	Number having precision <i>p</i> and scale <i>s</i> (Precision is the total number of decimal digits and scale is the number of digits to the right of the decimal point; precision can range from 1 through 38, and scale can range from -84 through 127.)
<code>DATE</code>	Date and time values to the nearest second between January 1, 4712 B.C., and December 31, 9999 A.D.

Data Type	Description
LONG	Variable-length character data (up to 2 GB)
CLOB	Character large object containing single-byte or multibyte characters. Maximum size is (4 gigabytes - 1) * (DB_BLOCK_SIZE); stores national character set data.
NCLOB	Character large object containing Unicode characters. Both fixed-width and variable-width character sets are supported, both using the database national character set. Maximum size is (4 gigabytes - 1) * (database block size); stores national character set data.
RAW(size)	Raw binary data of length <i>size</i> bytes. You must specify <i>size</i> for a RAW value. Maximum <i>size</i> is: 32767 bytes if MAX_SQL_STRING_SIZE = EXTENDED 4000 bytes if MAX_SQL_STRING_SIZE = LEGACY
LONG RAW	Raw binary data of variable length up to 2 gigabytes
BLOB	Binary large object. Maximum size is (4 gigabytes - 1) * (DB_BLOCK_SIZE initialization parameter (8 TB to 128 TB)).
BFILE	Binary data stored in an external file (up to 4 GB)
ROWID	Base 64 string representing the unique address of a row in its table. This data type is primarily for values returned by the ROWID pseudocolumn.

Guidelines

- A LONG column is not copied when a table is created using a subquery.
- A LONG column cannot be included in a GROUP BY or an ORDER BY clause.
- Only one LONG column can be used per table.
- No constraints can be defined on a LONG column.
- You might want to use a CLOB column rather than a LONG column.

Datetime Data Types

You can use several datetime data types:

Data Type	Description
TIMESTAMP	Date with fractional seconds
INTERVAL YEAR TO MONTH	Stored as an interval of years and months
INTERVAL DAY TO SECOND	Stored as an interval of days, hours, minutes, and seconds



Data Type	Description
TIMESTAMP	Enables storage of time as a date with fractional seconds. It stores the year, month, day, hour, minute, and the second value of the DATE data type, as well as the fractional seconds value. There are several variations of this data type, such as WITH TIMEZONE and WITH LOCALTIMEZONE.
INTERVAL YEAR TO MONTH	Enables storage of time as an interval of years and months; used to represent the difference between two datetime values in which the only significant portions are the year and month
INTERVAL DAY TO SECOND	Enables storage of time as an interval of days, hours, minutes, and seconds; used to represent the precise difference between two datetime values

Note: The datetime data types are discussed in detail in the lesson titled “Managing Data in Different Time Zones” in the *Oracle Database: SQL Workshop II* course.

Also, for more information about datetime data types, see the sections on “TIMESTAMP Datatype,” “INTERVAL YEAR TO MONTH Datatype,” and “INTERVAL DAY TO SECOND Datatype” in *Oracle Database SQL Language Reference* for 23c database.

SQL*Plus Support for Boolean Data Type in 23c

Enables `BOOLEAN` data type to return a different text literal, other than `'TRUE'` or `'FALSE'`

Benefits

- ✓ Increases SQL standard support
- ✓ Is compatible with features offered by other vendors



SQL Boolean data type was available in different vendor database/applications. With this feature, it makes the SQL*Plus tool easier to use and is similar to other existing database vendors. It allows application development that is simpler and less error prone.

The `DESCRIBE` command will be able to display metadata for an object that contains a Boolean data type like any data type.

Creating a Table with Boolean Type and Inserting Data: Example

```
CREATE TABLE example1 (  
    id NUMBER,  
    c2 boolean,  
    c3 boolean  
);  
  
INSERT INTO example1  
VALUES(1,'true','false'), (2,'no','yes'), (3,'n','y'), (4,'no','yes');
```

Querying the Table Containing BOOLEAN Values in SQL Developer: Example



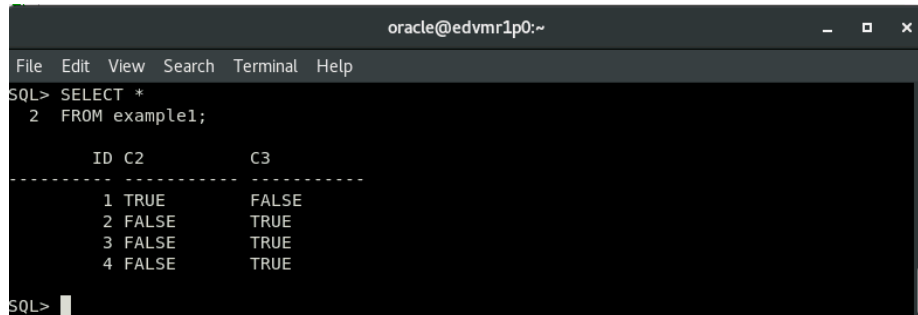
The screenshot shows the SQL Developer interface with a query window titled 'Sol_sql_boolean.sql'. The query is: `--Query the example1 table in SQL DEVELOPER`
`SELECT *`
`FROM example1;`

The 'Query Result' tab is active, showing a table with 4 rows and 3 columns. The columns are labeled 'ID', 'C2', and 'C3'. The data is as follows:

	ID	C2	C3
1	1	1	0
2	2	0	1
3	3	0	1
4	4	0	1

For Instructor Use Only.
This document should not be distributed

Default Output in SQL*Plus: Example



The screenshot shows a terminal window titled 'oracle@edvmr1p0:~'. The terminal contains the following text:

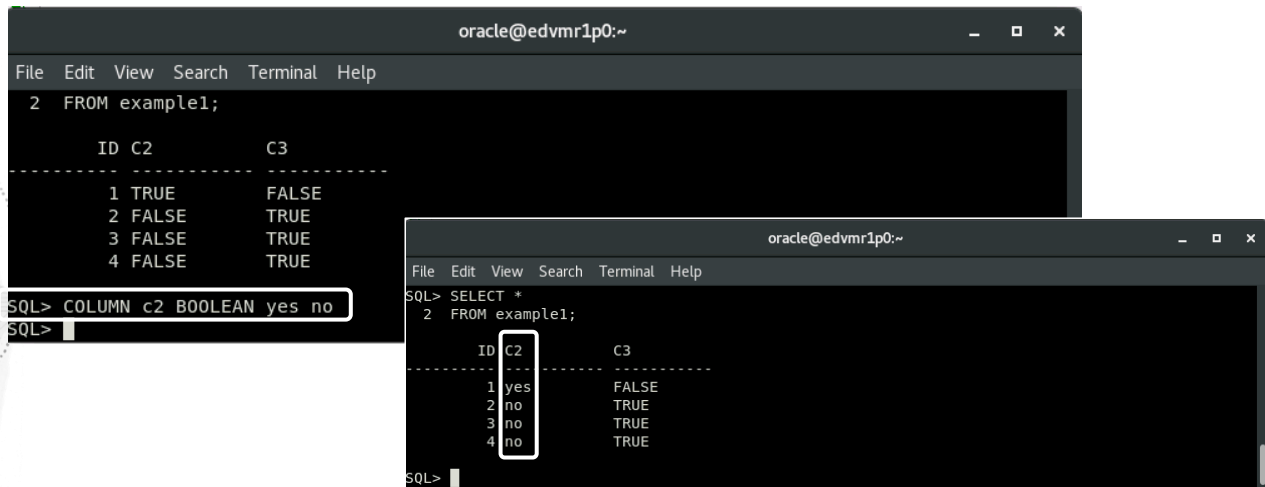
```
File Edit View Search Terminal Help
SQL> SELECT *
  2  FROM example1;

   ID C2      C3
-----
  1 TRUE     FALSE
  2 FALSE    TRUE
  3 FALSE    TRUE
  4 FALSE    TRUE
SQL>
```

The output is a table with three columns: ID, C2, and C3. The data rows are as follows:

ID	C2	C3
1	TRUE	FALSE
2	FALSE	TRUE
3	FALSE	TRUE
4	FALSE	TRUE

Using the COLUMN Command to Customize Output: Example



The image shows two terminal windows side-by-side, both titled 'oracle@edvmr1p0:~'. The top window shows the output of a query 'SELECT * FROM example1;'. The output is a table with columns ID, C2, and C3. The bottom window shows the same query result after using the 'COLUMN c2 BOOLEAN yes no' command to format the C2 column. The output is a table with columns ID, C2, and C3, where C2 is formatted as 'yes' or 'no'.

```
File Edit View Search Terminal Help
2 FROM example1;

  ID C2      C3
-----
  1 TRUE     FALSE
  2 FALSE    TRUE
  3 FALSE    TRUE
  4 FALSE    TRUE

SQL> COLUMN c2 BOOLEAN yes no
SQL>
```

```
File Edit View Search Terminal Help
SQL> SELECT *
2 FROM example1;

  ID C2      C3
-----
  1 yes     FALSE
  2 no      TRUE
  3 no      TRUE
  4 no      TRUE

SQL>
```

DEFAULT Option

- Specify a default value for a column in the `CREATE TABLE` statement.

```
... hire_date DATE DEFAULT SYSDATE, ...
```

- Literal values, expressions, or SQL functions are legal values.
- Another column's name or a pseudocolumn is an illegal value.
- Default data type must match the column data type.

```
CREATE TABLE hire_dates  
  (id      NUMBER(8),  
   hire_date DATE DEFAULT SYSDATE);
```

```
Table HIRE_DATES created.
```

When you define a table, you can specify that a column should be given a default value by using the `DEFAULT` option. This option prevents null values from entering the columns when a row is inserted without a value for the column.

The default value can be a literal, an expression, or a SQL function (such as `SYSDATE` or `USER`); however, the value cannot be the name of another column or a pseudocolumn (such as `NEXTVAL` or `CURRVAL`). The default expression must match the data type of the column.

Consider the following examples:

```
INSERT INTO hire_dates values(45, NULL);
```

The preceding statement will insert the null value rather than the default value.

```
INSERT INTO hire_dates(id) values(35);
```

The preceding statement will insert `SYSDATE` for the `HIRE_DATE` column.

Note: In SQL Developer, click the Run Script icon or press F5 to run the DDL statements. The feedback messages will be shown on the Script Output tab.

23c Enhancement to SQL Statements

DEFAULT ON NULL for UPDATE statements

- Allows column values to be automatically updated to a specific default value during an update operation
- Replaces previous methods used that caused extra overhead
- Can be used in UPDATE components in MERGE statements as well

Benefits

- ✓ Eliminates the need for triggers to implement similar functionality
- ✓ Simplifies migration from other platforms



Historically this was accomplished by using triggers, which added extra overhead for both performance and maintainability.

The DEFAULT ON NULL functionality was added in **12c** for INSERT; however, UPDATE still required the use of triggers or additional application code.

The first two of the following statements are equivalent:

```
create table t1(c2 varchar2(10) default on null 'abc');
```

```
create table t1(c2 varchar2(10) default on null for insert only 'abc');
```

```
create table t1(c2 varchar2(10) default on null for insert and update 'abc');
```

Existing tables can also be modified to support these features:

```
alter table t1 modify (c2 default on null for insert only 'abc');
```

```
alter table t1 modify (c2 default on null for insert and update 'abc');
```

UPDATE Option to the Existing DEFAULT ON NULL Clause: Example

```
create table demo_t(  
    id number,  
    ename varchar2(100) DEFAULT ON NULL for insert and UPDATE  
    'missing name'  
);
```

For Instructor Use Only.
This document should not be distributed

Updating demo_t and Setting ename to the 'missing name' Value: Example

```
UPDATE demo_t set ename = null where id = 100;
```

For Instructor Use Only.
This document should not be distributed

DEFAULT ON NULL with IDENTITY Columns and Update Column c2 to <sequence>.nextval: Example

```
CREATE table t1(c1 number,  
               c2 number generated by default on null for insert  
and update as identity);
```

```
update t1 set c2 = null where c1 = 100;
```

Identity columns can also have `INSERT` and `UPDATE DEFAULT ON NULL`. In this case, the internally defined `sequence.nextval` value will be used.



Constraints: Overview

For Instructor Use Only.
This document should not be distributed

Including Constraints

- Constraints enforce rules at the table level.
- Constraints ensure consistency and integrity of the database.
- Valid constraint types:
 - NOT NULL
 - UNIQUE
 - PRIMARY KEY
 - FOREIGN KEY
 - CHECK



The Oracle server uses constraints to prevent invalid data entry into tables.

You can use constraints to do the following:

- Enforce rules on the data in a table whenever a row is inserted, updated, or deleted from that table. The constraint must be satisfied for the operation to succeed.
- Prevent the dropping of a table if there are dependencies from other tables
- Provide rules for Oracle tools, such as Oracle Developer

Data Integrity Constraints

Constraint	Description
NOT NULL NOT NULL	Specifies that the column cannot contain a null value
UNIQUE	Specifies a column or combination of columns whose values must be unique for all rows in the table
PRIMARY KEY	Uniquely identifies each row of the table
FOREIGN KEY	Establishes and enforces a referential integrity between the column and a column of the referenced table such that values in one table match values in another table
CHECK	Specifies a condition that must be true

Constraint Guidelines

- You can name a constraint, or the Oracle server generates a name by using the `SYS_Cn` format.
- Create a constraint at either of the following times:
 - At the time of table creation
 - After the creation of the table
- Define a constraint at the column or table level.
- View a constraint in the data dictionary.



All constraints are stored in the data dictionary.

Constraints are easy to reference if you give them a meaningful name. Constraint names must follow the standard object-naming rules, except that the name cannot be the same as another object owned by the same user. If you do not name your constraint, the Oracle server generates a name with the format `SYS_Cn`, where *n* is an integer so that the constraint name is unique.

Constraints can be defined at the time of table creation or after the creation of the table. You can define a constraint at the column or table level. Functionally, a table-level constraint is the same as a column-level constraint.

For more information, see the section on “Constraints” in *Oracle Database SQL Language Reference* for 23c database.

Defining Constraints

- Syntax:

```
CREATE TABLE [schema.]table
  (column datatype [DEFAULT expr]
    [column_constraint],
    ...
    [table_constraint][,...]);
```

- Column-level constraint syntax:

```
column [CONSTRAINT constraint_name] constraint_type,
```

- Table-level constraint syntax:

```
column,...
[CONSTRAINT constraint_name] constraint_type
(column, ...),
```

The slide gives the syntax for defining constraints when creating a table. You can create constraints at the column level or the table level.

Constraints defined at the column level are included when the column is defined. Table-level constraints are defined at the end of the table definition and must refer to the column or columns to which the constraint pertains in a set of parentheses. It is mainly the syntax that differentiates the two; otherwise, functionally, a column-level constraint is the same as a table-level constraint. `NOT NULL` constraints can be defined only at the column level.

Constraints that apply to more than one column must be defined at the table level.

In the syntax:

schema	Same as the owner's name
table	Name of the table
DEFAULT expr	Specifies a default value to be used if a value is omitted in the INSERT statement
column	Name of the column
datatype	Column's data type and length
column_constraint	Integrity constraint as part of the column definition
table_constraint	Integrity constraint as part of the table definition

Defining Constraints: Example

- Example of a column-level constraint:

```
CREATE TABLE employees(  
  employee_id NUMBER(6)  
    CONSTRAINT emp_emp_id_pk PRIMARY KEY,  
  first_name VARCHAR2(20),  
  ...);
```

- Example of a table-level constraint:

```
CREATE TABLE employees(  
  employee_id NUMBER(6),  
  first_name VARCHAR2(20),  
  ...  
  job_id VARCHAR2(10) NOT NULL,  
  CONSTRAINT emp_emp_id_pk  
    PRIMARY KEY (EMPLOYEE_ID));
```

Constraints are usually created at the same time as the table. Constraints can be added to a table after its creation and also be temporarily disabled.

Both examples in the slide create a primary key constraint on the `EMPLOYEE_ID` column of the `EMPLOYEES` table.

1. The first example uses the column-level syntax to define the constraint.
2. The second example uses the table-level syntax to define the constraint.

More details about the primary key constraint are provided later in this lesson.

NOT NULL Constraint

Ensures that null values are not permitted for the column:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	COMMISSION_PCT	DEPARTMENT_ID	EMAIL	PHONE_NUMBER	HIRE_DATE
100	Steven	King	24000	(null)	90	SKING	515.123.4567	17-JUN-87
101	Neena	Kochhar	17000	(null)	90	NKOCHHAR	515.123.4568	21-SEP-89
102	Lex	De Haan	17000	(null)	90	LDEHAAN	515.123.4569	13-JAN-93
103	Alexander	Hunold	9000	(null)	60	AHUNOLD	590.423.4567	03-JAN-90
104	Bruce	Ernst	6000	(null)	60	BERNST	590.423.4568	21-MAY-91
107	Diana	Lorentz	4200	(null)	60	DLORENTZ	590.423.5567	07-FEB-99
124	Kevin	Mourgos	5800	(null)	50	KMOURGOS	650.123.5234	16-NOV-99
141	Trenna	Rajs	3500	(null)	50	TRAJS	650.121.8009	17-OCT-95
142	Curtis	Davies	3100	(null)	50	CDAVIES	650.121.2994	29-JAN-97
143	Randall	Matos	2600	(null)	50	RMATOS	650.121.2874	15-MAR-98
144	Peter	Vargas	2500	(null)	50	PVARGAS	650.121.2004	09-JUL-98
149	Eleni	Zlotkey	10500	0.2	80	EZLOTKEY	011.44.1344.429018	29-JAN-00
174	Ellen	Abel	11000	0.3	80	EABEL	011.44.1644.429267	11-MAY-96
176	Jonathon	Taylor	8600	0.2	80	JTAYLOR	011.44.1644.429265	24-MAR-98
178	Kimberely	Grant	7000	0.15	(null)	KGRANT	011.44.1644.429263	24-MAY-99
200	Jennifer	Whalen	4400	(null)	10	JWHALEN	515.123.4444	17-SEP-87
201	Michael	Hartstein	13000	(null)	20	MHARTSTE	515.123.5555	17-FEB-96
202	Pat	Fay	6000	(null)	20	PFAY	603.123.6666	17-AUG-97
205	Shelley	Higgins	12000	(null)	110	SHIGGINS	515.123.8080	07-JUN-94
206	William	Gietz	8300	(null)	110	WGIEZT	515.123.8181	07-JUN-94

NOT NULL constraint
(Primary key enforces
NOT NULL constraint.)

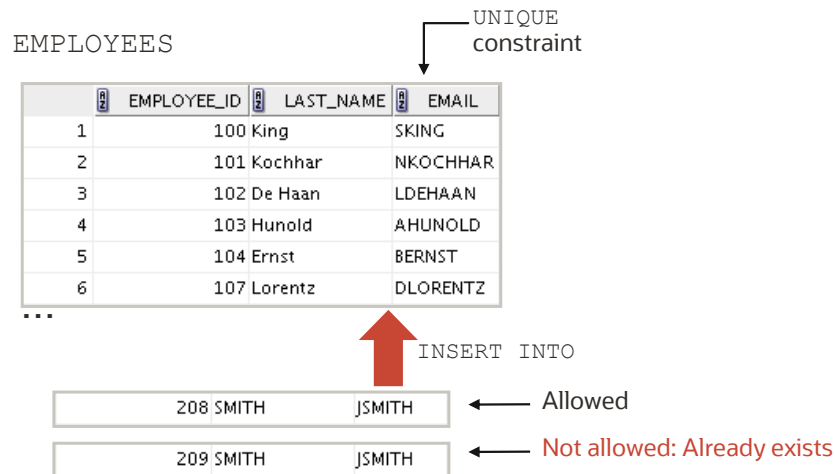
NOT NULL
constraint

Absence of NOT NULL constraint (Any row
can contain a null value for this column.)

The NOT NULL constraint ensures that the column contains no null values. Columns without the NOT NULL constraint can contain null values by default. NOT NULL constraints must be defined at the column level.

In the EMPLOYEES table, the EMPLOYEE_ID column inherits a NOT NULL constraint because it is defined as a primary key. Otherwise, the LAST_NAME, EMAIL, HIRE_DATE, and JOB_ID columns have the NOT NULL constraint enforced on them.

UNIQUE Constraint



A **UNIQUE** key integrity constraint requires that every value in a column or a set of columns (key) be unique—that is, no two rows of a table can have duplicate values in a specified column or a set of columns.

The column (or set of columns) included in the definition of the **UNIQUE** key constraint is called the *unique key*. If the **UNIQUE** constraint comprises more than one column, that group of columns is called a *composite unique key*.

UNIQUE constraints enable the input of nulls unless you also define **NOT NULL** constraints for the same columns. In fact, any number of rows can include nulls for columns without the **NOT NULL** constraints because nulls are not considered equal to anything. A null in a column (or in all columns of a composite **UNIQUE** key) always satisfies a **UNIQUE** constraint.

Note: Because of the search mechanism for the **UNIQUE** constraints on more than one column, you cannot have identical values in the non-null columns of a partially null composite **UNIQUE** key.

UNIQUE Constraint

Define at either the table level or the column level:

```
CREATE TABLE employees(  
    employee_id      NUMBER(6),  
    last_name        VARCHAR2(25) NOT NULL,  
    email            VARCHAR2(25),  
    salary           NUMBER(8,2),  
    commission_pct   NUMBER(2,2),  
    hire_date        DATE NOT NULL,  
    ...  
    CONSTRAINT emp_email_uk UNIQUE(email));
```

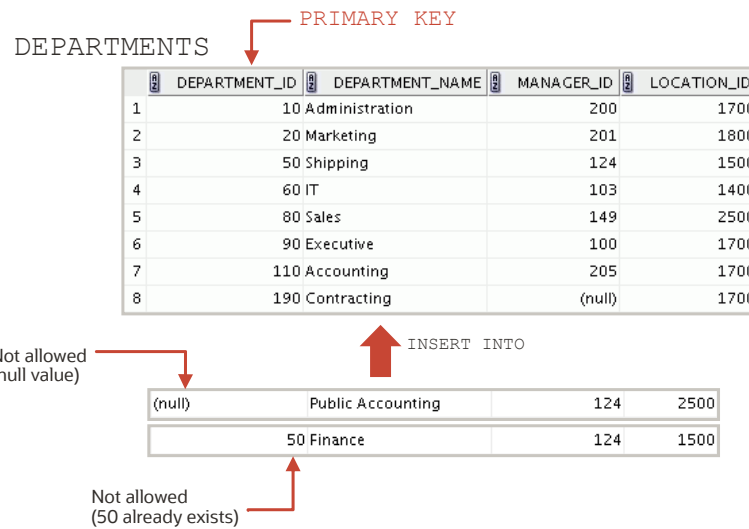
You can define `UNIQUE` constraints at the column level or table level.

You define the constraint at the table level when you want to create a composite unique key. A composite key is defined when there is not a single attribute that can uniquely identify a row. In that case, you can have a unique key that is composed of two or more columns, the combined value of which is always unique and can identify rows.

The example in the slide applies the `UNIQUE` constraint to the `EMAIL` column of the `EMPLOYEES` table. The name of the constraint is `EMP_EMAIL_UK`.

Note: The Oracle server enforces the `UNIQUE` constraint by implicitly creating a unique index on the unique key column or columns.

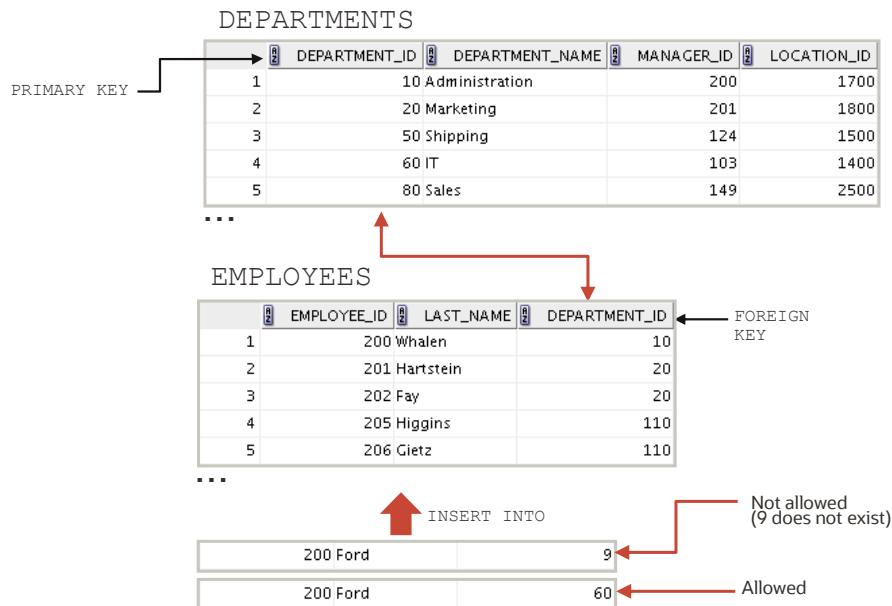
PRIMARY KEY Constraint



A **PRIMARY KEY** constraint creates a primary key for the table. You can create only one primary key for each table. The **PRIMARY KEY** constraint is a column or a set of columns that uniquely identifies each row in a table. This constraint enforces the uniqueness of the column or column combination and ensures that no column can contain a null value.

Note: Because uniqueness is part of the primary key constraint definition, the Oracle server enforces the uniqueness by implicitly creating a unique index on the primary key column or columns.

FOREIGN KEY Constraint



The **FOREIGN KEY** (or referential integrity) constraint designates a column or a combination of columns as a foreign key and establishes a relationship with a primary key or a unique key in the same table or a different table.

In the example in the slide, **DEPARTMENT_ID** has been defined as the foreign key in the **EMPLOYEES** table (dependent or child table); it references the **DEPARTMENT_ID** column of the **DEPARTMENTS** table (the referenced or parent table).

Guidelines

- A foreign key value must match an existing value in the parent table or be **NULL**.
- Foreign keys are based on data values and are purely logical, rather than physical, pointers.

FOREIGN KEY Constraint

Define at either the table level or the column level:

```
CREATE TABLE employees (  
    employee_id      NUMBER(6),  
    last_name        VARCHAR2(25) NOT NULL,  
    email            VARCHAR2(25),  
    salary           NUMBER(8,2),  
    commission_pct   NUMBER(2,2),  
    hire_date        DATE NOT NULL,  
    ...  
    department_id    NUMBER(4),  
    CONSTRAINT emp_dept_fk FOREIGN KEY (department_id)  
        REFERENCES departments(department_id),  
    CONSTRAINT emp_email_uk UNIQUE(email));
```

FOREIGN KEY constraints can be defined at the column or table constraint level. A composite foreign key must be created by using the table-level definition.

The example in the slide defines a FOREIGN KEY constraint on the DEPARTMENT_ID column of the EMPLOYEES table, using table-level syntax. The name of the constraint is EMP_DEPT_FK.

The foreign key can also be defined at the column level, provided that the constraint is based on a single column. The syntax differs in that the keywords FOREIGN KEY do not appear, as shown in the following example:

```
CREATE TABLE employees  
(  
    ...  
    department_id NUMBER(4) CONSTRAINT emp_deptid_fk  
        REFERENCES departments(department_id),  
    ...  
)
```

FOREIGN KEY Constraint: Keywords

- **FOREIGN KEY:** Defines the column in the child table at the table-constraint level
- **REFERENCES:** Identifies the table and column in the parent table
- **ON DELETE CASCADE:** Deletes the dependent rows in the child table when a row in the parent table is deleted
- **ON DELETE SET NULL:** Converts dependent foreign key values to null



The foreign key is defined in the child table, and the column it references is in the parent table. The foreign key is defined using a combination of the following keywords:

- **FOREIGN KEY** is used to define the column in the child table at the table-constraint level.
- **REFERENCES** identifies the table and the column in the parent table.
- **ON DELETE CASCADE** indicates that when a row in the parent table is deleted, the dependent rows in the child table are also deleted.
- **ON DELETE SET NULL** indicates that when a row in the parent table is deleted, the foreign key values are set to null.

The default behavior is called the *restrict rule*, which disallows the update or deletion of referenced data.

Without the **ON DELETE CASCADE** or the **ON DELETE SET NULL** options, the row in the parent table cannot be deleted if it is referenced in the child table. Also, these keywords cannot be used in column-level syntax.

CHECK Constraint

- Defines a condition that each row must satisfy
- Cannot reference columns from other tables

```
..., salary    NUMBER(2)
CONSTRAINT emp_salary_min
CHECK (salary > 0),...
```



When you define a `CHECK` constraint on a column, each row satisfies the condition. To satisfy the constraint, each row in the table must make the condition either `TRUE` or unknown (due to a null).

The condition can use the same constructs as the query conditions; however, they must not refer to other values in other rows.

A single column can have multiple `CHECK` constraints that refer to the column in its definition. There is no limit to the number of `CHECK` constraints that you can define on a column.

`CHECK` constraints can be defined at the column level or table level.

```
CREATE TABLE employees
(
    salary NUMBER(8,2) CONSTRAINT emp_salary_min
        CHECK (salary > 0),
    ...
)
```

CREATE TABLE: Example

```
CREATE TABLE teach_emp (  
    empno      NUMBER(5) PRIMARY KEY,  
    ename      VARCHAR2(15) NOT NULL,  
    job        VARCHAR2(10),  
    mgr        NUMBER(5),  
    hiredate   DATE DEFAULT (sysdate),  
    photo      BLOB,  
    sal        NUMBER(7,2),  
    deptno     NUMBER(3) NOT NULL  
    CONSTRAINT admin_dept_fkey  
    REFERENCES departments (department_id));
```

The example in the slide shows the statement that is used to create the `TEACH_EMP` table.

IF (NOT) EXISTS Enhancement

- Helps avoid errors when creating or dropping certain database objects
- Simplifies writing code

Benefits

- ✓ Writing less code
- ✓ Fewer errors
- ✓ Better developer experience
- ✓ Portability with other vendors



Starting in Oracle Database 23c

With this new feature, which completes the “`OR REPLACE`” syntax, developers can write code that avoids errors when creating or dropping certain database objects.

Note: *The “`OR REPLACE`” clause cannot coexist with the “`IF NOT EXISTS`” clause.*

IF NOT EXISTS Clause: Example

Creating the DEMO table if it does not currently exist

```
CREATE TABLE IF NOT EXISTS demo (id number, empname  
varchar2(100));
```

```
CREATE TABLE IF NOT EXISTS example.
```

Violating Constraints

Department 55 does not exist.

```
UPDATE employees
SET   department_id = 55
WHERE department_id = 110;
```



```
Error starting at line : 1 in command -
UPDATE employees
SET   department_id = 55
WHERE department_id = 110
Error report -
ORA-02291: integrity constraint (ORA1.EMP_DEPT_FK) violated - parent key not found
```

When you have constraints in place on columns, an error is returned if you try to violate the constraint rule. For example, if you try to update a record with a value that is tied to an integrity constraint, an error is returned.

In the example in the slide, department 55 does not exist in the parent table, `DEPARTMENTS`, and therefore, you receive the “parent key not found” violation `ORA-02291`.

Violating Constraints

You cannot delete a row that contains a primary key that is used as a foreign key in another table.

```
DELETE FROM departments
WHERE department_id = 60;
```



```
Error starting at line : 1 in command -
delete from departments
where department id = 60
Error report -
ORA-02292: integrity constraint (ORA1.EMP_DEPT_FK) violated - child record found
```

If you attempt to delete a record with a value that is tied to an integrity constraint, an error is returned.

The example in the slide tries to delete department 60 from the `DEPARTMENTS` table, but it results in an error because that department number is used as a foreign key in the `EMPLOYEES` table. If the parent record that you attempt to delete has child records, you receive the “child record found” violation `ORA-02292`.

The following statement works because there are no employees in department 70:

```
DELETE FROM departments
WHERE department_id = 70;
```

```
1 row deleted.
```




Creating a Table by Using a Subquery

—

For Instructor Use Only.
This document should not be distributed

Creating a Table by Using a Subquery

- Create a table and insert rows by combining the `CREATE TABLE` statement and the `AS subquery` option.

```
CREATE TABLE table [IF NOT EXISTS]
    [(column, column...)]
AS subquery;
```

- Match the number of specified columns to the number of subquery columns.
- Define columns with column names and default values.



A second method for creating a table is to apply the `AS subquery` clause, which both creates the table and inserts rows returned from the subquery.

In the syntax:

<code>table</code>	Name of the table
<code>column</code>	Name of the column, default value, and integrity constraint
<code>subquery</code>	<code>SELECT</code> statement that defines the set of rows to be inserted into the new table

Guidelines

- The table is created with the specified column names, and the rows retrieved by the `SELECT` statement are inserted into the table.
- The column definition can contain only the column name and default value.
- If column specifications are given, the number of columns must equal the number of columns in the subquery `SELECT` list.
- If no column specifications are given, the column names of the table are the same as the column names in the subquery.
- The column data type definitions and the `NOT NULL` constraint are passed to the new table. Note that only the explicit `NOT NULL` constraint will be inherited. The `PRIMARY KEY` column will not pass the `NOT NULL` feature to the new column. Any other constraint rules are not passed to the new table. However, you can add constraints in the column definition.

Creating a Table by Using a Subquery

```
CREATE TABLE dept80
AS
SELECT employee_id, last_name,
       salary*12 ANNSAL,
       hire_date
FROM employees
WHERE department_id = 80;
```

Table DEPT80 created.

```
DESCRIBE dept80
```

Name	Null	Type
EMPLOYEE_ID		NUMBER(6)
LAST_NAME	NOT NULL	VARCHAR2(25)
ANNSAL		NUMBER
HIRE_DATE	NOT NULL	DATE

The example in the slide creates a table named `DEPT80`, which contains details of all the employees working in department 80. Notice that the data for the `DEPT80` table comes from the `EMPLOYEES` table.

You can verify the existence of a database table and check the column definitions by using the `DESCRIBE` command.

However, be sure to provide a column alias when selecting an expression. The expression `SALARY*12` is given the alias `ANNSAL`. Without the alias, the following error is generated:

Error starting at line 1 in command:

```
CREATE TABLE dept80
AS
  SELECT employee_id, last_name,
         salary*12,
         hire_date
  FROM employees
  WHERE department_id = 80
```

Error at Command Line:4 Column:18

Error report:

SQL Error: ORA-00998: must name this expression with a column alias
00998. 00000 - "must name this expression with a column alias"

*Cause:

*Action:



ALTER TABLE Statement

—

For Instructor Use Only.
This document should not be distributed

ALTER TABLE Statement

Use the `ALTER TABLE` statement to:

- Add a new column
- Modify an existing column definition
- Define a default value for the new column
- Drop a column
- Rename a column
- Change table to read-only status



After you create a table, you may need to change the table structure for any of the following reasons:

- You omitted a column.
- Your column definition or its name needs to be changed.
- You need to remove columns.
- You want to put the table into read-only mode

You can do this by using the `ALTER TABLE` statement.

ALTER TABLE Statement

Use the ALTER TABLE statement to add, modify, or drop columns:

```
ALTER TABLE table
ADD          (column datatype [DEFAULT expr]
              [, column datatype]...);
```

```
ALTER TABLE table
MODIFY       (column datatype [DEFAULT expr]
              [, column datatype]...);
```

```
ALTER TABLE table
DROP (column [, column] ...);
```

You can add columns to a table, modify columns, and drop columns from a table by using the ALTER TABLE statement.

In the syntax:

<i>table</i>	Name of the table
ADD MODIFY DROP	Type of modification
<i>column</i>	Name of the column
<i>datatype</i>	Data type and length of the column
DEFAULT <i>expr</i>	Specifies the default value for a column

Adding a Column

- You use the `ADD` clause to add columns:

```
ALTER TABLE dept80
ADD      (job_id VARCHAR2(9));
```

Table DEPT80 altered.

- The new column becomes the last column:

EMPLOYEE_ID	LAST_NAME	ANNSAL	HIRE_DATE	JOB_ID
1	149 Zlotkey	126000	29-JAN-16	(null)
2	174 Abel	132000	11-MAY-12	(null)
3	176 Taylor	103200	24-MAR-14	(null)
4	206 Gietz	99600	07-JUN-10	(null)



Guidelines for Adding a Column

- You can add or modify columns.
- You cannot specify where the column is to appear. The new column becomes the last column.

The example in the slide adds a column named `JOB_ID` to the `DEPT80` table. The `JOB_ID` column becomes the last column in the table.

Note: If a table already contains rows when a column is added, the new column is initially null or takes the default value for all the rows. You can add a mandatory `NOT NULL` column to a table that already contains data in the other columns only if you specify a default value. You can add a `NOT NULL` column to an empty table without the default value.

Modifying a Column

- You can change a column's data type, size, and default value.

```
ALTER TABLE dept80  
MODIFY      (last_name VARCHAR2(30));
```

Table DEPT80 altered.

Size of the last_name
column is modified.

- A change to the default value of a column affects only subsequent insertions to the table.



You can modify a column definition by using the `ALTER TABLE` statement with the `MODIFY` clause. Column modification can include changes to a column's data type, size, and default value.

Guidelines

- You can increase the width or precision of a numeric column.
- You can increase the width of character columns.
- You can decrease the width of a column if:
 - The column contains only null values
 - The table has no rows
 - The decrease in column width is not less than the existing values in that column
- You can change the data type if the column contains only null values. The exception to this is `CHAR`-to-`VARCHAR2` conversions, which can be done with data in the columns.
- You can convert a `CHAR` column to the `VARCHAR2` data type or convert a `VARCHAR2` column to the `CHAR` data type only if the column contains null values or if you do not change the size.
- A change to the default value of a column affects only subsequent insertions to the table.

Dropping a Column

Use the `DROP COLUMN` clause to drop columns that you no longer need from the table:

```
ALTER TABLE dept80
DROP (job_id);
```

Table DEPT80 altered.

	EMPLOYEE_ID	LAST_NAME	ANNSAL	HIRE_DATE
1	149	Zlotkey	126000	29-JAN-16
2	174	Abel	132000	11-MAY-12
3	176	Taylor	103200	24-MAR-14
4	206	Gietz	99600	07-JUN-10



You can drop a column from a table by using the `ALTER TABLE` statement with the `DROP COLUMN` clause.

Guidelines

- The column may or may not contain data.
- Using the `ALTER TABLE DROP COLUMN` statement, only one column can be dropped at a time.
- The table must have at least one column remaining in it after it is altered.
- After a column is dropped, it cannot be recovered.
- A primary key that is referenced by another column cannot be dropped, unless the cascade option is added.
- Dropping a column can take a while if the column has a large number of values. In this case, it may be better to set it to be unused and drop it when there are fewer users on the system to avoid extended locks.

Note: Certain columns can never be dropped, such as columns that form part of the partitioning key of a partitioned table or columns that form part of the `PRIMARY KEY` of an index-organized table. For more information about index-organized tables and partitioned tables, refer to the *Oracle Database Concepts* and *Oracle Database Administrator's Guide*.

SET UNUSED Option

- You use the `SET UNUSED` option to mark one or more columns as unused.
- You use the `DROP UNUSED COLUMNS` option to remove the columns that are marked as unused.
- You can specify the `ONLINE` keyword to indicate that DML operations on the table will be allowed while marking the column or columns `UNUSED`.

```
ALTER TABLE <table_name>  
SET UNUSED (<column_name> [ , <column_name>] );  
OR  
ALTER TABLE <table_name>  
SET UNUSED COLUMN <column_name> [ , <column_name>];  
  
ALTER TABLE <table_name>  
DROP UNUSED COLUMNS;
```

You can use the `SET UNUSED` option to mark one or more columns as unused so that they can be dropped when the demand on system resources is lower. Specifying this clause does not actually remove the target columns from each row in the table (that is, it does not restore the disk space used by these columns). Therefore, the response time is faster than if you executed the `DROP` clause.

Unused columns are treated as if they were dropped, even though their column data remains in the table's rows. After a column has been marked as unused, a `SELECT *` query will not retrieve data from unused columns. In addition, the names and types of columns marked as unused will not be displayed during a `DESCRIBE` statement, and you can add to the table a new column with the same name as an unused column. The `SET UNUSED` information is stored in the `USER_UNUSED_COL_TABS` dictionary view.

You can specify the `ONLINE` keyword to indicate that DML operations on the table will be allowed while marking the column or columns `UNUSED`. The code example shows the use of `SET UNUSED COLUMN` that sets a column unused forever by using the `ONLINE` keyword.

```
ALTER TABLE dept80 SET UNUSED(hire_date) ONLINE;
```

Note: The guidelines for setting a column to be `UNUSED` are similar to those for dropping a column.

DROP UNUSED COLUMNS Option

DROP UNUSED COLUMNS removes from the table all columns that are currently marked as unused. You can use this statement when you want to reclaim the extra disk space from the unused columns in the table. If the table contains no unused columns, the statement returns with no errors.

```
ALTER TABLE dept80  
SET UNUSED (last_name);
```

```
ALTER TABLE dept80  
DROP UNUSED COLUMNS;
```

Note: A subsequent DROP UNUSED COLUMNS will physically remove all unused columns from a table, similar to a DROP COLUMN.

For Instructor Use Only.
This document should not be distributed

Read-Only Tables

You can use the `ALTER TABLE` syntax to:

- Put a table in read-only mode, which prevents DDL or DML changes during table maintenance
- Put the table back into read/write mode

```
ALTER TABLE employees READ ONLY;

-- perform table maintenance and then
-- return table back to read/write mode

ALTER TABLE employees READ WRITE;
```



You can specify `READ ONLY` to convert a table into read-only mode. When the table is in `READ ONLY` mode, you cannot issue any DML statements that affect the table or any `SELECT . . . FOR UPDATE` statements. You can issue DDL statements as long as they do not modify any data in the table. Operations on indexes associated with the table are allowed when the table is in `READ ONLY` mode.

Specify `READ/WRITE` to return a read-only table to read/write mode.

Note: You can drop a table that is in `READ ONLY` mode. The `DROP` command is executed only in the data dictionary, so access to the table contents is not required. The space used by the table will not be reclaimed until the tablespace is made read/write again, and then the required changes can be made to the block segment headers, and so on.



DROP TABLE Statement

For Instructor Use Only.
This document should not be distributed

Dropping a Table

- Moves a table to the recycle bin
- Removes the table and all its data entirely if the `PURGE` clause is specified
- Invalidates dependent objects and removes object privileges on the table

```
DROP TABLE dept80;
```

```
Table DEPT80 dropped.
```

```
DROP TABLE IF EXISTS dept80; --23c: optional syntax to avoid errors
```



The `DROP TABLE` statement moves a table to the recycle bin or removes the table and all its data from the database entirely. Unless you specify the `PURGE` clause, the `DROP TABLE` statement does not result in space being released back to the tablespace for use by other objects, and the space continues to count toward the user's space quota. Dropping a table invalidates the dependent objects and removes object privileges on the table.

When you drop a table, the database loses all the data in the table and all the indexes associated with it.

Syntax

```
DROP TABLE [IF EXISTS] table [PURGE]
```

In the syntax, *table* is the name of the table.

Guidelines

- All data is deleted from the table.
- Any views and synonyms remain, but are invalid.
- Any pending transactions are committed.
- Only the creator of the table or a user with the `DROP ANY TABLE` privilege can remove a table.
- The `IF EXISTS` clause is supported starting in the 23c version of the Oracle Database.

Note: Use the `FLASHBACK TABLE` statement to restore a dropped table from the recycle bin. This is discussed in detail later in this course.

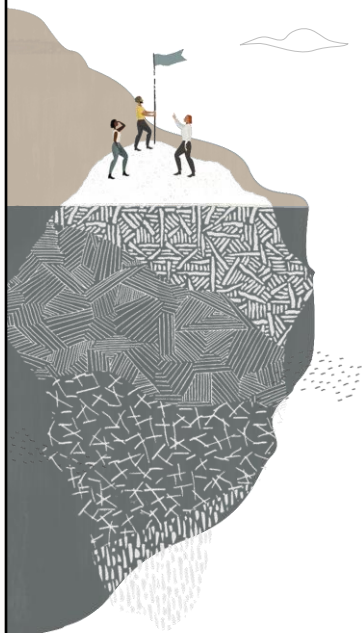
Dropping the DEMO Table If It Exists: Example



```
DROP TABLE IF EXISTS demo;
```

```
DROP TABLE IF EXISTS demo;
```

Summary



Database objects

`CREATE TABLE` statement

Data types

Overview of constraints

Creating a table by using a subquery

`ALTER TABLE` statement

`DROP TABLE` statement

In this lesson, you should have learned the following:

- **Database Objects**
- **Data Types**
- **Constraints**

`CREATE TABLE`

- Use the `CREATE TABLE` statement to create a table and include constraints.
- Create a table based on another table by using a subquery.

`ALTER TABLE`

`DROP TABLE`

- Remove rows and a table structure.
- When executed, this statement cannot be rolled back.

For Instructor Use Only.
This document should not be distributed

For Instructor Use Only.
This document should not be distributed

Introduction to Data Dictionary Views



Objectives



Introduction to the data dictionary

Querying the dictionary views

Adding a comment to a table and querying the dictionary views

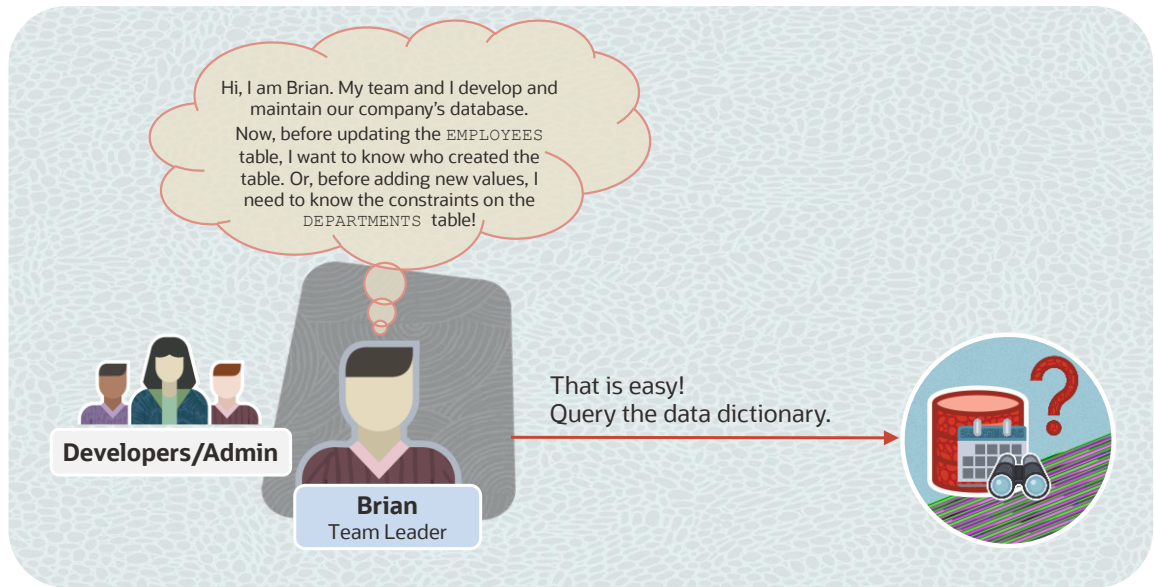
In this lesson, you are introduced to data dictionary views. You learn that dictionary views can be used to retrieve metadata and create reports about your schema objects.



Introduction to Data Dictionary

For Instructor Use Only.
This document should not be distributed

Why data dictionary?

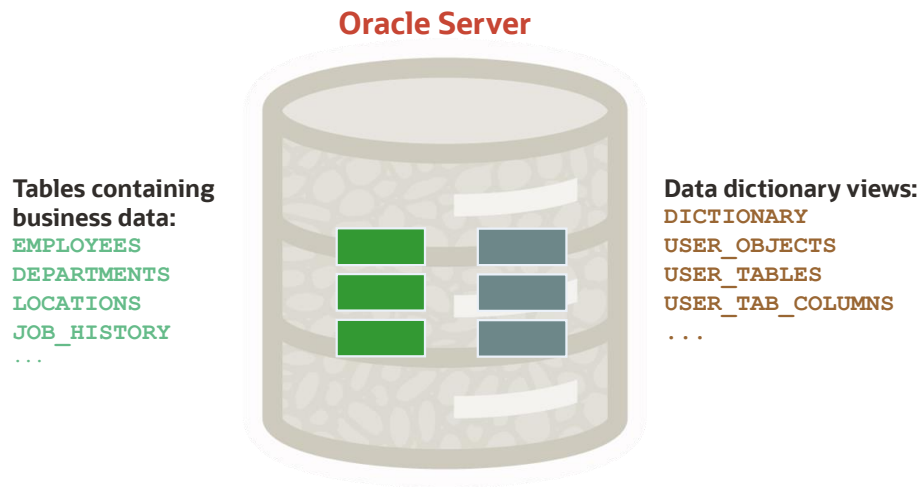


Brian and his team of developers are currently developing and maintaining his company's database. So, at some point in time, Brian (Team Leader) wants to alter the `EMPLOYEES` table by dropping/adding a column. Before altering the table, he wants to check with the owner of the table and discuss the effects of the change. He also wants to add a new department into the `DEPARTMENTS` table. Before inserting new values into the table, he needs to know the various constraints on the table so that they are not violated.

How do you think Brian will keep track of this information?

The good news is that Brian need not maintain any other document or table to store such information. All he needs to do is learn to query the data dictionary. The data dictionary is a list of in-built tables and views, which come along with the database. You learn more about data dictionary in the following slides.

Data Dictionary



User tables are tables created by you and contain business data, such as `EMPLOYEES`. Along with user tables, there is another collection of tables and views in the Oracle database known as the data dictionary. This collection is created and maintained by the Oracle Server and contains information about the database.

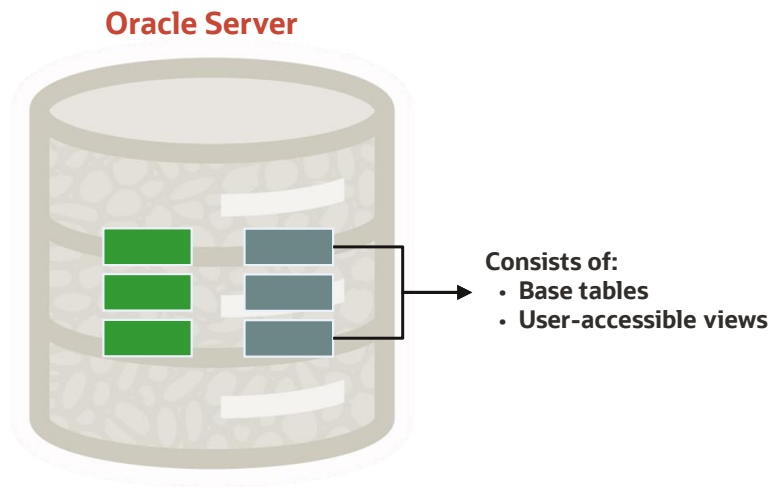
The data dictionary is structured in tables and views, just like other database data. The data dictionary is central to every Oracle database and is an important tool for all users, from end users to application designers and database administrators.

You use SQL statements to access the data dictionary. Remember that the data dictionary is read-only and, therefore, you can issue queries only against its tables and views.

You can query the dictionary views that are based on the dictionary tables to find information such as:

- Definitions of all schema objects in the database (tables, views, indexes, synonyms, sequences, procedures, functions, packages, triggers, and so on)
- Default values for columns
- Integrity constraint information
- Names of Oracle users
- Privileges and roles that each user has been granted
- Other general database information

Data Dictionary Structure



Underlying base tables store information about the associated database. Only the Oracle Server should write to and read from these tables. You rarely access them directly.

There are several views that summarize and display the information stored in the base tables of the data dictionary. These views decode the base table data into useful information (such as user or table names) using joins and `WHERE` clauses to simplify the information. You are mostly given access to the views rather than the base tables.

The Oracle user `SYS` owns all base tables and user-accessible views of the data dictionary. No Oracle user should *ever* alter (`UPDATE`, `DELETE`, or `INSERT`) any rows or schema objects contained in the `SYS` schema; doing so can compromise data integrity.

You will learn more about views and how to create them in the lesson titled “Creating Views.”

Data Dictionary Structure

View naming convention is as follows:

View Prefix	Purpose
USER	User's view (what is in your schema; what you own)
ALL	Expanded user's view (what you can access)
DBA	Database administrator's view (what is in everyone's schemas)
V\$	Performance-related data

The data dictionary consists of sets of views. In many cases, a set consists of three views containing similar information and distinguished from each other by their prefixes. For example, there is a view named `USER_OBJECTS`, another named `ALL_OBJECTS`, and a third named `DBA_OBJECTS`.

These three views contain similar information about objects in the database, except that the scope is different.

`USER_OBJECTS` contains information about objects that you own or you created. `ALL_OBJECTS` contains information about all objects to which you have access. `DBA_OBJECTS` contains information about all objects that are owned by all users.

For views that are prefixed with `ALL` or `DBA`, you will find an additional column in the view named `OWNER` to identify who owns the object.

There is also a set of views that is prefixed with `V$`. These views are dynamic in nature and hold information about performance. Dynamic performance tables are not true tables, and they should not be accessed by most users. However, database administrators can query and create views on the tables and grant access to those views to other users. This course does not go into details about these views.

How to Use Dictionary Views

Start with `DICTIONARY`. It contains the names and descriptions of the dictionary tables and views.

```
DESCRIBE DICTIONARY
```

```
DESCRIBE dictionary
Name      Null Type
-----
TABLE_NAME VARCHA2(128)
COMMENTS   VARCHA2(4000)
```

```
SELECT *
FROM dictionary
WHERE table_name = 'USER_OBJECTS';
```

	TABLE_NAME	COMMENTS
1	USER_OBJECTS	Objects owned by the user

To familiarize yourself with the dictionary views, you can use the dictionary view named `DICTIONARY`. It contains the name and short description of each dictionary view to which you have access.

You can write queries to search for information about a particular view name, or you can search the `COMMENTS` column for a word or phrase. In the example shown in the slide, the `DICTIONARY` view is described. It has two columns. The `SELECT` statement in the slide retrieves information about the dictionary view named `USER_OBJECTS`. The `USER_OBJECTS` view contains information about all the objects that you own.

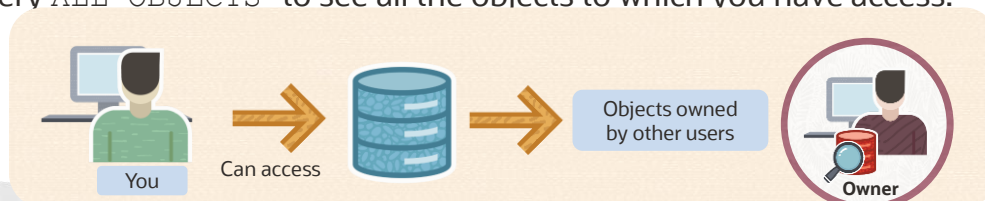
You can write queries to search the `COMMENTS` column for a word or phrase. For example, the following query returns the names of all views that you are permitted to access in which the `COMMENTS` column contains the word *columns*:

```
SELECT table_name
FROM dictionary
WHERE LOWER(comments) LIKE %columns% ;
```

Note: The table names in the data dictionary are in uppercase.

USER_OBJECTS and ALL_OBJECTS Views

- **USER_OBJECTS:**
 - Query `USER_OBJECTS` to see all the objects that you own.
 - Using `USER_OBJECTS`, you can obtain a listing of all object names and types in your schema, plus the following information:
 - Date created
 - Date of last modification
 - Status (valid or invalid)
- **ALL_OBJECTS:**
 - Query `ALL_OBJECTS` to see all the objects to which you have access.



You can query the `USER_OBJECTS` view to see the names and types of all the objects in your schema. There are several columns in this view:

- **OBJECT_NAME:** Name of the object
- **OBJECT_ID:** Dictionary object number of the object
- **OBJECT_TYPE:** Type of object (such as `TABLE`, `VIEW`, `INDEX`, `SEQUENCE`)
- **CREATED:** Time stamp for the creation of the object
- **LAST_DDL_TIME:** Time stamp for the last modification of the object resulting from a data definition language (DDL) command
- **STATUS:** Status of the object (`VALID`, `INVALID`, or `N/A`)
- **GENERATED:** Was the name of this object system generated? (Y | N)

Note: This is not a complete listing of the columns. For a complete listing, see “`USER_OBJECTS`” in *Oracle® Database Reference 23c*.

You can also query the `ALL_OBJECTS` view to see a listing of all objects to which you have access.

USER_OBJECTS View

```
SELECT object_name, object_type, created, status
FROM user_objects
ORDER BY object_type;
```

OBJECT_NAME	OBJECT_TYPE	CREATED	STATUS
1 LOC_COUNTRY_IX	INDEX	27-JUN-16	VALID
2 EMP_DEPARTMENT_IX	INDEX	27-JUN-16	VALID
3 LOC_STATE_PROVINCE_IX	INDEX	27-JUN-16	VALID
4 COUNTRY_C_ID_PK	INDEX	27-JUN-16	VALID
5 LOC_CITY_IX	INDEX	27-JUN-16	VALID
6 LOC_ID_PK	INDEX	27-JUN-16	VALID
7 JHIST_DEPARTMENT_IX	INDEX	27-JUN-16	VALID
8 JHIST_EMPLOYEE_IX	INDEX	27-JUN-16	VALID
9 DEPT_ID_PK	INDEX	27-JUN-16	VALID

...

Owned by the user

The example in the slide shows the name, type, date of creation, and status of all objects that are owned by this user.

The `OBJECT_TYPE` column holds the values of `TABLE`, `VIEW`, `SEQUENCE`, `INDEX`, `PROCEDURE`, `FUNCTION`, `PACKAGE`, or `TRIGGER`.

The `STATUS` column holds a value of `VALID`, `INVALID`, or `N/A`. Although tables are always valid, the views, procedures, functions, packages, and triggers may be invalid.

The CAT View

For a simplified query and output, you can query the `CAT` view. This view contains only two columns:

- `TABLE_NAME`
- `TABLE_TYPE`

It provides the names of all your `INDEX`, `TABLE`, `CLUSTER`, `VIEW`, `SYNONYM`, `SEQUENCE`, or `UNDEFINED` objects.

Note: `CAT` is a synonym for `USER_CATALOG`—a view that lists tables, views, synonyms, and sequences owned by the user.



Querying the Dictionary Views

—

For Instructor Use Only.
This document should not be distributed

Table Information

USER_TABLES to obtain the names of all your tables:

```
DESCRIBE user_tables
```

```
DESCRIBE user_tables
Name          Null    Type
-----
TABLE_NAME    NOT NULL VARCHAR2(128)
TABLESPACE_NAME          VARCHAR2(30)
CLUSTER_NAME             VARCHAR2(128)
IOT_NAME                VARCHAR2(128)
...
```

```
SELECT table_name
FROM   user_tables;
```

	TABLE_NAME
1	REGIONS
2	LOCATIONS
3	DEPARTMENTS
4	JOBS
5	EMPLOYEES
6	JOB_HISTORY

...

You can use the USER_TABLES view to obtain the names of all your tables. The USER_TABLES view contains information about your tables. In addition to providing the table name, it contains detailed information about the storage.

The TABS view is a synonym of the USER_TABLES view. You can query it to see a listing of tables that you own:

```
SELECT table_name
FROM   tabs;
```

Note: For a complete listing of the columns in the USER_TABLES view, see “USER_TABLES” in *Oracle® Database Reference 23c*.

You can also query the ALL_TABLES view to see a listing of all tables to which you have access.

Column Information

- `USER_TAB_COLUMNS` provides detailed information about the columns in your tables.

`DESCRIBE user_tab_columns`

Name	Null	Type
TABLE_NAME	NOT NULL	VARCHAR2(128)
COLUMN_NAME	NOT NULL	VARCHAR2(128)
DATA_TYPE		VARCHAR2(128)
DATA_TYPE_MOD		VARCHAR2(3)
DATA_TYPE_OWNER		VARCHAR2(128)
DATA_LENGTH	NOT NULL	NUMBER
DATA_PRECISION		NUMBER
DATA_SCALE		NUMBER
NULLABLE		VARCHAR2(1)

...

Get information
about a column.



You can query the `USER_TAB_COLUMNS` view to find detailed information about the columns in your tables. Although the `USER_TABLES` view provides information about your table names and storage, you will find detailed column information in the `USER_TAB_COLUMNS` view.

This view contains information such as:

- Column names
- Column data types
- Length of data types
- Precision and scale for `NUMBER` columns
- Whether nulls are allowed (Is there a `NOT NULL` constraint on the column?)
- Default value

Note: For a complete listing and description of the columns in the `USER_TAB_COLUMNS` view, see “`USER_TAB_COLUMNS`” in the *Oracle® Database Reference 23c*.

Column Information

```
SELECT column_name, data_type, data_length,  
       data_precision, data_scale, nullable  
FROM   user_tab_columns  
WHERE  table_name = 'EMPLOYEES';
```

	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	DATA_PRECISION	DATA_SCALE	NULLABLE
1	EMPLOYEE_ID	NUMBER	22	6	0	N
2	FIRST_NAME	VARCHAR2	20	(null)	(null)	Y
3	LAST_NAME	VARCHAR2	25	(null)	(null)	N
4	EMAIL	VARCHAR2	25	(null)	(null)	N
5	PHONE_NUMBER	VARCHAR2	20	(null)	(null)	Y
6	HIRE_DATE	DATE	7	(null)	(null)	N
7	JOB_ID	VARCHAR2	10	(null)	(null)	N
8	SALARY	NUMBER	22	8	2	Y
9	COMMISSION_PCT	NUMBER	22	2	2	Y
10	MANAGER_ID	NUMBER	22	6	0	Y
11	DEPARTMENT_ID	NUMBER	22	4	0	Y

By querying the `USER_TAB_COLUMNS` table, you can find details about your columns, such as the names, data types, data type lengths, null constraints, and default value for a column.

The example shown in the slide displays the columns, data types, data lengths, and null constraints for the `EMPLOYEES` table. Note that this information is similar to the output from the `DESCRIBE` command.

To view information about columns set as unused, you use the `USER_UNUSED_COL_TABS` dictionary view.

Note: Names of the objects in data dictionary are in uppercase.

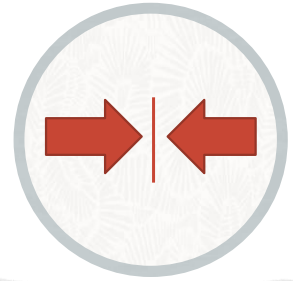
Constraint Information

- `USER_CONSTRAINTS` describes the constraint definitions on your tables.
- `USER_CONS_COLUMNS` describes columns that are owned by you and that are specified in constraints.

```
DESCRIBE user_constraints
```

Name	Null	Type
OWNER		VARCHAR2(128)
CONSTRAINT_NAME	NOT NULL	VARCHAR2(128)
CONSTRAINT_TYPE		VARCHAR2(1)
TABLE_NAME	NOT NULL	VARCHAR2(128)
SEARCH_CONDITION		LONG
SEARCH_CONDITION_VC		VARCHAR2(4000)
R_OWNER		VARCHAR2(128)
R_CONSTRAINT_NAME		VARCHAR2(128)
DELETE_RULE		VARCHAR2(9)
STATUS		VARCHAR2(8)

...



You can find out the names of your constraints, the type of constraint, the table name to which the constraint applies, the condition for check constraints, foreign key constraint information, deletion rule for foreign key constraints, the status, and many other types of information about your constraints.

Note: For a complete listing and description of the columns in the `USER_CONSTRAINTS` view, see “`USER_CONSTRAINTS`” in *Oracle® Database Reference 23c*.

USER_CONSTRAINTS: Example

```
SELECT constraint_name, constraint_type,  
       search_condition, r_constraint_name,  
       delete_rule, status  
FROM   user_constraints  
WHERE  table_name = 'EMPLOYEES';
```

	CONSTRAINT_NAME	CONSTRAINT_TYPE	SEARCH_CONDITION	R_CONSTRAINT_NAME	DELETE_RULE	STATUS
1	EMP_MANAGER_FK	R	(null)	EMP_EMP_ID_PK	NO ACTION	ENABLED
2	EMP_JOB_FK	R	(null)	JOB_ID_PK	NO ACTION	ENABLED
3	EMP_DEPT_FK	R	(null)	DEPT_ID_PK	NO ACTION	ENABLED
4	EMP_EMP_ID_PK	P	(null)	(null)	(null)	ENABLED
5	EMP_EMAIL_UK	U	(null)	(null)	(null)	ENABLED
6	EMP_SALARY_MIN	C	salary > 0	(null)	(null)	ENABLED
7	EMP_JOB_NN	C	"JOB_ID" IS NOT NULL	(null)	(null)	ENABLED
8	EMP_HIRE_DATE_NN	C	"HIRE_DATE" IS NOT NULL	(null)	(null)	ENABLED
9	EMP_EMAIL_NN	C	"EMAIL" IS NOT NULL	(null)	(null)	ENABLED
10	EMP_LAST_NAME_NN	C	"LAST_NAME" IS NOT NULL	(null)	(null)	ENABLED

In the example in the slide, the USER_CONSTRAINTS view is queried to find the names, types, check conditions, name of the unique constraint that the foreign key references, deletion rule for a foreign key, and status for constraints on the EMPLOYEES table.

The CONSTRAINT_TYPE can be:

- C (check constraint on a table, or NOT NULL)
- P (primary key)
- U (unique key)
- R (referential integrity)
- V (with check option, on a view)
- O (with read-only, on a view)

The DELETE_RULE can be:

- **CASCADE:** If the parent record is deleted, the child records are deleted too.
- **SET NULL:** If the parent record is deleted, change the respective child record to null.
- **NO ACTION:** A parent record can be deleted only if no child records exist.

The STATUS can be:

- **ENABLED:** Constraint is active.
- **DISABLED:** Constraint is made inactive.

Querying USER_CONS_COLUMNS

```
DESCRIBE user_cons_columns
```

```
DESCRIBE user_cons_columns
Name          Null    Type
-----
OWNER         NOT NULL VARCHAR2(128)
CONSTRAINT_NAME NOT NULL VARCHAR2(128)
TABLE_NAME    NOT NULL VARCHAR2(128)
COLUMN_NAME   VARCHAR2(4000)
POSITION      NUMBER
```

```
SELECT constraint_name, column_name
FROM   user_cons_columns
WHERE  table_name = 'EMPLOYEES';
```

	CONSTRAINT_NAME	COLUMN_NAME
1	EMP_LAST_NAME_NN	LAST_NAME
2	EMP_EMAIL_NN	EMAIL
3	EMP_HIRE_DATE_NN	HIRE_DATE
4	EMP_JOB_NN	JOB_ID
5	EMP_SALARY_MIN	SALARY
6	EMP_EMAIL_UK	EMAIL
7	EMP_EMP_ID_PK	EMPLOYEE_ID
8	EMP_DEPT_FK	DEPARTMENT_ID
9	EMP_JOB_FK	JOB_ID
10	EMP_MANAGER_FK	MANAGER_ID

...

To find the names of the columns to which a constraint applies, query the `USER_CONS_COLUMNS` dictionary view. This view tells you the name of the owner of a constraint, the name of the constraint, the table that the constraint is on, the names of the columns with the constraint, and the original position of column or attribute in the definition of the object.

Note: A constraint may apply to more than one column.

You can also write a join between `USER_CONSTRAINTS` and `USER_CONS_COLUMNS` to create customized output from both tables.



Adding a Comment to a Table and Querying the Dictionary Views

For Instructor Use Only.
This document should not be distributed

Adding Comments to a Table

- **COMMENT** statement: To add comments to a table or column

```
COMMENT ON TABLE employees  
IS 'Employee Information';
```

```
COMMENT ON COLUMN employees.first name  
IS 'First name of the employee';
```

- Comments can be viewed through the data dictionary views:
 - ALL_COL_COMMENTS
 - USER_COL_COMMENTS
 - ALL_TAB_COMMENTS
 - USER_TAB_COMMENTS

You can add a comment of up to 4,000 bytes about a column, table, view, or snapshot by using the **COMMENT** statement. The comment is stored in the data dictionary and can be viewed in one of the following data dictionary views in the **COMMENTS** column:

- ALL_COL_COMMENTS
- USER_COL_COMMENTS
- ALL_TAB_COMMENTS
- USER_TAB_COMMENTS

Syntax

```
COMMENT ON {TABLE table | COLUMN table.column}  
IS 'text';
```

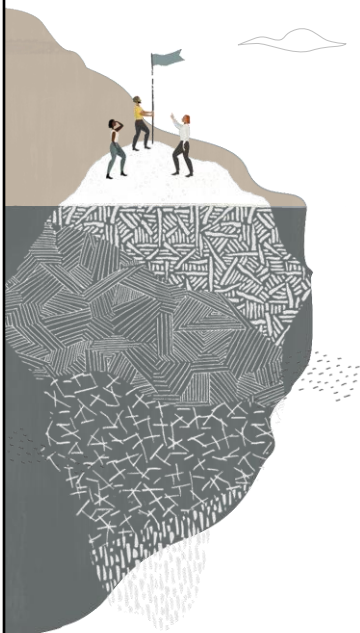
In the syntax:

<i>table</i>	Is the name of the table
<i>column</i>	Is the name of the column in a table
<i>text</i>	Is the text of the comment

You can drop a comment from the database by setting it to empty string (' '):

```
COMMENT ON TABLE employees IS ' ';
```

Summary



Introduction to data dictionary

Querying the dictionary views

Adding a comment to a table and querying the dictionary views

In this lesson, you learned about some of the dictionary views that are available to you. You can use these dictionary views to find information about your tables, constraints, views, sequences, and synonyms.

We also show you how to incorporate comments into the tables.

For Instructor Use Only.
This document should not be distributed

Creating Sequences, Synonyms, and Indexes



Objectives



Create, maintain, and use sequences

Create private and public synonyms

Create and maintain indexes

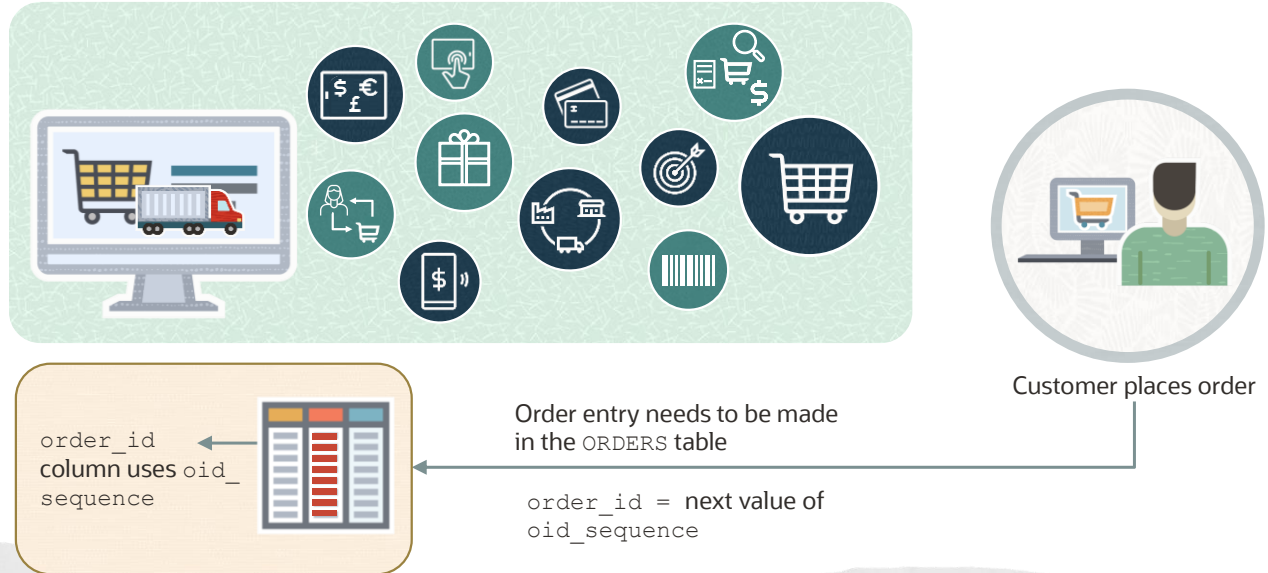
In this lesson, you are introduced to the sequence, synonyms, and index objects. You learn the basics of creating and using sequences, synonyms, and indexes.



Create, Maintain, and Use Sequences

For Instructor Use Only.
This document should not be distributed

E-Commerce Scenario



OracleKart is an online e-commerce company selling a variety of goods. The back-end database maintains a table for all orders placed by the customers. Each order requires a unique ID to store the order information correctly. How do you think OracleKart will generate the `order_id` in the `ORDERS` table? Should an administrator manually insert a unique `order_id` each time an order is placed?

Brian, the DBA, is aware that manually generating unique IDs is not efficient. An error can cause incorrect data mapping, which will lead to incorrect order delivery. For data integrity and work efficiency, Brian can make use of SQL sequence to generate order IDs automatically.

To solve scenarios like the above, SQL provides sequences. Using a sequence, you can generate a unique sequence of numbers according to your need. Let's look into some more advantages of a sequence and learn how to use it in a SQL query.

Database Objects

Object	Description
Table	Basic unit of storage; composed of rows
View	Logically represents subsets of data from one or more tables
Sequence	Generates numeric values
Index	Improves the performance of data retrieval queries
Synonym	Gives alternative names to objects

As shown in the slide, there are several other objects in a database in addition to **tables**.

With **views**, you can present and hide data from the tables.

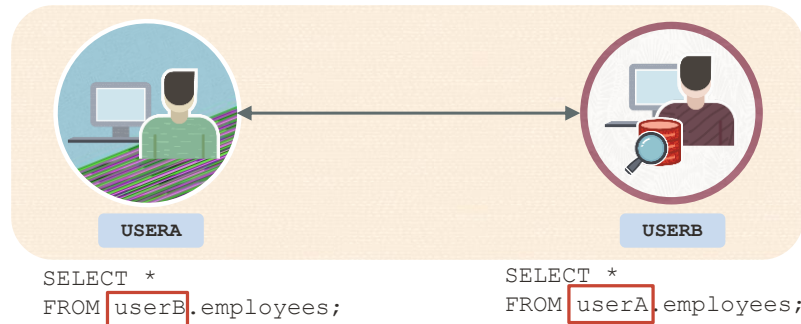
Many applications require the use of unique numbers as primary key values. You can either build code into the application to handle this requirement or use a **sequence** to generate unique numbers.

If you want to improve the performance of data retrieval queries, you should consider creating an **index**. You can also use indexes to enforce uniqueness on a column or a collection of columns.

You can provide alternative names for objects by using **synonyms**.

Referencing Another User's Tables

- Tables belonging to other users are not in the user's schema.
- You should use the owner's name as a prefix to those tables.



What is a schema?

A schema is a collection of logical structures of data or *schema objects*. A schema is owned by a database user and has the same name as that user. Each user owns a single schema.

Schema objects can be created and manipulated with SQL and include tables, views, synonyms, sequences, stored procedures, indexes, clusters, and database links.

If a table does not belong to the user, the owner's name must be prefixed to the table. For example, if there are schemas named `USERA` and `USERB`, and both have an `EMPLOYEES` table, then if `USERA` wants to access the `EMPLOYEES` table that belongs to `USERB`, `USERA` must prefix the table name with the schema name:

```
SELECT *  
FROM   userb.employees;
```

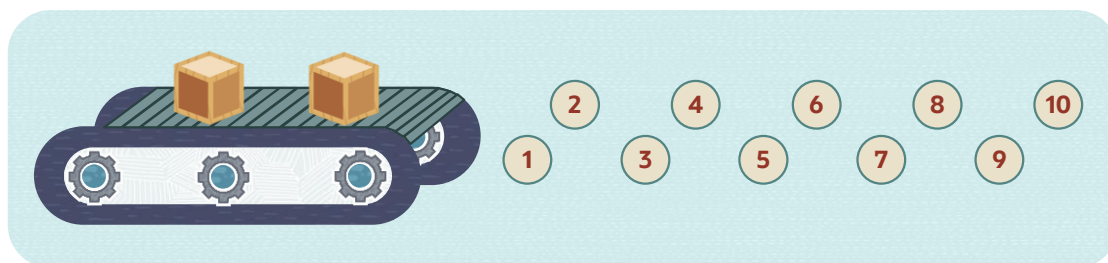
If `USERB` wants to access the `EMPLOYEES` table that is owned by `USERA`, `USERB` must prefix the table name with the schema name:

```
SELECT *  
FROM   usera.employees;
```

Sequences

A sequence:

- Can automatically generate unique numbers
- Is a shareable object
- Can be used to create a primary key value
- Replaces application code
- Speeds up the efficiency of accessing sequence values when cached in memory



A sequence is a user-created database object that can be shared by multiple users to generate integers. You can define a sequence to generate unique values or to recycle and use the same numbers again.

A typical usage for sequences is to create a primary key value, which must be unique for each row. A sequence is generated and incremented (or decremented) by an internal Oracle routine. This can be a time-saving object, because it can reduce the amount of application code needed to write a sequence-generating routine.

Sequence numbers are stored and generated independent of tables. Therefore, the same sequence can be used for multiple tables.

CREATE SEQUENCE Statement: Syntax

Define a sequence to generate sequential numbers automatically:

```
CREATE SEQUENCE [ schema. ] sequence
[ { START WITH|INCREMENT BY } integer
| { MAXVALUE integer | NOMAXVALUE }
| { MINVALUE integer | NOMINVALUE }
| { CYCLE | NOCYCLE }
| { CACHE integer | NOCACHE }
| { ORDER | NOORDER }
| { SCALE | NOSCALE }
| { SHARD | NOSHARD }
];
```



Automatically generate sequential numbers by using the CREATE SEQUENCE statement.

In the syntax:

Sequence	Is the name of the sequence generator
START WITH <i>n</i>	Specifies the first sequence number to be generated (If this clause is omitted, the sequence starts with 1.)
INCREMENT BY <i>n</i>	Specifies the interval between sequence numbers, where <i>n</i> is an integer (If this clause is omitted, the sequence increments by 1.)
MAXVALUE <i>n</i>	Specifies the maximum value the sequence can generate
NOMAXVALUE	Specifies a maximum value of 10^27 for an ascending sequence and -1 for a descending sequence (This is the default option.)
MINVALUE <i>n</i>	Specifies the minimum sequence value
NOMINVALUE	Specifies a minimum value of 1 for an ascending sequence and -(10^26) for a descending sequence (This is the default option.)

ORDER	If specified, guarantees that sequence numbers are generated in order of request. This clause is useful if you are using the sequence numbers as timestamps.
NOORDER	If specified, sequence numbers are not generated in order of request. This is the default.
CYCLE NOCYCLE	Specifies whether the sequence continues to generate values after reaching its maximum or minimum value (NOCYCLE is the default option.)
CACHE <i>n</i> NOCACHE	Specifies how many values the Oracle Server preallocates and keeps in memory (By default, the Oracle server caches 20 values.)
SCALE [EXTEND NOEXTEND] NOSCALE	If specified, a numeric offset is affixed to the beginning of the sequence, which removes all duplicates in generated values (NOSCALE is the default option.)
If EXTEND	If specified with SCALE, the generated sequence values are all of length (x+y), where x is the length of the scalable offset (default value is 6) and y is the maximum number of digits in the sequence (maxvalue/minvalue) (NOEXTEND is the default)
SHARD [EXTEND NOEXTEND] NOSHARD	If specified, the sequence generates unique sequence numbers across shards. If EXTEND is specified with the SHARD clause, the generated sequence values are all of length (x = y), where x is the length of a SHARD offset of size 4. (NOEXTEND is the default.)

Creating a Sequence

- Create a sequence named `DEPT_DEPTID_SEQ` to be used for the primary key of the `DEPARTMENTS` table.
- Do not use the `CYCLE` option.

```
CREATE SEQUENCE dept_deptid_seq  
  START WITH 280  
  INCREMENT BY 10  
  MAXVALUE 9999  
  NOCACHE  
  NOCYCLE  
  SCALE;  
  
Sequence DEPT_DEPTID_SEQ created.
```

The example in the slide creates a sequence named `DEPT_DEPTID_SEQ` to be used for the `DEPARTMENT_ID` column of the `DEPARTMENTS` table. The sequence starts at 280 and adds a 6-digit prefix to the sequence number (three-digit instance number followed by a three-digit session number). It does not allow caching and does not cycle.

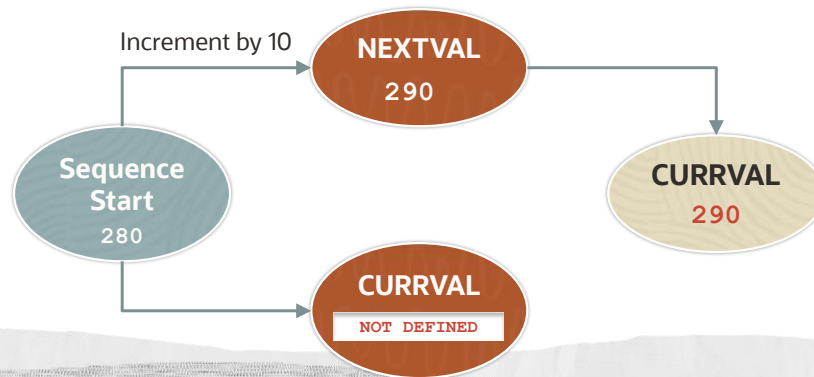
Do not use the `CYCLE` option if the sequence is used to generate primary key values, unless you have a reliable mechanism that purges old rows faster than the sequence cycles.

For more information, see the “`CREATE SEQUENCE`” section in *Oracle Database SQL Language Reference* for Oracle Database 23c.

Note: The sequence is not tied to a table. Generally, you should name the sequence after its intended use. However, the sequence can be used anywhere, regardless of its name.

NEXTVAL and CURRVAL Pseudocolumns

- NEXTVAL returns the next available sequence value. It returns a unique value every time it is referenced, even for different users.
- CURRVAL obtains the current sequence value.
- NEXTVAL must be issued for that sequence before CURRVAL can be referenced.



After you create your sequence, it generates sequential numbers for use in your tables. Reference the sequence values by using the NEXTVAL and CURRVAL pseudocolumns.

The NEXTVAL pseudocolumn is used to extract successive sequence numbers from a specified sequence. You must qualify NEXTVAL with the sequence name. When you reference *sequence*.NEXTVAL, a new sequence number is generated, and the current sequence number is placed in CURRVAL.

The CURRVAL pseudocolumn is used to refer to a sequence number that the current user has just generated. However, NEXTVAL must be used to generate a sequence number in the current user's session before CURRVAL can be referenced. You must qualify CURRVAL with the sequence name. When you reference *sequence*.CURRVAL, the last value returned to that user's process is displayed.

Rules for Using NEXTVAL and CURRVAL

You can use NEXTVAL and CURRVAL in the following contexts:

- The SELECT list of a SELECT statement that is not part of a subquery
- The SELECT list of a subquery in an INSERT statement
- The VALUES clause of an INSERT statement
- The SET clause of an UPDATE statement

You cannot use `NEXTVAL` and `CURRVAL` in the following contexts:

- The `SELECT` list of a view
- A `SELECT` statement with the `DISTINCT` keyword
- A `SELECT` statement with `GROUP BY`, `HAVING`, or `ORDER BY` clause
- A subquery in a `SELECT`, `DELETE`, or `UPDATE` statement

For more information, see the “Pseudocolumns” and “`CREATE SEQUENCE`” sections in *Oracle Database SQL Language Reference* for Oracle Database 23c.

Using a Sequence

- Insert a new department named “Support” in location ID 2500:

```
INSERT INTO departments(department_id,  
                        department_name, location_id)  
VALUES (dept_deptid_seq.NEXTVAL,  
      'Support', 2500);
```

1 row inserted.

- View the current value for the DEPT_DEPTID_SEQ sequence:

```
SELECT dept_deptid_seq.CURRVAL  
FROM dual;
```

CURRVAL
1 280

The example in the slide inserts a new department in the DEPARTMENTS table. It uses the DEPT_DEPTID_SEQ sequence to generate a new department number as shown in the slide.

You can view the current value of the sequence using the *sequence_name.CURRVAL*, as shown in the second example in the slide.

Suppose that you now want to hire employees to staff the new department. The INSERT statement to be executed for all new employees can include the following code:

```
INSERT INTO employees (employee_id, department_id, ...)  
VALUES (employees_seq.NEXTVAL, dept_deptid_seq.CURRVAL, ...);
```

Note: The preceding example assumes that a sequence called EMPLOYEES_SEQ has already been created to generate new employee numbers.

SQL Column Defaulting Using a Sequence

- You can use the SQL syntax `<sequence>.nextval`, `<sequence>.currval` as a SQL column defaulting expression for numeric columns, where `<sequence>` is an Oracle database sequence.
- The `DEFAULT` expression can include the sequence pseudocolumns `CURRVAL` and `NEXTVAL`, as long as the sequence exists, and you have the privileges necessary to access it.

```
CREATE SEQUENCE ID_SEQ START WITH 1;  
CREATE TABLE emp (ID NUMBER DEFAULT ID_SEQ.NEXTVAL NOT NULL,  
                   name VARCHAR2(10));  
INSERT INTO emp (name) VALUES ('john');  
INSERT INTO emp (name) VALUES ('mark');  
SELECT * FROM emp;
```

```
Sequence ID_SEQ created.  
Table EMP created.  
1 row inserted.  
1 row inserted.  
  
ID NAME  
-----  
1 john  
2 mark
```

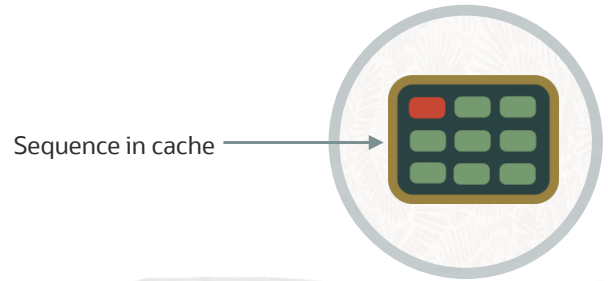
SQL syntax for column defaults has been enhanced so that it allows `<sequence>.nextval`, `<sequence>.currval` as a SQL column defaulting expression for numeric columns, where `<sequence>` is an Oracle database sequence.

The `DEFAULT` expression can include the sequence pseudocolumns `CURRVAL` and `NEXTVAL`, as long as the sequence exists, and you have the privileges necessary to access it. The user inserting into a table must have access privileges to the sequence. If the sequence is dropped, subsequent insert DMLs where *expr* is used for defaulting will result in a compilation error.

In the example in the slide, sequence `ID_SEQ` is created, which starts from 1.

Caching Sequence Values

- Caching sequence values in memory gives faster access to those values.
- Gaps in sequence values can occur when:
 - A rollback occurs
 - The system crashes
 - A sequence is used in another table



You can cache sequences in memory to provide faster access to those sequence values. The cache is populated the first time you refer to the sequence. Each request for the next sequence value is retrieved from the cached sequence. After the last sequence value is used, the next request for the sequence pulls another cache of sequences into memory.

Gaps in the Sequence

The sequence generators generally issue sequential numbers without gaps, and this action occurs independently of a commit or rollback. Therefore, if you roll back a statement containing a sequence, the number is lost.

Another event that can cause gaps in the sequence is a system crash. If the sequence caches values in memory, those values are lost if the system crashes.

Because sequences are not tied directly to tables, the same sequence can be used for multiple tables. However, if you do so, each table can contain gaps in the sequential numbers.

Modifying a Sequence

Change the increment value, maximum value, minimum value, cycle option, or cache option:

```
ALTER SEQUENCE dept_deptid_seq  
INCREMENT BY 20  
MAXVALUE 999999  
NOCACHE  
NOCYCLE  
NOSCALE;
```

Sequence DEPT_DEPTID_SEQ altered.

If you reach the `MAXVALUE` limit for your sequence, no additional values from the sequence are allocated and you will receive an error indicating that the sequence exceeds the `MAXVALUE`. To continue to use the sequence, you can modify it by using the `ALTER SEQUENCE` statement.

Syntax

```
ALTER SEQUENCE sequence  
[INCREMENT BY n]  
[{MAXVALUE n | NOMAXVALUE}]  
[{MINVALUE n | NOMINVALUE}]  
[{CYCLE | NOCYCLE}]  
[{CACHE n | NOCACHE}]  
[{SCALE [EXTEND | NOEXTEND] | NOSCALE}]  
[{SHARD [EXTEND | NOEXTEND] | NOSHARD];
```

In the syntax, *sequence* is the name of the sequence generator.

For more information, see the section on “`ALTER SEQUENCE`” in *Oracle Database SQL Language Reference* for Oracle Database 23c.

Guidelines for Modifying a Sequence

- You must be the owner or have the `ALTER` privilege for the sequence.
- Only future sequence numbers are affected.
- The sequence must be dropped and re-created to restart the sequence at a different number.
- Some validation is performed.
- To remove a sequence, use the `DROP` statement:

```
DROP SEQUENCE dept_deptid_seq;
```

```
Sequence DEPT_DEPTID_SEQ dropped.
```

You must be the owner or have the `ALTER` privilege for the sequence to modify it. You must be the owner or have the `DROP ANY SEQUENCE` privilege to remove it.

Only future sequence numbers are affected by the `ALTER SEQUENCE` statement.

The `START WITH` option cannot be changed by using `ALTER SEQUENCE`. The sequence must be dropped and re-created to restart the sequence at a different number.

Some validation is performed. For example, a new `MAXVALUE` that is less than the current sequence number cannot be imposed.

```
ALTER SEQUENCE dept_deptid_seq  
    INCREMENT BY 20  
    MAXVALUE 90  
    NOCACHE  
    NOCYCLE  
    NOSCALE;
```

SQL Error: ORA-04009: MAXVALUE cannot be made to be less than the current value 04009. 00000 - "MAXVALUE cannot be made to be less than the current value"

*Cause: the current value exceeds the given `MAXVALUE`

*Action: make sure that the new `MAXVALUE` is larger than the current value

Sequence Information

- The `USER_SEQUENCES` view describes all sequences that you own.

```
DESCRIBE user_sequences
```

SEQUENCE_NAME	VARCHAR2(128)
MIN_VALUE	NUMBER
MAX_VALUE	NUMBER
INCREMENT_BY	NUMBER
CYCLE_FLAG	VARCHAR2(1)
ORDER_FLAG	VARCHAR2(1)
CACHE_SIZE	NUMBER
LAST_NUMBER	NUMBER
SCALE_FLAG	VARCHAR2(1)
EXTEND_FLAG	VARCHAR2(1)
SHARDED_FLAG	VARCHAR2(1)
SESSION_FLAG	VARCHAR2(1)
KEEP_VALUE	VARCHAR2(1)

- Verify your sequence values in the `USER_SEQUENCES` data dictionary table.

```
SELECT sequence_name, min_value, max_value,  
       increment_by, last_number  
FROM user_sequences;
```

The `USER_SEQUENCES` view describes all sequences that you own. When you create the sequence, you specify criteria that are stored in the `USER_SEQUENCES` view. The following are the columns in this view:

- SEQUENCE_NAME:** Name of the sequence
- MIN_VALUE:** Minimum value of the sequence
- MAX_VALUE:** Maximum value of the sequence
- INCREMENT_BY:** Value by which the sequence is incremented
- CYCLE_FLAG:** Whether sequence wraps around on reaching the limit
- ORDER_FLAG:** Whether sequence numbers are generated in order
- CACHE_SIZE:** Number of sequence numbers to cache
- LAST_NUMBER:** Last sequence number written to disk. If a sequence uses caching, the number written to disk is the last number placed in the sequence cache. If `NOCACHE` is specified, the `LAST_NUMBER` column displays the next available sequence number
- SCALE_FLAG:** Indicates whether this is a scalable sequence (Y) or not (N)
- EXTEND_FLAG:** Indicates whether this scalable sequence's generated values extend beyond `MAX_VALUE` or `MIN_VALUE` (Y) or not (N)
- SHARDED_FLAG:** Indicates whether this is a sharded sequence (Y) or not (N)
- SESSION_FLAG:** Whether the order value is session private
- KEEP_VALUE:** Response time after an error

After creating your sequence, it is documented in the data dictionary. Because a sequence is a database object, you can identify it in the `USER_OBJECTS` data dictionary table.

You can also confirm the settings of the sequence by selecting from the `USER_SEQUENCES` data dictionary view.



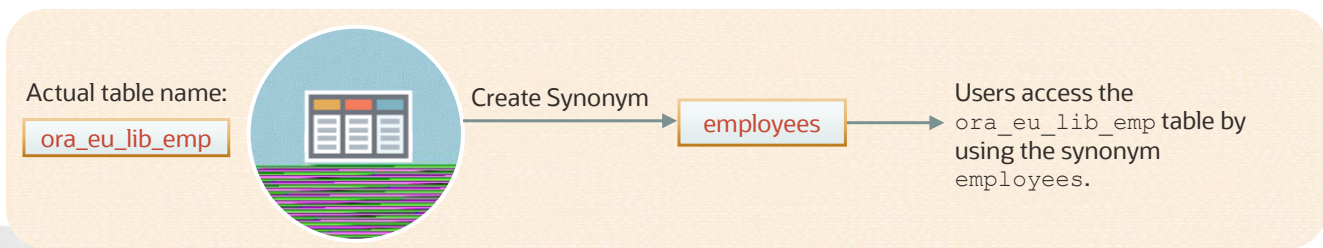
Create Private and Public Synonyms

For Instructor Use Only.
This document should not be distributed

Synonyms

A synonym:

- Is a database object
- Can be created to give an alternative name to a table or to another database object
- Requires no storage other than its definition in the data dictionary
- Is useful for hiding the identity and location of an underlying schema object



Synonyms are database objects that enable you to call a table by another name.

You can create synonyms to give an alternative name to a table or to another database object. For example, you can create a synonym for a table or view, sequence, PL/SQL program unit, user-defined object type, or another synonym.

Because a synonym is simply an alias, it requires no storage other than its definition in the data dictionary.

Synonyms can simplify SQL statements for database users. Synonyms are also useful for hiding the identity and location of an underlying schema object.

Creating a Synonym for an Object

- You can simplify access to objects by creating a synonym.
- With synonyms, you can:
 - Create an easier reference to a table that is owned by another user
 - Shorten lengthy object names

```
CREATE [PUBLIC] SYNONYM synonym  
FOR object;
```

To refer to a table that is owned by another user, you need to prefix the table name with the name of the user who created it, followed by a period. Creating a synonym eliminates the need to qualify the object name with the schema and provides you with an alternative name for a table, view, sequence, procedure, or other objects. This method can be especially useful with lengthy object names, such as views.

In the syntax:

<code>PUBLIC</code>	Creates a synonym that is accessible to all users
<code>synonym</code>	Is the name of the synonym to be created
<code>object</code>	Identifies the object for which the synonym is created

Guidelines

- The object cannot be contained in a package.
- A private synonym name must be distinct from all other objects that are owned by the same user.
- To create a `PUBLIC` synonym, you must have the `CREATE PUBLIC SYNONYM` system privilege.

For more information, see the section on “`CREATE SYNONYM`” in *Oracle Database SQL Language Reference* for Oracle Database 23c.

Creating and Removing Synonyms

- Create a shortened name for the `DEPARTMENTS` table:

```
CREATE SYNONYM dept
FOR departments;

Synonym DEPT created.
```

- Drop a synonym:

```
DROP SYNONYM dept;

Synonym DEPT dropped.
```

Creating a Synonym

The example in the slide creates a synonym for the `DEPARTMENTS` table for quicker reference.

The database administrator can create a public synonym that is accessible to all users. The following example creates a public synonym named `DEPT` for Alice's `DEPARTMENTS` table:

```
CREATE PUBLIC SYNONYM dept
FOR alice.departments;
```

Removing a Synonym

To remove a synonym, use the `DROP SYNONYM` statement. Only the database administrator can drop a public synonym.

```
DROP PUBLIC SYNONYM dept;
```

For more information, see the section on “`DROP SYNONYM`” in *Oracle Database SQL Language Reference* for Oracle Database 23c.

Synonym Information

```
DESCRIBE user_synonyms
```

Name	Null	Type
SYNONYM_NAME	NOT NULL	VARCHAR2(128)
TABLE_OWNER		VARCHAR2(128)
TABLE_NAME	NOT NULL	VARCHAR2(128)
DB_LINK		VARCHAR2(128)
ORIGIN_CON_ID		NUMBER

```
SELECT *  
FROM user_synonyms;
```

SYNONYM_NAME	TABLE_OWNER	TABLE_NAME	DB_LINK	ORIGIN_CON_ID
1 DEPT	TEACH_B	DEPARTMENTS	(null)	3

The USER_SYNONYMS dictionary view describes private synonyms (synonyms that you own).

You can query this view to find your synonyms. You can query ALL_SYNONYMS to find out the name of all the synonyms that are available to you and the objects on which these synonyms apply.

The columns in this view are:

- SYNONYM_NAME: Name of the synonym
- TABLE_OWNER: Owner of the object that is referenced by the synonym
- TABLE_NAME: Name of the table or view that is referenced by the synonym
- DB_LINK: Name of the database link reference (if any)
- ORIGIN_CON_ID: The container ID of the PDB where the data originates. In this case, it is 3.



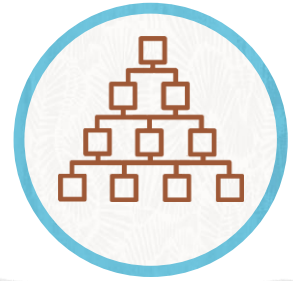
Create and Maintain Indexes

For Instructor Use Only.
This document should not be distributed

Indexes

An index:

- Is a schema object
- Can be used by the Oracle Server to speed up the retrieval of rows by using a pointer
- Can reduce disk input/output (I/O) by using a rapid path access method to locate data quickly
- Is dependent on the table that it indexes
- Is used and maintained automatically by the Oracle Server



You can use an index to speed up the retrieval of rows by using a pointer and improve the performance of some queries.

Indexes can be created explicitly or automatically. If you do not have an index on the column, a full table scan occurs.

An index provides direct and fast access to rows in a table. Its purpose is to reduce the disk I/O by using an indexed path to locate data quickly. An index is used and maintained automatically by the Oracle Server. After an index is created, no direct activity is required by the user.

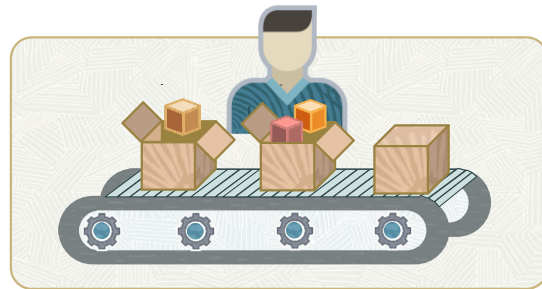
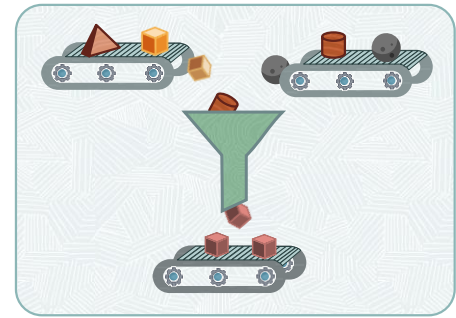
Indexes are logically and physically independent of the data in the objects with which they are associated. This means that they can be created or deleted at any time and have no effect on the base tables or other indexes.

Note: When you drop a table, the corresponding indexes are also dropped.

For more information, see the section on “Schema Objects: Indexes” in *Oracle Database Concepts 23c*.

How are indexes created?

- **Automatically:** A unique index is created automatically when you define a `PRIMARY KEY` or `UNIQUE` constraint in a table definition.
- **Manually:** You can create a unique or nonunique index on columns to speed up access to the rows.



You can create two types of indexes.

- **Unique index:** The Oracle Server automatically creates this index when you define a column in a table to have a `PRIMARY KEY` or a `UNIQUE` constraint. The name of the index is the name that is given to the constraint.
- **Nonunique index:** This is an index that a user can create. For example, you can create the `FOREIGN KEY` column index for a join in a query to improve the speed of retrieval.

Note: You can manually create a unique index, but it is recommended that you create a unique constraint, which implicitly creates a unique index.

Creating an Index

- Create an index on one or more columns:

```
CREATE [UNIQUE] INDEX index  
ON table (column[, column]...);
```

- Improve the speed of query access to the `LAST_NAME` column in the `EMPLOYEES` table:

```
CREATE INDEX emp_last_name_idx  
ON employees(last_name);
```

```
Index EMP_LAST_NAME_IDX created.
```

Create an index on one or more columns by issuing the `CREATE INDEX` statement.

In the syntax:

- `Index` Is the name of the index
- `Table` Is the name of the table
- `Column` Is the name of the column in the table to be indexed

Specify `UNIQUE` to indicate that the value of the column (or columns) on which the index is based must be unique. Specify `BITMAP` to indicate that the index is to be created with a bitmap for each distinct key, rather than indexing each row separately. Bitmap indexes store the `rowid` associated with a key value as a bitmap.

For more information, see the section on “`CREATE INDEX`” in *Oracle Database SQL Language Reference* for Oracle Database 23c.

CREATE INDEX with the CREATE TABLE Statement

```
CREATE TABLE NEW_EMP  
(employee_id NUMBER(6)  
    PRIMARY KEY USING INDEX  
    (CREATE INDEX emp_id_idx ON  
    NEW_EMP(employee_id)),  
first_name VARCHAR2(20),  
last_name VARCHAR2(25));
```

Table NEW_EMP created.

```
SELECT INDEX_NAME, TABLE_NAME  
FROM USER_INDEXES  
WHERE TABLE_NAME = 'NEW_EMP';
```

INDEX_NAME	TABLE_NAME
1 EMP_ID_IDX	NEW_EMP

In the example in the slide, the `CREATE INDEX` clause is used with the `CREATE TABLE` statement to create a `PRIMARY KEY` index explicitly. You can name your indexes at the time of `PRIMARY KEY` creation to be different from the name of the `PRIMARY KEY` constraint.

You can query the `USER_INDEXES` data dictionary view for information about your indexes.

The following example illustrates the database behavior if the index is not explicitly named:

```
CREATE TABLE EMP_UNNAMED_INDEX  
(employee_id NUMBER(6) PRIMARY KEY ,  
first_name VARCHAR2(20),  
last_name VARCHAR2(25));
```

```
SELECT INDEX_NAME, TABLE_NAME  
FROM USER_INDEXES  
WHERE TABLE_NAME = 'EMP_UNNAMED_INDEX';
```

Observe that the Oracle Server gives a generic name to the index that is created for the `PRIMARY KEY` column.

You can also use an existing index for your `PRIMARY KEY` column—for example, when you are expecting a large data load and want to speed up the operation. You may want to disable the constraints while performing the load and then enable them, in which case, having a unique index on the `PRIMARY KEY` will still cause the data to be verified during the load. Therefore, you can first create a nonunique index on the column designated as `PRIMARY KEY` and then create the `PRIMARY KEY` column and specify that it should use the existing index. The following examples illustrate this process:

Step 1: Create the table:

```
CREATE TABLE NEW_EMP2  
  
  (employee_id NUMBER(6),  
   first_name   VARCHAR2(20),  
   last_name    VARCHAR2(25)  
  
  );
```

Step 2: Create the index:

```
CREATE INDEX emp_id_idx2 ON  
  new_emp2(employee_id);
```

Step 3: Create the `PRIMARY KEY`:

```
ALTER TABLE new_emp2 ADD PRIMARY KEY   (employee_id) USING INDEX  
  emp_id_idx2;
```

Function-Based Indexes

- A function-based index is based on expressions.
- The index expression is built from table columns, constants, SQL functions, and user-defined functions.

```
CREATE INDEX upper_dept_name_idx  
ON dept2 (UPPER(department_name));
```

Index UPPER_DEPT_NAME_IDX created.

```
SELECT *  
FROM dept2  
WHERE UPPER(department_name) = 'SALES';
```

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	80 Sales	145	2500

1 row fetched in 0.012 seconds
*Note: The time may differ for you.

Function-based indexes defined with the `UPPER(column_name)` or `LOWER(column_name)` keywords allow non-case-sensitive searches. For example, consider the following index:

```
CREATE INDEX upper_last_name_idx ON emp2 (UPPER(last_name));
```

This facilitates processing queries such as:

```
SELECT * FROM emp2 WHERE UPPER(last_name) = 'KING';
```

The Oracle Server uses the index only when that particular function is used in a query. For example, the following statement may use the index; however, without the `WHERE` clause, the Oracle Server may perform a full table scan:

```
SELECT *  
FROM employees  
WHERE UPPER (last_name) IS NOT NULL  
ORDER BY UPPER (last_name);
```

Note: For creating a function-based index, you need the `QUERY REWRITE` system privilege. The `QUERY_REWRITE_ENABLED` initialization parameter must be set to `TRUE` for a function-based index to be used.

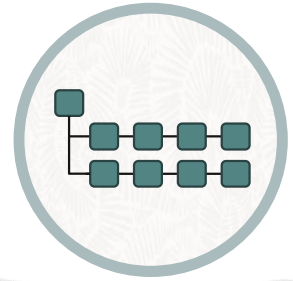
The Oracle Server treats indexes with columns marked `DESC` as function-based indexes. The columns marked `DESC` are sorted in descending order.

Observe that each time you run the same query on an index-based table, the time required to fetch the row(s) reduces.

Creating Multiple Indexes on the Same Set of Columns

You can create multiple indexes on the same set of columns if the indexes:

- Are of different types
- Use different partitioning
- Have different uniqueness properties



You can create multiple indexes on the same set of columns if the indexes are of different types, use different partitioning, or have different uniqueness properties. For example, you can create a B-tree index and a bitmap index on the same set of columns.

Similarly, you can create a unique and nonunique index on the same set of columns.

When you have multiple indexes on the same set of columns, only one of these indexes can be visible at a time.

Creating Multiple Indexes on the Same Set of Columns: Example

1
`CREATE INDEX emp_id_name_ix1`
`ON employees(employee_id, first_name);`
Index EMP_ID_NAME_IX1 created.

2
`ALTER INDEX emp_id_name_ix1 INVISIBLE;`
Index EMP_ID_NAME_IX1 altered.

3
`CREATE BITMAP INDEX emp_id_name_ix2`
`ON employees(employee_id, first_name);`
Bitmap index EMP_ID_NAME_IX2 created.

1. The code example shows the creation of a B-tree index, `emp_id_name_ix1`, on the `employee_id` and `first_name` column of the `employees` table in the `HR` schema. By default, Oracle Server creates a B-tree index. In a B-tree, you traverse the branches until you get to the node that has the data you are searching for.
2. After the creation of the index, it is altered to make it invisible.
3. Then a bitmap index is created on the `employee_id` and `first_name` column of the `employees` table in the `HR` schema. The bitmap index, `emp_id_name_ix2`, is visible by default.

Index Information

- `USER_INDEXES` provides information about your indexes.
- `USER_IND_COLUMNS` describes columns of indexes owned by you and columns of indexes on your tables.

```
DESCRIBE user_indexes
```

Name	Null	Type
INDEX_NAME	NOT NULL	VARCHAR2(128)
INDEX_TYPE		VARCHAR2(27)
TABLE_OWNER	NOT NULL	VARCHAR2(128)
TABLE_NAME	NOT NULL	VARCHAR2(128)
TABLE_TYPE		VARCHAR2(11)
UNIQUENESS		VARCHAR2(9)
COMPRESSION		VARCHAR2(13)
PREFIX_LENGTH		NUMBER
TABLESPACE_NAME		VARCHAR2(30)
...		



You query the `USER_INDEXES` view to find out the names of your indexes, the table name on which the index is created, and whether the index is unique.

Note: For a complete listing and description of the columns in the `USER_INDEXES` view, see “`USER_INDEXES`” in *Oracle® Database Reference 23c*.

USER_INDEXES: Examples

```

SELECT index_name, table_name, uniqueness
FROM   user_indexes
WHERE  table_name = 'EMPLOYEES';

```

1

	INDEX_NAME	TABLE_NAME	UNIQUENESS
1	EMP_EMP_ID_PK	EMPLOYEES	UNIQUE
2	EMP_DEPARTMENT_IX	EMPLOYEES	NONUNIQUE
3	EMP_JOB_IX	EMPLOYEES	NONUNIQUE
4	EMP_MANAGER_IX	EMPLOYEES	NONUNIQUE
5	EMP_NAME_IX	EMPLOYEES	NONUNIQUE

...

```

SELECT index_name, table_name
FROM   user_indexes
WHERE  table_name = 'EMP_LIB';

```

2

	INDEX_NAME	TABLE_NAME
1	SYS_C0010979	EMP_LIB

In example 1 in the slide, the USER_INDEXES view is queried to find the name of the index, name of the table on which the index is created, and whether the index is unique.

In example 2 in the slide, observe that the Oracle Server gives a generic name to the index that is created for the PRIMARY KEY column. The EMP_LIB table is created by using the following code:

```

CREATE TABLE emp_lib
(
    book_id NUMBER(6) PRIMARY KEY,
    title VARCHAR2(25),
    category VARCHAR2(20));

```

Querying USER_IND_COLUMNS

```
DESCRIBE user_ind_columns
```

Name	Null	Type
INDEX_NAME		VARCHAR2(128)
TABLE_NAME		VARCHAR2(128)
COLUMN_NAME		VARCHAR2(4000)
COLUMN_POSITION		NUMBER
COLUMN_LENGTH		NUMBER
CHAR_LENGTH		NUMBER
DESCEND		VARCHAR2(4)

```
SELECT index_name, column_name, table_name
FROM   user_ind_columns
WHERE  index_name = 'LNAME_IDX';
```

	INDEX_NAME	COLUMN_NAME	TABLE_NAME
1	LNAME_IDX	LAST_NAME	EMP_TEST

The USER_IND_COLUMNS dictionary view provides information, such as the name of the index, name of the indexed table, name of a column within the index, and the column's position within the index.

For the example in the slide, the emp_test table and LNAME_IDX index are created by using the following code:

```
CREATE TABLE emp_test AS SELECT * FROM employees;
```

```
CREATE INDEX lname_idx ON emp_test(last_name);
```


Removing an Index

- Remove an index from the data dictionary by using the `DROP INDEX` command:

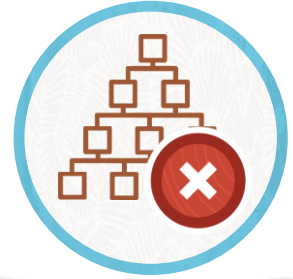
```
DROP INDEX index;
```

- Remove the `emp_last_name_idx` index from the data dictionary:

```
DROP INDEX emp_last_name_idx;
```

```
Index EMP_LAST_NAME_IDX dropped.
```

- To drop an index, you must be the owner of the index or have the `DROP ANY INDEX` privilege.



You cannot modify indexes. To change an index, you must drop it and then re-create it.

You can remove an index definition from the data dictionary by issuing the `DROP INDEX` statement. To drop an index, you must be the owner of the index or have the `DROP ANY INDEX` privilege.

In the syntax, *index* is the name of the index.

You can drop an index by using the `ONLINE` keyword:

```
DROP INDEX emp_idx ONLINE;
```

Specify `ONLINE` to indicate that DML operations on the table are allowed while dropping the index.

Note: If you drop a table, indexes and constraints are automatically dropped, but views remain.

Summary



Create, maintain, and use sequences

Create private and public synonyms

Create and maintain indexes

In this lesson, you should have learned about database objects, such as sequences, indexes, and synonyms.

For Instructor Use Only.
This document should not be distributed

For Instructor Use Only.
This document should not be distributed

Creating Views



Objectives



Overview of views

Creating, modifying, and retrieving data from a view

Data manipulation language (DML) operations on a view

Dropping a view

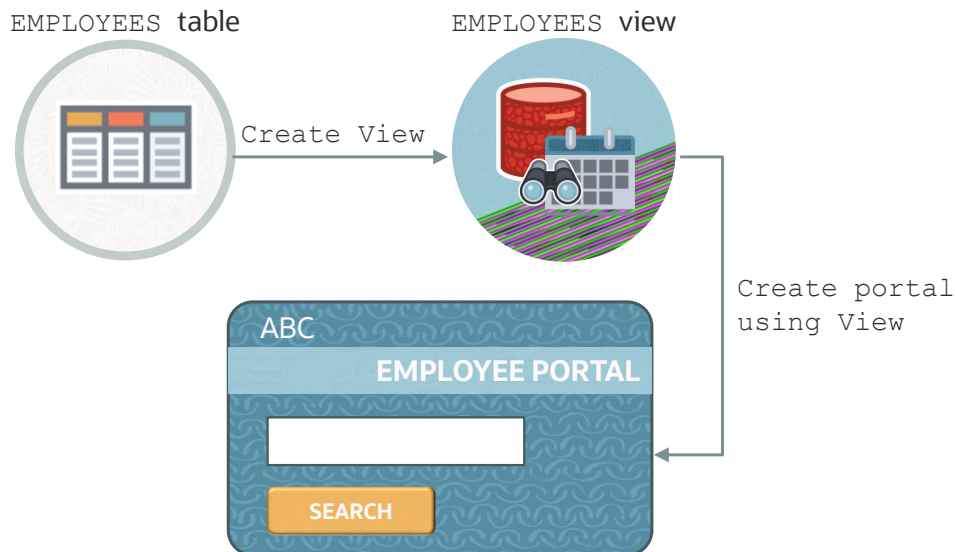
In this lesson, you are introduced to views, and you learn the basics of creating and using views.



Overview of Views

For Instructor Use Only.
This document should not be distributed

Why views?



ABC is a company, which consists of around 1000 employees. The HR department of the company maintains a database to store all the employee information, such as full name, date of birth, department, salary, manager, designation, phone number, hire date, and so on. This information is confidential and cannot be accessed by all the employees.

The company now decides to create an employee search portal for the use of employees. This portal should display only the necessary information (such as name, department, and manager) and should *not* display any confidential information (such as salary). To achieve this, the company need not create a separate table, which stores the same data with fewer columns.

Instead, a view is created with the columns required based on the underlying `EMPLOYEES` table. This helps in removing redundancy and hiding sensitive information securely. You learn more about views and how to use them in the following slides.

Database Objects

Object	Description
Table	Basic unit of storage; composed of rows
View	Logically represents subsets of data from one or more tables
Sequence	Generates numeric values
Index	Improves the performance of data retrieval queries
Synonym	Gives alternative names to objects

There are several other objects in a database in addition to tables.

With views, you can present and hide data from the tables.

Many applications require the use of unique numbers as primary key values. You can either build code into the application to handle this requirement or use a sequence to generate unique numbers.

If you want to improve the performance of data retrieval queries, you should consider creating an index. You can also use indexes to enforce uniqueness on a column or a collection of columns.

You can provide alternative names for objects by using synonyms.

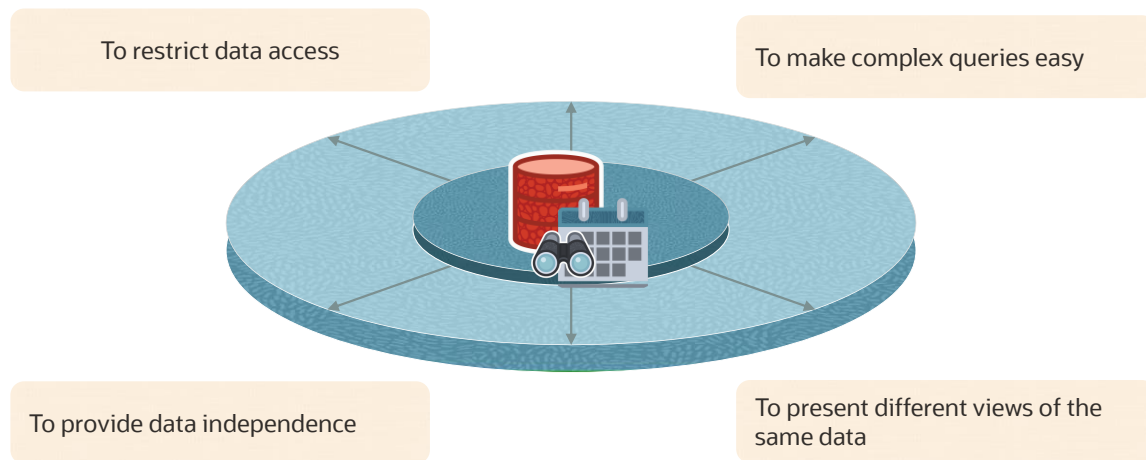
What is a view?

EMPLOYEES table

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY
100	Steven	King	SKING	515.123.4567	17-JUN-11	AD_PRES	24000
101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-09	AD_VP	17000
102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-09	AD_VP	17000
103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-14	IT_PROG	9000
104	Bruce	Ernst	BERNST	590.423.4568	21-MAY-15	IT_PROG	6000
105	David	Austin	DAUSTIN	590.423.4569	25-JUN-13	IT_PROG	4800
106	Valli	Pataballa	VPATABAL	590.423.4560	05-FEB-14	IT_PROG	4800
107	Diana	Lawrence	DLAWRENCE	590.423.5567	07-FEB-15	IT_PROG	4200
110	Clara	Favre	CFAVRE	590.423.4569	17-AUG-10	FI_MGR	12008
100	Steven	King		24000	16-AUG-10	FI_ACCOUNT	9000
101	Neena	Kochhar		17000	28-SEP-13	FI_ACCOUNT	8200
102	Lex	De Haan		17000	30-SEP-13	FI_ACCOUNT	7700
103	Alexander	Hunold		9000	07-MAR-14	FI_ACCOUNT	7800
104	Bruce	Ernst		6000	07-DEC-15	FI_ACCOUNT	6900
113	Luis	Popp	LPOPP	515.124.4567	07-DEC-10	PU_MAN	11000
114	Den	Raphaeely	DRAPHEAL	515.127.4561	18-MAY-11	PU_CLERK	3100
115	Alexander	Khoo	AKHOO	515.127.4562			

You can present logical subsets or combinations of data by creating views of tables. A view is a schema object and is stored as a `SELECT` statement based on a table or another view. A view contains no data of its own but is like a window through which data from tables can be viewed or changed. The tables on which a view is based are called *base tables*. The view is stored as a `SELECT` statement in the data dictionary.

Advantages of Views



The following are some of the advantages of views:

- Views restrict access to data because they display selected columns from the table.
- Views can be used to make simple queries to retrieve the results of complicated queries. For example, views can be used to query information from multiple tables without the user knowing how to write a join statement.
- Views provide data independence for ad hoc users and application programs. One view can be used to retrieve data from several tables.
- Views provide groups of users access to data according to their particular criteria.

For more information, see the “`CREATE VIEW`” section in *Oracle Database SQL Language Reference* for Oracle Database 20c.

Simple Views and Complex Views

Feature	Simple Views	Complex Views
Number of tables	One	One or more
Contain functions	No	Yes
Contain groups of data	No	Yes
DML operations through a view	Yes	Not always

There are two classifications for views: simple and complex. The basic difference is related to the DML (INSERT, UPDATE, and DELETE) operations.

- A simple view:
 - Derives data from only one table
 - Contains no functions or groups of data
 - Usually performs DML operations through the view
- A complex view:
 - Derives data from many tables
 - Contains functions or groups of data
 - Does not always allow DML operations through the view



Creating, Modifying, and Retrieving Data from a View

For Instructor Use Only.
This document should not be distributed

Creating a View

- You embed a subquery in the `CREATE VIEW` statement:

```
CREATE [OR REPLACE] [FORCE|NOFORCE] VIEW view
[(alias[, alias]...)]
AS subquery
[WITH CHECK OPTION [CONSTRAINT constraint]]
[WITH READ ONLY [CONSTRAINT constraint]];
```

- The subquery can contain complex `SELECT` syntax.



You can create a view by embedding a subquery in the `CREATE VIEW` statement.

In the syntax:

<code>OR REPLACE</code>	Re-creates the view if it already exists. You can use this clause to change the definition of an existing view without dropping, re-creating, and regranting object privileges previously granted on it.
<code>FORCE</code>	Creates the view regardless of whether or not the base tables exist
<code>NOFORCE</code>	Creates the view only if the base tables exist (This is the default.)
<code>view</code>	Is the name of the view
<code>alias</code>	Specifies names for the expressions selected by the view's query (The number of aliases must match the number of expressions selected by the view.)
<code>subquery</code>	Is a complete <code>SELECT</code> statement (You can use aliases for the columns in the <code>SELECT</code> list.)
<code>WITH CHECK OPTION</code>	Specifies that only those rows that are accessible to the view can be inserted or updated
<code>Constraint</code>	Is the name assigned to the <code>CHECK OPTION</code> constraint
<code>WITH READ ONLY</code>	Ensures that no DML operations can be performed on this view

Note: In SQL Developer, click the Run Script icon or press F5 to run the data definition language (DDL) statements. The feedback messages will be shown on the Script Output tabbed page.

Creating a View

- Create the EMPVU80 view, which contains details of the employees in department 80:

```
CREATE VIEW empvu80
AS SELECT employee_id, last_name, salary
FROM employees
WHERE department_id = 80;
```

View EMPVU80 created.

- Describe the structure of the view by using the SQL*Plus DESCRIBE command:

```
DESCRIBE empvu80;
```

The example in the slide creates a view that contains the employee number, last name, and salary for each employee in department 80.

You can display the structure of the view by using the DESCRIBE command.

Name	Null	Type
EMPLOYEE_ID	NOT NULL	NUMBER(6)
LAST_NAME	NOT NULL	VARCHAR2(25)
SALARY		NUMBER(8, 2)

Guidelines

- The subquery that defines a view can contain complex SELECT syntax, including joins, groups, and subqueries.
- If you do not specify a constraint name for the view created with the WITH CHECK OPTION, the system assigns a default name in the SYS_Cn format.
- You can use the OR REPLACE option to change the definition of the view without dropping and re-creating it, or regranting the object privileges previously granted on it.

Creating a View

- Create a view by using column aliases in the subquery:

```
CREATE VIEW salvu50
AS SELECT employee_id ID_NUMBER, last_name NAME,
          salary*12 ANN_SALARY
FROM employees
WHERE department_id = 50;
```

View SALVU50 created.

- Select the columns from this view by the given alias names:

```
SELECT ID_NUMBER, NAME, ANN_SALARY
FROM salvu50;
```

You can control the column names by including column aliases in the subquery.

The example in the slide creates a view containing the employee number (`EMPLOYEE_ID`) with the alias `ID_NUMBER`, name (`LAST_NAME`) with the alias `NAME`, and annual salary (`SALARY`) with the alias `ANN_SALARY` for every employee in department 50.

Alternatively, you can use an alias after the `CREATE` statement and before the `SELECT` subquery. The number of aliases listed must match the number of expressions selected in the subquery.

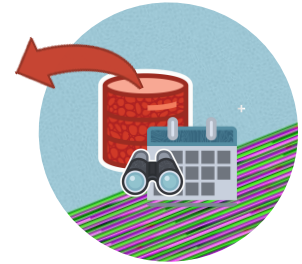
```
CREATE OR REPLACE VIEW salvu50 (ID_NUMBER, NAME, ANN_SALARY)
AS SELECT employee_id, last_name, salary*12
FROM employees
WHERE department_id = 50;
```

Retrieving Data from a View

```
SELECT *  
FROM salvu50;
```

	ID_NUMBER	NAME	ANN_SALARY
1	120	Weiss	96000
2	121	Fripp	98400
3	122	Kaufling	94800
4	123	Vollman	78000
5	124	Mourgos	69600
6	125	Nayer	38400
7	126	Mikkilineni	32400

...



You can retrieve data from a view as you would from any table. You can display either the contents of the entire view or just specific rows and columns.

Modifying a View

- Modify the EMPVU80 view by using a `CREATE OR REPLACE VIEW` clause. Add an alias for each column name:

```
CREATE OR REPLACE VIEW empvu80
(id_number, name, sal, department_id)
AS SELECT employee_id, first_name || ' '
        || last_name, salary, department_id
FROM employees
WHERE department_id = 80;
```

View EMPVU80 created.

- Column aliases in the `CREATE OR REPLACE VIEW` clause are listed in the same order as the columns in the subquery.

With the `REPLACE` option, a view can be created even if one exists with this name already, thus replacing the old version of the view for its owner. This means that the view can be altered without dropping, re-creating, and regranteeing object privileges.

Note: When assigning column aliases in the `CREATE OR REPLACE VIEW` clause, remember that the aliases are listed in the same order as the columns in the subquery.

Creating a Complex View

Create a complex view that contains group functions to display values from two tables:

```
CREATE OR REPLACE VIEW dept_sum_vu
(name, minsal, maxsal, avgsal)
AS SELECT  d.department_name, MIN(e.salary),
           MAX(e.salary), AVG(e.salary)
FROM      employees e JOIN departments d
USING     (department id)
GROUP BY  d.department_name;
```

View DEPT_SUM_VU created.

The example in the slide creates a complex view of department names, minimum salaries, maximum salaries, and the average salaries by department. Note that alternative names have been specified for the view. This is a requirement if any column of the view is derived from a function or an expression.

You can view the structure of the view by using the `DESCRIBE` command. Display the contents of the view by issuing a `SELECT` statement.

```
SELECT  *
FROM    dept_sum_vu;
```

View Information

DESCRIBE user_views

Name	Null	Type
VIEW_NAME	NOT NULL	VARCHAR2(128)
TEXT_LENGTH		NUMBER
TEXT		LONG
TEXT_VC		VARCHAR2(4000)
TYPE_TEXT_LENGTH		NUMBER
TYPE_TEXT		VARCHAR2(4000)

SELECT view_name FROM user_views;

VIEW_NAME
1 EMP_DETAILS_VIEW
2 SALVUSO
3 EMPVUSO
4 DEPT_SUM_VU

**SELECT text FROM user_views
WHERE view_name = 'EMP_DETAILS_VIEW';**

TEXT
1 SELECT e.employee_id, e.job_id, e.manager_id, e.department_id, d.location_id, l.co

AND c.region_id = r.region_id AND j.job_id = e.job_id WITH READ ONLY

After your view is created, you can query the data dictionary view called `USER_VIEWS` to see the name of the view and the view definition.

The text of the `SELECT` statement that constitutes your view is stored in a `LONG` column. The `TEXT_LENGTH` column is the number of characters in the `SELECT` statement. By default, when you select from a `LONG` column, only the first 80 characters of the column's value are displayed. To see more than 80 characters in SQL*Plus, use the `SET LONG` command:

```
SET LONG 1000
```

In the examples in the slide:

1. The `USER_VIEWS` columns are displayed. Note that this is a partial listing.
2. The names of your views are retrieved
3. The `SELECT` statement for the `EMP_DETAILS_VIEW` is displayed from the dictionary

Data Access Using Views

When you access data by using a view, the Oracle Server performs the following operations:

- It retrieves the view definition from the data dictionary table `USER_VIEWS`.
- It checks access privileges for the view base table.
- It converts the view query into an equivalent operation on the underlying base table or tables. That is, data is retrieved from, or an update is made to, the base tables.



Data Manipulation Language (DML) Operations on a View

For Instructor Use Only.
This document should not be distributed

Rules for Performing DML Operations on a View

- You can usually perform DML operations on simple views.
- You cannot remove a row if the view contains:
 - Group functions
 - A GROUP BY clause
 - The DISTINCT keyword
 - The pseudocolumn ROWNUM keyword



You can perform DML operations on data in a view if those operations follow certain rules.

You can remove a row from a view unless it contains any of the following:

- Group functions
- A GROUP BY clause
- The DISTINCT keyword
- The pseudocolumn ROWNUM keyword

Rules for Performing Modify Operations on a View

You cannot modify data in a view if it contains:

- Group functions
- A `GROUP BY` clause
- The `DISTINCT` keyword
- The pseudocolumn `ROWNUM` keyword
- Expressions

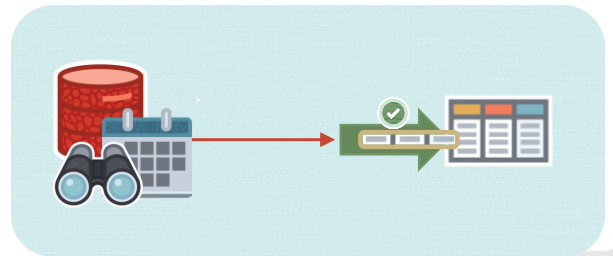


You can modify data in a view unless it contains any of the conditions mentioned in the slide.

Rules for Performing Insert Operations Through a View

You cannot add data in a view if the view includes:

- Group functions
- A `GROUP BY` clause
- The `DISTINCT` keyword
- The pseudocolumn `ROWNUM` keyword
- Columns defined by expressions
- `NOT NULL` columns without default value in the base tables that are not selected by the view



You can add data through a view unless it contains any of the items listed in the slide. You cannot add data to a view if the view contains `NOT NULL` columns without default values in the base table. All the required values must be present in the view. Remember that you are adding values directly to the underlying table through the view.

For more information, see the “CREATE VIEW” section in Oracle Database SQL Language.

Using the WITH CHECK OPTION Clause

```
CREATE OR REPLACE VIEW empvu20
AS SELECT *
   FROM   employees
  WHERE   department_id = 20
  WITH CHECK OPTION CONSTRAINT empvu20_ck ;
```

View EMPVU20 created.

It is possible to perform referential integrity checks through views. You can also enforce constraints at the database level. The view can be used to protect data integrity, but the use is very limited.

The `WITH CHECK OPTION` clause specifies that `INSERTS` and `UPDATES` performed through the view cannot create rows that the view cannot select. Therefore, it enables integrity constraints and data validation checks to be enforced on data being inserted or updated. If there is an attempt to perform DML operations on rows that the view has not selected, an error is displayed, along with the constraint name if that has been specified.

```
UPDATE empvu20
SET     department_id = 10
WHERE   employee_id = 201;
```

Error:

```
SQL Error: ORA-01402: view WITH CHECK OPTION where-clause violation
01402. 00000 - "view WITH CHECK OPTION where-clause violation"
*Cause:
*Action:
```

Note: No rows are updated because, if the department number were to change to 10, the view would no longer be able to see that employee. With the `WITH CHECK OPTION` clause, therefore, the view can see only the employees in department 20 and does not allow the department number for those employees to be changed through the view.

Denying DML Operations

- You can ensure that no DML operations occur on your view by adding the `WITH READ ONLY` option to your view definition.
- Any attempt to perform a DML operation on any row in the view results in an Oracle server error.



The example in the following slide modifies the EMPVU10 view to prevent any DML operations on the view.

Denying DML Operations

```
CREATE OR REPLACE VIEW empvu10
  (employee_number, employee_name, job_title)
AS SELECT employee_id, last_name, job_id
   FROM   employees
  WHERE   department_id = 10
  WITH READ ONLY ;
```

View EMPVU10 created.

Any attempt to remove a row from a view with a read-only constraint results in an error:

```
DELETE FROM empvu10
WHERE employee_number = 200;
```

Similarly, any attempt to insert a row or modify a row using the view with a read-only constraint results in the same error.

```
Error report:
SQL Error: ORA-42399: cannot perform a DML operation on a read-only view
```



Dropping a View

For Instructor Use Only.
This document should not be distributed

Removing a View

You can remove a view without losing data because a view is based on underlying tables in the database.

Syntax:

```
DROP VIEW view;
```

Example:

```
DROP VIEW empvu80;
```

```
View EMPVU80 dropped.
```



You use the `DROP VIEW` statement to remove a view. The statement removes the view definition from the database. However, dropping views has no effect on the tables on which the view was based. Alternatively, views or other applications based on the deleted views become invalid. Only the creator or a user with the `DROP ANY VIEW` privilege can remove a view.

In the syntax, *view* is the name of the view.

Summary



Overview of views

Creating, modifying, and retrieving data from a view

Data manipulation language (DML) operations on a view

Dropping a view

In this lesson, you should have learned about views.